



WebAssembly Specification

Release 2.0 (Draft 2023-07-11)

WebAssembly Community Group

Andreas Rossberg (editor)

Jul 11, 2023

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Overview	3
2	Structure	5
2.1	Conventions	5
2.2	Values	7
2.3	Types	9
2.4	Instructions	12
2.5	Modules	19
3	Validation	25
3.1	Conventions	25
3.2	Types	28
3.3	Instructions	31
3.4	Modules	47
4	Execution	55
4.1	Conventions	55
4.2	Runtime Structure	57
4.3	Numerics	64
4.4	Instructions	87
4.5	Modules	121
5	Binary Format	133
5.1	Conventions	133
5.2	Values	135
5.3	Types	136
5.4	Instructions	138
5.5	Modules	149
6	Text Format	157
6.1	Conventions	157
6.2	Lexical Format	159
6.3	Values	161
6.4	Types	163
6.5	Instructions	164
6.6	Modules	177
7	Appendix	185
7.1	Embedding	185

7.2	Implementation Limitations	192
7.3	Validation Algorithm	194
7.4	Custom Sections	198
7.5	Soundness	200
7.6	Change History	209
	Index	213

1.1 Introduction

WebAssembly (abbreviated Wasm²) is a *safe, portable, low-level code format* designed for efficient execution and compact representation. Its main goal is to enable high performance applications on the Web, but it does not make any Web-specific assumptions or provide Web-specific features, so it can be employed in other environments as well.

WebAssembly is an open standard developed by a [W3C Community Group](https://www.w3.org/community/webassembly/)¹.

This document describes version 2.0 (Draft 2023-07-11) of the [core](#) WebAssembly standard. It is intended that it will be superseded by new incremental releases with additional features in the future.

1.1.1 Design Goals

The design goals of WebAssembly are the following:

- Fast, safe, and portable *semantics*:
 - **Fast**: executes with near native code performance, taking advantage of capabilities common to all contemporary hardware.
 - **Safe**: code is validated and executes in a memory-safe³, sandboxed environment preventing data corruption or security breaches.
 - **Well-defined**: fully and precisely defines valid programs and their behavior in a way that is easy to reason about informally and formally.
 - **Hardware-independent**: can be compiled on all modern architectures, desktop or mobile devices and embedded systems alike.
 - **Language-independent**: does not privilege any particular language, programming model, or object model.
 - **Platform-independent**: can be embedded in browsers, run as a stand-alone VM, or integrated in other environments.

² A contraction of “WebAssembly”, not an acronym, hence not using all-caps.

¹ <https://www.w3.org/community/webassembly/>

³ No program can break WebAssembly’s memory model. Of course, it cannot guarantee that an unsafe language compiling to WebAssembly does not corrupt its own memory layout, e.g. inside WebAssembly’s linear memory.

- **Open:** programs can interoperate with their environment in a simple and universal manner.
- Efficient and portable *representation*:
 - **Compact:** has a binary format that is fast to transmit by being smaller than typical text or native code formats.
 - **Modular:** programs can be split up in smaller parts that can be transmitted, cached, and consumed separately.
 - **Efficient:** can be decoded, validated, and compiled in a fast single pass, equally with either just-in-time (JIT) or ahead-of-time (AOT) compilation.
 - **Streamable:** allows decoding, validation, and compilation to begin as soon as possible, before all data has been seen.
 - **Parallelizable:** allows decoding, validation, and compilation to be split into many independent parallel tasks.
 - **Portable:** makes no architectural assumptions that are not broadly supported across modern hardware.

WebAssembly code is also intended to be easy to inspect and debug, especially in environments like web browsers, but such features are beyond the scope of this specification.

1.1.2 Scope

At its core, WebAssembly is a *virtual instruction set architecture (virtual ISA)*. As such, it has many use cases and can be embedded in many different environments. To encompass their variety and enable maximum reuse, the WebAssembly specification is split and layered into several documents.

This document is concerned with the core ISA layer of WebAssembly. It defines the instruction set, binary encoding, validation, and execution semantics, as well as a textual representation. It does not, however, define how WebAssembly programs can interact with a specific environment they execute in, nor how they are invoked from such an environment.

Instead, this specification is complemented by additional documents defining interfaces to specific embedding environments such as the Web. These will each define a WebAssembly *application programming interface (API)* suitable for a given environment.

1.1.3 Security Considerations

WebAssembly provides no ambient access to the computing environment in which code is executed. Any interaction with the environment, such as I/O, access to resources, or operating system calls, can only be performed by invoking [functions](#) provided by the [embedder](#) and imported into a WebAssembly [module](#). An embedder can establish security policies suitable for a respective environment by controlling or limiting which functional capabilities it makes available for import. Such considerations are an embedder's responsibility and the subject of [API definitions](#) for a specific environment.

Because WebAssembly is designed to be translated into machine code running directly on the host's hardware, it is potentially vulnerable to side channel attacks on the hardware level. In environments where this is a concern, an embedder may have to put suitable mitigations into place to isolate WebAssembly computations.

1.1.4 Dependencies

WebAssembly depends on two existing standards:

- [IEEE 754⁴](#), for the representation of [floating-point data](#) and the semantics of respective [numeric operations](#).
- [Unicode⁵](#), for the representation of [import/export names](#) and the [text format](#).

However, to make this specification self-contained, relevant aspects of the aforementioned standards are defined and formalized as part of this specification, such as the [binary representation](#) and [rounding](#) of floating-point values, and the [value range](#) and [UTF-8 encoding](#) of Unicode characters.

Note: The aforementioned standards are the authoritative source of all respective definitions. Formalizations given in this specification are intended to match these definitions. Any discrepancy in the syntax or semantics described is to be considered an error.

1.2 Overview

1.2.1 Concepts

WebAssembly encodes a low-level, assembly-like programming language. This language is structured around the following concepts.

Values

WebAssembly provides only four basic *number types*. These are integers and [IEEE 754⁶](#) numbers, each in 32 and 64 bit width. 32 bit integers also serve as Booleans and as memory addresses. The usual operations on these types are available, including the full matrix of conversions between them. There is no distinction between signed and unsigned integer types. Instead, integers are interpreted by respective operations as either unsigned or signed in two's complement representation.

In addition to these basic number types, there is a single 128 bit wide vector type representing different types of packed data. The supported representations are 4 32-bit, or 2 64-bit [IEEE 754⁷](#) numbers, or different widths of packed integer values, specifically 2 64-bit integers, 4 32-bit integers, 8 16-bit integers, or 16 8-bit integers.

Finally, values can consist of opaque *references* that represent pointers towards different sorts of entities. Unlike with other types, their size or representation is not observable.

Instructions

The computational model of WebAssembly is based on a *stack machine*. Code consists of sequences of *instructions* that are executed in order. Instructions manipulate values on an implicit *operand stack*⁸ and fall into two main categories. *Simple* instructions perform basic operations on data. They pop arguments from the operand stack and push results back to it. *Control* instructions alter control flow. Control flow is *structured*, meaning it is expressed with well-nested constructs such as blocks, loops, and conditionals. Branches can only target such constructs.

Traps

Under some conditions, certain instructions may produce a *trap*, which immediately aborts execution. Traps cannot be handled by WebAssembly code, but are reported to the outside environment, where they typically can be caught.

Functions

Code is organized into separate *functions*. Each function takes a sequence of values as parameters and returns

⁴ <https://ieeexplore.ieee.org/document/8766229>

⁵ <https://www.unicode.org/versions/latest/>

⁶ <https://ieeexplore.ieee.org/document/8766229>

⁷ <https://ieeexplore.ieee.org/document/8766229>

⁸ In practice, implementations need not maintain an actual operand stack. Instead, the stack can be viewed as a set of anonymous registers that are implicitly referenced by instructions. The [type system](#) ensures that the stack height, and thus any referenced register, is always known statically.

a sequence of values as results. Functions can call each other, including recursively, resulting in an implicit call stack that cannot be accessed directly. Functions may also declare mutable *local variables* that are usable as virtual registers.

Tables

A *table* is an array of opaque values of a particular *element type*. It allows programs to select such values indirectly through a dynamic index operand. Currently, the only available element type is an untyped function reference or a reference to an external host value. Thereby, a program can call functions indirectly through a dynamic index into a table. For example, this allows emulating function pointers by way of table indices.

Linear Memory

A *linear memory* is a contiguous, mutable array of raw bytes. Such a memory is created with an initial size but can be grown dynamically. A program can load and store values from/to a linear memory at any byte address (including unaligned). Integer loads and stores can specify a *storage size* which is smaller than the size of the respective value type. A trap occurs if an access is not within the bounds of the current memory size.

Modules

A WebAssembly binary takes the form of a *module* that contains definitions for functions, tables, and linear memories, as well as mutable or immutable *global variables*. Definitions can also be *imported*, specifying a module/name pair and a suitable type. Each definition can optionally be *exported* under one or more names. In addition to definitions, modules can define initialization data for their memories or tables that takes the form of *segments* copied to given offsets. They can also define a *start function* that is automatically executed.

Embedder

A WebAssembly implementation will typically be *embedded* into a *host* environment. This environment defines how loading of modules is initiated, how imports are provided (including host-side definitions), and how exports can be accessed. However, the details of any particular embedding are beyond the scope of this specification, and will instead be provided by complementary, environment-specific API definitions.

1.2.2 Semantic Phases

Conceptually, the semantics of WebAssembly is divided into three phases. For each part of the language, the specification specifies each of them.

Decoding

WebAssembly modules are distributed in a *binary format*. *Decoding* processes that format and converts it into an internal representation of a module. In this specification, this representation is modelled by *abstract syntax*, but a real implementation could compile directly to machine code instead.

Validation

A decoded module has to be *valid*. Validation checks a number of well-formedness conditions to guarantee that the module is meaningful and safe. In particular, it performs *type checking* of functions and the instruction sequences in their bodies, ensuring for example that the operand stack is used consistently.

Execution

Finally, a valid module can be *executed*. Execution can be further divided into two phases:

Instantiation. A module *instance* is the dynamic representation of a module, complete with its own state and execution stack. Instantiation executes the module body itself, given definitions for all its imports. It initializes globals, memories and tables and invokes the module's start function if defined. It returns the instances of the module's exports.

Invocation. Once instantiated, further WebAssembly computations can be initiated by *invoking* an exported function on a module instance. Given the required arguments, that executes the respective function and returns its results.

Instantiation and invocation are operations within the embedding environment.

2.1 Conventions

WebAssembly is a programming language that has multiple concrete representations (its [binary format](#) and the [text format](#)). Both map to a common structure. For conciseness, this structure is described in the form of an *abstract syntax*. All parts of this specification are defined in terms of this abstract syntax.

2.1.1 Grammar Notation

The following conventions are adopted in defining grammar rules for abstract syntax.

- Terminal symbols (atoms) are written in sans-serif font or in symbolic form: `i32`, `end`, `→`, `[`, `]`.
- Nonterminal symbols are written in italic font: *valtype*, *instr*.
- A^n is a sequence of $n \geq 0$ iterations of A .
- A^* is a possibly empty sequence of iterations of A . (This is a shorthand for A^n used where n is not relevant.)
- A^+ is a non-empty sequence of iterations of A . (This is a shorthand for A^n where $n \geq 1$.)
- $A^?$ is an optional occurrence of A . (This is a shorthand for A^n where $n \leq 1$.)
- Productions are written $sym ::= A_1 \mid \dots \mid A_n$.
- Large productions may be split into multiple definitions, indicated by ending the first one with explicit ellipses, $sym ::= A_1 \mid \dots$, and starting continuations with ellipses, $sym ::= \dots \mid A_2$.
- Some productions are augmented with side conditions in parentheses, “(if *condition*)”, that provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production, then all those occurrences must have the same instantiation. (This is a shorthand for a side condition requiring multiple different variables to be equal.)

2.1.2 Auxiliary Notation

When dealing with syntactic constructs the following notation is also used:

- ϵ denotes the empty sequence.
- $|s|$ denotes the length of a sequence s .
- $s[i]$ denotes the i -th element of a sequence s , starting from 0.
- $s[i : n]$ denotes the sub-sequence $s[i] \dots s[i + n - 1]$ of a sequence s .
- $s \text{ with } [i] = A$ denotes the same sequence as s , except that the i -th element is replaced with A .
- $s \text{ with } [i : n] = A^n$ denotes the same sequence as s , except that the sub-sequence $s[i : n]$ is replaced with A^n .
- $\text{concat}(s^*)$ denotes the flat sequence formed by concatenating all sequences s_i in s^* .

Moreover, the following conventions are employed:

- The notation x^n , where x is a non-terminal symbol, is treated as a meta variable ranging over respective sequences of x (similarly for x^* , x^+ , $x^?$).
- When given a sequence x^n , then the occurrences of x in a sequence written $(A_1 x A_2)^n$ are assumed to be in point-wise correspondence with x^n (similarly for x^* , x^+ , $x^?$). This implicitly expresses a form of mapping syntactic constructions over a sequence.

Productions of the following form are interpreted as *records* that map a fixed set of fields field_i to “values” A_i , respectively:

$$r ::= \{ \text{field}_1 A_1, \text{field}_2 A_2, \dots \}$$

The following notation is adopted for manipulating such records:

- $r.\text{field}$ denotes the contents of the field component of r .
- $r \text{ with field} = A$ denotes the same record as r , except that the contents of the field component is replaced with A .
- $r_1 \oplus r_2$ denotes the composition of two records with the same fields of sequences by appending each sequence point-wise:

$$\{ \text{field}_1 A_1^*, \text{field}_2 A_2^*, \dots \} \oplus \{ \text{field}_1 B_1^*, \text{field}_2 B_2^*, \dots \} = \{ \text{field}_1 A_1^* B_1^*, \text{field}_2 A_2^* B_2^*, \dots \}$$

- $\bigoplus r^*$ denotes the composition of a sequence of records, respectively; if the sequence is empty, then all fields of the resulting record are empty.

The update notation for sequences and records generalizes recursively to nested components accessed by “paths” $pth ::= ([\dots] \mid \text{field})^+$:

- $s \text{ with } [i] pth = A$ is short for $s \text{ with } [i] = (s[i] \text{ with } pth = A)$,
- $r \text{ with field } pth = A$ is short for $r \text{ with field} = (r.\text{field} \text{ with } pth = A)$,

where $r \text{ with field} = A$ is shortened to $r \text{ with field} = A$.

2.1.3 Vectors

Vectors are bounded sequences of the form A^n (or A^*), where the A can either be values or complex constructions. A vector can have at most $2^{32} - 1$ elements.

$$\text{vec}(A) ::= A^n \quad (\text{if } n < 2^{32})$$

2.2 Values

WebAssembly programs operate on primitive numeric *values*. Moreover, in the definition of programs, immutable sequences of values occur to represent more complex data, such as text strings or other vectors.

2.2.1 Bytes

The simplest form of value are raw uninterpreted *bytes*. In the abstract syntax they are represented as hexadecimal literals.

$$\text{byte} ::= 0x00 \mid \dots \mid 0xFF$$

Conventions

- The meta variable b ranges over bytes.
- Bytes are sometimes interpreted as natural numbers $n < 256$.

2.2.2 Integers

Different classes of *integers* with different value ranges are distinguished by their *bit width* N and by whether they are *unsigned* or *signed*.

$$\begin{aligned} uN &::= 0 \mid 1 \mid \dots \mid 2^N - 1 \\ sN &::= -2^{N-1} \mid \dots \mid -1 \mid 0 \mid 1 \mid \dots \mid 2^{N-1} - 1 \\ iN &::= uN \end{aligned}$$

The class iN defines *uninterpreted* integers, whose signedness interpretation can vary depending on context. In the abstract syntax, they are represented as unsigned values. However, some operations *convert* them to signed based on a two's complement interpretation.

Note: The main integer types occurring in this specification are $u32$, $u64$, $s32$, $s64$, $i8$, $i16$, $i32$, $i64$. However, other sizes occur as auxiliary constructions, e.g., in the definition of *floating-point* numbers.

Conventions

- The meta variables m, n, i range over integers.
- Numbers may be denoted by simple arithmetics, as in the grammar above. In order to distinguish arithmetics like 2^N from sequences like $(1)^N$, the latter is distinguished with parentheses.

2.2.3 Floating-Point

Floating-point data represents 32 or 64 bit values that correspond to the respective binary formats of the [IEEE 754⁹](https://ieeexplore.ieee.org/document/8766229) standard (Section 3.3).

Every value has a *sign* and a *magnitude*. Magnitudes can either be expressed as *normal* numbers of the form $m_0.m_1m_2\dots m_M \cdot 2^e$, where e is the exponent and m is the *significand* whose most significant bit m_0 is 1, or as a *subnormal* number where the exponent is fixed to the smallest possible value and m_0 is 0; among the subnormals are positive and negative zero values. Since the significands are binary values, normals are represented in the form $(1 + m \cdot 2^{-M}) \cdot 2^e$, where M is the bit width of m ; similarly for subnormals.

⁹ <https://ieeexplore.ieee.org/document/8766229>

Possible magnitudes also include the special values ∞ (infinity) and `nan` (NaN, not a number). NaN values have a *payload* that describes the mantissa bits in the underlying [binary representation](#). No distinction is made between signalling and quiet NaNs.

$$\begin{aligned} fN &::= +fNmag \mid -fNmag \\ fNmag &::= \begin{array}{l} (1 + uM \cdot 2^{-M}) \cdot 2^e \quad (\text{if } -2^{E-1} + 2 \leq e \leq 2^{E-1} - 1) \\ (0 + uM \cdot 2^{-M}) \cdot 2^e \quad (\text{if } e = -2^{E-1} + 2) \\ \infty \\ \text{nan}(n) \quad (\text{if } 1 \leq n < 2^M) \end{array} \end{aligned}$$

where $M = \text{signif}(N)$ and $E = \text{expon}(N)$ with

$$\begin{array}{ll} \text{signif}(32) &= 23 & \text{expon}(32) &= 8 \\ \text{signif}(64) &= 52 & \text{expon}(64) &= 11 \end{array}$$

A *canonical NaN* is a floating-point value $\pm \text{nan}(\text{canon}_N)$ where canon_N is a payload whose most significant bit is 1 while all others are 0:

$$\text{canon}_N = 2^{\text{signif}(N)-1}$$

An *arithmetic NaN* is a floating-point value $\pm \text{nan}(n)$ with $n \geq \text{canon}_N$, such that the most significant bit is 1 while all others are arbitrary.

Note: In the abstract syntax, subnormals are distinguished by the leading 0 of the significand. The exponent of subnormals has the same value as the smallest possible exponent of a normal number. Only in the [binary representation](#) the exponent of a subnormal is encoded differently than the exponent of any normal number.

The notion of canonical NaN defined here is unrelated to the notion of canonical NaN that the [IEEE 754¹⁰](#) standard (Section 3.5.2) defines for decimal interchange formats.

Conventions

- The meta variable z ranges over floating-point values where clear from context.

2.2.4 Vectors

Numeric vectors are 128-bit values that are processed by vector instructions (also known as *SIMD* instructions, single instruction multiple data). They are represented in the abstract syntax using *i128*. The interpretation of lane types ([integer](#) or [floating-point](#) numbers) and lane sizes are determined by the specific instruction operating on them.

2.2.5 Names

Names are sequences of *characters*, which are *scalar values* as defined by [Unicode¹¹](#) (Section 2.4).

$$\begin{aligned} \text{name} &::= \text{char}^* \quad (\text{if } |\text{utf8}(\text{char}^*)| < 2^{32}) \\ \text{char} &::= \text{U+00} \mid \dots \mid \text{U+D7FF} \mid \text{U+E000} \mid \dots \mid \text{U+10FFFF} \end{aligned}$$

Due to the limitations of the [binary format](#), the length of a name is bounded by the length of its [UTF-8](#) encoding.

¹⁰ <https://ieeexplore.ieee.org/document/8766229>

¹¹ <https://www.unicode.org/versions/latest/>

Convention

- Characters (Unicode scalar values) are sometimes used interchangeably with natural numbers $n < 1114112$.

2.3 Types

Various entities in WebAssembly are classified by types. Types are checked during [validation](#), [instantiation](#), and possibly [execution](#).

2.3.1 Number Types

Number types classify numeric values.

$$\text{numtype} ::= \text{i32} \mid \text{i64} \mid \text{f32} \mid \text{f64}$$

The types [i32](#) and [i64](#) classify 32 and 64 bit integers, respectively. Integers are not inherently signed or unsigned, their interpretation is determined by individual operations.

The types [f32](#) and [f64](#) classify 32 and 64 bit floating-point data, respectively. They correspond to the respective binary floating-point representations, also known as *single* and *double* precision, as defined by the [IEEE 754](#)¹² standard (Section 3.3).

Number types are *transparent*, meaning that their bit patterns can be observed. Values of number type can be stored in [memories](#).

Conventions

- The notation $|t|$ denotes the *bit width* of a number type t . That is, $|\text{i32}| = |\text{f32}| = 32$ and $|\text{i64}| = |\text{f64}| = 64$.

2.3.2 Vector Types

Vector types classify vectors of [numeric](#) values processed by vector instructions (also known as *SIMD* instructions, single instruction multiple data).

$$\text{vectype} ::= \text{v128}$$

The type [v128](#) corresponds to a 128 bit vector of packed integer or floating-point data. The packed data can be interpreted as signed or unsigned integers, single or double precision floating-point values, or a single 128 bit type. The interpretation is determined by individual operations.

Vector types, like [number types](#) are *transparent*, meaning that their bit patterns can be observed. Values of vector type can be stored in [memories](#).

Conventions

- The notation $|t|$ for [bit width](#) extends to vector types as well, that is, $|\text{v128}| = 128$.

¹² <https://ieeexplore.ieee.org/document/8766229>

2.3.3 Reference Types

Reference types classify first-class references to objects in the runtime [store](#).

$$reftype ::= funcref \mid externref$$

The type [funcref](#) denotes the infinite union of all references to [functions](#), regardless of their [function types](#).

The type [externref](#) denotes the infinite union of all references to objects owned by the [embedder](#) and that can be passed into WebAssembly under this type.

Reference types are *opaque*, meaning that neither their size nor their bit pattern can be observed. Values of reference type can be stored in [tables](#).

2.3.4 Value Types

Value types classify the individual values that WebAssembly code can compute with and the values that a variable accepts. They are either [number types](#), [vector types](#), or [reference types](#).

$$valtype ::= numtype \mid vectype \mid reftype$$

Conventions

- The meta variable t ranges over value types or subclasses thereof where clear from context.

2.3.5 Result Types

Result types classify the result of [executing instructions](#) or [functions](#), which is a sequence of values, written with brackets.

$$resulttype ::= [vec(valtype)]$$

2.3.6 Function Types

Function types classify the signature of [functions](#), mapping a vector of parameters to a vector of results. They are also used to classify the inputs and outputs of [instructions](#).

$$functype ::= resulttype \rightarrow resulttype$$

2.3.7 Limits

Limits classify the size range of resizable storage associated with [memory types](#) and [table types](#).

$$limits ::= \{\min\ u32, \max\ u32^?\}$$

If no maximum is given, the respective storage can grow to any size.

2.3.8 Memory Types

Memory types classify linear [memories](#) and their size range.

$$\text{memtype} ::= \text{limits}$$

The limits constrain the minimum and optionally the maximum size of a memory. The limits are given in units of [page size](#).

2.3.9 Table Types

Table types classify [tables](#) over elements of [reference type](#) within a size range.

$$\text{tabletype} ::= \text{limits reftype}$$

Like memories, tables are constrained by limits for their minimum and optionally maximum size. The limits are given in numbers of entries.

Note: In future versions of WebAssembly, additional element types may be introduced.

2.3.10 Global Types

Global types classify [global](#) variables, which hold a value and can either be mutable or immutable.

$$\begin{aligned} \text{globaltype} &::= \text{mut valtype} \\ \text{mut} &::= \text{const} \mid \text{var} \end{aligned}$$

2.3.11 External Types

External types classify [imports](#) and [external values](#) with their respective types.

$$\text{externtype} ::= \text{func functype} \mid \text{table tabletype} \mid \text{mem memtype} \mid \text{global globaltype}$$

Conventions

The following auxiliary notation is defined for sequences of external types. It filters out entries of a specific kind in an order-preserving fashion:

- $\text{funcs}(\text{externtype}^*) = [\text{functype} \mid (\text{func functype}) \in \text{externtype}^*]$
- $\text{tables}(\text{externtype}^*) = [\text{tabletype} \mid (\text{table tabletype}) \in \text{externtype}^*]$
- $\text{mems}(\text{externtype}^*) = [\text{memtype} \mid (\text{mem memtype}) \in \text{externtype}^*]$
- $\text{globals}(\text{externtype}^*) = [\text{globaltype} \mid (\text{global globaltype}) \in \text{externtype}^*]$

2.4 Instructions

WebAssembly code consists of sequences of *instructions*. Its computational model is based on a *stack machine* in that instructions manipulate values on an implicit *operand stack*, consuming (popping) argument values and producing or returning (pushing) result values.

In addition to dynamic operands from the stack, some instructions also have static *immediate* arguments, typically *indices* or type annotations, which are part of the instruction itself.

Some instructions are *structured* in that they bracket nested sequences of instructions.

The following sections group instructions into a number of different categories.

2.4.1 Numeric Instructions

Numeric instructions provide basic operations over numeric *values* of specific *type*. These operations closely match respective operations available in hardware.

```

nn, mm ::= 32 | 64
sx      ::= u | s
instr   ::= inn.const un | fnn.const f nn
           | inn.iunop | fnn.funop
           | inn.ibinop | fnn.fbinop
           | inn.itestop
           | inn.irelop | fnn.frelop
           | inn.extend8_s | inn.extend16_s | i64.extend32_s
           | i32.wrap_i64 | i64.extend_i32_sx | inn.trunc_fmm_sx
           | inn.trunc_sat_fmm_sx
           | f32.demote_f64 | f64.promote_f32 | fnn.convert_imm_sx
           | inn.reinterpret_fnn | fnn.reinterpret_inn
           | ...
iunop   ::= clz | ctz | popcnt
ibinop  ::= add | sub | mul | div_sx | rem_sx
           | and | or | xor | shl | shr_sx | rotl | rotr
funop   ::= abs | neg | sqrt | ceil | floor | trunc | nearest
fbinop  ::= add | sub | mul | div | min | max | copysign
itestop ::= eqz
irelop  ::= eq | ne | lt_sx | gt_sx | le_sx | ge_sx
frelop  ::= eq | ne | lt | gt | le | ge

```

Numeric instructions are divided by *number type*. For each type, several subcategories can be distinguished:

- *Constants*: return a static constant.
- *Unary Operations*: consume one operand and produce one result of the respective type.
- *Binary Operations*: consume two operands and produce one result of the respective type.
- *Tests*: consume one operand of the respective type and produce a Boolean integer result.
- *Comparisons*: consume two operands of the respective type and produce a Boolean integer result.
- *Conversions*: consume a value of one type and produce a result of another (the source type of the conversion is the one after the “_”).

Some integer instructions come in two flavors, where a signedness annotation *sx* distinguishes whether the operands are to be *interpreted* as *unsigned* or *signed* integers. For the other integer instructions, the use of two’s complement for the signed interpretation means that they behave the same regardless of signedness.

Conventions

Occasionally, it is convenient to group operators together according to the following grammar shorthands:

```
unop ::= iunop | funop | extendN_s  
binop ::= ibinop | fbinop  
testop ::= itestop  
relop ::= irelop | frelop  
cvtop ::= wrap | extend | trunc | trunc_sat | convert | demote | promote | reinterpret
```

2.4.2 Vector Instructions

Vector instructions (also known as *SIMD* instructions, *single instruction multiple data*) provide basic operations over *values* of *vector* type.

```
ishape ::= i8x16 | i16x8 | i32x4 | i64x2  
fshape ::= f32x4 | f64x2  
shape ::= ishape | fshape  
half ::= low | high  
laneidx ::= u8
```

```

instr ::= ...
      v128.const i128
      v128.vvunop
      v128.vvbinop
      v128.vvternop
      v128.vvtestop
      i8x16.shuffle laneidx16
      i8x16.swizzle
      shape.splat
      i8x16.extract_lane_sx laneidx | i16x8.extract_lane_sx laneidx
      i32x4.extract_lane laneidx | i64x2.extract_lane laneidx
      fshape.extract_lane laneidx
      shape.replace_lane laneidx
      i8x16.virelop | i16x8.virelop | i32x4.virelop
      i64x2.eq | i64x2.ne | i64x2.lt_s | i64x2.gt_s | i64x2.le_s | i64x2.ge_s
      fshape.vfrelop
      ishape.viunop | i8x16.popcnt
      i16x8.q15mulr_sat_s
      i32x4.dot_i16x8_s
      fshape.vfunop
      ishape.vitestop
      ishape.bitmask
      i8x16.narrow_i16x8_sx | i16x8.narrow_i32x4_sx
      i16x8.extend_half_i8x16_sx | i32x4.extend_half_i16x8_sx
      i64x2.extend_half_i32x4_sx
      ishape.vishiftop
      ishape.vibinop
      i8x16.viminmaxop | i16x8.viminmaxop | i32x4.viminmaxop
      i8x16.visatbinop | i16x8.visatbinop
      i16x8.mul | i32x4.mul | i64x2.mul
      i8x16.avgr_u | i16x8.avgr_u
      i16x8.extmul_half_i8x16_sx | i32x4.extmul_half_i16x8_sx | i64x2.extmul_half_i32x4_sx
      i16x8.extadd_pairwise_i8x16_sx | i32x4.extadd_pairwise_i16x8_sx
      fshape.vfbinoop
      i32x4.trunc_sat_f32x4_sx | i32x4.trunc_sat_f64x2_sx_zero
      f32x4.convert_i32x4_sx | f32x4.demote_f64x2_zero
      f64x2.convert_low_i32x4_sx | f64x2.promote_low_f32x4
      ...

      vvunop      ::= not
      vvbinop     ::= and | andnot | or | xor
      vvternop    ::= bitselect
      vvtestop    ::= any_true
      vitestop    ::= all_true
      virelop     ::= eq | ne | lt_sx | gt_sx | le_sx | ge_sx
      vfrelop     ::= eq | ne | lt | gt | le | ge
      viunop      ::= abs | neg
      vibinop     ::= add | sub
      viminmaxop  ::= min_sx | max_sx
      visatbinop  ::= add_sat_sx | sub_sat_sx
      vishiftop   ::= shl | shr_sx
      vfunop      ::= abs | neg | sqrt | ceil | floor | trunc | nearest
      vfbinoop    ::= add | sub | mul | div | min | max | pmin | pmax

```

Vector instructions have a naming convention involving a prefix that determines how their operands will be interpreted. This prefix describes the *shape* of the operand, written *txN*, and consisting of a packed [numeric type](#) *t* and the number of *lanes* *N* of that type. Operations are performed point-wise on the values of each lane.

Note: For example, the shape *i32x4* interprets the operand as four *i32* values, packed into an *i128*. The bitwidth

of the numeric type t times N always is 128.

Instructions prefixed with `v128` do not involve a specific interpretation, and treat the `v128` as an `i128` value or a vector of 128 individual bits.

Vector instructions can be grouped into several subcategories:

- *Constants*: return a static constant.
- *Unary Operations*: consume one `v128` operand and produce one `v128` result.
- *Binary Operations*: consume two `v128` operands and produce one `v128` result.
- *Ternary Operations*: consume three `v128` operands and produce one `v128` result.
- *Tests*: consume one `v128` operand and produce a Boolean integer result.
- *Shifts*: consume a `v128` operand and a `i32` operand, producing one `v128` result.
- *Splats*: consume a value of numeric type and produce a `v128` result of a specified shape.
- *Extract lanes*: consume a `v128` operand and return the numeric value in a given lane.
- *Replace lanes*: consume a `v128` operand and a numeric value for a given lane, and produce a `v128` result.

Some vector instructions have a signedness annotation `sx` which distinguishes whether the elements in the operands are to be interpreted as `unsigned` or `signed` integers. For the other vector instructions, the use of two's complement for the signed interpretation means that they behave the same regardless of signedness.

Conventions

Occasionally, it is convenient to group operators together according to the following grammar shorthands:

```

vunop    ::= viunop | vfunop | popcnt
vbinop    ::= vibinop | vfbbinop
           | viminmaxop | visatbinop
           | mul | avgr_u | q15mulr_sat_s
vtestop   ::= vitestop
vrelop    ::= virelop | vfrelop
vcvtop    ::= extend | trunc_sat | convert | demote | promote

```

2.4.3 Reference Instructions

Instructions in this group are concerned with accessing `references`.

```

instr    ::= ...
           | ref.null reftype
           | ref.is_null
           | ref.func funcidx

```

These instructions produce a null value, check for a null value, or produce a reference to a given function, respectively.

2.4.4 Parametric Instructions

Instructions in this group can operate on operands of any [value type](#).

```
instr ::= ...  
        | drop  
        | select (valtype*)?
```

The [drop](#) instruction simply throws away a single operand.

The [select](#) instruction selects one of its first two operands based on whether its third operand is zero or not. It may include a [value type](#) determining the type of these operands. If missing, the operands must be of [numeric type](#).

Note: In future versions of WebAssembly, the type annotation on [select](#) may allow for more than a single value being selected at the same time.

2.4.5 Variable Instructions

Variable instructions are concerned with access to [local](#) or [global](#) variables.

```
instr ::= ...  
        | local.get localidx  
        | local.set localidx  
        | local.tee localidx  
        | global.get globalidx  
        | global.set globalidx
```

These instructions get or set the values of variables, respectively. The [local.tee](#) instruction is like [local.set](#) but also returns its argument.

2.4.6 Table Instructions

Instructions in this group are concerned with tables [table](#).

```
instr ::= ...  
        | table.get tableidx  
        | table.set tableidx  
        | table.size tableidx  
        | table.grow tableidx  
        | table.fill tableidx  
        | table.copy tableidx tableidx  
        | table.init tableidx elemidx  
        | elem.drop elemidx
```

The [table.get](#) and [table.set](#) instructions load or store an element in a table, respectively.

The [table.size](#) instruction returns the current size of a table. The [table.grow](#) instruction grows table by a given delta and returns the previous size, or -1 if enough space cannot be allocated. It also takes an initialization value for the newly allocated entries.

The [table.fill](#) instruction sets all entries in a range to a given value.

The [table.copy](#) instruction copies elements from a source table region to a possibly overlapping destination region; the first index denotes the destination. The [table.init](#) instruction copies elements from a [passive element segment](#) into a table. The [elem.drop](#) instruction prevents further use of a passive element segment. This instruction is intended to be used as an optimization hint. After an element segment is dropped its elements can no longer be retrieved, so the memory used by this segment may be freed.

An additional instruction that accesses a table is the [control instruction](#) [call_indirect](#).

2.4.7 Memory Instructions

Instructions in this group are concerned with linear [memory](#).

```

memarg ::= {offset u32, align u32}
ww      ::= 8 | 16 | 32 | 64
instr   ::= ...
            | inn.load memarg | fnn.load memarg | v128.load memarg
            | inn.store memarg | fnn.store memarg | v128.store memarg
            | inn.load8_sx memarg | inn.load16_sx memarg | i64.load32_sx memarg
            | inn.store8 memarg | inn.store16 memarg | i64.store32 memarg
            | v128.load8x8_sx memarg | v128.load16x4_sx memarg | v128.load32x2_sx memarg
            | v128.load32_zero memarg | v128.load64_zero memarg
            | v128.loadww_splat memarg
            | v128.loadww_lane memarg laneidx | v128.storeww_lane memarg laneidx
            | memory.size
            | memory.grow
            | memory.fill
            | memory.copy
            | memory.init dataidx
            | data.drop dataidx

```

Memory is accessed with [load](#) and [store](#) instructions for the different [number types](#). They all take a *memory immediate* [memarg](#) that contains an address *offset* and the expected *alignment* (expressed as the exponent of a power of 2). Integer loads and stores can optionally specify a *storage size* that is smaller than the [bit width](#) of the respective value type. In the case of loads, a sign extension mode *sx* is then required to select appropriate behavior.

Vector loads can specify a shape that is half the [bit width](#) of [v128](#). Each lane is half its usual size, and the sign extension mode *sx* then specifies how the smaller lane is extended to the larger lane. Alternatively, vector loads can perform a *splat*, such that only a single lane of the specified storage size is loaded, and the result is duplicated to all lanes.

The static address offset is added to the dynamic address operand, yielding a 33 bit *effective address* that is the zero-based index at which the memory is accessed. All values are read and written in [little endian](#)¹³ byte order. A [trap](#) results if any of the accessed memory bytes lies outside the address range implied by the memory's current size.

Note: Future versions of WebAssembly might provide memory instructions with 64 bit address ranges.

The [memory.size](#) instruction returns the current size of a memory. The [memory.grow](#) instruction grows memory by a given delta and returns the previous size, or -1 if enough memory cannot be allocated. Both instructions operate in units of [page size](#).

The [memory.fill](#) instruction sets all values in a region to a given byte. The [memory.copy](#) instruction copies data from a source memory region to a possibly overlapping destination region. The [memory.init](#) instruction copies data from a [passive data segment](#) into a memory. The [data.drop](#) instruction prevents further use of a passive data segment. This instruction is intended to be used as an optimization hint. After a data segment is dropped its data can no longer be retrieved, so the memory used by this segment may be freed.

Note: In the current version of WebAssembly, all memory instructions implicitly operate on [memory index 0](#). This restriction may be lifted in future versions.

¹³ <https://en.wikipedia.org/wiki/Endianness#Little-endian>

2.4.8 Control Instructions

Instructions in this group affect the flow of control.

```

blocktype ::= typeid | valtype?
instr      ::= ...
               | nop
               | unreachable
               | block blocktype instr* end
               | loop blocktype instr* end
               | if blocktype instr* else instr* end
               | br labelidx
               | br_if labelidx
               | br_table vec(labelidx) labelidx
               | return
               | call funcidx
               | call_indirect tableidx typeid

```

The *nop* instruction does nothing.

The *unreachable* instruction causes an unconditional *trap*.

The *block*, *loop* and *if* instructions are *structured* instructions. They bracket nested sequences of instructions, called *blocks*, terminated with, or separated by, *end* or *else* pseudo-instructions. As the grammar prescribes, they must be well-nested.

A structured instruction can consume *input* and produce *output* on the operand stack according to its annotated *block type*. It is given either as a *type index* that refers to a suitable *function type*, or as an optional *value type* inline, which is a shorthand for the function type $\square \rightarrow [\textit{valtype}]$.

Each structured control instruction introduces an implicit *label*. Labels are targets for branch instructions that reference them with *label indices*. Unlike with other *index spaces*, indexing of labels is relative by nesting depth, that is, label 0 refers to the innermost structured control instruction enclosing the referring branch instruction, while increasing indices refer to those farther out. Consequently, labels can only be referenced from *within* the associated structured control instruction. This also implies that branches can only be directed outwards, “breaking” from the block of the control construct they target. The exact effect depends on that control construct. In case of *block* or *if* it is a *forward jump*, resuming execution after the matching *end*. In case of *loop* it is a *backward jump* to the beginning of the loop.

Note: This enforces *structured control flow*. Intuitively, a branch targeting a *block* or *if* behaves like a break statement in most C-like languages, while a branch targeting a *loop* behaves like a continue statement.

Branch instructions come in several flavors: *br* performs an unconditional branch, *br_if* performs a conditional branch, and *br_table* performs an indirect branch through an operand indexing into the label vector that is an immediate to the instruction, or to a default target if the operand is out of bounds. The *return* instruction is a shortcut for an unconditional branch to the outermost block, which implicitly is the body of the current function. Taking a branch *unwinds* the operand stack up to the height where the targeted structured control instruction was entered. However, branches may additionally consume operands themselves, which they push back on the operand stack after unwinding. Forward branches require operands according to the output of the targeted block’s type, i.e., represent the values produced by the terminated block. Backward branches require operands according to the input of the targeted block’s type, i.e., represent the values consumed by the restarted block.

The *call* instruction invokes another *function*, consuming the necessary arguments from the stack and returning the result values of the call. The *call_indirect* instruction calls a function indirectly through an operand indexing into a *table* that is denoted by a *table index* and must have type *funcref*. Since it may contain functions of heterogeneous type, the callee is dynamically checked against the *function type* indexed by the instruction’s second immediate, and the call is aborted with a *trap* if it does not match.

2.4.9 Expressions

Function bodies, initialization values for `globals`, and offsets of `element` or `data` segments are given as expressions, which are sequences of `instructions` terminated by an `end` marker.

$$expr ::= instr^* end$$

In some places, validation `restricts` expressions to be *constant*, which limits the set of allowable instructions.

2.5 Modules

WebAssembly programs are organized into *modules*, which are the unit of deployment, loading, and compilation. A module collects definitions for `types`, `functions`, `tables`, `memories`, and `globals`. In addition, it can declare `imports` and `exports` and provide initialization in the form of `data` and `element` segments, or a `start` function.

$$module ::= \{ \begin{array}{l} \text{types } vec(func\ type), \\ \text{funcs } vec(func), \\ \text{tables } vec(table), \\ \text{mems } vec(mem), \\ \text{globals } vec(global), \\ \text{elems } vec(elem), \\ \text{datas } vec(data), \\ \text{start } start?, \\ \text{imports } vec(import), \\ \text{exports } vec(export) \end{array} \}$$

Each of the vectors – and thus the entire module – may be empty.

2.5.1 Indices

Definitions are referenced with zero-based *indices*. Each class of definition has its own *index space*, as distinguished by the following classes.

$$\begin{array}{ll} typeidx & ::= u32 \\ funcidx & ::= u32 \\ tableidx & ::= u32 \\ memidx & ::= u32 \\ globalidx & ::= u32 \\ elemidx & ::= u32 \\ dataidx & ::= u32 \\ localidx & ::= u32 \\ labelidx & ::= u32 \end{array}$$

The index space for `functions`, `tables`, `memories` and `globals` includes respective `imports` declared in the same module. The indices of these imports precede the indices of other definitions in the same index space.

Element indices reference `element segments` and data indices reference `data segments`.

The index space for `locals` is only accessible inside a `function` and includes the parameters of that function, which precede the local variables.

Label indices reference `structured control instructions` inside an instruction sequence.

Conventions

- The meta variable l ranges over label indices.
- The meta variables x, y range over indices in any of the other index spaces.
- The notation $\text{idx}(A)$ denotes the set of indices from index space idx occurring free in A . Sometimes this set is reinterpreted as the **vector** of its elements.

Note: For example, if instr^* is $(\text{data.drop } x)(\text{memory.init } y)$, then $\text{dataidx}(\text{instr}^*) = \{x, y\}$, or equivalently, the vector $x\ y$.

2.5.2 Types

The **types** component of a module defines a vector of **function types**.

All function types used in a module must be defined in this component. They are referenced by **type indices**.

Note: Future versions of WebAssembly may add additional forms of type definitions.

2.5.3 Functions

The **funcs** component of a module defines a vector of **functions** with the following structure:

$$\text{func} ::= \{\text{type } \text{typeid}, \text{locals } \text{vec}(\text{valtype}), \text{body } \text{expr}\}$$

The **type** of a function declares its signature by reference to a **type** defined in the module. The parameters of the function are referenced through 0-based **local indices** in the function's body; they are mutable.

The **locals** declare a vector of mutable local variables and their types. These variables are referenced through **local indices** in the function's body. The index of the first local is the smallest index not referencing a parameter.

The **body** is an **instruction** sequence that upon termination must produce a stack matching the function type's **result type**.

Functions are referenced through **function indices**, starting with the smallest index not referencing a function **import**.

2.5.4 Tables

The **tables** component of a module defines a vector of **tables** described by their **table type**:

$$\text{table} ::= \{\text{type } \text{tabletype}\}$$

A table is a vector of opaque values of a particular **reference type**. The **min** size in the **limits** of the table type specifies the initial size of that table, while its **max**, if present, restricts the size to which it can grow later.

Tables can be initialized through **element segments**.

Tables are referenced through **table indices**, starting with the smallest index not referencing a table **import**. Most constructs implicitly reference table index 0.

2.5.5 Memories

The `mems` component of a module defines a vector of *linear memories* (or *memories* for short) as described by their *memory type*:

$$mem ::= \{type\ memtype\}$$

A memory is a vector of raw uninterpreted bytes. The `min` size in the *limits* of the memory type specifies the initial size of that memory, while its `max`, if present, restricts the size to which it can grow later. Both are in units of *page size*.

Memories can be initialized through *data segments*.

Memories are referenced through *memory indices*, starting with the smallest index not referencing a memory *import*. Most constructs implicitly reference memory index 0.

Note: In the current version of WebAssembly, at most one memory may be defined or imported in a single module, and *all* constructs implicitly reference this memory 0. This restriction may be lifted in future versions.

2.5.6 Globals

The `globals` component of a module defines a vector of *global variables* (or *globals* for short):

$$global ::= \{type\ globaltype, init\ expr\}$$

Each global stores a single value of the given *global type*. Its *type* also specifies whether a global is immutable or mutable. Moreover, each global is initialized with an *init* value given by a *constant initializer expression*.

Globals are referenced through *global indices*, starting with the smallest index not referencing a global *import*.

2.5.7 Element Segments

The initial contents of a table is uninitialized. *Element segments* can be used to initialize a subrange of a table from a static *vector* of elements.

The `elems` component of a module defines a vector of element segments. Each element segment defines a *reference type* and a corresponding list of *constant element expressions*.

Element segments have a mode that identifies them as either *passive*, *active*, or *declarative*. A passive element segment's elements can be copied to a table using the `table.init` instruction. An active element segment copies its elements into a table during *instantiation*, as specified by a *table index* and a *constant expression* defining an offset into that table. A declarative element segment is not available at runtime but merely serves to forward-declare references that are formed in code with instructions like `ref.func`.

$$\begin{array}{ll} elem & ::= \{type\ reftype, init\ vec(expr), mode\ elemmode\} \\ elemmode & ::= \begin{array}{l} passive \\ | \\ active\ \{table\ tableidx, offset\ expr\} \\ | \\ declarative \end{array} \end{array}$$

The *offset* is given by a *constant expression*.

Element segments are referenced through *element indices*.

2.5.8 Data Segments

The initial contents of a [memory](#) are zero bytes. *Data segments* can be used to initialize a range of memory from a static [vector](#) of bytes.

The [datas](#) component of a module defines a vector of data segments.

Like element segments, data segments have a mode that identifies them as either *passive* or *active*. A passive data segment's contents can be copied into a memory using the [memory.init](#) instruction. An active data segment copies its contents into a memory during [instantiation](#), as specified by a [memory index](#) and a [constant expression](#) defining an offset into that memory.

```
data      ::= {init vec(byte), mode datamode}
datamode  ::= passive
           | active {memory memidx, offset expr}
```

Data segments are referenced through [data indices](#).

Note: In the current version of WebAssembly, at most one memory is allowed in a module. Consequently, the only valid *memidx* is 0.

2.5.9 Start Function

The [start](#) component of a module declares the [function index](#) of a *start function* that is automatically invoked when the module is [instantiated](#), after [tables](#) and [memories](#) have been initialized.

```
start ::= {func funcidx}
```

Note: The start function is intended for initializing the state of a module. The module and its exports are not accessible externally before this initialization has completed.

2.5.10 Exports

The [exports](#) component of a module defines a set of *exports* that become accessible to the host environment once the module has been [instantiated](#).

```
export      ::= {name name, desc exportdesc}
exportdesc  ::= func funcidx
           | table tableidx
           | mem memidx
           | global globalidx
```

Each export is labeled by a unique [name](#). Exportable definitions are [functions](#), [tables](#), [memories](#), and [globals](#), which are referenced through a respective descriptor.

Conventions

The following auxiliary notation is defined for sequences of exports, filtering out indices of a specific kind in an order-preserving fashion:

- $\text{funcs}(\text{export}^*) = [\text{funcidx} \mid \text{func } \text{funcidx} \in (\text{export}.\text{desc})^*]$
- $\text{tables}(\text{export}^*) = [\text{tableidx} \mid \text{table } \text{tableidx} \in (\text{export}.\text{desc})^*]$
- $\text{mems}(\text{export}^*) = [\text{memidx} \mid \text{mem } \text{memidx} \in (\text{export}.\text{desc})^*]$
- $\text{globals}(\text{export}^*) = [\text{globalidx} \mid \text{global } \text{globalidx} \in (\text{export}.\text{desc})^*]$

2.5.11 Imports

The `imports` component of a module defines a set of *imports* that are required for *instantiation*.

```
import      ::= { module name, name name, desc importdesc }  
importdesc ::= func typeidx  
                | table tabletype  
                | mem memtype  
                | global globaltype
```

Each import is labeled by a two-level *name* space, consisting of a *module* name and a *name* for an entity within that module. Importable definitions are *functions*, *tables*, *memories*, and *globals*. Each import is specified by a descriptor with a respective type that a definition provided during instantiation is required to match.

Every import defines an index in the respective *index space*. In each index space, the indices of imports go before the first index of any definition contained in the module itself.

Note: Unlike export names, import names are not necessarily unique. It is possible to import the same *module/name* pair multiple times; such imports may even have different type descriptions, including different kinds of entities. A module with such imports can still be instantiated depending on the specifics of how an *embedder* allows resolving and supplying imports. However, embedders are not required to support such overloading, and a WebAssembly module itself cannot implement an overloaded name.

3.1 Conventions

Validation checks that a WebAssembly module is well-formed. Only valid modules can be *instantiated*.

Validity is defined by a *type system* over the *abstract syntax* of a *module* and its contents. For each piece of abstract syntax, there is a typing rule that specifies the constraints that apply to it. All rules are given in two *equivalent* forms:

1. In *prose*, describing the meaning in intuitive form.
2. In *formal notation*, describing the rule in mathematical form.¹⁴

Note: The prose and formal rules are equivalent, so that understanding of the formal notation is *not* required to read this specification. The formalism offers a more concise description in notation that is used widely in programming languages semantics and is readily amenable to mathematical proof.

In both cases, the rules are formulated in a *declarative* manner. That is, they only formulate the constraints, they do not define an algorithm. The skeleton of a sound and complete algorithm for type-checking instruction sequences according to this specification is provided in the *appendix*.

3.1.1 Contexts

Validity of an individual definition is specified relative to a *context*, which collects relevant information about the surrounding *module* and the definitions in scope:

- *Types*: the list of types defined in the current module.
- *Functions*: the list of functions declared in the current module, represented by their function type.
- *Tables*: the list of tables declared in the current module, represented by their table type.
- *Memories*: the list of memories declared in the current module, represented by their memory type.
- *Globals*: the list of globals declared in the current module, represented by their global type.

¹⁴ The semantics is derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. *Bringing the Web up to Speed with WebAssembly*¹⁵. Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

¹⁵ <https://dl.acm.org/citation.cfm?doid=3062341.3062363>

- *Element Segments*: the list of element segments declared in the current module, represented by their element type.
- *Data Segments*: the list of data segments declared in the current module, each represented by an *ok* entry.
- *Locals*: the list of locals declared in the current function (including parameters), represented by their value type.
- *Labels*: the stack of labels accessible from the current position, represented by their result type.
- *Return*: the return type of the current function, represented as an optional result type that is absent when no return is allowed, as in free-standing expressions.
- *References*: the list of [function indices](#) that occur in the module outside functions and can hence be used to form references inside them.

In other words, a context contains a sequence of suitable [types](#) for each [index space](#), describing each defined entry in that space. Locals, labels and return type are only used for validating [instructions](#) in [function bodies](#), and are left empty elsewhere. The label stack is the only part of the context that changes as validation of an instruction sequence proceeds.

More concretely, contexts are defined as [records](#) C with abstract syntax:

$$C ::= \{ \begin{array}{ll} \text{types} & \text{functype}^*, \\ \text{funcs} & \text{functype}^*, \\ \text{tables} & \text{tabletype}^*, \\ \text{mems} & \text{memtype}^*, \\ \text{globals} & \text{globaltype}^*, \\ \text{elems} & \text{reftype}^*, \\ \text{datas} & \text{ok}^*, \\ \text{locals} & \text{valtype}^*, \\ \text{labels} & \text{resulttype}^*, \\ \text{return} & \text{resulttype}^?, \\ \text{refs} & \text{funcidx}^* \end{array} \}$$

In addition to field access written $C.\text{field}$ the following notation is adopted for manipulating contexts:

- When spelling out a context, empty fields are omitted.
- $C, \text{field } A^*$ denotes the same context as C but with the elements A^* prepended to its field component sequence.

Note: [Indexing notation](#) like $C.\text{labels}[i]$ is used to look up indices in their respective [index space](#) in the context. Context extension notation $C, \text{field } A$ is primarily used to locally extend *relative* index spaces, such as [label indices](#). Accordingly, the notation is defined to append at the *front* of the respective sequence, introducing a new relative index 0 and shifting the existing ones.

3.1.2 Prose Notation

Validation is specified by stylised rules for each relevant part of the [abstract syntax](#). The rules not only state constraints defining when a phrase is valid, they also classify it with a type. The following conventions are adopted in stating these rules.

- A phrase A is said to be “valid with type T ” if and only if all constraints expressed by the respective rules are met. The form of T depends on what A is.

Note: For example, if A is a [function](#), then T is a [function type](#); for an A that is a [global](#), T is a [global type](#); and so on.

- The rules implicitly assume a given [context](#) C .

- In some places, this context is locally extended to a context C' with additional entries. The formulation “Under context C' , ... *statement* ...” is adopted to express that the following statement must apply under the assumptions embodied in the extended context.

3.1.3 Formal Notation

Note: This section gives a brief explanation of the notation for specifying typing rules formally. For the interested reader, a more thorough introduction can be found in respective text books.¹⁶

The proposition that a phrase A has a respective type T is written $A : T$. In general, however, typing is dependent on a context C . To express this explicitly, the complete form is a *judgement* $C \vdash A : T$, which says that $A : T$ holds under the assumptions encoded in C .

The formal typing rules use a standard approach for specifying type systems, rendering them into *deduction rules*. Every rule has the following general form:

$$\frac{\text{premise}_1 \quad \text{premise}_2 \quad \dots \quad \text{premise}_n}{\text{conclusion}}$$

Such a rule is read as a big implication: if all premises hold, then the conclusion holds. Some rules have no premises; they are *axioms* whose conclusion holds unconditionally. The conclusion always is a judgment $C \vdash A : T$, and there is one respective rule for each relevant construct A of the abstract syntax.

Note: For example, the typing rule for the `i32.add` instruction can be given as an axiom:

$$\overline{C \vdash \text{i32.add} : [\text{i32 } \text{i32}] \rightarrow [\text{i32}]}$$

The instruction is always valid with type $[\text{i32 } \text{i32}] \rightarrow [\text{i32}]$ (saying that it consumes two `i32` values and produces one), independent of any side conditions.

An instruction like `local.get` can be typed as follows:

$$\frac{C.\text{locals}[x] = t}{C \vdash \text{local.get } x : [] \rightarrow [t]}$$

Here, the premise enforces that the immediate `local` index x exists in the context. The instruction produces a value of its respective type t (and does not consume any values). If $C.\text{locals}[x]$ does not exist then the premise does not hold, and the instruction is ill-typed.

Finally, a `structured` instruction requires a recursive rule, where the premise is itself a typing judgement:

$$\frac{C \vdash \text{blocktype} : [t_1^*] \rightarrow [t_2^*] \quad C, \text{label } [t_2^*] \vdash \text{instr}^* : [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{block } \text{blocktype } \text{instr}^* \text{ end} : [t_1^*] \rightarrow [t_2^*]}$$

A `block` instruction is only valid when the instruction sequence in its body is. Moreover, the result type must match the block’s annotation `blocktype`. If so, then the `block` instruction has the same type as the body. Inside the body an additional label of the corresponding result type is available, which is expressed by extending the context C with the additional label information for the premise.

¹⁶ For example: Benjamin Pierce. *Types and Programming Languages*^{Page 27, 17}. The MIT Press 2002

¹⁷ <https://www.cis.upenn.edu/~bcpierce/tapl/>

3.2 Types

Most **types** are universally valid. However, restrictions apply to **limits**, which must be checked during validation. Moreover, **block types** are converted to plain **function types** for ease of processing.

3.2.1 Limits

Limits must have meaningful bounds that are within a given range.

$\{\min n, \max m^?\}$

- The value of n must not be larger than k .
- If the maximum $m^?$ is not empty, then:
 - Its value must not be larger than k .
 - Its value must not be smaller than n .
- Then the limit is valid within range k .

$$\frac{n \leq k \quad (m \leq k)^? \quad (n \leq m)^?}{\vdash \{\min n, \max m^?\} : k}$$

3.2.2 Block Types

Block types may be expressed in one of two forms, both of which are converted to plain **function types** by the following rules.

typeid x

- The type $C.\text{types}[\text{typeid}x]$ must be defined in the context.
- Then the block type is valid as **function type** $C.\text{types}[\text{typeid}x]$.

$$\frac{C.\text{types}[\text{typeid}x] = \text{functype}}{C \vdash \text{typeid}x : \text{functype}}$$

$[\text{valtype}^?]$

- The block type is valid as **function type** $[] \rightarrow [\text{valtype}^?]$.

$$\overline{C \vdash [\text{valtype}^?] : [] \rightarrow [\text{valtype}^?]}$$

3.2.3 Function Types

Function types are always valid.

$$[t_1^n] \rightarrow [t_2^m]$$

- The function type is valid.

$$\overline{\vdash [t_1^*] \rightarrow [t_2^*] \text{ ok}}$$

3.2.4 Table Types

limits reftype

- The limits *limits* must be **valid** within range $2^{32} - 1$.
- Then the table type is valid.

$$\frac{\vdash \text{limits} : 2^{32} - 1}{\vdash \text{limits reftype ok}}$$

3.2.5 Memory Types

limits

- The limits *limits* must be **valid** within range 2^{16} .
- Then the memory type is valid.

$$\frac{\vdash \text{limits} : 2^{16}}{\vdash \text{limits ok}}$$

3.2.6 Global Types

mut valtype

- The global type is valid.

$$\overline{\vdash \text{mut valtype ok}}$$

3.2.7 External Types

func functype

- The function type *functype* must be valid.
- Then the external type is valid.

$$\frac{\vdash \text{functype ok}}{\vdash \text{func functype ok}}$$

table *tabletype*

- The table type *tabletype* must be valid.
- Then the external type is valid.

$$\frac{\vdash \text{tabletype ok}}{\vdash \text{table } \text{tabletype ok}}$$

mem *memtype*

- The memory type *memtype* must be valid.
- Then the external type is valid.

$$\frac{\vdash \text{memtype ok}}{\vdash \text{mem } \text{memtype ok}}$$

global *globaltype*

- The global type *globaltype* must be valid.
- Then the external type is valid.

$$\frac{\vdash \text{globaltype ok}}{\vdash \text{global } \text{globaltype ok}}$$

3.2.8 Import Subtyping

When *instantiating* a module, *external values* must be provided whose *types* are *matched* against the respective *external types* classifying each import. In some cases, this allows for a simple form of subtyping (written “ $\leq$ ” formally), as defined here.

Limits

Limits $\{\min n_1, \max m_1^?\}$ match limits $\{\min n_2, \max m_2^?\}$ if and only if:

- n_1 is larger than or equal to n_2 .
- Either:
 - $m_2^?$ is empty.
- Or:
 - Both $m_1^?$ and $m_2^?$ are non-empty.
 - m_1 is smaller than or equal to m_2 .

$$\frac{n_1 \geq n_2}{\vdash \{\min n_1, \max m_1^?\} \leq \{\min n_2, \max \epsilon\}} \quad \frac{n_1 \geq n_2 \quad m_1 \leq m_2}{\vdash \{\min n_1, \max m_1\} \leq \{\min n_2, \max m_2\}}$$

Functions

An external type `func functype1` matches `func functype2` if and only if:

- Both *functype*₁ and *functype*₂ are the same.

$$\frac{}{\vdash \text{func } \textit{functype} \leq \text{func } \textit{functype}}$$

Tables

An external type `table (limits1 reftype1)` matches `table (limits2 reftype2)` if and only if:

- Limits *limits*₁ match *limits*₂.
- Both *reftype*₁ and *reftype*₂ are the same.

$$\frac{\vdash \textit{limits}_1 \leq \textit{limits}_2}{\vdash \text{table } (\textit{limits}_1 \textit{reftype}) \leq \text{table } (\textit{limits}_2 \textit{reftype})}$$

Memories

An external type `mem limits1` matches `mem limits2` if and only if:

- Limits *limits*₁ match *limits*₂.

$$\frac{\vdash \textit{limits}_1 \leq \textit{limits}_2}{\vdash \text{mem } \textit{limits}_1 \leq \text{mem } \textit{limits}_2}$$

Globals

An external type `global globaltype1` matches `global globaltype2` if and only if:

- Both *globaltype*₁ and *globaltype*₂ are the same.

$$\frac{}{\vdash \text{global } \textit{globaltype} \leq \text{global } \textit{globaltype}}$$

3.3 Instructions

Instructions are classified by *stack types* $[t_1^*] \rightarrow [t_2^*]$ that describe how instructions manipulate the *operand stack*.

$$\begin{aligned} \textit{stacktype} &::= [\textit{opdtype}^*] \rightarrow [\textit{opdtype}^*] \\ \textit{opdtype} &::= \textit{valtype} \mid \perp \end{aligned}$$

The types describe the required input stack with *operand types* t_1^* that an instruction pops off and the provided output stack with result values of types t_2^* that it pushes back. Stack types are akin to *function types*, except that they allow individual operands to be classified as $\perp$ (*bottom*), indicating that the type is unconstrained. As an auxiliary notion, an operand type t_1 *matches* another operand type t_2 , if t_1 is either $\perp$ or equal to t_2 . This is extended to stack types in a point-wise manner.

$$\begin{aligned} \frac{}{\vdash t \leq t} \quad \frac{}{\vdash \perp \leq t} \\ \frac{(\vdash t \leq t')^*}{\vdash [t^*] \leq [t'^*]} \end{aligned}$$

Note: For example, the instruction `i32.add` has type $[i32\ i32] \rightarrow [i32]$, consuming two `i32` values and producing one.

Typing extends to *instruction sequences* $instr^*$. Such a sequence has a *stack type* $[t_1^*] \rightarrow [t_2^*]$ if the accumulative effect of executing the instructions is consuming values of types t_1^* off the operand stack and pushing new values of types t_2^* .

For some instructions, the typing rules do not fully constrain the type, and therefore allow for multiple types. Such instructions are called *polymorphic*. Two degrees of polymorphism can be distinguished:

- *value-polymorphic*: the *value type* t of one or several individual operands is unconstrained. That is the case for all *parametric instructions* like `drop` and `select`.
- *stack-polymorphic*: the entire (or most of the) *stack type* $[t_1^*] \rightarrow [t_2^*]$ of the instruction is unconstrained. That is the case for all *control instructions* that perform an *unconditional control transfer*, such as `unreachable`, `br`, `br_table`, and `return`.

In both cases, the unconstrained types or type sequences can be chosen arbitrarily, as long as they meet the constraints imposed for the surrounding parts of the program.

Note: For example, the `select` instruction is valid with type $[t\ t\ i32] \rightarrow [t]$, for any possible *number type* t . Consequently, both instruction sequences

`(i32.const 1) (i32.const 2) (i32.const 3) select`

and

`(f64.const 1.0) (f64.const 2.0) (i32.const 3) select`

are valid, with t in the typing of `select` being instantiated to `i32` or `f64`, respectively.

The `unreachable` instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$ for any possible sequences of *operand types* t_1^* and t_2^* . Consequently,

`unreachable i32.add`

is valid by assuming type $[] \rightarrow [i32\ i32]$ for the `unreachable` instruction. In contrast,

`unreachable (i64.const 0) i32.add`

is invalid, because there is no possible type to pick for the `unreachable` instruction that would make the sequence well-typed.

The [Appendix](#) describes a type checking *algorithm* that efficiently implements validation of instruction sequences as prescribed by the rules given here.

3.3.1 Numeric Instructions

`t.const c`

- The instruction is valid with type $[] \rightarrow [t]$.

$$\overline{C \vdash t.\text{const } c : [] \rightarrow [t]}$$

t.unop

- The instruction is valid with type $[t] \rightarrow [t]$.

$$\overline{C \vdash t.unop : [t] \rightarrow [t]}$$

t.binop

- The instruction is valid with type $[t\ t] \rightarrow [t]$.

$$\overline{C \vdash t.binop : [t\ t] \rightarrow [t]}$$

t.testop

- The instruction is valid with type $[t] \rightarrow [i32]$.

$$\overline{C \vdash t.testop : [t] \rightarrow [i32]}$$

t.relop

- The instruction is valid with type $[t\ t] \rightarrow [i32]$.

$$\overline{C \vdash t.relop : [t\ t] \rightarrow [i32]}$$

t₂.cvt_{top}_t₁_sx[?]

- The instruction is valid with type $[t_1] \rightarrow [t_2]$.

$$\overline{C \vdash t_2.cvt_{top_t_1_sx} : [t_1] \rightarrow [t_2]}$$

3.3.2 Reference Instructions

ref.null t

- The instruction is valid with type $[] \rightarrow [t]$.

$$\overline{C \vdash \text{ref.null } t : [] \rightarrow [t]}$$

Note: In future versions of WebAssembly, there may be reference types for which no null reference is allowed.

ref.is_null

- The instruction is valid with type $[t] \rightarrow [i32]$, for any reference type *t*.

$$\frac{t = \text{reftype}}{C \vdash \text{ref.is_null} : [t] \rightarrow [i32]}$$

`ref.func x`

- The function $C.\text{funcs}[x]$ must be defined in the context.
- The function index x must be contained in $C.\text{refs}$.
- The instruction is valid with type $[] \rightarrow [\text{funcref}]$.

$$\frac{C.\text{funcs}[x] = \text{functype} \quad x \in C.\text{refs}}{C \vdash \text{ref.func } x : [] \rightarrow [\text{funcref}]}$$

3.3.3 Vector Instructions

Vector instructions can have a prefix to describe the *shape* of the operand. Packed numeric types, `i8` and `i16`, are not *value types*. An auxiliary function maps such packed type shapes to value types:

$$\begin{aligned} \text{unpacked}(i8 \times 16) &= i32 \\ \text{unpacked}(i16 \times 8) &= i32 \\ \text{unpacked}(t \times N) &= t \end{aligned}$$

The following auxiliary function denotes the number of lanes in a vector shape, i.e., its *dimension*:

$$\text{dim}(t \times N) = N$$

`v128.const c`

- The instruction is valid with type $[] \rightarrow [v128]$.

$$\overline{C \vdash \text{v128.const } c : [] \rightarrow [v128]}$$

`v128.vvunop`

- The instruction is valid with type $[v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{v128.vvunop} : [v128] \rightarrow [v128]}$$

`v128.vvbinop`

- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{v128.vvbinop} : [v128 \ v128] \rightarrow [v128]}$$

`v128.vvternop`

- The instruction is valid with type $[v128 \ v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{v128.vvternop} : [v128 \ v128 \ v128] \rightarrow [v128]}$$

v128.vtestop

- The instruction is valid with type $[v128] \rightarrow [i32]$.

$$\overline{C \vdash v128.vtestop : [v128] \rightarrow [i32]}$$

i8x16.swizzle

- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash i8x16.swizzle : [v128 \ v128] \rightarrow [v128]}$$

i8x16.shuffle laneidx¹⁶

- For all $laneidx_i$, in $laneidx^{16}$, $laneidx_i$ must be smaller than 32.
- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\frac{(laneidx < 32)^{16}}{\overline{C \vdash i8x16.shuffle \ laneidx^{16} : [v128 \ v128] \rightarrow [v128]}}$$

shape.splat

- Let t be $\text{unpacked}(shape)$.
- The instruction is valid with type $[t] \rightarrow [v128]$.

$$\overline{C \vdash shape.splat : [\text{unpacked}(shape)] \rightarrow [v128]}$$

shape.extract_lane_sx[?] laneidx

- The lane index $laneidx$ must be smaller than $\text{dim}(shape)$.
- The instruction is valid with type $[v128] \rightarrow [\text{unpacked}(shape)]$.

$$\frac{laneidx < \text{dim}(shape)}{\overline{C \vdash txN.extract_lane_sx^? \ laneidx : [v128] \rightarrow [\text{unpacked}(shape)]}}$$

shape.replace_lane laneidx

- The lane index $laneidx$ must be smaller than $\text{dim}(shape)$.
- Let t be $\text{unpacked}(shape)$.
- The instruction is valid with type $[v128 \ t] \rightarrow [v128]$.

$$\frac{laneidx < \text{dim}(shape)}{\overline{C \vdash shape.replace_lane \ laneidx : [v128 \ \text{unpacked}(shape)] \rightarrow [v128]}}$$

shape.vunop

- The instruction is valid with type $[v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{shape.vunop} : [v128] \rightarrow [v128]}$$

shape.vbinop

- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{shape.vbinop} : [v128 \ v128] \rightarrow [v128]}$$

shape.vrelop

- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{shape.vrelop} : [v128 \ v128] \rightarrow [v128]}$$

ishape.vishiftp

- The instruction is valid with type $[v128 \ i32] \rightarrow [v128]$.

$$\overline{C \vdash \text{ishape.vishiftp} : [v128 \ i32] \rightarrow [v128]}$$

shape.vtestop

- The instruction is valid with type $[v128] \rightarrow [i32]$.

$$\overline{C \vdash \text{shape.vtestop} : [v128] \rightarrow [i32]}$$

shape.vcvtop_half?_shape_sx?_zero?

- The instruction is valid with type $[v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{shape.vcvtop_half?_shape_sx?_zero?} : [v128] \rightarrow [v128]}$$

ishape₁.narrow_ishape₂_sx

- The instruction is valid with type $[v128 \ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{ishape}_1.\text{narrow_ishape}_2.\text{sx} : [v128 \ v128] \rightarrow [v128]}$$

ishape.bitmask

- The instruction is valid with type $[v128] \rightarrow [i32]$.

$$\overline{C \vdash \text{ishape.bitmask} : [v128] \rightarrow [i32]}$$

ishape₁.dot_ishape₂_s

- The instruction is valid with type $[v128\ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{ishape}_1.\text{dot_ishape}_2_s : [v128\ v128] \rightarrow [v128]}$$

ishape₁.extmul_half_ishape₂_sx

- The instruction is valid with type $[v128\ v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{ishape}_1.\text{extmul_half_ishape}_2_sx : [v128\ v128] \rightarrow [v128]}$$

ishape₁.extadd_pairwise_ishape₂_sx

- The instruction is valid with type $[v128] \rightarrow [v128]$.

$$\overline{C \vdash \text{ishape}_1.\text{extadd_pairwise_ishape}_2_sx : [v128] \rightarrow [v128]}$$

3.3.4 Parametric Instructions

drop

- The instruction is valid with type $[t] \rightarrow []$, for any operand type t .

$$\overline{C \vdash \text{drop} : [t] \rightarrow []}$$

Note: Both *drop* and *select* without annotation are value-polymorphic instructions.

select (t)?*

- If t^* is present, then:
 - The length of t^* must be 1.
 - Then the instruction is valid with type $[t^*\ t^*\ i32] \rightarrow [t^*]$.
- Else:
 - The instruction is valid with type $[t\ t\ i32] \rightarrow [t]$, for any operand type t that matches some number type or vector type.

$$\overline{C \vdash \text{select } t : [t\ t\ i32] \rightarrow [t]} \quad \overline{C \vdash \text{select} : [t\ t\ i32] \rightarrow [t]} \quad \overline{C \vdash \text{select} : [t\ t\ i32] \rightarrow [t]}$$

Note: In future versions of WebAssembly, *select* may allow more than one value per choice.

3.3.5 Variable Instructions

`local.get` x

- The local $C.\text{locals}[x]$ must be defined in the context.
- Let t be the [value type](#) $C.\text{locals}[x]$.
- Then the instruction is valid with type $[] \rightarrow [t]$.

$$\frac{C.\text{locals}[x] = t}{C \vdash \text{local.get } x : [] \rightarrow [t]}$$

`local.set` x

- The local $C.\text{locals}[x]$ must be defined in the context.
- Let t be the [value type](#) $C.\text{locals}[x]$.
- Then the instruction is valid with type $[t] \rightarrow []$.

$$\frac{C.\text{locals}[x] = t}{C \vdash \text{local.set } x : [t] \rightarrow []}$$

`local.tee` x

- The local $C.\text{locals}[x]$ must be defined in the context.
- Let t be the [value type](#) $C.\text{locals}[x]$.
- Then the instruction is valid with type $[t] \rightarrow [t]$.

$$\frac{C.\text{locals}[x] = t}{C \vdash \text{local.tee } x : [t] \rightarrow [t]}$$

`global.get` x

- The global $C.\text{globals}[x]$ must be defined in the context.
- Let $\text{mut } t$ be the [global type](#) $C.\text{globals}[x]$.
- Then the instruction is valid with type $[] \rightarrow [t]$.

$$\frac{C.\text{globals}[x] = \text{mut } t}{C \vdash \text{global.get } x : [] \rightarrow [t]}$$

`global.set` x

- The global $C.\text{globals}[x]$ must be defined in the context.
- Let $\text{mut } t$ be the [global type](#) $C.\text{globals}[x]$.
- The mutability mut must be `var`.
- Then the instruction is valid with type $[t] \rightarrow []$.

$$\frac{C.\text{globals}[x] = \text{var } t}{C \vdash \text{global.set } x : [t] \rightarrow []}$$

3.3.6 Table Instructions

`table.get` x

- The table $C.tables[x]$ must be defined in the context.
- Let *limits* t be the `table type` $C.tables[x]$.
- Then the instruction is valid with type $[i32] \rightarrow [t]$.

$$\frac{C.tables[x] = \text{limits } t}{C \vdash \text{table.get } x : [i32] \rightarrow [t]}$$

`table.set` x

- The table $C.tables[x]$ must be defined in the context.
- Let *limits* t be the `table type` $C.tables[x]$.
- Then the instruction is valid with type $[i32\ t] \rightarrow []$.

$$\frac{C.tables[x] = \text{limits } t}{C \vdash \text{table.set } x : [i32\ t] \rightarrow []}$$

`table.size` x

- The table $C.tables[x]$ must be defined in the context.
- Then the instruction is valid with type $[] \rightarrow [i32]$.

$$\frac{C.tables[x] = \text{tabletype}}{C \vdash \text{table.size } x : [] \rightarrow [i32]}$$

`table.grow` x

- The table $C.tables[x]$ must be defined in the context.
- Let *limits* t be the `table type` $C.tables[x]$.
- Then the instruction is valid with type $[t\ i32] \rightarrow [i32]$.

$$\frac{C.tables[x] = \text{limits } t}{C \vdash \text{table.grow } x : [t\ i32] \rightarrow [i32]}$$

`table.fill` x

- The table $C.tables[x]$ must be defined in the context.
- Let *limits* t be the `table type` $C.tables[x]$.
- Then the instruction is valid with type $[i32\ t\ i32] \rightarrow []$.

$$\frac{C.tables[x] = \text{limits } t}{C \vdash \text{table.fill } x : [i32\ t\ i32] \rightarrow []}$$

`table.copy x y`

- The table $C.tables[x]$ must be defined in the context.
- Let $limits_1 t_1$ be the table type $C.tables[x]$.
- The table $C.tables[y]$ must be defined in the context.
- Let $limits_2 t_2$ be the table type $C.tables[y]$.
- The reference type t_1 must be the same as t_2 .
- Then the instruction is valid with type $[i32\ i32\ i32] \rightarrow []$.

$$\frac{C.tables[x] = limits_1 t \quad C.tables[y] = limits_2 t}{C \vdash \text{table.copy } x\ y : [i32\ i32\ i32] \rightarrow []}$$

`table.init x y`

- The table $C.tables[x]$ must be defined in the context.
- Let $limits t_1$ be the table type $C.tables[x]$.
- The element segment $C.elems[y]$ must be defined in the context.
- Let t_2 be the reference type $C.elems[y]$.
- The reference type t_1 must be the same as t_2 .
- Then the instruction is valid with type $[i32\ i32\ i32] \rightarrow []$.

$$\frac{C.tables[x] = limits\ t \quad C.elems[y] = t}{C \vdash \text{table.init } x\ y : [i32\ i32\ i32] \rightarrow []}$$

`elem.drop x`

- The element segment $C.elems[x]$ must be defined in the context.
- Then the instruction is valid with type $[] \rightarrow []$.

$$\frac{C.elems[x] = t}{C \vdash \text{elem.drop } x : [] \rightarrow []}$$

3.3.7 Memory Instructions

`t.load memarg`

- The memory $C.mems[0]$ must be defined in the context.
- The alignment $2^{memarg.align}$ must not be larger than the bit width of t divided by 8.
- Then the instruction is valid with type $[i32] \rightarrow [t]$.

$$\frac{C.mems[0] = memtype \quad 2^{memarg.align} \leq |t|/8}{C \vdash \text{t.load } memarg : [i32] \rightarrow [t]}$$

t.loadN_{sx} memarg

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[i32] \rightarrow [t]$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq N/8}{C \vdash t.\text{loadN}_{sx} \text{ memarg} : [i32] \rightarrow [t]}$$

t.store memarg

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than the **bit width** of t divided by 8.
- Then the instruction is valid with type $[i32 \ t] \rightarrow []$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq |t|/8}{C \vdash t.\text{store} \text{ memarg} : [i32 \ t] \rightarrow []}$$

t.storeN memarg

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[i32 \ t] \rightarrow []$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq N/8}{C \vdash t.\text{storeN} \text{ memarg} : [i32 \ t] \rightarrow []}$$

v128.loadN_{xM}_{sx} memarg

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8 \cdot M$.
- Then the instruction is valid with type $[i32] \rightarrow [v128]$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq N/8 \cdot M}{C \vdash v128.\text{loadN}_{xM}_{sx} \text{ memarg} : [i32] \rightarrow [v128]}$$

v128.loadN_{splat} memarg

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[i32] \rightarrow [v128]$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq N/8}{C \vdash v128.\text{loadN}_{splat} \text{ memarg} : [i32] \rightarrow [v128]}$$

`v128.loadN_zero memarg`

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[\text{i32}] \rightarrow [\text{v128}]$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} \leq N/8}{C \vdash \text{v128.loadN_zero memarg} : [\text{i32}] \rightarrow [\text{v128}]}$$

`v128.loadN_lane memarg laneidx`

- The lane index laneidx must be smaller than $128/N$.
- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[\text{i32 v128}] \rightarrow [\text{v128}]$.

$$\frac{\text{laneidx} < 128/N \quad C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} < N/8}{C \vdash \text{v128.loadN_lane memarg laneidx} : [\text{i32 v128}] \rightarrow [\text{v128}]}$$

`v128.storeN_lane memarg laneidx`

- The lane index laneidx must be smaller than $128/N$.
- The memory $C.\text{mems}[0]$ must be defined in the context.
- The alignment $2^{\text{memarg.align}}$ must not be larger than $N/8$.
- Then the instruction is valid with type $[\text{i32 v128}] \rightarrow []$.

$$\frac{\text{laneidx} < 128/N \quad C.\text{mems}[0] = \text{memtype} \quad 2^{\text{memarg.align}} < N/8}{C \vdash \text{v128.storeN_lane memarg laneidx} : [\text{i32 v128}] \rightarrow []}$$

`memory.size`

- The memory $C.\text{mems}[0]$ must be defined in the context.
- Then the instruction is valid with type $[] \rightarrow [\text{i32}]$.

$$\frac{C.\text{mems}[0] = \text{memtype}}{C \vdash \text{memory.size} : [] \rightarrow [\text{i32}]}$$

`memory.grow`

- The memory $C.\text{mems}[0]$ must be defined in the context.
- Then the instruction is valid with type $[\text{i32}] \rightarrow [\text{i32}]$.

$$\frac{C.\text{mems}[0] = \text{memtype}}{C \vdash \text{memory.grow} : [\text{i32}] \rightarrow [\text{i32}]}$$

memory.fill

- The memory $C.\text{mems}[0]$ must be defined in the context.
- Then the instruction is valid with type $[i32\ i32\ i32] \rightarrow []$.

$$\frac{C.\text{mems}[0] = \text{memtype}}{C \vdash \text{memory.fill} : [i32\ i32\ i32] \rightarrow []}$$

memory.copy

- The memory $C.\text{mems}[0]$ must be defined in the context.
- Then the instruction is valid with type $[i32\ i32\ i32] \rightarrow []$.

$$\frac{C.\text{mems}[0] = \text{memtype}}{C \vdash \text{memory.copy} : [i32\ i32\ i32] \rightarrow []}$$

memory.init x

- The memory $C.\text{mems}[0]$ must be defined in the context.
- The data segment $C.\text{datas}[x]$ must be defined in the context.
- Then the instruction is valid with type $[i32\ i32\ i32] \rightarrow []$.

$$\frac{C.\text{mems}[0] = \text{memtype} \quad C.\text{datas}[x] = \text{ok}}{C \vdash \text{memory.init } x : [i32\ i32\ i32] \rightarrow []}$$

data.drop x

- The data segment $C.\text{datas}[x]$ must be defined in the context.
- Then the instruction is valid with type $[] \rightarrow []$.

$$\frac{C.\text{datas}[x] = \text{ok}}{C \vdash \text{data.drop } x : [] \rightarrow []}$$

3.3.8 Control Instructions**nop**

- The instruction is valid with type $[] \rightarrow []$.

$$\overline{C \vdash \text{nop} : [] \rightarrow []}$$

unreachable

- The instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$, for any sequences of operand types t_1^* and t_2^* .

$$\overline{C \vdash \text{unreachable} : [t_1^*] \rightarrow [t_2^*]}$$

Note: The `unreachable` instruction is *stack-polymorphic*.

block blocktype instr end*

- The **block type** must be **valid** as some **function type** $[t_1^*] \rightarrow [t_2^*]$.
- Let C' be the same **context** as C , but with the **result type** $[t_2^*]$ prepended to the **labels** vector.
- Under context C' , the instruction sequence $instr^*$ must be **valid** with type $[t_1^*] \rightarrow [t_2^*]$.
- Then the compound instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$.

$$\frac{C \vdash \text{blocktype} : [t_1^*] \rightarrow [t_2^*] \quad C, \text{labels}[t_2^*] \vdash instr^* : [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{block blocktype instr}^* \text{ end} : [t_1^*] \rightarrow [t_2^*]}$$

Note: The notation $C, \text{labels}[t^*]$ inserts the new label type at index 0, shifting all others.

loop blocktype instr end*

- The **block type** must be **valid** as some **function type** $[t_1^*] \rightarrow [t_2^*]$.
- Let C' be the same **context** as C , but with the **result type** $[t_1^*]$ prepended to the **labels** vector.
- Under context C' , the instruction sequence $instr^*$ must be **valid** with type $[t_1^*] \rightarrow [t_2^*]$.
- Then the compound instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$.

$$\frac{C \vdash \text{blocktype} : [t_1^*] \rightarrow [t_2^*] \quad C, \text{labels}[t_1^*] \vdash instr^* : [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{loop blocktype instr}^* \text{ end} : [t_1^*] \rightarrow [t_2^*]}$$

Note: The notation $C, \text{labels}[t^*]$ inserts the new label type at index 0, shifting all others.

if blocktype instr₁ else instr₂* end*

- The **block type** must be **valid** as some **function type** $[t_1^*] \rightarrow [t_2^*]$.
- Let C' be the same **context** as C , but with the **result type** $[t_2^*]$ prepended to the **labels** vector.
- Under context C' , the instruction sequence $instr_1^*$ must be **valid** with type $[t_1^*] \rightarrow [t_2^*]$.
- Under context C' , the instruction sequence $instr_2^*$ must be **valid** with type $[t_1^*] \rightarrow [t_2^*]$.
- Then the compound instruction is valid with type $[t_1^* \text{ i32}] \rightarrow [t_2^*]$.

$$\frac{C \vdash \text{blocktype} : [t_1^*] \rightarrow [t_2^*] \quad C, \text{labels}[t_2^*] \vdash instr_1^* : [t_1^*] \rightarrow [t_2^*] \quad C, \text{labels}[t_2^*] \vdash instr_2^* : [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{if blocktype instr}_1^* \text{ else instr}_2^* \text{ end} : [t_1^* \text{ i32}] \rightarrow [t_2^*]}$$

Note: The notation $C, \text{labels}[t^*]$ inserts the new label type at index 0, shifting all others.

br l

- The label $C.\text{labels}[l]$ must be defined in the context.
- Let $[t^*]$ be the **result type** $C.\text{labels}[l]$.
- Then the instruction is valid with type $[t_1^* t^*] \rightarrow [t_2^*]$, for any sequences of **operand types** t_1^* and t_2^* .

$$\frac{C.\text{labels}[l] = [t^*]}{C \vdash \text{br } l : [t_1^* t^*] \rightarrow [t_2^*]}$$

Note: The [label index](#) space in the [context](#) C contains the most recent label first, so that $C.\text{labels}[l]$ performs a relative lookup as expected.

The `br` instruction is [stack-polymorphic](#).

`br_if l`

- The label $C.\text{labels}[l]$ must be defined in the context.
- Let $[t^*]$ be the [result type](#) $C.\text{labels}[l]$.
- Then the instruction is valid with type $[t^* \text{ i32}] \rightarrow [t^*]$.

$$\frac{C.\text{labels}[l] = [t^*]}{C \vdash \text{br_if } l : [t^* \text{ i32}] \rightarrow [t^*]}$$

Note: The [label index](#) space in the [context](#) C contains the most recent label first, so that $C.\text{labels}[l]$ performs a relative lookup as expected.

`br_table l* lN`

- The label $C.\text{labels}[l_N]$ must be defined in the context.
- For all l_i in l^* , the label $C.\text{labels}[l_i]$ must be defined in the context.
- There must be a sequence t^* of [operand types](#), such that:
 - For each [operand type](#) t_j in t^* and corresponding type t'_{Nj} in $C.\text{labels}[l_N]$, t_j matches t'_{Nj} .
 - For all l_i in l^* , and for each [operand type](#) t_j in t^* and corresponding type t'_{ij} in $C.\text{labels}[l_i]$, t_j matches t'_{ij} .
- Then the instruction is valid with type $[t_1^* t^* \text{ i32}] \rightarrow [t_2^*]$, for any sequences of [operand types](#) t_1^* and t_2^* .

$$\frac{(\vdash [t^*] \leq C.\text{labels}[l])^* \quad \vdash [t^*] \leq C.\text{labels}[l_N]}{C \vdash \text{br_table } l^* l_N : [t_1^* t^* \text{ i32}] \rightarrow [t_2^*]}$$

Note: The [label index](#) space in the [context](#) C contains the most recent label first, so that $C.\text{labels}[l_i]$ performs a relative lookup as expected.

The `br_table` instruction is [stack-polymorphic](#).

`return`

- The return type $C.\text{return}$ must not be absent in the context.
- Let $[t^*]$ be the [result type](#) of $C.\text{return}$.
- Then the instruction is valid with type $[t_1^* t^*] \rightarrow [t_2^*]$, for any sequences of [operand types](#) t_1^* and t_2^* .

$$\frac{C.\text{return} = [t^*]}{C \vdash \text{return} : [t_1^* t^*] \rightarrow [t_2^*]}$$

Note: The `return` instruction is [stack-polymorphic](#).

$C.\text{return}$ is absent (set to ϵ) when validating an [expression](#) that is not a function body. This differs from it being set to the empty result type ($[\epsilon]$), which is the case for functions not returning anything.

$\text{call } x$

- The function $C.\text{funcs}[x]$ must be defined in the context.
- Then the instruction is valid with type $C.\text{funcs}[x]$.

$$\frac{C.\text{funcs}[x] = [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{call } x : [t_1^*] \rightarrow [t_2^*]}$$

$\text{call_indirect } x \ y$

- The table $C.\text{tables}[x]$ must be defined in the context.
- Let $\text{limits } t$ be the [table type](#) $C.\text{tables}[x]$.
- The [reference type](#) t must be [funcref](#).
- The type $C.\text{types}[y]$ must be defined in the context.
- Let $[t_1^*] \rightarrow [t_2^*]$ be the [function type](#) $C.\text{types}[y]$.
- Then the instruction is valid with type $[t_1^* \text{ i32}] \rightarrow [t_2^*]$.

$$\frac{C.\text{tables}[x] = \text{limits funcref} \quad C.\text{types}[y] = [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{call_indirect } x \ y : [t_1^* \text{ i32}] \rightarrow [t_2^*]}$$

3.3.9 Instruction Sequences

Typing of instruction sequences is defined recursively.

Empty Instruction Sequence: ϵ

- The empty instruction sequence is valid with type $[t^*] \rightarrow [t^*]$, for any sequence of [operand types](#) t^* .

$$\overline{C \vdash \epsilon : [t^*] \rightarrow [t^*]}$$

Non-empty Instruction Sequence: $\text{instr}^* \text{ instr}_N$

- The instruction sequence instr^* must be valid with type $[t_1^*] \rightarrow [t_2^*]$, for some sequences of [operand types](#) t_1^* and t_2^* .
- The instruction instr_N must be valid with type $[t^*] \rightarrow [t_3^*]$, for some sequences of [operand types](#) t^* and t_3^* .
- There must be a sequence of [operand types](#) t_0^* , such that $t_2^* = t_0^* t'^*$ where the type sequence t'^* is as long as t^* .
- For each [operand type](#) t'_i in t'^* and corresponding type t_i in t^* , t'_i matches t_i .
- Then the combined instruction sequence is valid with type $[t_1^*] \rightarrow [t_0^* t_3^*]$.

$$\frac{C \vdash \text{instr}^* : [t_1^*] \rightarrow [t_0^* t'^*] \quad \vdash [t'^*] \leq [t^*] \quad C \vdash \text{instr}_N : [t^*] \rightarrow [t_3^*]}{C \vdash \text{instr}^* \text{ instr}_N : [t_1^*] \rightarrow [t_0^* t_3^*]}$$

3.3.10 Expressions

Expressions *expr* are classified by **result types** of the form $[t^*]$.

instr^{*} end

- The instruction sequence *instr*^{*} must be **valid** with some **stack type** $[] \rightarrow [t'^*]$.
- For each **operand type** t'_i in t'^* and corresponding **value type** t_i in t^* , t'_i **matches** t_i .
- Then the expression is valid with **result type** $[t^*]$.

$$\frac{C \vdash \text{instr}^* : [] \rightarrow [t'^*] \quad \vdash [t'^*] \leq [t^*]}{C \vdash \text{instr}^* \text{ end} : [t^*]}$$

Constant Expressions

- In a **constant** expression *instr*^{*} end all instructions in *instr*^{*} must be constant.
- A constant instruction *instr* must be:
 - either of the form *t.const* *c*,
 - or of the form *ref.null*,
 - or of the form *ref.func* *x*,
 - or of the form *global.get* *x*, in which case $C.\text{globals}[x]$ must be a **global type** of the form *const* *t*.

$$\frac{(C \vdash \text{instr} \text{ const})^*}{C \vdash \text{instr}^* \text{ end} \text{ const}}$$

$$\frac{}{C \vdash \text{t.const } c \text{ const}} \quad \frac{}{C \vdash \text{ref.null } t \text{ const}} \quad \frac{}{C \vdash \text{ref.func } x \text{ const}}$$

$$\frac{C.\text{globals}[x] = \text{const } t}{C \vdash \text{global.get } x \text{ const}}$$

Note: Currently, constant expressions occurring in **globals**, **element**, or **data** segments are further constrained in that contained *global.get* instructions are only allowed to refer to *imported* globals. This is enforced in the **validation rule for modules** by constraining the context *C* accordingly.

The definition of constant expression may be extended in future versions of WebAssembly.

3.4 Modules

Modules are valid when all the components they contain are valid. Furthermore, most definitions are themselves classified with a suitable type.

3.4.1 Functions

Functions *func* are classified by function types of the form $[t_1^*] \rightarrow [t_2^*]$.

$\{\text{type } x, \text{locals } t^*, \text{body } \textit{expr}\}$

- The type $C.\text{types}[x]$ must be defined in the context.
- Let $[t_1^*] \rightarrow [t_2^*]$ be the function type $C.\text{types}[x]$.
- Let C' be the same context as C , but with:
 - locals set to the sequence of value types $t_1^* t^*$, concatenating parameters and locals,
 - labels set to the singular sequence containing only result type $[t_2^*]$.
 - return set to the result type $[t_2^*]$.
- Under the context C' , the expression *expr* must be valid with type $[t_2^*]$.
- Then the function definition is valid with type $[t_1^*] \rightarrow [t_2^*]$.

$$\frac{C.\text{types}[x] = [t_1^*] \rightarrow [t_2^*] \quad C, \text{locals } t_1^* t^*, \text{labels } [t_2^*], \text{return } [t_2^*] \vdash \textit{expr} : [t_2^*]}{C \vdash \{\text{type } x, \text{locals } t^*, \text{body } \textit{expr}\} : [t_1^*] \rightarrow [t_2^*]}$$

3.4.2 Tables

Tables *table* are classified by table types.

$\{\text{type } \textit{tabletype}\}$

- The table type *tabletype* must be valid.
- Then the table definition is valid with type *tabletype*.

$$\frac{\vdash \textit{tabletype} \text{ ok}}{C \vdash \{\text{type } \textit{tabletype}\} : \textit{tabletype}}$$

3.4.3 Memories

Memories *mem* are classified by memory types.

$\{\text{type } \textit{memtype}\}$

- The memory type *memtype* must be valid.
- Then the memory definition is valid with type *memtype*.

$$\frac{\vdash \textit{memtype} \text{ ok}}{C \vdash \{\text{type } \textit{memtype}\} : \textit{memtype}}$$

3.4.4 Globals

Globals *global* are classified by global types of the form *mut t*.

$\{\text{type } \text{mut } t, \text{init } \text{expr}\}$

- The global type *mut t* must be valid.
- The expression *expr* must be valid with result type $[t]$.
- The expression *expr* must be constant.
- Then the global definition is valid with type *mut t*.

$$\frac{\vdash \text{mut } t \text{ ok} \quad C \vdash \text{expr} : [t] \quad C \vdash \text{expr} \text{ const}}{C \vdash \{\text{type } \text{mut } t, \text{init } \text{expr}\} : \text{mut } t}$$

3.4.5 Element Segments

Element segments *elem* are classified by the reference type of their elements.

$\{\text{type } t, \text{init } e^*, \text{mode } \text{elemmode}\}$

- For each e_i in e^* :
 - The expression e_i must be valid with some result type $[t]$.
 - The expression e_i must be constant.
- The element mode *elemmode* must be valid with reference type *t*.
- Then the element segment is valid with reference type *t*.

$$\frac{(C \vdash e : [t])^* \quad (C \vdash e \text{ const})^* \quad C \vdash \text{elemmode} : t}{C \vdash \{\text{type } t, \text{init } e^*, \text{mode } \text{elemmode}\} : t}$$

passive

- The element mode is valid with any reference type.

$$\overline{C \vdash \text{passive} : \text{reftype}}$$

active $\{\text{table } x, \text{offset } \text{expr}\}$

- The table $C.\text{tables}[x]$ must be defined in the context.
- Let *limits t* be the table type $C.\text{tables}[x]$.
- The expression *expr* must be valid with result type $[i32]$.
- The expression *expr* must be constant.
- Then the element mode is valid with reference type *t*.

$$\frac{C.\text{tables}[x] = \text{limits } t \quad C \vdash \text{expr} : [i32] \quad C \vdash \text{expr} \text{ const}}{C \vdash \text{active } \{\text{table } x, \text{offset } \text{expr}\} : t}$$

declarative

- The element mode is valid with any [reference type](#).

$$\overline{C \vdash \text{declarative} : \text{ref}\text{type}}$$

3.4.6 Data Segments

Data segments *data* are not classified by any type but merely checked for well-formedness.

$\{\text{init } b^*, \text{mode } \text{datamode}\}$

- The data mode *datamode* must be valid.
- Then the data segment is valid.

$$\frac{C \vdash \text{datamode} \text{ ok}}{C \vdash \{\text{init } b^*, \text{mode } \text{datamode}\} \text{ ok}}$$

passive

- The data mode is valid.

$$\overline{C \vdash \text{passive} \text{ ok}}$$

active $\{\text{memory } x, \text{offset } \text{expr}\}$

- The memory $C.\text{mems}[x]$ must be defined in the context.
- The expression *expr* must be [valid](#) with [result type](#) $[i32]$.
- The expression *expr* must be [constant](#).
- Then the data mode is valid.

$$\frac{C.\text{mems}[x] = \text{limits} \quad C \vdash \text{expr} : [i32] \quad C \vdash \text{expr} \text{ const}}{C \vdash \text{active} \{\text{memory } x, \text{offset } \text{expr}\} \text{ ok}}$$

3.4.7 Start Function

Start function declarations *start* are not classified by any type.

$\{\text{func } x\}$

- The function $C.\text{funcs}[x]$ must be defined in the context.
- The type of $C.\text{funcs}[x]$ must be $[] \rightarrow []$.
- Then the start function is valid.

$$\frac{C.\text{funcs}[x] = [] \rightarrow []}{C \vdash \{\text{func } x\} \text{ ok}}$$

3.4.8 Exports

Exports *export* and export descriptions *exportdesc* are classified by their external type.

$\{\text{name } name, \text{desc } exportdesc\}$

- The export description *exportdesc* must be valid with external type *externtype*.
- Then the export is valid with external type *externtype*.

$$\frac{C \vdash exportdesc : externtype}{C \vdash \{\text{name } name, \text{desc } exportdesc\} : externtype}$$

func *x*

- The function $C.\text{funcs}[x]$ must be defined in the context.
- Then the export description is valid with external type func $C.\text{funcs}[x]$.

$$\frac{C.\text{funcs}[x] = functype}{C \vdash \text{func } x : \text{func } functype}$$

table *x*

- The table $C.\text{tables}[x]$ must be defined in the context.
- Then the export description is valid with external type table $C.\text{tables}[x]$.

$$\frac{C.\text{tables}[x] = tabletype}{C \vdash \text{table } x : \text{table } tabletype}$$

mem *x*

- The memory $C.\text{mems}[x]$ must be defined in the context.
- Then the export description is valid with external type mem $C.\text{mems}[x]$.

$$\frac{C.\text{mems}[x] = memtype}{C \vdash \text{mem } x : \text{mem } memtype}$$

global *x*

- The global $C.\text{globals}[x]$ must be defined in the context.
- Then the export description is valid with external type global $C.\text{globals}[x]$.

$$\frac{C.\text{globals}[x] = globaltype}{C \vdash \text{global } x : \text{global } globaltype}$$

3.4.9 Imports

Imports *import* and import descriptions *importdesc* are classified by external types.

$\{\text{module } name_1, \text{name } name_2, \text{desc } importdesc\}$

- The import description *importdesc* must be valid with type *externtype*.
- Then the import is valid with type *externtype*.

$$\frac{C \vdash importdesc : externtype}{C \vdash \{\text{module } name_1, \text{name } name_2, \text{desc } importdesc\} : externtype}$$

func x

- The function $C.\text{types}[x]$ must be defined in the context.
- Let $[t_1^*] \rightarrow [t_2^*]$ be the function type $C.\text{types}[x]$.
- Then the import description is valid with type *func* $[t_1^*] \rightarrow [t_2^*]$.

$$\frac{C.\text{types}[x] = [t_1^*] \rightarrow [t_2^*]}{C \vdash \text{func } x : \text{func } [t_1^*] \rightarrow [t_2^*]}$$

table tabletype

- The table type *tabletype* must be valid.
- Then the import description is valid with type *table tabletype*.

$$\frac{\vdash tabletype \text{ ok}}{C \vdash \text{table } tabletype : \text{table } tabletype}$$

mem memtype

- The memory type *memtype* must be valid.
- Then the import description is valid with type *mem memtype*.

$$\frac{\vdash memtype \text{ ok}}{C \vdash \text{mem } memtype : \text{mem } memtype}$$

global globaltype

- The global type *globaltype* must be valid.
- Then the import description is valid with type *global globaltype*.

$$\frac{\vdash globaltype \text{ ok}}{C \vdash \text{global } globaltype : \text{global } globaltype}$$

3.4.10 Modules

Modules are classified by their mapping from the **external types** of their **imports** to those of their **exports**.

A module is entirely *closed*, that is, its components can only refer to definitions that appear in the module itself. Consequently, no initial **context** is required. Instead, the context C for validation of the module's content is constructed from the definitions in the module.

- Let $module$ be the module to validate.
- Let C be a **context** where:
 - $C.types$ is $module.types$,
 - $C.funcs$ is $funcs(it^*)$ concatenated with ft^* , with the import's **external types** it^* and the internal **function types** ft^* as determined below,
 - $C.tables$ is $tables(it^*)$ concatenated with tt^* , with the import's **external types** it^* and the internal **table types** tt^* as determined below,
 - $C.mems$ is $mems(it^*)$ concatenated with mt^* , with the import's **external types** it^* and the internal **memory types** mt^* as determined below,
 - $C.globals$ is $globals(it^*)$ concatenated with gt^* , with the import's **external types** it^* and the internal **global types** gt^* as determined below,
 - $C.elems$ is rt^* as determined below,
 - $C.datas$ is ok^n , where n is the length of the vector $module.datas$,
 - $C.locals$ is empty,
 - $C.labels$ is empty,
 - $C.return$ is empty.
 - $C.refs$ is the set $funcidx(module \text{ with } funcs = \epsilon \text{ with } start = \epsilon)$, i.e., the set of **function indices** occurring in the module, except in its **functions** or **start function**.
- Let C' be the **context** where:
 - $C'.globals$ is the sequence $globals(it^*)$,
 - $C'.funcs$ is the same as $C.funcs$,
 - $C'.refs$ is the same as $C.refs$,
 - all other fields are empty.
- For each $functype_i$ in $module.types$, the **function type** $functype_i$ must be **valid**.
- Under the context C :
 - For each $func_i$ in $module.funcs$, the definition $func_i$ must be **valid** with a **function type** ft_i .
 - If $module.start$ is non-empty, then $module.start$ must be **valid**.
 - For each $import_i$ in $module.imports$, the segment $import_i$ must be **valid** with an **external type** it_i .
 - For each $export_i$ in $module.exports$, the segment $export_i$ must be **valid** with **external type** et_i .
- Under the context C' :
 - For each $table_i$ in $module.tables$, the definition $table_i$ must be **valid** with a **table type** tt_i .
 - For each mem_i in $module.mems$, the definition mem_i must be **valid** with a **memory type** mt_i .
 - For each $global_i$ in $module.globals$, the definition $global_i$ must be **valid** with a **global type** gt_i .
 - For each $elem_i$ in $module.elems$, the segment $elem_i$ must be **valid** with **reference type** rt_i .
 - For each $data_i$ in $module.datas$, the segment $data_i$ must be **valid**.
- The length of $C.mems$ must not be larger than 1.

- All export names $export_i.name$ must be different.
- Let ft^* be the concatenation of the internal function types ft_i , in index order.
- Let tt^* be the concatenation of the internal table types tt_i , in index order.
- Let mt^* be the concatenation of the internal memory types mt_i , in index order.
- Let gt^* be the concatenation of the internal global types gt_i , in index order.
- Let rt^* be the concatenation of the reference types rt_i , in index order.
- Let it^* be the concatenation of external types it_i of the imports, in index order.
- Let et^* be the concatenation of external types et_i of the exports, in index order.
- Then the module is valid with external types $it^* \rightarrow et^*$.

$$\begin{array}{c}
(\vdash \text{type ok})^* \quad (C \vdash \text{func} : ft)^* \quad (C' \vdash \text{table} : tt)^* \quad (C' \vdash \text{mem} : mt)^* \quad (C' \vdash \text{global} : gt)^* \\
(C' \vdash \text{elem} : rt)^* \quad (C' \vdash \text{data ok})^n \quad (C \vdash \text{start ok})^? \quad (C \vdash \text{import} : it)^* \quad (C \vdash \text{export} : et)^* \\
ift^* = \text{funcs}(it^*) \quad itt^* = \text{tables}(it^*) \quad imt^* = \text{mems}(it^*) \quad igt^* = \text{globals}(it^*) \\
x^* = \text{funcidx}(\text{module with funcs} = \epsilon \text{ with start} = \epsilon) \\
C = \{\text{types } type^*, \text{funcs } ift^* ft^*, \text{tables } itt^* tt^*, \text{mems } imt^* mt^*, \text{globals } igt^* gt^*, \text{elems } rt^*, \text{datas ok}^n, \text{refs } x^*\} \\
C' = \{\text{globals } igt^*, \text{funcs } (C.\text{funcs}), \text{refs } (C.\text{refs})\} \quad |C.\text{mems}| \leq 1 \quad (export.name)^* \text{ disjoint} \\
\text{module} = \{\text{types } type^*, \text{funcs } func^*, \text{tables } table^*, \text{mems } mem^*, \text{globals } global^*, \\
\text{elems } elem^*, \text{datas } data^n, \text{start } start^?, \text{imports } import^*, \text{exports } export^*\} \\
\hline
\vdash \text{module} : it^* \rightarrow et^*
\end{array}$$

Note: Most definitions in a module – particularly functions – are mutually recursive. Consequently, the definition of the context C in this rule is recursive: it depends on the outcome of validation of the function, table, memory, and global definitions contained in the module, which itself depends on C . However, this recursion is just a specification device. All types needed to construct C can easily be determined from a simple pre-pass over the module that does not perform any actual validation.

Globals, however, are not recursive and not accessible within constant expressions when they are defined locally. The effect of defining the limited context C' for validating certain definitions is that they can only access functions and imported globals and nothing else.

Note: The restriction on the number of memories may be lifted in future versions of WebAssembly.

4.1 Conventions

WebAssembly code is *executed* when *instantiating* a module or *invoking* an *exported* function on the resulting module *instance*.

Execution behavior is defined in terms of an *abstract machine* that models the *program state*. It includes a *stack*, which records operand values and control constructs, and an abstract *store* containing global state.

For each instruction, there is a rule that specifies the effect of its execution on the program state. Furthermore, there are rules describing the instantiation of a module. As with *validation*, all rules are given in two *equivalent* forms:

1. In *prose*, describing the execution in intuitive form.
2. In *formal notation*, describing the rule in mathematical form.¹⁸

Note: As with validation, the prose and formal rules are equivalent, so that understanding of the formal notation is *not* required to read this specification. The formalism offers a more concise description in notation that is used widely in programming languages semantics and is readily amenable to mathematical proof.

4.1.1 Prose Notation

Execution is specified by stylised, step-wise rules for each *instruction* of the *abstract syntax*. The following conventions are adopted in stating these rules.

- The execution rules implicitly assume a given *store* *S*.
- The execution rules also assume the presence of an implicit *stack* that is modified by *pushing* or *popping* values, labels, and frames.
- Certain rules require the stack to contain at least one frame. The most recent frame is referred to as the *current* frame.

¹⁸ The semantics is derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titze, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. *Bringing the Web up to Speed with WebAssembly* ^{Page 55}.¹⁹ Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

¹⁹ <https://dl.acm.org/citation.cfm?doid=3062341.3062363>

- Both the store and the current frame are mutated by *replacing* some of their components. Such replacement is assumed to apply globally.
- The execution of an instruction may *trap*, in which case the entire computation is aborted and no further modifications to the store are performed by it. (Other computations can still be initiated afterwards.)
- The execution of an instruction may also end in a *jump* to a designated target, which defines the next instruction to execute.
- Execution can *enter* and *exit* [instruction sequences](#) that form [blocks](#).
- [Instruction sequences](#) are implicitly executed in order, unless a trap or jump occurs.
- In various places the rules contain *assertions* expressing crucial invariants about the program state.

4.1.2 Formal Notation

Note: This section gives a brief explanation of the notation for specifying execution formally. For the interested reader, a more thorough introduction can be found in respective text books.²⁰

The formal execution rules use a standard approach for specifying operational semantics, rendering them into *reduction rules*. Every rule has the following general form:

$$\text{configuration} \hookrightarrow \text{configuration}$$

A *configuration* is a syntactic description of a program state. Each rule specifies one *step* of execution. As long as there is at most one reduction rule applicable to a given configuration, reduction – and thereby execution – is *deterministic*. WebAssembly has only very few exceptions to this, which are noted explicitly in this specification.

For WebAssembly, a configuration typically is a tuple $(S; F; \text{instr}^*)$ consisting of the current [store](#) S , the [call frame](#) F of the current function, and the sequence of [instructions](#) that is to be executed. (A more precise definition is given [later](#).)

To avoid unnecessary clutter, the store S and the frame F are omitted from reduction rules that do not touch them.

There is no separate representation of the [stack](#). Instead, it is conveniently represented as part of the configuration’s instruction sequence. In particular, [values](#) are defined to coincide with [const](#) instructions, and a sequence of [const](#) instructions can be interpreted as an operand “stack” that grows to the right.

Note: For example, the [reduction rule](#) for the `i32.add` instruction can be given as follows:

$$(\text{i32.const } n_1) (\text{i32.const } n_2) \text{i32.add} \hookrightarrow (\text{i32.const } (n_1 + n_2) \bmod 2^{32})$$

Per this rule, two [const](#) instructions and the `add` instruction itself are removed from the instruction stream and replaced with one new [const](#) instruction. This can be interpreted as popping two values off the stack and pushing the result.

When no result is produced, an instruction reduces to the empty sequence:

$$\text{nop} \hookrightarrow \epsilon$$

[Labels](#) and [frames](#) are similarly [defined](#) to be part of an instruction sequence.

The order of reduction is determined by the definition of an appropriate [evaluation context](#).

Reduction *terminates* when no more reduction rules are applicable. [Soundness](#) of the WebAssembly [type system](#) guarantees that this is only the case when the original instruction sequence has either been reduced to a sequence of [const](#) instructions, which can be interpreted as the [values](#) of the resulting operand stack, or if a [trap](#) occurred.

²⁰ For example: Benjamin Pierce. [Types and Programming Languages](#) Page 56, 21. The MIT Press 2002

²¹ <https://www.cis.upenn.edu/~bcpierce/tapl/>

Note: For example, the following instruction sequence,

(f64.const x_1) (f64.const x_2) f64.neg (f64.const x_3) f64.add f64.mul

terminates after three steps:

(f64.const x_1) (f64.const x_2) f64.neg (f64.const x_3) f64.add f64.mul
 $\hookrightarrow$ (f64.const x_1) (f64.const x_4) (f64.const x_3) f64.add f64.mul
 $\hookrightarrow$ (f64.const x_1) (f64.const x_5) f64.mul
 $\hookrightarrow$ (f64.const x_6)

where $x_4 = -x_2$ and $x_5 = -x_2 + x_3$ and $x_6 = x_1 \cdot (-x_2 + x_3)$.

4.2 Runtime Structure

Store, stack, and other *runtime structure* forming the WebAssembly abstract machine, such as *values* or *module instances*, are made precise in terms of additional auxiliary syntax.

4.2.1 Values

WebAssembly computations manipulate *values* of either the four basic *number types*, i.e., *integers* and *floating-point data* of 32 or 64 bit width each, or *vectors* of 128 bit width, or of *reference type*.

In most places of the semantics, values of different types can occur. In order to avoid ambiguities, values are therefore represented with an abstract syntax that makes their type explicit. It is convenient to reuse the same notation as for the *const instructions* and *ref.null* producing them.

References other than null are represented with additional *administrative instructions*. They either are *function references*, pointing to a specific *function address*, or *external references* pointing to an uninterpreted form of *extern address* that can be defined by the *embedder* to represent its own objects.

$$\begin{array}{ll}
 \text{num} & ::= \text{i32.const } i_{32} \\
 & \quad | \text{i64.const } i_{64} \\
 & \quad | \text{f32.const } f_{32} \\
 & \quad | \text{f64.const } f_{64} \\
 \text{vec} & ::= \text{v128.const } i_{128} \\
 \text{ref} & ::= \text{ref.null } t \\
 & \quad | \text{ref.funcaddr} \\
 & \quad | \text{ref.extern } \text{externaddr} \\
 \text{val} & ::= \text{num} \mid \text{vec} \mid \text{ref}
 \end{array}$$

Note: Future versions of WebAssembly may add additional forms of reference.

Each *value type* has an associated *default value*; it is the respective value 0 for *number types*, 0 for *vector types*, and null for *reference types*.

$$\begin{array}{lll}
 \text{default}_t & = & t.\text{const } 0 \quad (\text{if } t = \text{numtype}) \\
 \text{default}_t & = & t.\text{const } 0 \quad (\text{if } t = \text{vectype}) \\
 \text{default}_t & = & \text{ref.null } t \quad (\text{if } t = \text{reftype})
 \end{array}$$

Convention

- The meta variable r ranges over reference values where clear from context.

4.2.2 Results

A *result* is the outcome of a computation. It is either a sequence of *values* or a *trap*.

$$\begin{array}{lcl} \textit{result} & ::= & \textit{val}^* \\ & | & \textit{trap} \end{array}$$

4.2.3 Store

The *store* represents all global state that can be manipulated by WebAssembly programs. It consists of the runtime representation of all *instances* of *functions*, *tables*, *memories*, and *globals*, *element segments*, and *data segments* that have been *allocated* during the life time of the abstract machine.²²

It is an invariant of the semantics that no element or data instance is *addressed* from anywhere else but the owning module instances.

Syntactically, the store is defined as a *record* listing the existing instances of each category:

$$\textit{store} ::= \{ \begin{array}{ll} \textit{funcs} & \textit{funcinst}^*, \\ \textit{tables} & \textit{tableinst}^*, \\ \textit{mems} & \textit{meminst}^*, \\ \textit{globals} & \textit{globalinst}^*, \\ \textit{elems} & \textit{eleminst}^*, \\ \textit{datas} & \textit{datainst}^* \end{array} \}$$

Convention

- The meta variable S ranges over stores where clear from context.

4.2.4 Addresses

Function instances, table instances, memory instances, and global instances, element instances, and data instances in the *store* are referenced with abstract *addresses*. These are simply indices into the respective store component. In addition, an *embedder* may supply an uninterpreted set of *host addresses*.

$$\begin{array}{lcl} \textit{addr} & ::= & 0 \mid 1 \mid 2 \mid \dots \\ \textit{funcaddr} & ::= & \textit{addr} \\ \textit{tableaddr} & ::= & \textit{addr} \\ \textit{memaddr} & ::= & \textit{addr} \\ \textit{globaladdr} & ::= & \textit{addr} \\ \textit{elemaddr} & ::= & \textit{addr} \\ \textit{dataaddr} & ::= & \textit{addr} \\ \textit{externaddr} & ::= & \textit{addr} \end{array}$$

An *embedder* may assign identity to *exported* store objects corresponding to their addresses, even where this identity is not observable from within WebAssembly code itself (such as for *function instances* or immutable *globals*).

Note: Addresses are *dynamic*, globally unique references to runtime objects, in contrast to *indices*, which are *static*, module-local references to their original definitions. A *memory address* *memaddr* denotes the abstract address *of* a memory *instance* in the store, not an offset *inside* a memory instance.

²² In practice, implementations may apply techniques like garbage collection to remove objects from the store that are no longer referenced. However, such techniques are not semantically observable, and hence outside the scope of this specification.

There is no specific limit on the number of allocations of store objects, hence logical addresses can be arbitrarily large natural numbers.

4.2.5 Module Instances

A *module instance* is the runtime representation of a [module](#). It is created by [instantiating](#) a module, and collects runtime representations of all entities that are imported, defined, or exported by the module.

$$\begin{aligned} \text{moduleinst} \quad ::= \quad & \{ \text{types} \quad \text{functype}^*, \\ & \text{funcaddrs} \quad \text{funcaddr}^*, \\ & \text{tableaddrs} \quad \text{tableaddr}^*, \\ & \text{memaddrs} \quad \text{memaddr}^*, \\ & \text{globaladdrs} \quad \text{globaladdr}^*, \\ & \text{elemaddrs} \quad \text{elemaddr}^*, \\ & \text{dataaddrs} \quad \text{dataaddr}^*, \\ & \text{exports} \quad \text{exportinst}^* \} \end{aligned}$$

Each component references runtime instances corresponding to respective declarations from the original module – whether imported or defined – in the order of their static [indices](#). [Function instances](#), [table instances](#), [memory instances](#), and [global instances](#) are referenced with an indirection through their respective [addresses](#) in the [store](#).

It is an invariant of the semantics that all [export instances](#) in a given module instance have different [names](#).

4.2.6 Function Instances

A *function instance* is the runtime representation of a [function](#). It effectively is a *closure* of the original function over the runtime [module instance](#) of its originating [module](#). The module instance is used to resolve references to other definitions during execution of the function.

$$\begin{aligned} \text{funcinst} \quad & ::= \quad \{ \text{type } \text{functype}, \text{module } \text{moduleinst}, \text{code } \text{func} \} \\ & \mid \quad \{ \text{type } \text{functype}, \text{hostcode } \text{hostfunc} \} \\ \text{hostfunc} \quad & ::= \quad \dots \end{aligned}$$

A *host function* is a function expressed outside WebAssembly but passed to a [module](#) as an [import](#). The definition and behavior of host functions are outside the scope of this specification. For the purpose of this specification, it is assumed that when [invoked](#), a host function behaves non-deterministically, but within certain [constraints](#) that ensure the integrity of the runtime.

Note: Function instances are immutable, and their identity is not observable by WebAssembly code. However, the [embedder](#) might provide implicit or explicit means for distinguishing their [addresses](#).

4.2.7 Table Instances

A *table instance* is the runtime representation of a [table](#). It records its [type](#) and holds a vector of [reference values](#).

$$\text{tableinst} \quad ::= \quad \{ \text{type } \text{tabletype}, \text{elem } \text{vec}(\text{ref}) \}$$

Table elements can be mutated through [table instructions](#), the execution of an active [element segment](#), or by external means provided by the [embedder](#).

It is an invariant of the semantics that all table elements have a type equal to the element type of [tabletype](#). It also is an invariant that the length of the element vector never exceeds the maximum size of [tabletype](#), if present.

4.2.8 Memory Instances

A *memory instance* is the runtime representation of a linear [memory](#). It records its [type](#) and holds a vector of [bytes](#).

$$meminst ::= \{\text{type } memtype, \text{data } vec(byte)\}$$

The length of the vector always is a multiple of the WebAssembly *page size*, which is defined to be the constant 65536 – abbreviated 64 Ki.

The bytes can be mutated through [memory instructions](#), the execution of an active [data segment](#), or by external means provided by the [embedder](#).

It is an invariant of the semantics that the length of the byte vector, divided by page size, never exceeds the maximum size of *memtype*, if present.

4.2.9 Global Instances

A *global instance* is the runtime representation of a [global](#) variable. It records its [type](#) and holds an individual [value](#).

$$globalinst ::= \{\text{type } globaltype, \text{value } val\}$$

The value of mutable globals can be mutated through [variable instructions](#) or by external means provided by the [embedder](#).

It is an invariant of the semantics that the value has a type equal to the [value type](#) of *globaltype*.

4.2.10 Element Instances

An *element instance* is the runtime representation of an [element segment](#). It holds a vector of references and their common [type](#).

$$eleminst ::= \{\text{type } reftype, \text{elem } vec(ref)\}$$

4.2.11 Data Instances

An *data instance* is the runtime representation of a [data segment](#). It holds a vector of [bytes](#).

$$datainst ::= \{\text{data } vec(byte)\}$$

4.2.12 Export Instances

An *export instance* is the runtime representation of an [export](#). It defines the export's [name](#) and the associated [external value](#).

$$exportinst ::= \{\text{name } name, \text{value } externval\}$$

4.2.13 External Values

An *external value* is the runtime representation of an entity that can be imported or exported. It is an [address](#) denoting either a [function instance](#), [table instance](#), [memory instance](#), or [global instances](#) in the shared [store](#).

$$\begin{array}{lcl} \textit{externval} & ::= & \textit{func } \textit{funcaddr} \\ & & | \textit{table } \textit{tableaddr} \\ & & | \textit{mem } \textit{memaddr} \\ & & | \textit{global } \textit{globaladdr} \end{array}$$

Conventions

The following auxiliary notation is defined for sequences of external values. It filters out entries of a specific kind in an order-preserving fashion:

- $\textit{funcs}(\textit{externval}^*) = [\textit{funcaddr} \mid (\textit{func } \textit{funcaddr}) \in \textit{externval}^*]$
- $\textit{tables}(\textit{externval}^*) = [\textit{tableaddr} \mid (\textit{table } \textit{tableaddr}) \in \textit{externval}^*]$
- $\textit{mems}(\textit{externval}^*) = [\textit{memaddr} \mid (\textit{mem } \textit{memaddr}) \in \textit{externval}^*]$
- $\textit{globals}(\textit{externval}^*) = [\textit{globaladdr} \mid (\textit{global } \textit{globaladdr}) \in \textit{externval}^*]$

4.2.14 Stack

Besides the [store](#), most [instructions](#) interact with an implicit *stack*. The stack contains three kinds of entries:

- *Values*: the *operands* of instructions.
- *Labels*: active [structured control instructions](#) that can be targeted by branches.
- *Activations*: the *call frames* of active [function](#) calls.

These entries can occur on the stack in any order during the execution of a program. Stack entries are described by abstract syntax as follows.

Note: It is possible to model the WebAssembly semantics using separate stacks for operands, control constructs, and calls. However, because the stacks are interdependent, additional book keeping about associated stack heights would be required. For the purpose of this specification, an interleaved representation is simpler.

Values

Values are represented by [themselves](#).

Labels

Labels carry an argument arity n and their associated branch *target*, which is expressed syntactically as an [instruction](#) sequence:

$$\textit{label} ::= \textit{label}_n\{\textit{instr}^*\}$$

Intuitively, $\textit{instr}^*$ is the *continuation* to execute when the branch is taken, in place of the original control construct.

Note: For example, a loop label has the form

$$\textit{label}_n\{\textit{loop} \dots \textit{end}\}$$

When performing a branch to this label, this executes the loop, effectively restarting it from the beginning. Conversely, a simple block label has the form

$$\text{label}_n\{\epsilon\}$$

When branching, the empty continuation ends the targeted block, such that execution can proceed with consecutive instructions.

Activation Frames

Activation frames carry the return arity n of the respective function, hold the values of its [locals](#) (including arguments) in the order corresponding to their static [local indices](#), and a reference to the function’s own [module instance](#):

$$\text{frame} ::= \{\text{locals } \text{val}^*, \text{module } \text{moduleinst}\}$$

The values of the locals are mutated by respective [variable instructions](#).

Conventions

- The meta variable L ranges over labels where clear from context.
- The meta variable F ranges over frames where clear from context.
- The following auxiliary definition takes a [block type](#) and looks up the [function type](#) that it denotes in the current frame:

$$\begin{aligned} \text{expand}_F(\text{typeid}x) &= F.\text{module.types}[\text{typeid}x] \\ \text{expand}_F([\text{valtype}^?]) &= [] \rightarrow [\text{valtype}^?] \end{aligned}$$

4.2.15 Administrative Instructions

Note: This section is only relevant for the [formal notation](#).

In order to express the reduction of [traps](#), [calls](#), and [control instructions](#), the syntax of instructions is extended to include the following *administrative instructions*:

$$\begin{aligned} \text{instr} ::= & \dots \\ & | \text{trap} \\ & | \text{ref } \text{funcaddr} \\ & | \text{ref.extern } \text{externaddr} \\ & | \text{invoke } \text{funcaddr} \\ & | \text{label}_n\{\text{instr}^*\} \text{ instr}^* \text{ end} \\ & | \text{frame}_n\{\text{frame}\} \text{ instr}^* \text{ end} \end{aligned}$$

The [trap](#) instruction represents the occurrence of a trap. Traps are bubbled up through nested instruction sequences, ultimately reducing the entire program to a single [trap](#) instruction, signalling abrupt termination.

The [ref](#) instruction represents [function reference values](#). Similarly, [ref.extern](#) represents [external references](#).

The [invoke](#) instruction represents the imminent invocation of a [function instance](#), identified by its [address](#). It unifies the handling of different forms of calls.

The [label](#) and [frame](#) instructions model [labels](#) and [frames](#) “on the stack”. Moreover, the administrative syntax maintains the nesting structure of the original [structured control instruction](#) or [function body](#) and their [instruction sequences](#) with an [end](#) marker. That way, the end of the inner instruction sequence is known when part of an outer sequence.

Note: For example, the *reduction rule* for *block* is:

$$\text{block } [t^n] \text{ instr}^* \text{ end} \quad \hookrightarrow \quad \text{label}_n\{\epsilon\} \text{ instr}^* \text{ end}$$

This replaces the block with a label instruction, which can be interpreted as “pushing” the label on the stack. When *end* is reached, i.e., the inner instruction sequence has been reduced to the empty sequence – or rather, a sequence of n *const* instructions representing the resulting values – then the *label* instruction is eliminated courtesy of its own *reduction rule*:

$$\text{label}_m\{\text{instr}^*\} \text{ val}^n \text{ end} \quad \hookrightarrow \quad \text{val}^n$$

This can be interpreted as removing the label from the stack and only leaving the locally accumulated operand values.

Block Contexts

In order to specify the reduction of *branches*, the following syntax of *block contexts* is defined, indexed by the count k of labels surrounding a *hole* $[_]$ that marks the place where the next step of computation is taking place:

$$\begin{aligned} B^0 &::= \text{val}^* [_] \text{instr}^* \\ B^{k+1} &::= \text{val}^* \text{label}_n\{\text{instr}^*\} B^k \text{end instr}^* \end{aligned}$$

This definition allows to index active labels surrounding a *branch* or *return* instruction.

Note: For example, the *reduction* of a simple branch can be defined as follows:

$$\text{label}_0\{\text{instr}^*\} B^l[\text{br } l] \text{end} \quad \hookrightarrow \quad \text{instr}^*$$

Here, the hole $[_]$ of the context is instantiated with a branch instruction. When a branch occurs, this rule replaces the targeted label and associated instruction sequence with the label’s continuation. The selected label is identified through the *label index* l , which corresponds to the number of surrounding *label* instructions that must be hopped over – which is exactly the count encoded in the index of a block context.

Configurations

A *configuration* consists of the current *store* and an executing *thread*.

A thread is a computation over *instructions* that operates relative to a current *frame* referring to the *module instance* in which the computation runs, i.e., where the current function originates from.

$$\begin{aligned} \text{config} &::= \text{store}; \text{thread} \\ \text{thread} &::= \text{frame}; \text{instr}^* \end{aligned}$$

Note: The current version of WebAssembly is single-threaded, but configurations with multiple threads may be supported in the future.

Evaluation Contexts

Finally, the following definition of *evaluation context* and associated structural rules enable reduction inside instruction sequences and administrative forms as well as the propagation of traps:

$$\begin{aligned}
 E &::= \quad __ \mid \text{val}^* E \text{ instr}^* \mid \text{label}_n\{\text{instr}^*\} E \text{ end} \\
 S; F; E[\text{instr}^*] &\hookrightarrow S'; F'; E[\text{instr}'^*] \\
 &\quad (\text{if } S; F; \text{instr}^* \hookrightarrow S'; F'; \text{instr}'^*) \\
 S; F; \text{frame}_n\{F'\} \text{ instr}^* \text{ end} &\hookrightarrow S'; F'; \text{frame}_n\{F''\} \text{ instr}'^* \text{ end} \\
 &\quad (\text{if } S; F'; \text{instr}^* \hookrightarrow S'; F''; \text{instr}'^*) \\
 S; F; E[\text{trap}] &\hookrightarrow S; F; \text{trap} \quad (\text{if } E \neq __) \\
 S; F; \text{frame}_n\{F'\} \text{ trap end} &\hookrightarrow S; F; \text{trap}
 \end{aligned}$$

Reduction terminates when a thread's instruction sequence has been reduced to a **result**, that is, either a sequence of **values** or to a **trap**.

Note: The restriction on evaluation contexts rules out contexts like $__$ and $\epsilon __ \epsilon$ for which $E[\text{trap}] = \text{trap}$.

For an example of reduction under evaluation contexts, consider the following instruction sequence.

(f64.const x_1) (f64.const x_2) f64.neg (f64.const x_3) f64.add f64.mul

This can be decomposed into $E[(\text{f64.const } x_2) \text{ f64.neg}]$ where

$$E = (\text{f64.const } x_1) __ (\text{f64.const } x_3) \text{ f64.add f64.mul}$$

Moreover, this is the *only* possible choice of evaluation context where the contents of the hole matches the left-hand side of a reduction rule.

4.3 Numerics

Numeric primitives are defined in a generic manner, by operators indexed over a bit width N .

Some operators are *non-deterministic*, because they can return one of several possible results (such as different NaN values). Technically, each operator thus returns a *set* of allowed values. For convenience, deterministic results are expressed as plain values, which are assumed to be identified with a respective singleton set.

Some operators are *partial*, because they are not defined on certain inputs. Technically, an empty set of results is returned for these inputs.

In formal notation, each operator is defined by equational clauses that apply in decreasing order of precedence. That is, the first clause that is applicable to the given arguments defines the result. In some cases, similar clauses are combined into one by using the notation $\pm$ or $\mp$. When several of these placeholders occur in a single clause, then they must be resolved consistently: either the upper sign is chosen for all of them or the lower sign.

Note: For example, the `fcopysignN` operator is defined as follows:

$$\begin{aligned}
 \text{fcopysign}_N(\pm p_1, \pm p_2) &= \pm p_1 \\
 \text{fcopysign}_N(\pm p_1, \mp p_2) &= \mp p_1
 \end{aligned}$$

This definition is to be read as a shorthand for the following expansion of each clause into two separate ones:

$$\begin{aligned}
 \text{fcopysign}_N(+p_1, +p_2) &= +p_1 \\
 \text{fcopysign}_N(-p_1, -p_2) &= -p_1 \\
 \text{fcopysign}_N(+p_1, -p_2) &= -p_1 \\
 \text{fcopysign}_N(-p_1, +p_2) &= +p_1
 \end{aligned}$$

Numeric operators are lifted to input sequences by applying the operator element-wise, returning a sequence of results. When there are multiple inputs, they must be of equal length.

$$op(c_1^n, \dots, c_k^n) = op(c_1^n[0], \dots, c_k^n[0]) \dots op(c_1^n[n-1], \dots, c_k^n[n-1])$$

Note: For example, the unary operator `fabs`, when given a sequence of floating-point values, return a sequence of floating-point results:

$$\text{fabs}_N(z^n) = \text{fabs}_N(z[0]) \dots \text{fabs}_N(z[n])$$

The binary operator `iadd`, when given two sequences of integers of the same length, n , return a sequence of integer results:

$$\text{iadd}_N(i_1^n, i_2^n) = \text{iadd}_N(i_1[0], i_2[0]) \dots \text{iadd}_N(i_1[n], i_2[n])$$

Conventions:

- The meta variable d is used to range over single bits.
- The meta variable p is used to range over (signless) **magnitudes** of floating-point values, including `nan` and ∞ .
- The meta variable q is used to range over (signless) **rational magnitudes**, excluding `nan` or ∞ .
- The notation f^{-1} denotes the inverse of a bijective function f .
- Truncation of rational values is written `trunc`($\pm q$), with the usual mathematical definition:

$$\text{trunc}(\pm q) = \pm i \quad (\text{if } i \in \mathbb{N} \wedge +q - 1 < i \leq +q)$$

- Saturation of integers is written `sat_uN`(i) and `sat_sN`(i). The arguments to these two functions range over arbitrary signed integers.

- Unsigned saturation, `sat_uN`(i) clamps i to between 0 and $2^N - 1$:

$$\begin{aligned} \text{sat_u}_N(i) &= 2^N - 1 & (\text{if } i > 2^N - 1) \\ \text{sat_u}_N(i) &= 0 & (\text{if } i < 0) \\ \text{sat_u}_N(i) &= i & (\text{otherwise}) \end{aligned}$$

- Signed saturation, `sat_sN`(i) clamps i to between -2^{N-1} and $2^{N-1} - 1$:

$$\begin{aligned} \text{sat_s}_N(i) &= \text{signed}_N^{-1}(-2^{N-1}) & (\text{if } i < -2^{N-1}) \\ \text{sat_s}_N(i) &= \text{signed}_N^{-1}(2^{N-1} - 1) & (\text{if } i > 2^{N-1} - 1) \\ \text{sat_s}_N(i) &= i & (\text{otherwise}) \end{aligned}$$

4.3.1 Representations

Numbers and numeric vectors have an underlying binary representation as a sequence of bits:

$$\begin{aligned} \text{bits}_{iN}(i) &= \text{ibits}_N(i) \\ \text{bits}_{fN}(z) &= \text{fbits}_N(z) \\ \text{bits}_{vN}(i) &= \text{ibits}_N(i) \end{aligned}$$

Each of these functions is a bijection, hence they are invertible.

Integers

Integers are represented as base two unsigned numbers:

$$\text{ibits}_N(i) = d_{N-1} \dots d_0 \quad (i = 2^{N-1} \cdot d_{N-1} + \dots + 2^0 \cdot d_0)$$

Boolean operators like $\wedge$, $\vee$, or $\underline{\vee}$ are lifted to bit sequences of equal length by applying them pointwise.

Floating-Point

Floating-point values are represented in the respective binary format defined by [IEEE 754](#)²³ (Section 3.4):

$$\begin{aligned} \text{fbits}_N(\pm(1 + m \cdot 2^{-M}) \cdot 2^e) &= \text{fsign}(\pm) \text{ibits}_E(e + \text{fbias}_N) \text{ibits}_M(m) \\ \text{fbits}_N(\pm(0 + m \cdot 2^{-M}) \cdot 2^e) &= \text{fsign}(\pm) (0)^E \text{ibits}_M(m) \\ \text{fbits}_N(\pm\infty) &= \text{fsign}(\pm) (1)^E (0)^M \\ \text{fbits}_N(\pm\text{nan}(n)) &= \text{fsign}(\pm) (1)^E \text{ibits}_M(n) \\ \text{fbias}_N &= 2^{E-1} - 1 \\ \text{fsign}(+) &= 0 \\ \text{fsign}(-) &= 1 \end{aligned}$$

where $M = \text{signif}(N)$ and $E = \text{expon}(N)$.

Vectors

Numeric vectors have the same underlying representation as an *i128*. They can also be interpreted as a sequence of numeric values packed into a *v128* with a particular *shape*.

$$\begin{aligned} \text{lanes}_{t \times N}(c) &= c_0 \dots c_{N-1} \\ (\text{where } B &= |t|/8 \\ \wedge b^{16} &= \text{bytes}_{i128}(c) \\ \wedge c_i &= \text{bytes}_t^{-1}(b^{16}[i \cdot B : B])) \end{aligned}$$

These functions are bijections, so they are invertible.

Storage

When a number is stored into *memory*, it is converted into a sequence of *bytes* in *little endian*²⁴ byte order:

$$\begin{aligned} \text{bytes}_t(i) &= \text{littleendian}(\text{bits}_t(i)) \\ \text{littleendian}(\epsilon) &= \epsilon \\ \text{littleendian}(d^8 d'^*) &= \text{littleendian}(d'^*) \text{ibits}_8^{-1}(d^8) \end{aligned}$$

Again these functions are invertible bijections.

4.3.2 Integer Operations

Sign Interpretation

Integer operators are defined on *iN* values. Operators that use a signed interpretation convert the value using the following definition, which takes the two's complement when the value lies in the upper half of the value range (i.e., its most significant bit is 1):

$$\begin{aligned} \text{signed}_N(i) &= i & (0 \leq i < 2^{N-1}) \\ \text{signed}_N(i) &= i - 2^N & (2^{N-1} \leq i < 2^N) \end{aligned}$$

This function is bijective, and hence invertible.

²³ <https://ieeexplore.ieee.org/document/8766229>

²⁴ <https://en.wikipedia.org/wiki/Endianness#Little-endian>

Boolean Interpretation

The integer result of predicates – i.e., `tests` and `relational` operators – is defined with the help of the following auxiliary function producing the value 1 or 0 depending on a condition.

$$\begin{aligned}\text{bool}(C) &= 1 && (\text{if } C) \\ \text{bool}(C) &= 0 && (\text{otherwise})\end{aligned}$$

$\text{iadd}_N(i_1, i_2)$

- Return the result of adding i_1 and i_2 modulo 2^N .

$$\text{iadd}_N(i_1, i_2) = (i_1 + i_2) \bmod 2^N$$

$\text{isub}_N(i_1, i_2)$

- Return the result of subtracting i_2 from i_1 modulo 2^N .

$$\text{isub}_N(i_1, i_2) = (i_1 - i_2 + 2^N) \bmod 2^N$$

$\text{imul}_N(i_1, i_2)$

- Return the result of multiplying i_1 and i_2 modulo 2^N .

$$\text{imul}_N(i_1, i_2) = (i_1 \cdot i_2) \bmod 2^N$$

$\text{idiv}_u(i_1, i_2)$

- If i_2 is 0, then the result is undefined.
- Else, return the result of dividing i_1 by i_2 , truncated toward zero.

$$\begin{aligned}\text{idiv}_u(i_1, 0) &= \{\} \\ \text{idiv}_u(i_1, i_2) &= \text{trunc}(i_1/i_2)\end{aligned}$$

Note: This operator is `partial`.

$\text{idiv}_s(i_1, i_2)$

- Let j_1 be the `signed interpretation` of i_1 .
- Let j_2 be the `signed interpretation` of i_2 .
- If j_2 is 0, then the result is undefined.
- Else if j_1 divided by j_2 is 2^{N-1} , then the result is undefined.
- Else, return the result of dividing j_1 by j_2 , truncated toward zero.

$$\begin{aligned}\text{idiv}_s(i_1, 0) &= \{\} \\ \text{idiv}_s(i_1, i_2) &= \{\} && (\text{if } \text{signed}_N(i_1)/\text{signed}_N(i_2) = 2^{N-1}) \\ \text{idiv}_s(i_1, i_2) &= \text{signed}_N^{-1}(\text{trunc}(\text{signed}_N(i_1)/\text{signed}_N(i_2)))\end{aligned}$$

Note: This operator is `partial`. Besides division by 0, the result of $(-2^{N-1})/(-1) = +2^{N-1}$ is not representable as an N -bit signed integer.

$\text{irem_u}_N(i_1, i_2)$

- If i_2 is 0, then the result is undefined.
- Else, return the remainder of dividing i_1 by i_2 .

$$\begin{aligned}\text{irem_u}_N(i_1, 0) &= \{\} \\ \text{irem_u}_N(i_1, i_2) &= i_1 - i_2 \cdot \text{trunc}(i_1/i_2)\end{aligned}$$

Note: This operator is [partial](#).As long as both operators are defined, it holds that $i_1 = i_2 \cdot \text{idiv_u}(i_1, i_2) + \text{irem_u}(i_1, i_2)$.

 $\text{irem_s}_N(i_1, i_2)$

- Let j_1 be the [signed interpretation](#) of i_1 .
- Let j_2 be the [signed interpretation](#) of i_2 .
- If i_2 is 0, then the result is undefined.
- Else, return the remainder of dividing j_1 by j_2 , with the sign of the dividend j_1 .

$$\begin{aligned}\text{irem_s}_N(i_1, 0) &= \{\} \\ \text{irem_s}_N(i_1, i_2) &= \text{signed}_N^{-1}(j_1 - j_2 \cdot \text{trunc}(j_1/j_2)) \\ &\quad (\text{where } j_1 = \text{signed}_N(i_1) \wedge j_2 = \text{signed}_N(i_2))\end{aligned}$$

Note: This operator is [partial](#).As long as both operators are defined, it holds that $i_1 = i_2 \cdot \text{idiv_s}(i_1, i_2) + \text{irem_s}(i_1, i_2)$.

 $\text{inot}_N(i)$

- Return the bitwise negation of i .

$$\text{inot}_N(i) = \text{ibits}_N^{-1}(\text{ibits}_N(i) \vee \text{ibits}_N(2^N - 1))$$

 $\text{iand}_N(i_1, i_2)$

- Return the bitwise conjunction of i_1 and i_2 .

$$\text{iand}_N(i_1, i_2) = \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \wedge \text{ibits}_N(i_2))$$

 $\text{iandnot}_N(i_1, i_2)$

- Return the bitwise conjunction of i_1 and the bitwise negation of i_2 .

$$\text{iandnot}_N(i_1, i_2) = \text{iand}_N(i_1, \text{inot}_N(i_2))$$

$\text{ior}_N(i_1, i_2)$

- Return the bitwise disjunction of i_1 and i_2 .

$$\text{ior}_N(i_1, i_2) = \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \vee \text{ibits}_N(i_2))$$

$\text{ixor}_N(i_1, i_2)$

- Return the bitwise exclusive disjunction of i_1 and i_2 .

$$\text{ixor}_N(i_1, i_2) = \text{ibits}_N^{-1}(\text{ibits}_N(i_1) \vee \text{ibits}_N(i_2))$$

$\text{ishl}_N(i_1, i_2)$

- Let k be i_2 modulo N .
- Return the result of shifting i_1 left by k bits, modulo 2^N .

$$\text{ishl}_N(i_1, i_2) = \text{ibits}_N^{-1}(d_2^{N-k} 0^k) \quad (\text{if } \text{ibits}_N(i_1) = d_1^k d_2^{N-k} \wedge k = i_2 \bmod N)$$

$\text{ishr_u}_N(i_1, i_2)$

- Let k be i_2 modulo N .
- Return the result of shifting i_1 right by k bits, extended with 0 bits.

$$\text{ishr_u}_N(i_1, i_2) = \text{ibits}_N^{-1}(0^k d_1^{N-k}) \quad (\text{if } \text{ibits}_N(i_1) = d_1^{N-k} d_2^k \wedge k = i_2 \bmod N)$$

$\text{ishr_s}_N(i_1, i_2)$

- Let k be i_2 modulo N .
- Return the result of shifting i_1 right by k bits, extended with the most significant bit of the original value.

$$\text{ishr_s}_N(i_1, i_2) = \text{ibits}_N^{-1}(d_0^{k+1} d_1^{N-k-1}) \quad (\text{if } \text{ibits}_N(i_1) = d_0 d_1^{N-k-1} d_2^k \wedge k = i_2 \bmod N)$$

$\text{irotl}_N(i_1, i_2)$

- Let k be i_2 modulo N .
- Return the result of rotating i_1 left by k bits.

$$\text{irotl}_N(i_1, i_2) = \text{ibits}_N^{-1}(d_2^{N-k} d_1^k) \quad (\text{if } \text{ibits}_N(i_1) = d_1^k d_2^{N-k} \wedge k = i_2 \bmod N)$$

$\text{irotr}_N(i_1, i_2)$

- Let k be i_2 modulo N .
- Return the result of rotating i_1 right by k bits.

$$\text{irotr}_N(i_1, i_2) = \text{ibits}_N^{-1}(d_2^k d_1^{N-k}) \quad (\text{if } \text{ibits}_N(i_1) = d_1^{N-k} d_2^k \wedge k = i_2 \bmod N)$$

$\text{iclz}_N(i)$

- Return the count of leading zero bits in i ; all bits are considered leading zeros if i is 0.

$$\text{iclz}_N(i) = k \quad (\text{if } \text{ibits}_N(i) = 0^k (1 \ d^*)^?)$$

 $\text{ictz}_N(i)$

- Return the count of trailing zero bits in i ; all bits are considered trailing zeros if i is 0.

$$\text{ictz}_N(i) = k \quad (\text{if } \text{ibits}_N(i) = (d^* 1)^? 0^k)$$

 $\text{ipopcnt}_N(i)$

- Return the count of non-zero bits in i .

$$\text{ipopcnt}_N(i) = k \quad (\text{if } \text{ibits}_N(i) = (0^* 1)^k 0^*)$$

 $\text{ieqz}_N(i)$

- Return 1 if i is zero, 0 otherwise.

$$\text{ieqz}_N(i) = \text{bool}(i = 0)$$

 $\text{ieq}_N(i_1, i_2)$

- Return 1 if i_1 equals i_2 , 0 otherwise.

$$\text{ieq}_N(i_1, i_2) = \text{bool}(i_1 = i_2)$$

 $\text{ine}_N(i_1, i_2)$

- Return 1 if i_1 does not equal i_2 , 0 otherwise.

$$\text{ine}_N(i_1, i_2) = \text{bool}(i_1 \neq i_2)$$

 $\text{ilt_u}_N(i_1, i_2)$

- Return 1 if i_1 is less than i_2 , 0 otherwise.

$$\text{ilt_u}_N(i_1, i_2) = \text{bool}(i_1 < i_2)$$

 $\text{ilt_s}_N(i_1, i_2)$

- Let j_1 be the [signed interpretation](#) of i_1 .
- Let j_2 be the [signed interpretation](#) of i_2 .
- Return 1 if j_1 is less than j_2 , 0 otherwise.

$$\text{ilt_s}_N(i_1, i_2) = \text{bool}(\text{signed}_N(i_1) < \text{signed}_N(i_2))$$

$\text{igt_u}_N(i_1, i_2)$

- Return 1 if i_1 is greater than i_2 , 0 otherwise.

$$\text{igt_u}_N(i_1, i_2) = \text{bool}(i_1 > i_2)$$

 $\text{igt_s}_N(i_1, i_2)$

- Let j_1 be the [signed interpretation](#) of i_1 .
- Let j_2 be the [signed interpretation](#) of i_2 .
- Return 1 if j_1 is greater than j_2 , 0 otherwise.

$$\text{igt_s}_N(i_1, i_2) = \text{bool}(\text{signed}_N(i_1) > \text{signed}_N(i_2))$$

 $\text{ile_u}_N(i_1, i_2)$

- Return 1 if i_1 is less than or equal to i_2 , 0 otherwise.

$$\text{ile_u}_N(i_1, i_2) = \text{bool}(i_1 \leq i_2)$$

 $\text{ile_s}_N(i_1, i_2)$

- Let j_1 be the [signed interpretation](#) of i_1 .
- Let j_2 be the [signed interpretation](#) of i_2 .
- Return 1 if j_1 is less than or equal to j_2 , 0 otherwise.

$$\text{ile_s}_N(i_1, i_2) = \text{bool}(\text{signed}_N(i_1) \leq \text{signed}_N(i_2))$$

 $\text{ige_u}_N(i_1, i_2)$

- Return 1 if i_1 is greater than or equal to i_2 , 0 otherwise.

$$\text{ige_u}_N(i_1, i_2) = \text{bool}(i_1 \geq i_2)$$

 $\text{ige_s}_N(i_1, i_2)$

- Let j_1 be the [signed interpretation](#) of i_1 .
- Let j_2 be the [signed interpretation](#) of i_2 .
- Return 1 if j_1 is greater than or equal to j_2 , 0 otherwise.

$$\text{ige_s}_N(i_1, i_2) = \text{bool}(\text{signed}_N(i_1) \geq \text{signed}_N(i_2))$$

$\text{iextend}_{M,N}(i)$

- Return $\text{extend}_{M,N}^s(i)$.

$$\text{iextend}_{M,N}(i) = \text{extend}_{M,N}^s(i)$$

$\text{ibitselect}_N(i_1, i_2, i_3)$

- Let j_1 be the bitwise conjunction of i_1 and i_3 .
- Let j'_3 be the bitwise negation of i_3 .
- Let j_2 be the bitwise conjunction of i_2 and j'_3 .
- Return the bitwise disjunction of j_1 and j_2 .

$$\text{ibitselect}_N(i_1, i_2, i_3) = \text{ior}_N(\text{iand}_N(i_1, i_3), \text{iand}_N(i_2, \text{inot}_N(i_3)))$$

$\text{iabs}_N(i)$

- Let j be the [signed interpretation](#) of i .
- If j is greater than or equal to 0, then return i .
- Else return the negation of j , modulo 2^N .

$$\begin{aligned} \text{iabs}_N(i) &= i && (\text{if } \text{signed}_N(i) \geq 0) \\ \text{iabs}_N(i) &= -\text{signed}_N(i) \bmod 2^N && (\text{otherwise}) \end{aligned}$$

$\text{ineg}_N(i)$

- Return the result of negating i , modulo 2^N .

$$\text{ineg}_N(i) = (2^N - i) \bmod 2^N$$

$\text{imin}_{uN}(i_1, i_2)$

- Return i_1 if $\text{ilt}_{uN}(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{aligned} \text{imin}_{uN}(i_1, i_2) &= i_1 && (\text{if } \text{ilt}_{uN}(i_1, i_2) = 1) \\ \text{imin}_{uN}(i_1, i_2) &= i_2 && (\text{otherwise}) \end{aligned}$$

$\text{imin}_{sN}(i_1, i_2)$

- Return i_1 if $\text{ilt}_{sN}(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{aligned} \text{imin}_{sN}(i_1, i_2) &= i_1 && (\text{if } \text{ilt}_{sN}(i_1, i_2) = 1) \\ \text{imin}_{sN}(i_1, i_2) &= i_2 && (\text{otherwise}) \end{aligned}$$

$\text{imax_u}_N(i_1, i_2)$

- Return i_1 if $\text{igt_u}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{aligned}\text{imax_u}_N(i_1, i_2) &= i_1 && (\text{if } \text{igt_u}_N(i_1, i_2) = 1) \\ \text{imax_u}_N(i_1, i_2) &= i_2 && (\text{otherwise})\end{aligned}$$

$\text{imax_s}_N(i_1, i_2)$

- Return i_1 if $\text{igt_s}_N(i_1, i_2)$ is 1, return i_2 otherwise.

$$\begin{aligned}\text{imax_s}_N(i_1, i_2) &= i_1 && (\text{if } \text{igt_s}_N(i_1, i_2) = 1) \\ \text{imax_s}_N(i_1, i_2) &= i_2 && (\text{otherwise})\end{aligned}$$

$\text{iaddsat_u}_N(i_1, i_2)$

- Let i be the result of adding i_1 and i_2 .
- Return $\text{sat_u}_N(i)$.

$$\text{iaddsat_u}_N(i_1, i_2) = \text{sat_u}_N(i_1 + i_2)$$

$\text{iaddsat_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1
- Let j_2 be the signed interpretation of i_2
- Let j be the result of adding j_1 and j_2 .
- Return $\text{sat_s}_N(j)$.

$$\text{iaddsat_s}_N(i_1, i_2) = \text{sat_s}_N(\text{signed}_N(i_1) + \text{signed}_N(i_2))$$

$\text{isubsat_u}_N(i_1, i_2)$

- Let i be the result of subtracting i_2 from i_1 .
- Return $\text{sat_u}_N(i)$.

$$\text{isubsat_u}_N(i_1, i_2) = \text{sat_u}_N(i_1 - i_2)$$

$\text{isubsat_s}_N(i_1, i_2)$

- Let j_1 be the signed interpretation of i_1
- Let j_2 be the signed interpretation of i_2
- Let j be the result of subtracting j_2 from j_1 .
- Return $\text{sat_s}_N(j)$.

$$\text{isubsat_s}_N(i_1, i_2) = \text{sat_s}_N(\text{signed}_N(i_1) - \text{signed}_N(i_2))$$

$\text{iavgr_u}_N(i_1, i_2)$

- Let j be the result of adding i_1 , i_2 , and 1.
- Return the result of dividing j by 2, truncated toward zero.

$$\text{iavgr_u}_N(i_1, i_2) = \text{trunc}((i_1 + i_2 + 1)/2)$$

$\text{iq15mulrsat_s}_N(i_1, i_2)$

- Return the result of $\text{sat_s}_N(\text{ishr_s}_N(i_1 \cdot i_2 + 2^{14}, 15))$.

$$\text{iq15mulrsat_s}_N(i_1, i_2) = \text{sat_s}_N(\text{ishr_s}_N(i_1 \cdot i_2 + 2^{14}, 15))$$

4.3.3 Floating-Point Operations

Floating-point arithmetic follows the [IEEE 754²⁵](#) standard, with the following qualifications:

- All operators use round-to-nearest ties-to-even, except where otherwise specified. Non-default directed rounding attributes are not supported.
- Following the recommendation that operators propagate NaN payloads from their operands is permitted but not required.
- All operators use “non-stop” mode, and floating-point exceptions are not otherwise observable. In particular, neither alternate floating-point exception handling attributes nor operators on status flags are supported. There is no observable difference between quiet and signalling NaNs.

Note: Some of these limitations may be lifted in future versions of WebAssembly.

Rounding

Rounding always is round-to-nearest ties-to-even, in correspondence with [IEEE 754²⁶](#) (Section 4.3.1).

An *exact* floating-point number is a rational number that is exactly representable as a floating-point number of given bit width N .

A *limit* number for a given floating-point bit width N is a positive or negative number whose magnitude is the smallest power of 2 that is not exactly representable as a floating-point number of width N (that magnitude is 2^{128} for $N = 32$ and 2^{1024} for $N = 64$).

A *candidate* number is either an exact floating-point number or a positive or negative limit number for the given bit width N .

A *candidate pair* is a pair z_1, z_2 of candidate numbers, such that no candidate number exists that lies between the two.

A real number r is converted to a floating-point value of bit width N as follows:

- If r is 0, then return $+0$.
- Else if r is an exact floating-point number, then return r .
- Else if r greater than or equal to the positive limit, then return $+\infty$.
- Else if r is less than or equal to the negative limit, then return $-\infty$.
- Else if z_1 and z_2 are a candidate pair such that $z_1 < r < z_2$, then:

²⁵ <https://ieeexplore.ieee.org/document/8766229>

²⁶ <https://ieeexplore.ieee.org/document/8766229>

- If $|r - z_1| < |r - z_2|$, then let z be z_1 .
- Else if $|r - z_1| > |r - z_2|$, then let z be z_2 .
- Else if $|r - z_1| = |r - z_2|$ and the **significand** of z_1 is even, then let z be z_1 .
- Else, let z be z_2 .
- If z is 0, then:
 - If $r < 0$, then return -0 .
 - Else, return $+0$.
- Else if z is a limit number, then:
 - If $r < 0$, then return $-\infty$.
 - Else, return $+\infty$.

- Else, return z .

$\text{float}_N(0)$	$=$	$+0$	
$\text{float}_N(r)$	$=$	r	(if $r \in \text{exact}_N$)
$\text{float}_N(r)$	$=$	$+\infty$	(if $r \geq +\text{limit}_N$)
$\text{float}_N(r)$	$=$	$-\infty$	(if $r \leq -\text{limit}_N$)
$\text{float}_N(r)$	$=$	$\text{closest}_N(r, z_1, z_2)$	(if $z_1 < r < z_2 \wedge (z_1, z_2) \in \text{candidatepair}_N$)
$\text{closest}_N(r, z_1, z_2)$	$=$	$\text{rectify}_N(r, z_1)$	(if $ r - z_1 < r - z_2 $)
$\text{closest}_N(r, z_1, z_2)$	$=$	$\text{rectify}_N(r, z_2)$	(if $ r - z_1 > r - z_2 $)
$\text{closest}_N(r, z_1, z_2)$	$=$	$\text{rectify}_N(r, z_1)$	(if $ r - z_1 = r - z_2 \wedge \text{even}_N(z_1)$)
$\text{closest}_N(r, z_1, z_2)$	$=$	$\text{rectify}_N(r, z_2)$	(if $ r - z_1 = r - z_2 \wedge \text{even}_N(z_2)$)
$\text{rectify}_N(r, \pm\text{limit}_N)$	$=$	$\pm\infty$	
$\text{rectify}_N(r, 0)$	$=$	$+0$	($r \geq 0$)
$\text{rectify}_N(r, 0)$	$=$	-0	($r < 0$)
$\text{rectify}_N(r, z)$	$=$	z	

where:

exact_N	$=$	$fN \cap \mathbb{Q}$
limit_N	$=$	$2^{\text{expon}(N)-1}$
candidate_N	$=$	$\text{exact}_N \cup \{+\text{limit}_N, -\text{limit}_N\}$
candidatepair_N	$=$	$\{(z_1, z_2) \in \text{candidate}_N^2 \mid z_1 < z_2 \wedge \forall z \in \text{candidate}_N, z \leq z_1 \vee z \geq z_2\}$
$\text{even}_N((d + m \cdot 2^{-M}) \cdot 2^e)$	$\Leftrightarrow$	$m \bmod 2 = 0$
$\text{even}_N(\pm\text{limit}_N)$	$\Leftrightarrow$	true

NaN Propagation

When the result of a floating-point operator other than **fneg**, **fabs**, or **fcopysign** is a NaN, then its sign is non-deterministic and the **payload** is computed as follows:

- If the payload of all NaN inputs to the operator is **canonical** (including the case that there are no NaN inputs), then the payload of the output is canonical as well.
- Otherwise the payload is picked non-deterministically among all **arithmetic NaNs**; that is, its most significant bit is 1 and all others are unspecified.

This non-deterministic result is expressed by the following auxiliary function producing a set of allowed outputs from a set of inputs:

$$\begin{aligned} \text{nans}_N\{z^*\} &= \{+\text{nan}(n), -\text{nan}(n) \mid n = \text{canon}_N\} && (\text{if } \forall \text{nan}(n) \in z^*, n = \text{canon}_N) \\ \text{nans}_N\{z^*\} &= \{+\text{nan}(n), -\text{nan}(n) \mid n \geq \text{canon}_N\} && (\text{otherwise}) \end{aligned}$$

$\text{fadd}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities of opposite signs, then return an element of $\text{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of equal sign, then return that infinity.
- Else if either z_1 or z_2 is an infinity, then return that infinity.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of equal sign, then return that zero.
- Else if either z_1 or z_2 is a zero, then return the other operand.
- Else if both z_1 and z_2 are values with the same magnitude but opposite signs, then return positive zero.
- Else return the result of adding z_1 and z_2 , **rounded** to the nearest representable value.

$$\begin{aligned}
 \text{fadd}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\
 \text{fadd}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n), z_1\} \\
 \text{fadd}_N(\pm\infty, \mp\infty) &= \text{nans}_N\{\} \\
 \text{fadd}_N(\pm\infty, \pm\infty) &= \pm\infty \\
 \text{fadd}_N(z_1, \pm\infty) &= \pm\infty \\
 \text{fadd}_N(\pm\infty, z_2) &= \pm\infty \\
 \text{fadd}_N(\pm 0, \mp 0) &= +0 \\
 \text{fadd}_N(\pm 0, \pm 0) &= \pm 0 \\
 \text{fadd}_N(z_1, \pm 0) &= z_1 \\
 \text{fadd}_N(\pm 0, z_2) &= z_2 \\
 \text{fadd}_N(\pm q, \mp q) &= +0 \\
 \text{fadd}_N(z_1, z_2) &= \text{float}_N(z_1 + z_2)
 \end{aligned}$$

$\text{fsub}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities of equal signs, then return an element of $\text{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of opposite sign, then return z_1 .
- Else if z_1 is an infinity, then return that infinity.
- Else if z_2 is an infinity, then return that infinity negated.
- Else if both z_1 and z_2 are zeroes of equal sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return z_1 .
- Else if z_2 is a zero, then return z_1 .
- Else if z_1 is a zero, then return z_2 negated.
- Else if both z_1 and z_2 are the same value, then return positive zero.
- Else return the result of subtracting z_2 from z_1 , **rounded** to the nearest representable value.

$$\begin{array}{ll}
\text{fsub}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\
\text{fsub}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n), z_1\} \\
\text{fsub}_N(\pm\infty, \pm\infty) &= \text{nans}_N\{\} \\
\text{fsub}_N(\pm\infty, \mp\infty) &= \pm\infty \\
\text{fsub}_N(z_1, \pm\infty) &= \mp\infty \\
\text{fsub}_N(\pm\infty, z_2) &= \pm\infty \\
\text{fsub}_N(\pm 0, \pm 0) &= +0 \\
\text{fsub}_N(\pm 0, \mp 0) &= \pm 0 \\
\text{fsub}_N(z_1, \pm 0) &= z_1 \\
\text{fsub}_N(\pm 0, \pm q_2) &= \mp q_2 \\
\text{fsub}_N(\pm q, \pm q) &= +0 \\
\text{fsub}_N(z_1, z_2) &= \text{float}_N(z_1 - z_2)
\end{array}$$

Note: Up to the non-determinism regarding NaNs, it always holds that $\text{fsub}_N(z_1, z_2) = \text{fadd}_N(z_1, \text{fneg}_N(z_2))$.

$\text{fmul}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if one of z_1 and z_2 is a zero and the other an infinity, then return an element of $\text{nans}_N\{\}$.
- Else if both z_1 and z_2 are infinities of equal sign, then return positive infinity.
- Else if both z_1 and z_2 are infinities of opposite sign, then return negative infinity.
- Else if either z_1 or z_2 is an infinity and the other a value with equal sign, then return positive infinity.
- Else if either z_1 or z_2 is an infinity and the other a value with opposite sign, then return negative infinity.
- Else if both z_1 and z_2 are zeroes of equal sign, then return positive zero.
- Else if both z_1 and z_2 are zeroes of opposite sign, then return negative zero.
- Else return the result of multiplying z_1 and z_2 , rounded to the nearest representable value.

$$\begin{array}{ll}
\text{fmul}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\
\text{fmul}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n), z_1\} \\
\text{fmul}_N(\pm\infty, \pm 0) &= \text{nans}_N\{\} \\
\text{fmul}_N(\pm\infty, \mp 0) &= \text{nans}_N\{\} \\
\text{fmul}_N(\pm 0, \pm\infty) &= \text{nans}_N\{\} \\
\text{fmul}_N(\pm 0, \mp\infty) &= \text{nans}_N\{\} \\
\text{fmul}_N(\pm\infty, \pm\infty) &= +\infty \\
\text{fmul}_N(\pm\infty, \mp\infty) &= -\infty \\
\text{fmul}_N(\pm q_1, \pm\infty) &= +\infty \\
\text{fmul}_N(\pm q_1, \mp\infty) &= -\infty \\
\text{fmul}_N(\pm\infty, \pm q_2) &= +\infty \\
\text{fmul}_N(\pm\infty, \mp q_2) &= -\infty \\
\text{fmul}_N(\pm 0, \pm 0) &= +0 \\
\text{fmul}_N(\pm 0, \mp 0) &= -0 \\
\text{fmul}_N(z_1, z_2) &= \text{float}_N(z_1 \cdot z_2)
\end{array}$$

$\text{fdiv}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if both z_1 and z_2 are infinities, then return an element of $\text{nans}_N\{\}$.
- Else if both z_1 and z_2 are zeroes, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if z_1 is an infinity and z_2 a value with equal sign, then return positive infinity.
- Else if z_1 is an infinity and z_2 a value with opposite sign, then return negative infinity.
- Else if z_2 is an infinity and z_1 a value with equal sign, then return positive zero.
- Else if z_2 is an infinity and z_1 a value with opposite sign, then return negative zero.
- Else if z_1 is a zero and z_2 a value with equal sign, then return positive zero.
- Else if z_1 is a zero and z_2 a value with opposite sign, then return negative zero.
- Else if z_2 is a zero and z_1 a value with equal sign, then return positive infinity.
- Else if z_2 is a zero and z_1 a value with opposite sign, then return negative infinity.
- Else return the result of dividing z_1 by z_2 , **rounded** to the nearest representable value.

$$\begin{aligned} \text{fdiv}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\ \text{fdiv}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n), z_1\} \\ \text{fdiv}_N(\pm\infty, \pm\infty) &= \text{nans}_N\{\} \\ \text{fdiv}_N(\pm\infty, \mp\infty) &= \text{nans}_N\{\} \\ \text{fdiv}_N(\pm 0, \pm 0) &= \text{nans}_N\{\} \\ \text{fdiv}_N(\pm 0, \mp 0) &= \text{nans}_N\{\} \\ \text{fdiv}_N(\pm\infty, \pm q_2) &= +\infty \\ \text{fdiv}_N(\pm\infty, \mp q_2) &= -\infty \\ \text{fdiv}_N(\pm q_1, \pm\infty) &= +0 \\ \text{fdiv}_N(\pm q_1, \mp\infty) &= -0 \\ \text{fdiv}_N(\pm 0, \pm q_2) &= +0 \\ \text{fdiv}_N(\pm 0, \mp q_2) &= -0 \\ \text{fdiv}_N(\pm q_1, \pm 0) &= +\infty \\ \text{fdiv}_N(\pm q_1, \mp 0) &= -\infty \\ \text{fdiv}_N(z_1, z_2) &= \text{float}_N(z_1/z_2) \end{aligned}$$
 $\text{fmin}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if either z_1 or z_2 is a negative infinity, then return negative infinity.
- Else if either z_1 or z_2 is a positive infinity, then return the other value.
- Else if both z_1 and z_2 are zeroes of opposite signs, then return negative zero.
- Else return the smaller value of z_1 and z_2 .

$$\begin{aligned} \text{fmin}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\ \text{fmin}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n), z_1\} \\ \text{fmin}_N(+\infty, z_2) &= z_2 \\ \text{fmin}_N(-\infty, z_2) &= -\infty \\ \text{fmin}_N(z_1, +\infty) &= z_1 \\ \text{fmin}_N(z_1, -\infty) &= -\infty \\ \text{fmin}_N(\pm 0, \mp 0) &= -0 \\ \text{fmin}_N(z_1, z_2) &= z_1 && (\text{if } z_1 \leq z_2) \\ \text{fmin}_N(z_1, z_2) &= z_2 && (\text{if } z_2 \leq z_1) \end{aligned}$$

$\text{fmax}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return an element of $\text{nans}_N\{z_1, z_2\}$.
- Else if either z_1 or z_2 is a positive infinity, then return positive infinity.
- Else if either z_1 or z_2 is a negative infinity, then return the other value.
- Else if both z_1 and z_2 are zeroes of opposite signs, then return positive zero.
- Else return the larger value of z_1 and z_2 .

$$\begin{aligned}
 \text{fmax}_N(\pm\text{nan}(n), z_2) &= \text{nans}_N\{\pm\text{nan}(n), z_2\} \\
 \text{fmax}_N(z_1, \pm\text{nan}(n)) &= \text{nans}_N\{z_1, \pm\text{nan}(n)\} \\
 \text{fmax}_N(+\infty, z_2) &= +\infty \\
 \text{fmax}_N(-\infty, z_2) &= z_2 \\
 \text{fmax}_N(z_1, +\infty) &= +\infty \\
 \text{fmax}_N(z_1, -\infty) &= z_1 \\
 \text{fmax}_N(\pm 0, \mp 0) &= +0 \\
 \text{fmax}_N(z_1, z_2) &= z_1 && (\text{if } z_1 \geq z_2) \\
 \text{fmax}_N(z_1, z_2) &= z_2 && (\text{if } z_2 \geq z_1)
 \end{aligned}$$

$\text{fcopysign}_N(z_1, z_2)$

- If z_1 and z_2 have the same sign, then return z_1 .
- Else return z_1 with negated sign.

$$\begin{aligned}
 \text{fcopysign}_N(\pm p_1, \pm p_2) &= \pm p_1 \\
 \text{fcopysign}_N(\pm p_1, \mp p_2) &= \mp p_1
 \end{aligned}$$

$\text{fabs}_N(z)$

- If z is a NaN, then return z with positive sign.
- Else if z is an infinity, then return positive infinity.
- Else if z is a zero, then return positive zero.
- Else if z is a positive value, then z .
- Else return z negated.

$$\begin{aligned}
 \text{fabs}_N(\pm\text{nan}(n)) &= +\text{nan}(n) \\
 \text{fabs}_N(\pm\infty) &= +\infty \\
 \text{fabs}_N(\pm 0) &= +0 \\
 \text{fabs}_N(\pm q) &= +q
 \end{aligned}$$

$\text{fneg}_N(z)$

- If z is a NaN, then return z with negated sign.
- Else if z is an infinity, then return that infinity negated.
- Else if z is a zero, then return that zero negated.
- Else return z negated.

$$\begin{aligned}
 \text{fneg}_N(\pm\text{nan}(n)) &= \mp\text{nan}(n) \\
 \text{fneg}_N(\pm\infty) &= \mp\infty \\
 \text{fneg}_N(\pm 0) &= \mp 0 \\
 \text{fneg}_N(\pm q) &= \mp q
 \end{aligned}$$

$\text{fsqrt}_N(z)$

- If z is a NaN, then return an element of $\text{nans}_N\{z\}$.
- Else if z is negative infinity, then return an element of $\text{nans}_N\{\}$.
- Else if z is positive infinity, then return positive infinity.
- Else if z is a zero, then return that zero.
- Else if z has a negative sign, then return an element of $\text{nans}_N\{\}$.
- Else return the square root of z .

$$\begin{aligned}
 \text{fsqrt}_N(\pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n)\} \\
 \text{fsqrt}_N(-\infty) &= \text{nans}_N\{\} \\
 \text{fsqrt}_N(+\infty) &= +\infty \\
 \text{fsqrt}_N(\pm 0) &= \pm 0 \\
 \text{fsqrt}_N(-q) &= \text{nans}_N\{\} \\
 \text{fsqrt}_N(+q) &= \text{float}_N(\sqrt{q})
 \end{aligned}$$

 $\text{fceil}_N(z)$

- If z is a NaN, then return an element of $\text{nans}_N\{z\}$.
- Else if z is an infinity, then return z .
- Else if z is a zero, then return z .
- Else if z is smaller than 0 but greater than -1 , then return negative zero.
- Else return the smallest integral value that is not smaller than z .

$$\begin{aligned}
 \text{fceil}_N(\pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n)\} \\
 \text{fceil}_N(\pm\infty) &= \pm\infty \\
 \text{fceil}_N(\pm 0) &= \pm 0 \\
 \text{fceil}_N(-q) &= -0 && (\text{if } -1 < -q < 0) \\
 \text{fceil}_N(\pm q) &= \text{float}_N(i) && (\text{if } \pm q \leq i < \pm q + 1)
 \end{aligned}$$

 $\text{ffloor}_N(z)$

- If z is a NaN, then return an element of $\text{nans}_N\{z\}$.
- Else if z is an infinity, then return z .
- Else if z is a zero, then return z .
- Else if z is greater than 0 but smaller than 1, then return positive zero.
- Else return the largest integral value that is not larger than z .

$$\begin{aligned}
 \text{ffloor}_N(\pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n)\} \\
 \text{ffloor}_N(\pm\infty) &= \pm\infty \\
 \text{ffloor}_N(\pm 0) &= \pm 0 \\
 \text{ffloor}_N(+q) &= +0 && (\text{if } 0 < +q < 1) \\
 \text{ffloor}_N(\pm q) &= \text{float}_N(i) && (\text{if } \pm q - 1 < i \leq \pm q)
 \end{aligned}$$

$\text{ftrunc}_N(z)$

- If z is a NaN, then return an element of $\text{nans}_N\{z\}$.
- Else if z is an infinity, then return z .
- Else if z is a zero, then return z .
- Else if z is greater than 0 but smaller than 1, then return positive zero.
- Else if z is smaller than 0 but greater than -1 , then return negative zero.
- Else return the integral value with the same sign as z and the largest magnitude that is not larger than the magnitude of z .

$$\begin{aligned}
 \text{ftrunc}_N(\pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n)\} \\
 \text{ftrunc}_N(\pm\infty) &= \pm\infty \\
 \text{ftrunc}_N(\pm 0) &= \pm 0 \\
 \text{ftrunc}_N(+q) &= +0 && (\text{if } 0 < +q < 1) \\
 \text{ftrunc}_N(-q) &= -0 && (\text{if } -1 < -q < 0) \\
 \text{ftrunc}_N(\pm q) &= \text{float}_N(\pm i) && (\text{if } +q - 1 < i \leq +q)
 \end{aligned}$$

$\text{fnearest}_N(z)$

- If z is a NaN, then return an element of $\text{nans}_N\{z\}$.
- Else if z is an infinity, then return z .
- Else if z is a zero, then return z .
- Else if z is greater than 0 but smaller than or equal to 0.5, then return positive zero.
- Else if z is smaller than 0 but greater than or equal to -0.5 , then return negative zero.
- Else return the integral value that is nearest to z ; if two values are equally near, return the even one.

$$\begin{aligned}
 \text{fnearest}_N(\pm\text{nan}(n)) &= \text{nans}_N\{\pm\text{nan}(n)\} \\
 \text{fnearest}_N(\pm\infty) &= \pm\infty \\
 \text{fnearest}_N(\pm 0) &= \pm 0 \\
 \text{fnearest}_N(+q) &= +0 && (\text{if } 0 < +q \leq 0.5) \\
 \text{fnearest}_N(-q) &= -0 && (\text{if } -0.5 \leq -q < 0) \\
 \text{fnearest}_N(\pm q) &= \text{float}_N(\pm i) && (\text{if } |i - q| < 0.5) \\
 \text{fnearest}_N(\pm q) &= \text{float}_N(\pm i) && (\text{if } |i - q| = 0.5 \wedge i \text{ even})
 \end{aligned}$$

$\text{feq}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if both z_1 and z_2 are the same value, then return 1.
- Else return 0.

$$\begin{aligned}
 \text{feq}_N(\pm\text{nan}(n), z_2) &= 0 \\
 \text{feq}_N(z_1, \pm\text{nan}(n)) &= 0 \\
 \text{feq}_N(\pm 0, \mp 0) &= 1 \\
 \text{feq}_N(z_1, z_2) &= \text{bool}(z_1 = z_2)
 \end{aligned}$$

$\text{fne}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if both z_1 and z_2 are the same value, then return 0.
- Else return 1.

$$\begin{aligned}
 \text{fne}_N(\pm\text{nan}(n), z_2) &= 1 \\
 \text{fne}_N(z_1, \pm\text{nan}(n)) &= 1 \\
 \text{fne}_N(\pm 0, \mp 0) &= 0 \\
 \text{fne}_N(z_1, z_2) &= \text{bool}(z_1 \neq z_2)
 \end{aligned}$$

 $\text{flt}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 0.
- Else if z_1 is positive infinity, then return 0.
- Else if z_1 is negative infinity, then return 1.
- Else if z_2 is positive infinity, then return 1.
- Else if z_2 is negative infinity, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if z_1 is smaller than z_2 , then return 1.
- Else return 0.

$$\begin{aligned}
 \text{flt}_N(\pm\text{nan}(n), z_2) &= 0 \\
 \text{flt}_N(z_1, \pm\text{nan}(n)) &= 0 \\
 \text{flt}_N(z, z) &= 0 \\
 \text{flt}_N(+\infty, z_2) &= 0 \\
 \text{flt}_N(-\infty, z_2) &= 1 \\
 \text{flt}_N(z_1, +\infty) &= 1 \\
 \text{flt}_N(z_1, -\infty) &= 0 \\
 \text{flt}_N(\pm 0, \mp 0) &= 0 \\
 \text{flt}_N(z_1, z_2) &= \text{bool}(z_1 < z_2)
 \end{aligned}$$

 $\text{fgt}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 0.
- Else if z_1 is positive infinity, then return 1.
- Else if z_1 is negative infinity, then return 0.
- Else if z_2 is positive infinity, then return 0.
- Else if z_2 is negative infinity, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 0.
- Else if z_1 is larger than z_2 , then return 1.
- Else return 0.

$\text{fgt}_N(\pm\text{nan}(n), z_2)$	$=$	0
$\text{fgt}_N(z_1, \pm\text{nan}(n))$	$=$	0
$\text{fgt}_N(z, z)$	$=$	0
$\text{fgt}_N(+\infty, z_2)$	$=$	1
$\text{fgt}_N(-\infty, z_2)$	$=$	0
$\text{fgt}_N(z_1, +\infty)$	$=$	0
$\text{fgt}_N(z_1, -\infty)$	$=$	1
$\text{fgt}_N(\pm 0, \mp 0)$	$=$	0
$\text{fgt}_N(z_1, z_2)$	$=$	$\text{bool}(z_1 > z_2)$

$\text{fle}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 1.
- Else if z_1 is positive infinity, then return 0.
- Else if z_1 is negative infinity, then return 1.
- Else if z_2 is positive infinity, then return 1.
- Else if z_2 is negative infinity, then return 0.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if z_1 is smaller than or equal to z_2 , then return 1.
- Else return 0.

$\text{fle}_N(\pm\text{nan}(n), z_2)$	$=$	0
$\text{fle}_N(z_1, \pm\text{nan}(n))$	$=$	0
$\text{fle}_N(z, z)$	$=$	1
$\text{fle}_N(+\infty, z_2)$	$=$	0
$\text{fle}_N(-\infty, z_2)$	$=$	1
$\text{fle}_N(z_1, +\infty)$	$=$	1
$\text{fle}_N(z_1, -\infty)$	$=$	0
$\text{fle}_N(\pm 0, \mp 0)$	$=$	1
$\text{fle}_N(z_1, z_2)$	$=$	$\text{bool}(z_1 \leq z_2)$

$\text{fge}_N(z_1, z_2)$

- If either z_1 or z_2 is a NaN, then return 0.
- Else if z_1 and z_2 are the same value, then return 1.
- Else if z_1 is positive infinity, then return 1.
- Else if z_1 is negative infinity, then return 0.
- Else if z_2 is positive infinity, then return 0.
- Else if z_2 is negative infinity, then return 1.
- Else if both z_1 and z_2 are zeroes, then return 1.
- Else if z_1 is smaller than or equal to z_2 , then return 1.
- Else return 0.

$$\begin{aligned}
\text{fge}_N(\pm\text{nan}(n), z_2) &= 0 \\
\text{fge}_N(z_1, \pm\text{nan}(n)) &= 0 \\
\text{fge}_N(z, z) &= 1 \\
\text{fge}_N(+\infty, z_2) &= 1 \\
\text{fge}_N(-\infty, z_2) &= 0 \\
\text{fge}_N(z_1, +\infty) &= 0 \\
\text{fge}_N(z_1, -\infty) &= 1 \\
\text{fge}_N(\pm 0, \mp 0) &= 1 \\
\text{fge}_N(z_1, z_2) &= \text{bool}(z_1 \geq z_2)
\end{aligned}$$

$\text{fpmin}_N(z_1, z_2)$

- If z_2 is less than z_1 then return z_2 .
- Else return z_1 .

$$\begin{aligned}
\text{fpmin}_N(z_1, z_2) &= z_2 \quad (\text{if } \text{flt}_N(z_2, z_1) = 1) \\
\text{fpmin}_N(z_1, z_2) &= z_1 \quad (\text{otherwise})
\end{aligned}$$

$\text{fpmax}_N(z_1, z_2)$

- If z_1 is less than z_2 then return z_2 .
- Else return z_1 .

$$\begin{aligned}
\text{fpmax}_N(z_1, z_2) &= z_2 \quad (\text{if } \text{flt}_N(z_1, z_2) = 1) \\
\text{fpmax}_N(z_1, z_2) &= z_1 \quad (\text{otherwise})
\end{aligned}$$

4.3.4 Conversions

$\text{extend}^u_{M,N}(i)$

- Return i .

$$\text{extend}^u_{M,N}(i) = i$$

Note: In the abstract syntax, unsigned extension just reinterprets the same value.

$\text{extend}^s_{M,N}(i)$

- Let j be the signed interpretation of i of size M .
- Return the two's complement of j relative to size N .

$$\text{extend}^s_{M,N}(i) = \text{signed}_N^{-1}(\text{signed}_M(i))$$

$\text{wrap}_{M,N}(i)$

- Return i modulo 2^N .

$$\text{wrap}_{M,N}(i) = i \bmod 2^N$$

$\text{trunc}^u_{M,N}(z)$

- If z is a NaN, then the result is undefined.
- Else if z is an infinity, then the result is undefined.
- Else if z is a number and $\text{trunc}(z)$ is a value within range of the target type, then return that value.
- Else the result is undefined.

$$\begin{aligned} \text{trunc}^u_{M,N}(\pm\text{nan}(n)) &= \{\} \\ \text{trunc}^u_{M,N}(\pm\infty) &= \{\} \\ \text{trunc}^u_{M,N}(\pm q) &= \text{trunc}(\pm q) && (\text{if } -1 < \text{trunc}(\pm q) < 2^N) \\ \text{trunc}^u_{M,N}(\pm q) &= \{\} && (\text{otherwise}) \end{aligned}$$

Note: This operator is [partial](#). It is not defined for NaNs, infinities, or values for which the result is out of range.

$\text{trunc}^s_{M,N}(z)$

- If z is a NaN, then the result is undefined.
- Else if z is an infinity, then the result is undefined.
- If z is a number and $\text{trunc}(z)$ is a value within range of the target type, then return that value.
- Else the result is undefined.

$$\begin{aligned} \text{trunc}^s_{M,N}(\pm\text{nan}(n)) &= \{\} \\ \text{trunc}^s_{M,N}(\pm\infty) &= \{\} \\ \text{trunc}^s_{M,N}(\pm q) &= \text{trunc}(\pm q) && (\text{if } -2^{N-1} - 1 < \text{trunc}(\pm q) < 2^{N-1}) \\ \text{trunc}^s_{M,N}(\pm q) &= \{\} && (\text{otherwise}) \end{aligned}$$

Note: This operator is [partial](#). It is not defined for NaNs, infinities, or values for which the result is out of range.

$\text{trunc_sat_u}_{M,N}(z)$

- If z is a NaN, then return 0.
- Else if z is negative infinity, then return 0.
- Else if z is positive infinity, then return $2^N - 1$.
- Else, return $\text{sat_u}_N(\text{trunc}(z))$.

$$\begin{aligned} \text{trunc_sat_u}_{M,N}(\pm\text{nan}(n)) &= 0 \\ \text{trunc_sat_u}_{M,N}(-\infty) &= 0 \\ \text{trunc_sat_u}_{M,N}(+\infty) &= 2^N - 1 \\ \text{trunc_sat_u}_{M,N}(z) &= \text{sat_u}_N(\text{trunc}(z)) \end{aligned}$$

$\text{trunc_sat_s}_{M,N}(z)$

- If z is a NaN, then return 0.
- Else if z is negative infinity, then return -2^{N-1} .
- Else if z is positive infinity, then return $2^{N-1} - 1$.
- Else, return $\text{sat_s}_N(\text{trunc}(z))$.

$$\begin{aligned}\text{trunc_sat_s}_{M,N}(\pm\text{nan}(n)) &= 0 \\ \text{trunc_sat_s}_{M,N}(-\infty) &= -2^{N-1} \\ \text{trunc_sat_s}_{M,N}(+\infty) &= 2^{N-1} - 1 \\ \text{trunc_sat_s}_{M,N}(z) &= \text{sat_s}_N(\text{trunc}(z))\end{aligned}$$

$\text{promote}_{M,N}(z)$

- If z is a canonical NaN, then return an element of $\text{nans}_N\{\}$ (i.e., a canonical NaN of size N).
- Else if z is a NaN, then return an element of $\text{nans}_N\{\pm\text{nan}(1)\}$ (i.e., any arithmetic NaN of size N).
- Else, return z .

$$\begin{aligned}\text{promote}_{M,N}(\pm\text{nan}(n)) &= \text{nans}_N\{\} && (\text{if } n = \text{canon}_N) \\ \text{promote}_{M,N}(\pm\text{nan}(n)) &= \text{nans}_N\{+\text{nan}(1)\} && (\text{otherwise}) \\ \text{promote}_{M,N}(z) &= z\end{aligned}$$

$\text{demote}_{M,N}(z)$

- If z is a canonical NaN, then return an element of $\text{nans}_N\{\}$ (i.e., a canonical NaN of size N).
- Else if z is a NaN, then return an element of $\text{nans}_N\{\pm\text{nan}(1)\}$ (i.e., any NaN of size N).
- Else if z is an infinity, then return that infinity.
- Else if z is a zero, then return that zero.
- Else, return $\text{float}_N(z)$.

$$\begin{aligned}\text{demote}_{M,N}(\pm\text{nan}(n)) &= \text{nans}_N\{\} && (\text{if } n = \text{canon}_N) \\ \text{demote}_{M,N}(\pm\text{nan}(n)) &= \text{nans}_N\{+\text{nan}(1)\} && (\text{otherwise}) \\ \text{demote}_{M,N}(\pm\infty) &= \pm\infty \\ \text{demote}_{M,N}(\pm 0) &= \pm 0 \\ \text{demote}_{M,N}(\pm q) &= \text{float}_N(\pm q)\end{aligned}$$

$\text{convert}^u_{M,N}(i)$

- Return $\text{float}_N(i)$.

$$\text{convert}^u_{M,N}(i) = \text{float}_N(i)$$

$\text{convert}_{M,N}^s(i)$

- Let j be the signed interpretation of i .
- Return $\text{float}_N(j)$.

$$\text{convert}_{M,N}^s(i) = \text{float}_N(\text{signed}_M(i))$$

$\text{reinterpret}_{t_1,t_2}(c)$

- Let d^* be the bit sequence $\text{bits}_{t_1}(c)$.
- Return the constant c' for which $\text{bits}_{t_2}(c') = d^*$.

$$\text{reinterpret}_{t_1,t_2}(c) = \text{bits}_{t_2}^{-1}(\text{bits}_{t_1}(c))$$

$\text{narrow}_{M,N}^s(i)$

- Let j be the signed interpretation of i of size M .
- Return $\text{sat}_{sN}(j)$.

$$\text{narrow}_{M,N}^s(i) = \text{sat}_{sN}(\text{signed}_M(i))$$

$\text{narrow}_{M,N}^u(i)$

- Let j be the signed interpretation of i of size M .
- Return $\text{sat}_{uN}(j)$.

$$\text{narrow}_{M,N}^u(i) = \text{sat}_{uN}(\text{signed}_M(i))$$

4.4 Instructions

WebAssembly computation is performed by executing individual **instructions**.

4.4.1 Numeric Instructions

Numeric instructions are defined in terms of the generic **numeric operators**. The mapping of numeric instructions to their underlying operators is expressed by the following definition:

$$\begin{aligned} op_{iN}(i_1, \dots, i_k) &= iop_N(i_1, \dots, i_k) \\ op_{fN}(z_1, \dots, z_k) &= fop_N(z_1, \dots, z_k) \\ op_{vN}(i_1, \dots, i_k) &= iop_N(i_1, \dots, i_k) \end{aligned}$$

And for **conversion operators**:

$$cvtop_{t_1,t_2}^{sx?}(c) = cvtop_{|t_1|,|t_2|}^{sx?}(c)$$

Where the underlying operators are partial, the corresponding instruction will **trap** when the result is not defined. Where the underlying operators are non-deterministic, because they may return one of multiple possible NaN values, so are the corresponding instructions.

Note: For example, the result of instruction `i32.add` applied to operands i_1, i_2 invokes `addi32( $i_1, i_2$ )`, which maps to the generic `iadd32( $i_1, i_2$ )` via the above definition. Similarly, `i64.trunc_f32_s` applied to z invokes `truncf32,i64s( $z$ )`, which maps to the generic `trunc32,64s( $z$ )`.

t.const c

1. Push the value *t.const c* to the stack.

Note: No formal reduction rule is required for this instruction, since *const* instructions already are *values*.

t.unop

1. Assert: due to *validation*, a value of *value type t* is on the top of the stack.
2. Pop the value *t.const c₁* from the stack.
3. If *unop_t(c₁)* is defined, then:
 - a. Let *c* be a possible result of computing *unop_t(c₁)*.
 - b. Push the value *t.const c* to the stack.
4. Else:
 - a. Trap.

$$\begin{array}{ll} (t.\text{const } c_1) \text{ } t.\text{unop} & \hookrightarrow (t.\text{const } c) & (\text{if } c \in \text{unop}_t(c_1)) \\ (t.\text{const } c_1) \text{ } t.\text{unop} & \hookrightarrow \text{trap} & (\text{if } \text{unop}_t(c_1) = \{\}) \end{array}$$

t.binop

1. Assert: due to *validation*, two values of *value type t* are on the top of the stack.
2. Pop the value *t.const c₂* from the stack.
3. Pop the value *t.const c₁* from the stack.
4. If *binop_t(c₁, c₂)* is defined, then:
 - a. Let *c* be a possible result of computing *binop_t(c₁, c₂)*.
 - b. Push the value *t.const c* to the stack.
5. Else:
 - a. Trap.

$$\begin{array}{ll} (t.\text{const } c_1) (t.\text{const } c_2) \text{ } t.\text{binop} & \hookrightarrow (t.\text{const } c) & (\text{if } c \in \text{binop}_t(c_1, c_2)) \\ (t.\text{const } c_1) (t.\text{const } c_2) \text{ } t.\text{binop} & \hookrightarrow \text{trap} & (\text{if } \text{binop}_t(c_1, c_2) = \{\}) \end{array}$$

t.testop

1. Assert: due to *validation*, a value of *value type t* is on the top of the stack.
2. Pop the value *t.const c₁* from the stack.
3. Let *c* be the result of computing *testop_t(c₁)*.
4. Push the value *i32.const c* to the stack.

$$(t.\text{const } c_1) \text{ } t.\text{testop} \hookrightarrow (i32.\text{const } c) \quad (\text{if } c = \text{testop}_t(c_1))$$

t.relop

1. Assert: due to **validation**, two values of **value type** *t* are on the top of the stack.
2. Pop the value *t.const* *c*₂ from the stack.
3. Pop the value *t.const* *c*₁ from the stack.
4. Let *c* be the result of computing *relop*_{*t*}(*c*₁, *c*₂).
5. Push the value *i32.const* *c* to the stack.

$$(t.\text{const } c_1) (t.\text{const } c_2) t.\text{relop} \hookrightarrow (i32.\text{const } c) \quad (\text{if } c = \text{relop}_t(c_1, c_2))$$

t₂.cvttop_t1_sx?

1. Assert: due to **validation**, a value of **value type** *t*₁ is on the top of the stack.
2. Pop the value *t₁.const* *c*₁ from the stack.
3. If *cvttop*_{*t*₁,*t*₂}^{*sx?*}(*c*₁) is defined:
 - a. Let *c*₂ be a possible result of computing *cvttop*_{*t*₁,*t*₂}^{*sx?*}(*c*₁).
 - b. Push the value *t₂.const* *c*₂ to the stack.
4. Else:
 - a. Trap.

$$\begin{aligned} (t_1.\text{const } c_1) t_2.\text{cvttop_}t_1_sx? &\hookrightarrow (t_2.\text{const } c_2) && (\text{if } c_2 \in \text{cvttop}_{t_1, t_2}^{sx?}(c_1)) \\ (t_1.\text{const } c_1) t_2.\text{cvttop_}t_1_sx? &\hookrightarrow \text{trap} && (\text{if } \text{cvttop}_{t_1, t_2}^{sx?}(c_1) = \{\}) \end{aligned}$$

4.4.2 Reference Instructions

ref.null t

1. Push the value *ref.null* *t* to the stack.

Note: No formal reduction rule is required for this instruction, since the *ref.null* instruction is already a **value**.

ref.is_null

1. Assert: due to **validation**, a **reference value** is on the top of the stack.
2. Pop the value *val* from the stack.
3. If *val* is *ref.null* *t*, then:
 - a. Push the value *i32.const* 1 to the stack.
4. Else:
 - a. Push the value *i32.const* 0 to the stack.

$$\begin{aligned} val \text{ ref.is_null} &\hookrightarrow (i32.\text{const } 1) && (\text{if } val = \text{ref.null } t) \\ val \text{ ref.is_null} &\hookrightarrow (i32.\text{const } 0) && (\text{otherwise}) \end{aligned}$$

`ref.func x`

1. Let F be the current frame.
2. Assert: due to validation, $F.\text{module.funcaddrs}[x]$ exists.
3. Let a be the function address $F.\text{module.funcaddrs}[x]$.
4. Push the value `ref a` to the stack.

$$F; (\text{ref.func } x) \hookrightarrow F; (\text{ref } a) \quad (\text{if } a = F.\text{module.funcaddrs}[x])$$

4.4.3 Vector Instructions

Most vector instructions are defined in terms of generic numeric operators applied lane-wise based on the `shape`.

$$op_{txN}(n_1, \dots, n_k) = \text{lanes}_{txN}^{-1}(op_t(\text{lanes}_{txN}(n_1) \dots \text{lanes}_{txN}(n_k)))$$

Note: For example, the result of instruction `i32x4.add` applied to operands i_1, i_2 invokes `add_i32x4( $i_1, i_2$ )`, which maps to $\text{lanes}_{i32x4}^{-1}(\text{add}_{i32}(i_1^+, i_2^+))$, where i_1^+ and i_2^+ are sequences resulting from invoking `lanes_i32x4( $i_1$ )` and `lanes_i32x4( $i_2$ )` respectively.

`v128.const c`

1. Push the value `v128.const c` to the stack.

Note: No formal reduction rule is required for this instruction, since `const` instructions coincide with `values`.

`v128.vvunop`

1. Assert: due to validation, a value of value type `v128` is on the top of the stack.
2. Pop the value `v128.const c1` from the stack.
3. Let c be the result of computing `vvunop_v128( $c_1$ )`.
4. Push the value `v128.const c` to the stack.

$$(\text{v128.const } c_1) \text{ v128.vvunop} \hookrightarrow (\text{v128.const } c) \quad (\text{if } c = \text{vvunop}_{v128}(c_1))$$

`v128.vvbinop`

1. Assert: due to validation, two values of value type `v128` are on the top of the stack.
2. Pop the value `v128.const c2` from the stack.
3. Pop the value `v128.const c1` from the stack.
4. Let c be the result of computing `vvbinop_v128( $c_1, c_2$ )`.
5. Push the value `v128.const c` to the stack.

$$(\text{v128.const } c_1) (\text{v128.const } c_2) \text{ v128.vvbinop} \hookrightarrow (\text{v128.const } c) \quad (\text{if } c = \text{vvbinop}_{v128}(c_1, c_2))$$

v128.vternop

1. Assert: due to **validation**, three values of **value type v128** are on the top of the stack.
2. Pop the value **v128.const** c_3 from the stack.
3. Pop the value **v128.const** c_2 from the stack.
4. Pop the value **v128.const** c_1 from the stack.
5. Let c be the result of computing $vternop_{v128}(c_1, c_2, c_3)$.
6. Push the value **v128.const** c to the stack.

$$(v128.const\ c_1)\ (v128.const\ c_2)\ (v128.const\ c_3)\ v128.vternop \hookrightarrow (v128.const\ c) \quad (\text{if } c = vternop_{v128}(c_1, c_2, c_3))$$
v128.any_true

1. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
2. Pop the value **v128.const** c_1 from the stack.
3. Let i be the result of computing $ine_{128}(c_1, 0)$.
4. Push the value **i32.const** i onto the stack.

$$(v128.const\ c_1)\ v128.any_true \hookrightarrow (i32.const\ i) \quad (\text{if } i = ine_{128}(c_1, 0))$$
i8x16.swizzle

1. Assert: due to **validation**, two values of **value type v128** are on the top of the stack.
2. Pop the value **v128.const** c_2 from the stack.
3. Let i^* be the result of computing $lanes_{i8x16}(c_2)$.
4. Pop the value **v128.const** c_1 from the stack.
5. Let j^* be the result of computing $lanes_{i8x16}(c_1)$.
6. Let c^* be the concatenation of the two sequences j^* and 0^{240} .
7. Let c' be the result of computing $lanes_{i8x16}^{-1}(c^*[i^*[0]] \dots c^*[i^*[15]])$.
8. Push the value **v128.const** c' onto the stack.

$$(v128.const\ c_1)\ (v128.const\ c_2)\ i8x16.swizzle \hookrightarrow (v128.const\ c') \\ (\text{if } i^* = lanes_{i8x16}(c_2) \\ \wedge c^* = lanes_{i8x16}(c_1)\ 0^{240} \\ \wedge c' = lanes_{i8x16}^{-1}(c^*[i^*[0]] \dots c^*[i^*[15]]))$$
*i8x16.shuffle x^**

1. Assert: due to **validation**, two values of **value type v128** are on the top of the stack.
2. Assert: due to **validation**, for all x_i in x^* it holds that $x_i < 32$.
3. Pop the value **v128.const** c_2 from the stack.
4. Let i_2^* be the result of computing $lanes_{i8x16}(c_2)$.
5. Pop the value **v128.const** c_1 from the stack.
6. Let i_1^* be the result of computing $lanes_{i8x16}(c_1)$.
7. Let i^* be the concatenation of the two sequences i_1^* and i_2^* .
8. Let c be the result of computing $lanes_{i8x16}^{-1}(i^*[x^*[0]] \dots i^*[x^*[15]])$.

9. Push the value `v128.const c` onto the stack.

$$\begin{aligned} & (\text{v128.const } c_1) (\text{v128.const } c_2) (\text{i8x16.shuffle } x^*) \hookrightarrow (\text{v128.const } c) \\ & \quad (\text{if } i^* = \text{lanes}_{\text{i8x16}}(c_1) \text{ lanes}_{\text{i8x16}}(c_2) \\ & \quad \wedge c = \text{lanes}_{\text{i8x16}}^{-1}(i^*[x^*[0]] \dots i^*[x^*[15]])) \end{aligned}$$

`shape.splat`

1. Let t be the type `unpacked(shape)`.
2. Assert: due to **validation**, a value of **value type** t is on the top of the stack.
3. Pop the value $t.\text{const } c_1$ from the stack.
4. Let N be the integer `dim(shape)`.
5. Let c be the result of computing $\text{lanes}_{\text{shape}}^{-1}(c_1^N)$.
6. Push the value `v128.const c` to the stack.

$$(t.\text{const } c_1) \text{ shape.splat} \hookrightarrow (\text{v128.const } c) \quad (\text{if } t = \text{unpacked}(\text{shape}) \wedge c = \text{lanes}_{\text{shape}}^{-1}(c_1^{\text{dim}(\text{shape})}))$$

`t1×N.extract_lanesx? x`

1. Assert: due to **validation**, $x < N$.
2. Assert: due to **validation**, a value of **value type** `v128` is on the top of the stack.
3. Pop the value `v128.const c1` from the stack.
4. Let i^* be the result of computing $\text{lanes}_{t_1 \times N}(c_1)$.
5. Let t_2 be the type `unpacked(t1×N)`.
6. Let c_2 be the result of computing $\text{extend}_{t_1, t_2}^{sx?}(i^*[x])$.
7. Push the value $t_2.\text{const } c_2$ to the stack.

$$\begin{aligned} & (\text{v128.const } c_1) (t_1 \times N.\text{extract_lane } x) \hookrightarrow (t_2.\text{const } c_2) \\ & \quad (\text{if } t_2 = \text{unpacked}(t_1 \times N) \\ & \quad \wedge c_2 = \text{extend}_{t_1, t_2}^{sx?}(\text{lanes}_{t_1 \times N}(c_1)[x])) \end{aligned}$$

`shape.replace_lane x`

1. Assert: due to **validation**, $x < \text{dim}(\text{shape})$.
2. Let t_1 be the type `unpacked(shape)`.
3. Assert: due to **validation**, a value of **value type** t_1 is on the top of the stack.
4. Pop the value $t_1.\text{const } c_1$ from the stack.
5. Assert: due to **validation**, a value of **value type** `v128` is on the top of the stack.
6. Pop the value `v128.const c2` from the stack.
7. Let i^* be the result of computing $\text{lanes}_{\text{shape}}(c_2)$.
8. Let c be the result of computing $\text{lanes}_{\text{shape}}^{-1}(i^* \text{ with } [x] = c_1)$.
9. Push `v128.const c` on the stack.

$$\begin{aligned} & (t_1.\text{const } c_1) (\text{v128.const } c_2) (\text{shape.replace_lane } x) \hookrightarrow (\text{v128.const } c) \\ & \quad (\text{if } i^* = \text{lanes}_{\text{shape}}(c_2) \\ & \quad \wedge c = \text{lanes}_{\text{shape}}^{-1}(i^* \text{ with } [x] = c_1)) \end{aligned}$$

shape.vunop

1. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
2. Pop the value **v128.const** c_1 from the stack.
3. Let c be the result of computing $\text{vunop}_{\text{shape}}(c_1)$.
4. Push the value **v128.const** c to the stack.

$$(\text{v128.const } c_1) \text{ v128.vunop} \hookrightarrow (\text{v128.const } c) \quad (\text{if } c = \text{vunop}_{\text{shape}}(c_1))$$

shape.vbinop

1. Assert: due to **validation**, two values of **value type v128** are on the top of the stack.
2. Pop the value **v128.const** c_2 from the stack.
3. Pop the value **v128.const** c_1 from the stack.
4. If $\text{vbinop}_{\text{shape}}(c_1, c_2)$ is defined:
 - a. Let c be a possible result of computing $\text{vbinop}_{\text{shape}}(c_1, c_2)$.
 - b. Push the value **v128.const** c to the stack.
5. Else:
 - a. Trap.

$$\begin{aligned} (\text{v128.const } c_1) (\text{v128.const } c_2) \text{ shape.vbinop} &\hookrightarrow (\text{v128.const } c) && (\text{if } c \in \text{vbinop}_{\text{shape}}(c_1, c_2)) \\ (\text{v128.const } c_1) (\text{v128.const } c_2) \text{ shape.vbinop} &\hookrightarrow \text{trap} && (\text{if } \text{vbinop}_{\text{shape}}(c_1, c_2) = \{\}) \end{aligned}$$

txN.vrelop

1. Assert: due to **validation**, two values of **value type v128** are on the top of the stack.
2. Pop the value **v128.const** c_2 from the stack.
3. Pop the value **v128.const** c_1 from the stack.
4. Let i_1^* be the result of computing $\text{lanes}_{\text{txN}}(c_1)$.
5. Let i_2^* be the result of computing $\text{lanes}_{\text{txN}}(c_2)$.
6. Let i^* be the result of computing $\text{vrelop}_t(i_1^*, i_2^*)$.
7. Let j^* be the result of computing $\text{extend}_{1,|t|}^s(i^*)$.
8. Let c be the result of computing $\text{lanes}_{\text{txN}}^{-1}(j^*)$.
9. Push the value **v128.const** c to the stack.

$$\begin{aligned} (\text{v128.const } c_1) (\text{v128.const } c_2) \text{ txN.vrelop} &\hookrightarrow (\text{v128.const } c) \\ &(\text{if } c = \text{lanes}_{\text{txN}}^{-1}(\text{extend}_{1,|t|}^s(\text{vrelop}_t(\text{lanes}_{\text{txN}}(c_1), \text{lanes}_{\text{txN}}(c_2)))))) \end{aligned}$$

txN.vishiftop

1. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
2. Pop the value **i32.const** s from the stack.
3. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
4. Pop the value **v128.const** c_1 from the stack.
5. Let i^* be the result of computing $\text{lanes}_{txN}(c_1)$.
6. Let j^* be the result of computing $\text{vishiftop}_t(i^*, s^N)$.
7. Let c be the result of computing $\text{lanes}_{txN}^{-1}(j^*)$.
8. Push the value **v128.const** c to the stack.

$$(\text{v128.const } c_1) (\text{i32.const } s) \text{txN.vishiftop} \hookrightarrow (\text{v128.const } c) \\ (\text{if } i^* = \text{lanes}_{txN}(c_1) \\ \wedge c = \text{lanes}_{txN}^{-1}(\text{vishiftop}_t(i^*, s^N)))$$

shape.all_true

1. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
2. Pop the value **v128.const** c_1 from the stack.
3. Let i_1^* be the result of computing $\text{lanes}_{shape}(c_1)$.
4. Let i be the result of computing $\text{bool}(\bigwedge (i_1 \neq 0)^*)$.
5. Push the value **i32.const** i onto the stack.

$$(\text{v128.const } c_1) \text{shape.all_true} \hookrightarrow (\text{i32.const } i) \\ (\text{if } i_1^* = \text{lanes}_{shape}(c) \\ \wedge i = \text{bool}(\bigwedge (i_1 \neq 0)^*))$$

txN.bitmask

1. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
2. Pop the value **v128.const** c_1 from the stack.
3. Let i_1^N be the result of computing $\text{lanes}_{txN}(c)$.
4. Let B be the **bit width** $|t|$ of **value type** t .
5. Let i_2^N be the result of computing $\text{ilt}_{sB}(i_1^N, 0^N)$.
6. Let j^* be the concatenation of the two sequences i_2^N and 0^{32-N} .
7. Let c be the result of computing $\text{ibits}_{32}^{-1}(j^*)$.
8. Push the value **i32.const** c onto the stack.

$$(\text{v128.const } c_1) \text{txN.bitmask} \hookrightarrow (\text{i32.const } c) \quad (\text{if } c = \text{ibits}_{32}^{-1}(\text{ilt}_{s|t|}(\text{lanes}_{txN}(c), 0^N)))$$

$t_2 \times N$.*narrow* _{$t_1 \times M$} *_{sx}*

1. Assert: due to **syntax**, $N = 2 \cdot M$.
2. Assert: due to **validation**, two values of **value type v128** are on the top of the stack.
3. Pop the value **v128.const** c_2 from the stack.
4. Let i_2^M be the result of computing $\text{lanes}_{t_1 \times M}(c_2)$.
5. Let d_2^M be the result of computing $\text{narrow}_{|t_1|, |t_2|}^{sx}(i_2^M)$.
6. Pop the value **v128.const** c_1 from the stack.
7. Let i_1^M be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
8. Let d_1^M be the result of computing $\text{narrow}_{|t_1|, |t_2|}^{sx}(i_1^M)$.
9. Let j^N be the concatenation of the two sequences d_1^M and d_2^M .
10. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(j^N)$.
11. Push the value **v128.const** c onto the stack.

$$\begin{aligned}
 &(\text{v128.const } c_1) (\text{v128.const } c_2) t_2 \times N.\text{narrow}_{t_1 \times M_sx} \hookrightarrow (\text{v128.const } c) \\
 &(\text{if } d_1^M = \text{narrow}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_1)) \\
 &\quad \wedge d_2^M = \text{narrow}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_2)) \\
 &\quad \wedge c = \text{lanes}_{t_2 \times N}^{-1}(d_1^M d_2^M))
 \end{aligned}$$

$t_2 \times N$.*vcvtop* _{$t_1 \times M$} *_{sx}*

1. Assert: due to **syntax**, $N = M$.
2. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
3. Pop the value **v128.const** c_1 from the stack.
4. Let i^* be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
5. Let j^* be the result of computing $\text{vcvtop}_{|t_1|, |t_2|}^{sx}(i^*)$.
6. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(j^*)$.
7. Push the value **v128.const** c onto the stack.

$$\begin{aligned}
 &(\text{v128.const } c_1) t_2 \times N.\text{vcvtop}_{t_1 \times M_sx} \hookrightarrow (\text{v128.const } c) \\
 &(\text{if } c = \text{lanes}_{t_2 \times N}^{-1}(\text{vcvtop}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_1))))
 \end{aligned}$$

$t_2 \times N$.*vcvtop_half* _{$t_1 \times M$} *_{sx}?*

1. Assert: due to **syntax**, $N = M/2$.
2. Assert: due to **validation**, a value of **value type v128** is on the top of the stack.
3. Pop the value **v128.const** c_1 from the stack.
4. Let i^* be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
5. If *half* is low, then:
 - a. Let j^* be the sequence $i^*[0 : N]$.
6. Else:
 - a. Let j^* be the sequence $i^*[N : N]$.
7. Let k^* be the result of computing $\text{vcvtop}_{|t_1|, |t_2|}^{sx?}(j^*)$.
8. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(k^*)$.

9. Push the value `v128.const` c onto the stack.

$$(\text{v128.const } c_1) \ t_2 \times N . \text{vcvtop_half_} t_1 \times M _sx \ ? \ \hookrightarrow \ (\text{v128.const } c) \\ (\text{if } c = \text{lanes}_{t_2 \times N}^{-1}(\text{vcvtop}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_1)[\text{half}(0, N) : N])))$$

where:

$$\begin{aligned} \text{low}(x, y) &= x \\ \text{high}(x, y) &= y \end{aligned}$$

$t_2 \times N . \text{vcvtop_} t_1 \times M _sx _zero$

1. Assert: due to `syntax`, $N = 2 \cdot M$.
2. Assert: due to `validation`, a value of `value type v128` is on the top of the stack.
3. Pop the value `v128.const` c_1 from the stack.
4. Let i^* be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
5. Let j^* be the result of computing $\text{vcvtop}_{|t_1|, |t_2|}^{sx}(i^*)$.
6. Let k^* be the concatenation of the two sequences j^* and 0^M .
7. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(k^*)$.
8. Push the value `v128.const` c onto the stack.

$$(\text{v128.const } c_1) \ t_2 \times N . \text{vcvtop_} t_1 \times M _sx _zero \ \hookrightarrow \ (\text{v128.const } c) \\ (\text{if } c = \text{lanes}_{t_2 \times N}^{-1}(\text{vcvtop}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_1)) \ 0^M))$$

`i32x4.dot_i16x8_s`

1. Assert: due to `validation`, two values of `value type v128` are on the top of the stack.
2. Pop the value `v128.const` c_2 from the stack.
3. Pop the value `v128.const` c_1 from the stack.
4. Let i_1^* be the result of computing $\text{lanes}_{i16 \times 8}(c_1)$.
5. Let j_1^* be the result of computing $\text{extend}_{16, 32}^5(i_1^*)$.
6. Let i_2^* be the result of computing $\text{lanes}_{i16 \times 8}(c_2)$.
7. Let j_2^* be the result of computing $\text{extend}_{16, 32}^5(i_2^*)$.
8. Let $(k_1 \ k_2)^*$ be the result of computing $\text{imul}_{32}(j_1^*, j_2^*)$.
9. Let k^* be the result of computing $\text{iadd}_{32}(k_1, k_2)^*$.
10. Let c be the result of computing $\text{lanes}_{i32 \times 4}^{-1}(k^*)$.
11. Push the value `v128.const` c onto the stack.

$$(\text{v128.const } c_1) \ (\text{v128.const } c_2) \ \text{i32x4.dot_i16x8_s} \ \hookrightarrow \ (\text{v128.const } c) \\ (\text{if } (i_1 \ i_2)^* = \text{imul}_{32}(\text{extend}_{16, 32}^5(\text{lanes}_{i16 \times 8}(c_1)), \text{extend}_{16, 32}^5(\text{lanes}_{i16 \times 8}(c_2))) \\ \wedge j^* = \text{iadd}_{32}(i_1, i_2)^* \\ \wedge c = \text{lanes}_{i32 \times 4}^{-1}(j^*))$$

$t_2 \times N.\text{extmul_half_}t_1 \times M_sx$

1. Assert: due to **syntax**, $N = M/2$.
 2. Assert: due to **validation**, two values of **value type** `v128` are on the top of the stack.
 3. Pop the value `v128.const` c_2 from the stack.
 4. Pop the value `v128.const` c_1 from the stack.
 5. Let i_1^* be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
 6. Let i_2^* be the result of computing $\text{lanes}_{t_1 \times M}(c_2)$.
 7. If `half` is low, then:
 - a. Let j_1^* be the sequence $i_1^*[0 : N]$.
 - b. Let j_2^* be the sequence $i_2^*[0 : N]$.
 8. Else:
 - a. Let j_1^* be the sequence $i_1^*[N : N]$.
 - b. Let j_2^* be the sequence $i_2^*[N : N]$.
 9. Let k_1^* be the result of computing $\text{extend}_{|t_1|, |t_2|}^{sx}(j_1^*)$.
 10. Let k_2^* be the result of computing $\text{extend}_{|t_1|, |t_2|}^{sx}(j_2^*)$.
 11. Let k^* be the result of computing $\text{imul}_{t_2 \times N}(k_1^*, k_2^*)$.
 12. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(k^*)$.
 13. Push the value `v128.const` c onto the stack.
- $$(\text{v128.const } c_1) (\text{v128.const } c_2) t_2 \times N.\text{extmul_half_}t_1 \times M_sx \hookrightarrow (\text{v128.const } c)$$
- $$(\text{if } i^* = \text{lanes}_{t_1 \times M}(c_1)[\text{half}(0, N) : N] \\ \wedge j^* = \text{lanes}_{t_1 \times M}(c_2)[\text{half}(0, N) : N] \\ \wedge c = \text{lanes}_{t_2 \times N}^{-1}(\text{imul}_{t_2 \times N}(\text{extend}_{|t_1|, |t_2|}^{sx}(i^*), \text{extend}_{|t_1|, |t_2|}^{sx}(j^*))))$$

where:

$$\begin{aligned} \text{low}(x, y) &= x \\ \text{high}(x, y) &= y \end{aligned}$$

 $t_2 \times N.\text{extadd_pairwise_}t_1 \times M_sx$

1. Assert: due to **syntax**, $N = M/2$.
 2. Assert: due to **validation**, a value of **value type** `v128` is on the top of the stack.
 3. Pop the value `v128.const` c_1 from the stack.
 4. Let i^* be the result of computing $\text{lanes}_{t_1 \times M}(c_1)$.
 5. Let $(j_1 \ j_2)^*$ be the result of computing $\text{extend}_{|t_1|, |t_2|}^{sx}(i^*)$.
 6. Let k^* be the result of computing $\text{iadd}_N(j_1, j_2)^*$.
 7. Let c be the result of computing $\text{lanes}_{t_2 \times N}^{-1}(k^*)$.
 8. Push the value `v128.const` c to the stack.
- $$(\text{v128.const } c_1) t_2 \times N.\text{extadd_pairwise_}t_1 \times M_sx \hookrightarrow (\text{v128.const } c)$$
- $$(\text{if } (i_1 \ i_2)^* = \text{extend}_{|t_1|, |t_2|}^{sx}(\text{lanes}_{t_1 \times M}(c_1)) \\ \wedge j^* = \text{iadd}_N(i_1, i_2)^* \\ \wedge c = \text{lanes}_{t_2 \times N}^{-1}(j^*))$$

4.4.4 Parametric Instructions

drop

1. Assert: due to **validation**, a value is on the top of the stack.
2. Pop the value *val* from the stack.

$$val \text{ drop} \hookrightarrow \epsilon$$

select (*t**)?

1. Assert: due to **validation**, a value of **value type** i32 is on the top of the stack.
2. Pop the value i32.const *c* from the stack.
3. Assert: due to **validation**, two more values (of the same **value type**) are on the top of the stack.
4. Pop the value *val*₂ from the stack.
5. Pop the value *val*₁ from the stack.
6. If *c* is not 0, then:
 - a. Push the value *val*₁ back to the stack.
7. Else:
 - a. Push the value *val*₂ back to the stack.

$$\begin{aligned} val_1 \ val_2 \ (i32.\text{const } c) \ (\text{select } t^?) &\hookrightarrow val_1 && (\text{if } c \neq 0) \\ val_1 \ val_2 \ (i32.\text{const } c) \ (\text{select } t^?) &\hookrightarrow val_2 && (\text{if } c = 0) \end{aligned}$$

Note: In future versions of WebAssembly, **select** may allow more than one value per choice.

4.4.5 Variable Instructions

local.get *x*

1. Let *F* be the **current frame**.
2. Assert: due to **validation**, *F*.locals[*x*] exists.
3. Let *val* be the value *F*.locals[*x*].
4. Push the value *val* to the stack.

$$F; (\text{local.get } x) \hookrightarrow F; val \quad (\text{if } F.\text{locals}[x] = val)$$

local.set *x*

1. Let *F* be the **current frame**.
2. Assert: due to **validation**, *F*.locals[*x*] exists.
3. Assert: due to **validation**, a value is on the top of the stack.
4. Pop the value *val* from the stack.
5. Replace *F*.locals[*x*] with the value *val*.

$$F; val \ (\text{local.set } x) \hookrightarrow F'; \epsilon \quad (\text{if } F' = F \text{ with locals}[x] = val)$$

`local.tee x`

1. Assert: due to [validation](#), a value is on the top of the stack.
2. Pop the value *val* from the stack.
3. Push the value *val* to the stack.
4. Push the value *val* to the stack.
5. [Execute](#) the instruction `local.set x`.

$$val \text{ (local.tee } x) \hookrightarrow val \text{ val (local.set } x)$$

`global.get x`

1. Let *F* be the [current frame](#).
2. Assert: due to [validation](#), *F.module.globaladdrs*[*x*] exists.
3. Let *a* be the [global address](#) *F.module.globaladdrs*[*x*].
4. Assert: due to [validation](#), *S.globals*[*a*] exists.
5. Let *glob* be the [global instance](#) *S.globals*[*a*].
6. Let *val* be the value *glob.value*.
7. Push the value *val* to the stack.

$$S; F; (\text{global.get } x) \hookrightarrow S; F; val \\ (\text{if } S.globals[F.module.globaladdrs[x]].value = val)$$

`global.set x`

1. Let *F* be the [current frame](#).
2. Assert: due to [validation](#), *F.module.globaladdrs*[*x*] exists.
3. Let *a* be the [global address](#) *F.module.globaladdrs*[*x*].
4. Assert: due to [validation](#), *S.globals*[*a*] exists.
5. Let *glob* be the [global instance](#) *S.globals*[*a*].
6. Assert: due to [validation](#), a value is on the top of the stack.
7. Pop the value *val* from the stack.
8. Replace *glob.value* with the value *val*.

$$S; F; val \text{ (global.set } x) \hookrightarrow S'; F; \epsilon \\ (\text{if } S' = S \text{ with } globals[F.module.globaladdrs[x]].value = val)$$

Note: [Validation](#) ensures that the global is, in fact, marked as mutable.

4.4.6 Table Instructions

`table.get` x

1. Let F be the current frame.
2. Assert: due to validation, $F.\text{module}.\text{tableaddrs}[x]$ exists.
3. Let a be the table address $F.\text{module}.\text{tableaddrs}[x]$.
4. Assert: due to validation, $S.\text{tables}[a]$ exists.
5. Let tab be the table instance $S.\text{tables}[a]$.
6. Assert: due to validation, a value of value type `i32` is on the top of the stack.
7. Pop the value `i32.const` i from the stack.
8. If i is not smaller than the length of $tab.\text{elem}$, then:
 - a. Trap.
9. Let val be the value $tab.\text{elem}[i]$.
10. Push the value val to the stack.

$$\begin{aligned}
 S; F; (\text{i32.const } i) (\text{table.get } x) &\hookrightarrow S; F; val \\
 &\quad (\text{if } S.\text{tables}[F.\text{module}.\text{tableaddrs}[x]].\text{elem}[i] = val) \\
 S; F; (\text{i32.const } i) (\text{table.get } x) &\hookrightarrow S; F; \text{trap} \\
 &\quad (\text{otherwise})
 \end{aligned}$$

`table.set` x

1. Let F be the current frame.
2. Assert: due to validation, $F.\text{module}.\text{tableaddrs}[x]$ exists.
3. Let a be the table address $F.\text{module}.\text{tableaddrs}[x]$.
4. Assert: due to validation, $S.\text{tables}[a]$ exists.
5. Let tab be the table instance $S.\text{tables}[a]$.
6. Assert: due to validation, a reference value is on the top of the stack.
7. Pop the value val from the stack.
8. Assert: due to validation, a value of value type `i32` is on the top of the stack.
9. Pop the value `i32.const` i from the stack.
10. If i is not smaller than the length of $tab.\text{elem}$, then:
 - a. Trap.
11. Replace the element $tab.\text{elem}[i]$ with val .

$$\begin{aligned}
 S; F; (\text{i32.const } i) val (\text{table.set } x) &\hookrightarrow S'; F; \epsilon \\
 &\quad (\text{if } S' = S \text{ with } \text{tables}[F.\text{module}.\text{tableaddrs}[x]].\text{elem}[i] = val) \\
 S; F; (\text{i32.const } i) val (\text{table.set } x) &\hookrightarrow S; F; \text{trap} \\
 &\quad (\text{otherwise})
 \end{aligned}$$

`table.size  $x$`

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module.tableaddrs}[x]$ exists.
3. Let a be the **table address** $F.\text{module.tableaddrs}[x]$.
4. Assert: due to **validation**, $S.\text{tables}[a]$ exists.
5. Let tab be the **table instance** $S.\text{tables}[a]$.
6. Let sz be the length of $tab.\text{elem}$.
7. Push the value `i32.const  $sz$` to the stack.

$$S; F; (\text{table.size } x) \hookrightarrow S; F; (\text{i32.const } sz) \\ (\text{if } |S.\text{tables}[F.\text{module.tableaddrs}[x]].\text{elem}| = sz)$$

`table.grow  $x$`

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module.tableaddrs}[x]$ exists.
3. Let a be the **table address** $F.\text{module.tableaddrs}[x]$.
4. Assert: due to **validation**, $S.\text{tables}[a]$ exists.
5. Let tab be the **table instance** $S.\text{tables}[a]$.
6. Let sz be the length of $S.\text{tables}[a]$.
7. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
8. Pop the value `i32.const  $n$` from the stack.
9. Assert: due to **validation**, a **reference value** is on the top of the stack.
10. Pop the value val from the stack.
11. Let err be the **i32** value $2^{32} - 1$, for which `signed32( $err$ )` is -1 .
12. Either:
 - a. If **growing** tab by n entries with initialization value val succeeds, then:
 - i. Push the value `i32.const  $sz$` to the stack.
 - b. Else:
 - i. Push the value `i32.const  $err$` to the stack.
13. Or:
 - a. push the value `i32.const  $err$` to the stack.

$$S; F; val (\text{i32.const } n) (\text{table.grow } x) \hookrightarrow S'; F; (\text{i32.const } sz) \\ (\text{if } F.\text{module.tableaddrs}[x] = a \\ \wedge sz = |S.\text{tables}[a].\text{elem}| \\ \wedge S' = S \text{ with } \text{tables}[a] = \text{growtable}(S.\text{tables}[a], n, val)) \\ S; F; val (\text{i32.const } n) (\text{table.grow } x) \hookrightarrow S; F; (\text{i32.const signed}_{32}^{-1}(-1))$$

Note: The `table.grow` instruction is non-deterministic. It may either succeed, returning the old table size sz , or fail, returning -1 . Failure *must* occur if the referenced table instance has a maximum size defined that would be exceeded. However, failure *can* occur in other cases as well. In practice, the choice depends on the **resources** available to the **embedder**.

`table.fill x`

1. Let F be the current frame.
2. Assert: due to validation, $F.module.tableaddrs[x]$ exists.
3. Let ta be the table address $F.module.tableaddrs[x]$.
4. Assert: due to validation, $S.tables[ta]$ exists.
5. Let tab be the table instance $S.tables[ta]$.
6. Assert: due to validation, a value of value type `i32` is on the top of the stack.
7. Pop the value `i32.const n` from the stack.
8. Assert: due to validation, a reference value is on the top of the stack.
9. Pop the value val from the stack.
10. Assert: due to validation, a value of value type `i32` is on the top of the stack.
11. Pop the value `i32.const i` from the stack.
12. If $i + n$ is larger than the length of $tab.elem$, then:
 - a. Trap.
12. If n is 0, then:
 - a. Return.
13. Push the value `i32.const i` to the stack.
14. Push the value val to the stack.
15. Execute the instruction `table.set x`.
16. Push the value `i32.const (i + 1)` to the stack.
17. Push the value val to the stack.
18. Push the value `i32.const (n - 1)` to the stack.
19. Execute the instruction `table.fill x`.

$$\begin{aligned}
 S; F; (i32.const\ i)\ val\ (i32.const\ n)\ (table.fill\ x) &\hookrightarrow S; F; trap \\
 &\quad (if\ i + n > |S.tables[F.module.tableaddrs[x]].elem|) \\
 S; F; (i32.const\ i)\ val\ (i32.const\ 0)\ (table.fill\ x) &\hookrightarrow S; F; \epsilon \\
 &\quad (otherwise) \\
 S; F; (i32.const\ i)\ val\ (i32.const\ n + 1)\ (table.fill\ x) &\hookrightarrow \\
 S; F; (i32.const\ i)\ val\ (table.set\ x) & \\
 (i32.const\ i + 1)\ val\ (i32.const\ n)\ (table.fill\ x) & \\
 (otherwise) &
 \end{aligned}$$

`table.copy x y`

1. Let F be the current frame.
2. Assert: due to validation, $F.module.tableaddrs[x]$ exists.
3. Let ta_x be the table address $F.module.tableaddrs[x]$.
4. Assert: due to validation, $S.tables[ta_x]$ exists.
5. Let tab_x be the table instance $S.tables[ta_x]$.
6. Assert: due to validation, $F.module.tableaddrs[y]$ exists.
7. Let ta_y be the table address $F.module.tableaddrs[y]$.

8. Assert: due to **validation**, $S.tables[ta_y]$ exists.
9. Let tab_y be the **table instance** $S.tables[ta_y]$.
10. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
11. Pop the value **i32.const** n from the stack.
12. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
13. Pop the value **i32.const** s from the stack.
14. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
15. Pop the value **i32.const** d from the stack.
16. If $s + n$ is larger than the length of $tab_y.elem$ or $d + n$ is larger than the length of $tab_x.elem$, then:
 - a. Trap.
17. If $n = 0$, then:
 - a. Return.
18. If $d \leq s$, then:
 - a. Push the value **i32.const** d to the stack.
 - b. Push the value **i32.const** s to the stack.
 - c. Execute the instruction **table.get** y .
 - d. Execute the instruction **table.set** x .
 - e. Assert: due to the earlier check against the table size, $d + 1 < 2^{32}$.
 - f. Push the value **i32.const** $(d + 1)$ to the stack.
 - g. Assert: due to the earlier check against the table size, $s + 1 < 2^{32}$.
 - h. Push the value **i32.const** $(s + 1)$ to the stack.
19. Else:
 - a. Assert: due to the earlier check against the table size, $d + n - 1 < 2^{32}$.
 - b. Push the value **i32.const** $(d + n - 1)$ to the stack.
 - c. Assert: due to the earlier check against the table size, $s + n - 1 < 2^{32}$.
 - d. Push the value **i32.const** $(s + n - 1)$ to the stack.
 - e. Execute the instruction **table.get** y .
 - f. Execute the instruction **table.set** x .
 - g. Push the value **i32.const** d to the stack.
 - h. Push the value **i32.const** s to the stack.
20. Push the value **i32.const** $(n - 1)$ to the stack.
21. Execute the instruction **table.copy** $x y$.

$$\begin{aligned}
& S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n) (\text{table.copy } x \ y) \hookrightarrow S; F; \text{trap} \\
& \quad (\text{if } s + n > |S.\text{tables}[F.\text{module}.\text{tableaddrs}[y]].\text{elem}| \\
& \quad \vee d + n > |S.\text{tables}[F.\text{module}.\text{tableaddrs}[x]].\text{elem}|) \\
& S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } 0) (\text{table.copy } x \ y) \hookrightarrow S; F; \epsilon \\
& \quad (\text{otherwise}) \\
& S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n + 1) (\text{table.copy } x \ y) \hookrightarrow \\
& \quad S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{table.get } y) (\text{table.set } x) \\
& \quad (\text{i32.const } d + 1) (\text{i32.const } s + 1) (\text{i32.const } n) (\text{table.copy } x \ y) \\
& \quad (\text{otherwise, if } d \leq s) \\
& S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n + 1) (\text{table.copy } x \ y) \hookrightarrow \\
& \quad S; F; (\text{i32.const } d + n) (\text{i32.const } s + n) (\text{table.get } y) (\text{table.set } x) \\
& \quad (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n) (\text{table.copy } x \ y) \\
& \quad (\text{otherwise, if } d > s)
\end{aligned}$$

`table.init` $x \ y$

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module}.\text{tableaddrs}[x]$ exists.
3. Let ta be the **table address** $F.\text{module}.\text{tableaddrs}[x]$.
4. Assert: due to **validation**, $S.\text{tables}[ta]$ exists.
5. Let tab be the **table instance** $S.\text{tables}[ta]$.
6. Assert: due to **validation**, $F.\text{module}.\text{elemaddrs}[y]$ exists.
7. Let ea be the **element address** $F.\text{module}.\text{elemaddrs}[y]$.
8. Assert: due to **validation**, $S.\text{elems}[ea]$ exists.
9. Let $elem$ be the **element instance** $S.\text{elems}[ea]$.
10. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
11. Pop the value $\text{i32.const } n$ from the stack.
12. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
13. Pop the value $\text{i32.const } s$ from the stack.
14. Assert: due to **validation**, a value of **value type i32** is on the top of the stack.
15. Pop the value $\text{i32.const } d$ from the stack.
16. If $s + n$ is larger than the length of $elem.\text{elem}$ or $d + n$ is larger than the length of $tab.\text{elem}$, then:
 - a. Trap.
17. If $n = 0$, then:
 - a. Return.
18. Let val be the **reference value** $elem.\text{elem}[s]$.
19. Push the value $\text{i32.const } d$ to the stack.
20. Push the value val to the stack.
21. Execute the instruction `table.set` x .
22. Assert: due to the earlier check against the table size, $d + 1 < 2^{32}$.
23. Push the value $\text{i32.const } (d + 1)$ to the stack.
24. Assert: due to the earlier check against the segment size, $s + 1 < 2^{32}$.
25. Push the value $\text{i32.const } (s + 1)$ to the stack.

26. Push the value `i32.const` $(n - 1)$ to the stack.
27. Execute the instruction `table.init` x y .

$$\begin{aligned}
S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n) (\text{table.init } x \ y) &\hookrightarrow S; F; \text{trap} \\
&\quad (\text{if } s + n > |S.\text{elems}[F.\text{module}.\text{elemaddrs}[y]].\text{elem}| \\
&\quad \vee d + n > |S.\text{tables}[F.\text{module}.\text{tableaddrs}[x]].\text{elem}|) \\
S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } 0) (\text{table.init } x \ y) &\hookrightarrow S; F; \epsilon \\
&\quad (\text{otherwise}) \\
S; F; (\text{i32.const } d) (\text{i32.const } s) (\text{i32.const } n + 1) (\text{table.init } x \ y) &\hookrightarrow \\
S; F; (\text{i32.const } d) \text{ val } (\text{table.set } x) & \\
&\quad (\text{i32.const } d + 1) (\text{i32.const } s + 1) (\text{i32.const } n) (\text{table.init } x \ y) \\
&\quad (\text{otherwise, if } \text{val} = S.\text{elems}[F.\text{module}.\text{elemaddrs}[y]].\text{elem}[s])
\end{aligned}$$

`elem.drop` x

1. Let F be the `current frame`.
2. Assert: due to `validation`, $F.\text{module}.\text{elemaddrs}[x]$ exists.
3. Let a be the `element address` $F.\text{module}.\text{elemaddrs}[x]$.
4. Assert: due to `validation`, $S.\text{elems}[a]$ exists.
5. Replace $S.\text{elems}[a].\text{elem}$ with ϵ .

$$\begin{aligned}
S; F; (\text{elem.drop } x) &\hookrightarrow S'; F; \epsilon \\
&\quad (\text{if } S' = S \text{ with } \text{elems}[F.\text{module}.\text{elemaddrs}[x]].\text{elem} = \epsilon)
\end{aligned}$$

4.4.7 Memory Instructions

Note: The alignment `memarg.align` in load and store instructions does not affect the semantics. It is an indication that the offset ea at which the memory is accessed is intended to satisfy the property $ea \bmod 2^{\text{memarg.align}} = 0$. A WebAssembly implementation can use this hint to optimize for the intended use. Unaligned access violating that property is still allowed and must succeed regardless of the annotation. However, it may be substantially slower on some hardware.

`t.load memarg` and `t.loadN_sx memarg`

1. Let F be the `current frame`.
2. Assert: due to `validation`, $F.\text{module}.\text{memaddrs}[0]$ exists.
3. Let a be the `memory address` $F.\text{module}.\text{memaddrs}[0]$.
4. Assert: due to `validation`, $S.\text{mems}[a]$ exists.
5. Let mem be the `memory instance` $S.\text{mems}[a]$.
6. Assert: due to `validation`, a value of `value type` `i32` is on the top of the stack.
7. Pop the value `i32.const` i from the stack.
8. Let ea be the integer $i + \text{memarg.offset}$.
9. If N is not part of the instruction, then:
 - a. Let N be the `bit width` $|t|$ of `number type` t .

10. If $ea + N/8$ is larger than the length of $mem.data$, then:

a. Trap.

11. Let b^* be the byte sequence $mem.data[ea : N/8]$.

12. If N and sx are part of the instruction, then:

a. Let n be the integer for which $bytes_{iN}(n) = b^*$.

b. Let c be the result of computing $extend_{N,|t|}^{sx}(n)$.

13. Else:

a. Let c be the constant for which $bytes_t(c) = b^*$.

14. Push the value $t.const\ c$ to the stack.

$$\begin{aligned}
 S; F; (i32.const\ i)\ (t.load\ memarg) &\hookrightarrow S; F; (t.const\ c) \\
 &\text{(if } ea = i + memarg.offset \\
 &\quad \wedge ea + |t|/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge bytes_t(c) = S.mems[F.module.memaddrs[0]].data[ea : |t|/8]) \\
 S; F; (i32.const\ i)\ (t.loadN_sx\ memarg) &\hookrightarrow S; F; (t.const\ extend_{N,|t|}^{sx}(n)) \\
 &\text{(if } ea = i + memarg.offset \\
 &\quad \wedge ea + N/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge bytes_{iN}(n) = S.mems[F.module.memaddrs[0]].data[ea : N/8]) \\
 S; F; (i32.const\ i)\ (t.load(N_sx)^2\ memarg) &\hookrightarrow S; F; trap \\
 &\text{(otherwise)}
 \end{aligned}$$

$v128.loadM \times N_sx\ memarg$

1. Let F be the current frame.

2. Assert: due to validation, $F.module.memaddrs[0]$ exists.

3. Let a be the memory address $F.module.memaddrs[0]$.

4. Assert: due to validation, $S.mems[a]$ exists.

5. Let mem be the memory instance $S.mems[a]$.

6. Assert: due to validation, a value of value type $i32$ is on the top of the stack.

7. Pop the value $i32.const\ i$ from the stack.

8. Let ea be the integer $i + memarg.offset$.

9. If $ea + M \cdot N/8$ is larger than the length of $mem.data$, then:

a. Trap.

10. Let b^* be the byte sequence $mem.data[ea : M \cdot N/8]$.

11. Let m_k be the integer for which $bytes_{iM}(m_k) = b^*[k \cdot M/8 : M/8]$.

12. Let W be the integer $M \cdot 2$.

13. Let n_k be the result of computing $extend_{M,W}^{sx}(m_k)$.

14. Let c be the result of computing $lanes_{iW \times N}^{-1}(n_0 \dots n_{N-1})$.

15. Push the value $v128.const\ c$ to the stack.

$$\begin{aligned}
& S; F; (\text{i32.const } i) (\text{v128.loadMxN_sx memarg}) \hookrightarrow S; F; (\text{v128.const } c) \\
& \quad (\text{if } ea = i + \text{memarg.offset} \\
& \quad \wedge ea + M \cdot N/8 \leq |S.\text{mems}[F.\text{module.memaddrs}[0]].\text{data}| \\
& \quad \wedge \text{bytes}_{iM}(m_k) = S.\text{mems}[F.\text{module.memaddrs}[0]].\text{data}[ea + k \cdot M/8 : M/8] \\
& \quad \wedge W = M \cdot 2 \\
& \quad \wedge c = \text{lanes}_{iW \times N}^{-1}(\text{extend}_{M,W}^{sx}(m_0) \dots \text{extend}_{M,W}^{sx}(m_{N-1}))) \\
& S; F; (\text{i32.const } i) (\text{v128.loadMxN_sx memarg}) \hookrightarrow S; F; \text{trap} \\
& \quad (\text{otherwise})
\end{aligned}$$

v128.loadN_splat memarg

1. Let F be the [current frame](#).
2. Assert: due to [validation](#), $F.\text{module.memaddrs}[0]$ exists.
3. Let a be the [memory address](#) $F.\text{module.memaddrs}[0]$.
4. Assert: due to [validation](#), $S.\text{mems}[a]$ exists.
5. Let mem be the [memory instance](#) $S.\text{mems}[a]$.
6. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
7. Pop the value [i32.const](#) i from the stack.
8. Let ea be the integer $i + \text{memarg.offset}$.
9. If $ea + N/8$ is larger than the length of $mem.\text{data}$, then:
 - a. Trap.
10. Let b^* be the byte sequence $mem.\text{data}[ea : N/8]$.
11. Let n be the integer for which $\text{bytes}_{iN}(n) = b^*$.
12. Let L be the integer $128/N$.
13. Let c be the result of computing $\text{lanes}_{iN \times L}^{-1}(n^L)$.
14. Push the value [v128.const](#) c to the stack.

$$\begin{aligned}
& S; F; (\text{i32.const } i) (\text{v128.loadN_splat memarg}) \hookrightarrow S; F; (\text{v128.const } c) \\
& \quad (\text{if } ea = i + \text{memarg.offset} \\
& \quad \wedge ea + N/8 \leq |S.\text{mems}[F.\text{module.memaddrs}[0]].\text{data}| \\
& \quad \wedge \text{bytes}_{iN}(n) = S.\text{mems}[F.\text{module.memaddrs}[0]].\text{data}[ea : N/8] \\
& \quad \wedge c = \text{lanes}_{iN \times L}^{-1}(n^L)) \\
& S; F; (\text{i32.const } i) (\text{v128.loadN_splat memarg}) \hookrightarrow S; F; \text{trap} \\
& \quad (\text{otherwise})
\end{aligned}$$

v128.loadN_zero memarg

1. Let F be the [current frame](#).
2. Assert: due to [validation](#), $F.\text{module.memaddrs}[0]$ exists.
3. Let a be the [memory address](#) $F.\text{module.memaddrs}[0]$.
4. Assert: due to [validation](#), $S.\text{mems}[a]$ exists.
5. Let mem be the [memory instance](#) $S.\text{mems}[a]$.
6. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
7. Pop the value [i32.const](#) i from the stack.
8. Let ea be the integer $i + \text{memarg.offset}$.

9. If $ea + N/8$ is larger than the length of $mem.data$, then:
 - a. Trap.
10. Let b^* be the byte sequence $mem.data[ea : N/8]$.
11. Let n be the integer for which $bytes_{iN}(n) = b^*$.
12. Let c be the result of computing $extend^u_{N,128}(n)$.
13. Push the value `v128.const` c to the stack.

$$\begin{aligned}
 S; F; (i32.const\ i) (v128.loadN_zero\ memarg) &\hookrightarrow S; F; (v128.const\ c) \\
 &\quad (\text{if } ea = i + memarg.offset \\
 &\quad \wedge ea + N/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge bytes_{iN}(n) = S.mems[F.module.memaddrs[0]].data[ea : N/8] \\
 &\quad \wedge c = extend^u_{N,128}(n)) \\
 S; F; (i32.const\ i) (v128.loadN_zero\ memarg) &\hookrightarrow S; F; trap \\
 (\text{otherwise})
 \end{aligned}$$

`v128.loadN_lane` $memarg\ x$

1. Let F be the `current frame`.
2. Assert: due to `validation`, $F.module.memaddrs[0]$ exists.
3. Let a be the `memory address` $F.module.memaddrs[0]$.
4. Assert: due to `validation`, $S.mems[a]$ exists.
5. Let mem be the `memory instance` $S.mems[a]$.
6. Assert: due to `validation`, a value of `value type` `v128` is on the top of the stack.
7. Pop the value `v128.const` v from the stack.
8. Assert: due to `validation`, a value of `value type` `i32` is on the top of the stack.
9. Pop the value `i32.const` i from the stack.
10. Let ea be the integer $i + memarg.offset$.
11. If $ea + N/8$ is larger than the length of $mem.data$, then:
 - a. Trap.
12. Let b^* be the byte sequence $mem.data[ea : N/8]$.
13. Let r be the constant for which $bytes_{iN}(r) = b^*$.
14. Let L be $128/N$.
15. Let j^* be the result of computing $lanes_{iN \times L}(v)$.
16. Let c be the result of computing $lanes_{iN \times L}^{-1}(j^* \text{ with } [x] = r)$.
17. Push the value `v128.const` c to the stack.

$$\begin{aligned}
 S; F; (i32.const\ i) (v128.const\ v) (v128.loadN_lane\ memarg\ x) &\hookrightarrow S; F; (v128.const\ c) \\
 &\quad (\text{if } ea = i + memarg.offset \\
 &\quad \wedge ea + N/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge bytes_{iN}(r) = S.mems[F.module.memaddrs[0]].data[ea : N/8] \\
 &\quad \wedge L = 128/N \\
 &\quad \wedge c = lanes_{iN \times L}^{-1}(lanes_{iN \times L}(v) \text{ with } [x] = r)) \\
 S; F; (i32.const\ i) (v128.const\ v) (v128.loadN_lane\ memarg\ x) &\hookrightarrow S; F; trap \\
 (\text{otherwise})
 \end{aligned}$$

t.store memarg and t.storeN memarg

1. Let F be the current frame.
2. Assert: due to validation, $F.module.memaddrs[0]$ exists.
3. Let a be the memory address $F.module.memaddrs[0]$.
4. Assert: due to validation, $S.mems[a]$ exists.
5. Let mem be the memory instance $S.mems[a]$.
6. Assert: due to validation, a value of value type t is on the top of the stack.
7. Pop the value $t.const\ c$ from the stack.
8. Assert: due to validation, a value of value type $i32$ is on the top of the stack.
9. Pop the value $i32.const\ i$ from the stack.
10. Let ea be the integer $i + memarg.offset$.
11. If N is not part of the instruction, then:
 - a. Let N be the bit width $|t|$ of number type t .
12. If $ea + N/8$ is larger than the length of $mem.data$, then:
 - a. Trap.
13. If N is part of the instruction, then:
 - a. Let n be the result of computing $wrap_{|t|,N}(c)$.
 - b. Let b^* be the byte sequence $bytes_{iN}(n)$.
14. Else:
 - a. Let b^* be the byte sequence $bytes_t(c)$.
15. Replace the bytes $mem.data[ea : N/8]$ with b^* .

$$\begin{aligned}
 S; F; (i32.const\ i)\ (t.const\ c)\ (t.store\ memarg) &\hookrightarrow S'; F; \epsilon \\
 &\text{(if } ea = i + memarg.offset \\
 &\quad \wedge ea + |t|/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge S' = S \text{ with } mems[F.module.memaddrs[0]].data[ea : |t|/8] = bytes_t(c)) \\
 S; F; (i32.const\ i)\ (t.const\ c)\ (t.storeN\ memarg) &\hookrightarrow S'; F; \epsilon \\
 &\text{(if } ea = i + memarg.offset \\
 &\quad \wedge ea + N/8 \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge S' = S \text{ with } mems[F.module.memaddrs[0]].data[ea : N/8] = bytes_{iN}(wrap_{|t|,N}(c))) \\
 S; F; (i32.const\ i)\ (t.const\ c)\ (t.storeN^?\ memarg) &\hookrightarrow S; F; trap \\
 &\text{(otherwise)}
 \end{aligned}$$
v128.storeN_lane memarg x

1. Let F be the current frame.
2. Assert: due to validation, $F.module.memaddrs[0]$ exists.
3. Let a be the memory address $F.module.memaddrs[0]$.
4. Assert: due to validation, $S.mems[a]$ exists.
5. Let mem be the memory instance $S.mems[a]$.
6. Assert: due to validation, a value of value type $v128$ is on the top of the stack.
7. Pop the value $v128.const\ c$ from the stack.

8. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
9. Pop the value `i32.const i` from the stack.
10. Let ea be the integer $i + memarg.offset$.
11. If $ea + N/8$ is larger than the length of `mem.data`, then:
 - a. Trap.
12. Let L be $128/N$.
13. Let j^* be the result of computing `lanesiN×L(c)`.
14. Let b^* be the result of computing `bytesiN(j*[x])`.
15. Replace the bytes `mem.data[ea : N/8]` with b^* .

$$\begin{aligned}
 S; F; (i32.const\ i) (v128.const\ c) (v128.storeN_lane\ memarg\ x) &\hookrightarrow S'; F; \epsilon \\
 &(\text{if } ea = i + memarg.offset \\
 &\quad \wedge ea + N \leq |S.mems[F.module.memaddrs[0]].data| \\
 &\quad \wedge L = 128/N \\
 &\quad \wedge S' = S \text{ with } mems[F.module.memaddrs[0]].data[ea : N/8] = bytes_{iN}(lanes_{iN \times L}(c)[x])) \\
 S; F; (i32.const\ i) (v128.const\ c) (v128.storeN_lane\ memarg\ x) &\hookrightarrow S; F; trap \\
 &(\text{otherwise})
 \end{aligned}$$

memory.size

1. Let F be the current frame.
2. Assert: due to [validation](#), `F.module.memaddrs[0]` exists.
3. Let a be the [memory address](#) `F.module.memaddrs[0]`.
4. Assert: due to [validation](#), `S.mems[a]` exists.
5. Let mem be the [memory instance](#) `S.mems[a]`.
6. Let sz be the length of `mem.data` divided by the [page size](#).
7. Push the value `i32.const sz` to the stack.

$$\begin{aligned}
 S; F; memory.size &\hookrightarrow S; F; (i32.const\ sz) \\
 &(\text{if } |S.mems[F.module.memaddrs[0]].data| = sz \cdot 64\text{ Ki})
 \end{aligned}$$

memory.grow

1. Let F be the current frame.
2. Assert: due to [validation](#), `F.module.memaddrs[0]` exists.
3. Let a be the [memory address](#) `F.module.memaddrs[0]`.
4. Assert: due to [validation](#), `S.mems[a]` exists.
5. Let mem be the [memory instance](#) `S.mems[a]`.
6. Let sz be the length of `S.mems[a]` divided by the [page size](#).
7. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
8. Pop the value `i32.const n` from the stack.
9. Let err be the *i32* value $2^{32} - 1$, for which `signed32(err)` is -1 .
10. Either:
 - a. If [growing](#) mem by n [pages](#) succeeds, then:

- i. Push the value `i32.const sz` to the stack.
- b. Else:
 - i. Push the value `i32.const err` to the stack.
- 11. Or:
 - a. Push the value `i32.const err` to the stack.

$$\begin{aligned}
 S; F; (i32.const\ n)\ \text{memory.grow} &\hookrightarrow S'; F; (i32.const\ sz) \\
 &\quad (\text{if } F.\text{module.memaddrs}[0] = a \\
 &\quad \wedge\ sz = |S.\text{mems}[a].\text{data}|/64\text{ Ki} \\
 &\quad \wedge\ S' = S\ \text{with}\ \text{mems}[a] = \text{growmem}(S.\text{mems}[a], n)) \\
 S; F; (i32.const\ n)\ \text{memory.grow} &\hookrightarrow S; F; (i32.const\ \text{signed}_{32}^{-1}(-1))
 \end{aligned}$$

Note: The `memory.grow` instruction is non-deterministic. It may either succeed, returning the old memory size `sz`, or fail, returning `-1`. Failure *must* occur if the referenced memory instance has a maximum size defined that would be exceeded. However, failure *can* occur in other cases as well. In practice, the choice depends on the `resources` available to the `embedder`.

`memory.fill`

1. Let F be the current frame.
2. Assert: due to `validation`, $F.\text{module.memaddrs}[0]$ exists.
3. Let ma be the memory address $F.\text{module.memaddrs}[0]$.
4. Assert: due to `validation`, $S.\text{mems}[ma]$ exists.
5. Let mem be the memory instance $S.\text{mems}[ma]$.
6. Assert: due to `validation`, a value of value type `i32` is on the top of the stack.
7. Pop the value `i32.const n` from the stack.
8. Assert: due to `validation`, a value of value type `i32` is on the top of the stack.
9. Pop the value val from the stack.
10. Assert: due to `validation`, a value of value type `i32` is on the top of the stack.
11. Pop the value `i32.const d` from the stack.
12. If $d + n$ is larger than the length of $mem.\text{data}$, then:
 - a. Trap.
13. If $n = 0$, then:
 - a. Return.
14. Push the value `i32.const d` to the stack.
15. Push the value val to the stack.
16. Execute the instruction `i32.store8 {offset 0, align 0}`.
17. Assert: due to the earlier check against the memory size, $d + 1 < 2^{32}$.
18. Push the value `i32.const (d + 1)` to the stack.
19. Push the value val to the stack.
20. Push the value `i32.const (n - 1)` to the stack.
21. Execute the instruction `memory.fill`.

$$\begin{aligned}
& S; F; (i32.\text{const } d) \text{ val } (i32.\text{const } n) \text{ memory.fill} \hookrightarrow S; F; \text{trap} \\
& \quad (\text{if } d + n > |S.\text{mems}[F.\text{module}.\text{memaddrs}[0]].\text{data}|) \\
& S; F; (i32.\text{const } d) \text{ val } (i32.\text{const } 0) \text{ memory.fill} \hookrightarrow S; F; \epsilon \\
& \quad (\text{otherwise}) \\
& S; F; (i32.\text{const } d) \text{ val } (i32.\text{const } n + 1) \text{ memory.fill} \hookrightarrow \\
& \quad S; F; (i32.\text{const } d) \text{ val } (i32.\text{store8 } \{\text{offset } 0, \text{align } 0\}) \\
& \quad (i32.\text{const } d + 1) \text{ val } (i32.\text{const } n) \text{ memory.fill} \\
& \quad (\text{otherwise})
\end{aligned}$$

memory.copy

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module}.\text{memaddrs}[0]$ exists.
3. Let ma be the **memory address** $F.\text{module}.\text{memaddrs}[0]$.
4. Assert: due to **validation**, $S.\text{mems}[ma]$ exists.
5. Let mem be the **memory instance** $S.\text{mems}[ma]$.
6. Assert: due to **validation**, a value of **value type** **i32** is on the top of the stack.
7. Pop the value **i32.const** n from the stack.
8. Assert: due to **validation**, a value of **value type** **i32** is on the top of the stack.
9. Pop the value **i32.const** s from the stack.
10. Assert: due to **validation**, a value of **value type** **i32** is on the top of the stack.
11. Pop the value **i32.const** d from the stack.
12. If $s + n$ is larger than the length of $mem.\text{data}$ or $d + n$ is larger than the length of $mem.\text{data}$, then:
 - a. Trap.
13. If $n = 0$, then:
 - a. Return.
14. If $d \leq s$, then:
 - a. Push the value **i32.const** d to the stack.
 - b. Push the value **i32.const** s to the stack.
 - c. Execute the instruction **i32.load8_u** $\{\text{offset } 0, \text{align } 0\}$.
 - d. Execute the instruction **i32.store8** $\{\text{offset } 0, \text{align } 0\}$.
 - e. Assert: due to the earlier check against the memory size, $d + 1 < 2^{32}$.
 - f. Push the value **i32.const** $(d + 1)$ to the stack.
 - g. Assert: due to the earlier check against the memory size, $s + 1 < 2^{32}$.
 - h. Push the value **i32.const** $(s + 1)$ to the stack.
15. Else:
 - a. Assert: due to the earlier check against the memory size, $d + n - 1 < 2^{32}$.
 - b. Push the value **i32.const** $(d + n - 1)$ to the stack.
 - c. Assert: due to the earlier check against the memory size, $s + n - 1 < 2^{32}$.
 - d. Push the value **i32.const** $(s + n - 1)$ to the stack.
 - e. Execute the instruction **i32.load8_u** $\{\text{offset } 0, \text{align } 0\}$.

- f. Execute the instruction `i32.store8 {offset 0, align 0}`.
- g. Push the value `i32.const d` to the stack.
- h. Push the value `i32.const s` to the stack.
- 16. Push the value `i32.const (n - 1)` to the stack.
- 17. Execute the instruction `memory.copy`.

$$\begin{aligned}
& S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n)\ \text{memory.copy} \quad \hookrightarrow \quad S; F; \text{trap} \\
& \quad (\text{if } s + n > |S.mems[F.module.memaddrs[0]].data| \\
& \quad \quad \vee d + n > |S.mems[F.module.memaddrs[0]].data|) \\
& S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ 0)\ \text{memory.copy} \quad \hookrightarrow \quad S; F; \epsilon \\
& \quad (\text{otherwise}) \\
& S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n + 1)\ \text{memory.copy} \quad \hookrightarrow \\
& \quad S; F; (i32.const\ d) \\
& \quad \quad (i32.const\ s)\ (i32.load8_u\ \{\text{offset } 0, \text{align } 0\}) \\
& \quad \quad (i32.store8\ \{\text{offset } 0, \text{align } 0\}) \\
& \quad \quad (i32.const\ d + 1)\ (i32.const\ s + 1)\ (i32.const\ n)\ \text{memory.copy} \\
& \quad (\text{otherwise, if } d \leq s) \\
& S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n + 1)\ \text{memory.copy} \quad \hookrightarrow \\
& \quad S; F; (i32.const\ d + n) \\
& \quad \quad (i32.const\ s + n)\ (i32.load8_u\ \{\text{offset } 0, \text{align } 0\}) \\
& \quad \quad (i32.store8\ \{\text{offset } 0, \text{align } 0\}) \\
& \quad \quad (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n)\ \text{memory.copy} \\
& \quad (\text{otherwise, if } d > s)
\end{aligned}$$

`memory.init x`

1. Let F be the current frame.
2. Assert: due to validation, $F.module.memaddrs[0]$ exists.
3. Let ma be the memory address $F.module.memaddrs[0]$.
4. Assert: due to validation, $S.mems[ma]$ exists.
5. Let mem be the memory instance $S.mems[ma]$.
6. Assert: due to validation, $F.module.dataaddrs[x]$ exists.
7. Let da be the data address $F.module.dataaddrs[x]$.
8. Assert: due to validation, $S.datas[da]$ exists.
9. Let $data$ be the data instance $S.datas[da]$.
10. Assert: due to validation, a value of value type `i32` is on the top of the stack.
11. Pop the value `i32.const n` from the stack.
12. Assert: due to validation, a value of value type `i32` is on the top of the stack.
13. Pop the value `i32.const s` from the stack.
14. Assert: due to validation, a value of value type `i32` is on the top of the stack.
15. Pop the value `i32.const d` from the stack.
16. If $s + n$ is larger than the length of $data.data$ or $d + n$ is larger than the length of $mem.data$, then:
 - a. Trap.
17. If $n = 0$, then:
 - a. Return.

18. Let b be the byte $data.data[s]$.
19. Push the value $i32.const\ d$ to the stack.
20. Push the value $i32.const\ b$ to the stack.
21. Execute the instruction $i32.store8\ \{\text{offset } 0, \text{align } 0\}$.
22. Assert: due to the earlier check against the memory size, $d + 1 < 2^{32}$.
23. Push the value $i32.const\ (d + 1)$ to the stack.
24. Assert: due to the earlier check against the memory size, $s + 1 < 2^{32}$.
25. Push the value $i32.const\ (s + 1)$ to the stack.
26. Push the value $i32.const\ (n - 1)$ to the stack.
27. Execute the instruction $memory.init\ x$.

$$\begin{aligned}
 S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n)\ (memory.init\ x) &\hookrightarrow S; F; \text{trap} \\
 &\quad (\text{if } s + n > |S.datas[F.module.dataaddrs[x]].data| \\
 &\quad \vee d + n > |S.mems[F.module.memaddrs[0]].data|) \\
 S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ 0)\ (memory.init\ x) &\hookrightarrow S; F; \epsilon \\
 &\quad (\text{otherwise}) \\
 S; F; (i32.const\ d)\ (i32.const\ s)\ (i32.const\ n + 1)\ (memory.init\ x) &\hookrightarrow \\
 S; F; (i32.const\ d)\ (i32.const\ b)\ (i32.store8\ \{\text{offset } 0, \text{align } 0\}) & \\
 (i32.const\ d + 1)\ (i32.const\ s + 1)\ (i32.const\ n)\ (memory.init\ x) & \\
 (\text{otherwise, if } b = S.datas[F.module.dataaddrs[x]].data[s]) &
 \end{aligned}$$

`data.drop` x

1. Let F be the `current frame`.
2. Assert: due to `validation`, $F.module.dataaddrs[x]$ exists.
3. Let a be the `data address` $F.module.dataaddrs[x]$.
4. Assert: due to `validation`, $S.datas[a]$ exists.
5. Replace $S.datas[a]$ with the `data instance` $\{\text{data } \epsilon\}$.

$$\begin{aligned}
 S; F; (\text{data.drop } x) &\hookrightarrow S'; F; \epsilon \\
 &\quad (\text{if } S' = S \text{ with } datas[F.module.dataaddrs[x]] = \{\text{data } \epsilon\})
 \end{aligned}$$

4.4.8 Control Instructions

`nop`

1. Do nothing.

$$\text{nop} \hookrightarrow \epsilon$$

unreachable

1. Trap.

$$\text{unreachable} \hookrightarrow \text{trap}$$

block *blocktype* *instr** end

1. Let F be the **current frame**.
2. Assert: due to **validation**, $\text{expand}_F(\text{blocktype})$ is defined.
3. Let $[t_1^m] \rightarrow [t_2^n]$ be the **function type** $\text{expand}_F(\text{blocktype})$.
4. Let L be the label whose arity is n and whose continuation is the end of the block.
5. Assert: due to **validation**, there are at least m values on the top of the stack.
6. Pop the values val^m from the stack.
7. **Enter** the block $\text{val}^m \text{ instr}^*$ with label L .

$$F; \text{val}^m \text{ block } bt \text{ instr}^* \text{ end} \hookrightarrow F; \text{label}_n\{\epsilon\} \text{val}^m \text{ instr}^* \text{ end} \\ (\text{if } \text{expand}_F(bt) = [t_1^m] \rightarrow [t_2^n])$$

loop *blocktype* *instr** end

1. Let F be the **current frame**.
2. Assert: due to **validation**, $\text{expand}_F(\text{blocktype})$ is defined.
3. Let $[t_1^m] \rightarrow [t_2^n]$ be the **function type** $\text{expand}_F(\text{blocktype})$.
4. Let L be the label whose arity is m and whose continuation is the start of the loop.
5. Assert: due to **validation**, there are at least m values on the top of the stack.
6. Pop the values val^m from the stack.
7. **Enter** the block $\text{val}^m \text{ instr}^*$ with label L .

$$F; \text{val}^m \text{ loop } bt \text{ instr}^* \text{ end} \hookrightarrow F; \text{label}_m\{\text{loop } bt \text{ instr}^* \text{ end}\} \text{val}^m \text{ instr}^* \text{ end} \\ (\text{if } \text{expand}_F(bt) = [t_1^m] \rightarrow [t_2^n])$$

if *blocktype* *instr**₁ else *instr**₂ end

1. Assert: due to **validation**, a value of **value type** i32 is on the top of the stack.
2. Pop the value **i32.const** c from the stack.
3. If c is non-zero, then:
 - a. Execute the block instruction **block** *blocktype* *instr**₁ end.
4. Else:
 - a. Execute the block instruction **block** *blocktype* *instr**₂ end.

$$(\text{i32.const } c) \text{ if } bt \text{ instr}_1^* \text{ else } \text{instr}_2^* \text{ end} \hookrightarrow \text{block } bt \text{ instr}_1^* \text{ end} \\ (\text{if } c \neq 0) \\ (\text{i32.const } c) \text{ if } bt \text{ instr}_1^* \text{ else } \text{instr}_2^* \text{ end} \hookrightarrow \text{block } bt \text{ instr}_2^* \text{ end} \\ (\text{if } c = 0)$$

`br l`

1. Assert: due to [validation](#), the stack contains at least $l + 1$ labels.
2. Let L be the l -th label appearing on the stack, starting from the top and counting from zero.
3. Let n be the arity of L .
4. Assert: due to [validation](#), there are at least n values on the top of the stack.
5. Pop the values val^n from the stack.
6. Repeat $l + 1$ times:
 - a. While the top of the stack is a value, do:
 - i. Pop the value from the stack.
 - b. Assert: due to [validation](#), the top of the stack now is a label.
 - c. Pop the label from the stack.
7. Push the values val^n to the stack.
8. Jump to the continuation of L .

$$\text{label}_n\{instr^*\} B^l[val^n (\text{br } l)] \text{ end} \hookrightarrow val^n instr^*$$

`br_if l`

1. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
2. Pop the value `i32.const c` from the stack.
3. If c is non-zero, then:
 - a. [Execute](#) the instruction `br l`.
4. Else:
 - a. Do nothing.

$$\begin{aligned} (\text{i32.const } c) (\text{br_if } l) &\hookrightarrow (\text{br } l) && (\text{if } c \neq 0) \\ (\text{i32.const } c) (\text{br_if } l) &\hookrightarrow \epsilon && (\text{if } c = 0) \end{aligned}$$

`br_table l* lN`

1. Assert: due to [validation](#), a value of [value type i32](#) is on the top of the stack.
2. Pop the value `i32.const i` from the stack.
3. If i is smaller than the length of l^* , then:
 - a. Let l_i be the label $l^*[i]$.
 - b. [Execute](#) the instruction `br li`.
4. Else:
 - a. [Execute](#) the instruction `br lN`.

$$\begin{aligned} (\text{i32.const } i) (\text{br_table } l^* l_N) &\hookrightarrow (\text{br } l_i) && (\text{if } l^*[i] = l_i) \\ (\text{i32.const } i) (\text{br_table } l^* l_N) &\hookrightarrow (\text{br } l_N) && (\text{if } |l^*| \leq i) \end{aligned}$$

return

1. Let F be the **current frame**.
2. Let n be the arity of F .
3. Assert: due to **validation**, there are at least n values on the top of the stack.
4. Pop the results val^n from the stack.
5. Assert: due to **validation**, the stack contains at least one **frame**.
6. While the top of the stack is not a frame, do:
 - a. Pop the top element from the stack.
7. Assert: the top of the stack is the frame F .
8. Pop the frame from the stack.
9. Push val^n to the stack.
10. Jump to the instruction after the original call that pushed the frame.

$$\text{frame}_n\{F\} B^k[val^n \text{ return}] \text{ end} \hookrightarrow val^n$$

call x

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module.funcaddrs}[x]$ exists.
3. Let a be the **function address** $F.\text{module.funcaddrs}[x]$.
4. **Invoke** the function instance at address a .

$$F; (\text{call } x) \hookrightarrow F; (\text{invoke } a) \quad (\text{if } F.\text{module.funcaddrs}[x] = a)$$

call_indirect x y

1. Let F be the **current frame**.
2. Assert: due to **validation**, $F.\text{module.tableaddrs}[x]$ exists.
3. Let ta be the **table address** $F.\text{module.tableaddrs}[x]$.
4. Assert: due to **validation**, $S.\text{tables}[ta]$ exists.
5. Let tab be the **table instance** $S.\text{tables}[ta]$.
6. Assert: due to **validation**, $F.\text{module.types}[y]$ exists.
7. Let ft_{expect} be the **function type** $F.\text{module.types}[y]$.
8. Assert: due to **validation**, a value with **value type** **i32** is on the top of the stack.
9. Pop the value **i32.const** i from the stack.
10. If i is not smaller than the length of $tab.\text{elem}$, then:
 - a. Trap.
11. Let r be the **reference** $tab.\text{elem}[i]$.
12. If r is **ref.null** t , then:
 - a. Trap.
13. Assert: due to **validation of table mutation**, r is a **function reference**.

14. Let $\text{ref } a$ be the function reference r .
15. Assert: due to validation of table mutation, $S.\text{funcs}[a]$ exists.
16. Let f be the function instance $S.\text{funcs}[a]$.
17. Let ft_{actual} be the function type $f.\text{type}$.
18. If ft_{actual} and ft_{expect} differ, then:
 - a. Trap.
19. Invoke the function instance at address a .

$$\begin{aligned}
 S; F; (\text{i32.const } i) (\text{call_indirect } x \ y) &\hookrightarrow S; F; (\text{invoke } a) \\
 &\quad (\text{if } S.\text{tables}[F.\text{module.tableaddrs}[x]].\text{elem}[i] = \text{ref } a \\
 &\quad \wedge S.\text{funcs}[a] = f \\
 &\quad \wedge F.\text{module.types}[y] = f.\text{type}) \\
 S; F; (\text{i32.const } i) (\text{call_indirect } x \ y) &\hookrightarrow S; F; \text{trap} \\
 &\quad (\text{otherwise})
 \end{aligned}$$

4.4.9 Blocks

The following auxiliary rules define the semantics of executing an instruction sequence that forms a block.

Entering instr^* with label L

1. Push L to the stack.
2. Jump to the start of the instruction sequence instr^* .

Note: No formal reduction rule is needed for entering an instruction sequence, because the label L is embedded in the administrative instruction that structured control instructions reduce to directly.

Exiting instr^* with label L

When the end of a block is reached without a jump or trap aborting it, then the following steps are performed.

1. Pop all values val^* from the top of the stack.
2. Assert: due to validation, the label L is now on the top of the stack.
3. Pop the label from the stack.
4. Push val^* back to the stack.
5. Jump to the position after the end of the structured control instruction associated with the label L .

$$\text{label}_n\{\text{instr}^*\} \ \text{val}^* \ \text{end} \hookrightarrow \text{val}^*$$

Note: This semantics also applies to the instruction sequence contained in a loop instruction. Therefore, execution of a loop falls off the end, unless a backwards branch is performed explicitly.

4.4.10 Function Calls

The following auxiliary rules define the semantics of invoking a **function** instance through one of the **call instructions** and returning from it.

Invocation of function address a

1. Assert: due to **validation**, $S.\text{funcs}[a]$ exists.
2. Let f be the **function** instance, $S.\text{funcs}[a]$.
3. Let $[t_1^n] \rightarrow [t_2^m]$ be the **function type** $f.\text{type}$.
4. Let t^* be the list of **value types** $f.\text{code}.\text{locals}$.
5. Let $\text{instr}^* \text{ end}$ be the **expression** $f.\text{code}.\text{body}$.
6. Assert: due to **validation**, n values are on the top of the stack.
7. Pop the values val^n from the stack.
8. Let F be the **frame** $\{\text{module } f.\text{module}, \text{locals } \text{val}^n (\text{default}_t)^*\}$.
9. Push the activation of F with arity m to the stack.
10. Let L be the **label** whose arity is m and whose continuation is the end of the function.
11. **Enter** the instruction sequence instr^* with label L .

$$\begin{aligned}
 S; \text{val}^n (\text{invoke } a) &\hookrightarrow S; \text{frame}_m\{F\} \text{ label}_m\{\} \text{ instr}^* \text{ end end} \\
 &\quad (\text{if } S.\text{funcs}[a] = f \\
 &\quad \wedge f.\text{type} = [t_1^n] \rightarrow [t_2^m] \\
 &\quad \wedge f.\text{code} = \{\text{type } x, \text{locals } t^k, \text{body } \text{instr}^* \text{ end}\} \\
 &\quad \wedge F = \{\text{module } f.\text{module}, \text{locals } \text{val}^n (\text{default}_t)^k\})
 \end{aligned}$$

Returning from a function

When the end of a function is reached without a jump (i.e., **return**) or trap aborting it, then the following steps are performed.

1. Let F be the **current frame**.
2. Let n be the arity of the activation of F .
3. Assert: due to **validation**, there are n values on the top of the stack.
4. Pop the results val^n from the stack.
5. Assert: due to **validation**, the frame F is now on the top of the stack.
6. Pop the frame from the stack.
7. Push val^n back to the stack.
8. Jump to the instruction after the original call.

$$\text{frame}_n\{F\} \text{ val}^n \text{ end} \hookrightarrow \text{val}^n$$

Host Functions

Invoking a [host function](#) has non-deterministic behavior. It may either terminate with a [trap](#) or return regularly. However, in the latter case, it must consume and produce the right number and types of WebAssembly [values](#) on the stack, according to its [function type](#).

A host function may also modify the [store](#). However, all store modifications must result in an [extension](#) of the original store, i.e., they must only modify mutable contents and must not have instances removed. Furthermore, the resulting store must be [valid](#), i.e., all data and code in it is well-typed.

$$\begin{aligned}
 S; \text{val}^n (\text{invoke } a) &\hookrightarrow S'; \text{result} \\
 &\quad (\text{if } S.\text{funcs}[a] = \{\text{type } [t_1^n] \rightarrow [t_2^m], \text{hostcode } hf\} \\
 &\quad \quad \wedge (S'; \text{result}) \in hf(S; \text{val}^n)) \\
 S; \text{val}^n (\text{invoke } a) &\hookrightarrow S; \text{val}^n (\text{invoke } a) \\
 &\quad (\text{if } S.\text{funcs}[a] = \{\text{type } [t_1^n] \rightarrow [t_2^m], \text{hostcode } hf\} \\
 &\quad \quad \wedge \perp \in hf(S; \text{val}^n))
 \end{aligned}$$

Here, $hf(S; \text{val}^n)$ denotes the implementation-defined execution of host function hf in current store S with arguments val^n . It yields a set of possible outcomes, where each element is either a pair of a modified store S' and a [result](#) or the special value $\perp$ indicating divergence. A host function is non-deterministic if there is at least one argument for which the set of outcomes is not singular.

For a WebAssembly implementation to be [sound](#) in the presence of host functions, every [host function instance](#) must be [valid](#), which means that it adheres to suitable pre- and post-conditions: under a [valid store](#) S , and given arguments val^n matching the ascribed parameter types t_1^n , executing the host function must yield a non-empty set of possible outcomes each of which is either divergence or consists of a valid store S' that is an [extension](#) of S and a result matching the ascribed return types t_2^m . All these notions are made precise in the [Appendix](#).

Note: A host function can call back into WebAssembly by [invoking](#) a function [exported](#) from a [module](#). However, the effects of any such call are subsumed by the non-deterministic behavior allowed for the host function.

4.4.11 Expressions

An [expression](#) is *evaluated* relative to a [current frame](#) pointing to its containing [module instance](#).

1. Jump to the start of the instruction sequence instr^* of the expression.
2. Execute the instruction sequence.
3. Assert: due to [validation](#), the top of the stack contains a [value](#).
4. Pop the [value](#) val from the stack.

The value val is the result of the evaluation.

$$S; F; \text{instr}^* \hookrightarrow S'; F'; \text{instr}'^* \quad (\text{if } S; F; \text{instr}^* \text{ end} \hookrightarrow S'; F'; \text{instr}'^* \text{ end})$$

Note: Evaluation iterates this reduction rule until reaching a value. Expressions constituting [function bodies](#) are executed during [function invocation](#).

4.5 Modules

For modules, the execution semantics primarily defines [instantiation](#), which [allocates](#) instances for a module and its contained definitions, initializes [tables](#) and [memories](#) from contained [element](#) and [data](#) segments, and invokes the [start function](#) if present. It also includes [invocation](#) of exported functions.

Instantiation depends on a number of auxiliary notions for [type-checking imports](#) and [allocating](#) instances.

4.5.1 External Typing

For the purpose of checking [external values](#) against [imports](#), such values are classified by [external types](#). The following auxiliary typing rules specify this typing relation relative to a [store](#) S in which the referenced instances live.

[func](#) a

- The store entry $S.\text{funcs}[a]$ must exist.
- Then [func](#) a is valid with [external type](#) $\text{func } S.\text{funcs}[a].\text{type}$.

$$\frac{}{S \vdash \text{func } a : \text{func } S.\text{funcs}[a].\text{type}}$$

[table](#) a

- The store entry $S.\text{tables}[a]$ must exist.
- Then [table](#) a is valid with [external type](#) $\text{table } S.\text{tables}[a].\text{type}$.

$$\frac{}{S \vdash \text{table } a : \text{table } S.\text{tables}[a].\text{type}}$$

[mem](#) a

- The store entry $S.\text{mems}[a]$ must exist.
- Then [mem](#) a is valid with [external type](#) $\text{mem } S.\text{mems}[a].\text{type}$.

$$\frac{}{S \vdash \text{mem } a : \text{mem } S.\text{mems}[a].\text{type}}$$

[global](#) a

- The store entry $S.\text{globals}[a]$ must exist.
- Then [global](#) a is valid with [external type](#) $\text{global } S.\text{globals}[a].\text{type}$.

$$\frac{}{S \vdash \text{global } a : \text{global } S.\text{globals}[a].\text{type}}$$

4.5.2 Value Typing

For the purpose of checking argument **values** against the parameter types of exported **functions**, values are classified by **value types**. The following auxiliary typing rules specify this typing relation relative to a **store** S in which possibly referenced addresses live.

Numeric Values $t.\text{const } c$

- The value is valid with **number type** t .

$$\frac{}{S \vdash t.\text{const } c : t}$$

Null References $\text{ref.null } t$

- The value is valid with **reference type** t .

$$\frac{}{S \vdash \text{ref.null } t : t}$$

Function References $\text{ref } a$

- The **external value** $\text{func } a$ must be valid.
- Then the value is valid with **reference type** funcref .

$$\frac{S \vdash \text{func } a : \text{func } \text{func_type}}{S \vdash \text{ref } a : \text{funcref}}$$

External References $\text{ref.extern } a$

- The value is valid with **reference type** externref .

$$\frac{}{S \vdash \text{ref.extern } a : \text{externref}}$$

4.5.3 Allocation

New instances of **functions**, **tables**, **memories**, and **globals** are *allocated* in a **store** S , as defined by the following auxiliary functions.

Functions

1. Let func be the **function** to allocate and moduleinst its **module instance**.
2. Let a be the first free **function address** in S .
3. Let func_type be the **function type** $\text{moduleinst.types}[\text{func.type}]$.
4. Let funcinst be the **function instance** $\{\text{type } \text{func_type}, \text{module } \text{moduleinst}, \text{code } \text{func}\}$.
5. Append funcinst to the **funcs** of S .
6. Return a .

$$\text{allocfunc}(S, \text{func}, \text{moduleinst}) = S', \text{funcaddr}$$

where:

$$\begin{aligned} \text{funcaddr} &= |S.\text{funcs}| \\ \text{functype} &= \text{moduleinst}.\text{types}[\text{func.type}] \\ \text{funcinst} &= \{\text{type } \text{functype}, \text{module } \text{moduleinst}, \text{code } \text{func}\} \\ S' &= S \oplus \{\text{funcs } \text{funcinst}\} \end{aligned}$$

Host Functions

1. Let *hostfunc* be the host function to allocate and *functype* its function type.
2. Let *a* be the first free function address in *S*.
3. Let *funcinst* be the function instance $\{\text{type } \text{functype}, \text{hostcode } \text{hostfunc}\}$.
4. Append *funcinst* to the funcs of *S*.
5. Return *a*.

$$\text{allochostfunc}(S, \text{functype}, \text{hostfunc}) = S', \text{funcaddr}$$

where:

$$\begin{aligned} \text{funcaddr} &= |S.\text{funcs}| \\ \text{funcinst} &= \{\text{type } \text{functype}, \text{hostcode } \text{hostfunc}\} \\ S' &= S \oplus \{\text{funcs } \text{funcinst}\} \end{aligned}$$

Note: Host functions are never allocated by the WebAssembly semantics itself, but may be allocated by the embedder.

Tables

1. Let *tabletype* be the table type to allocate and *ref* the initialization value.
2. Let $(\{\min n, \max m^?\} \text{ reftype})$ be the structure of table type *tabletype*.
3. Let *a* be the first free table address in *S*.
4. Let *tableinst* be the table instance $\{\text{type } \text{tabletype}, \text{elem } \text{ref}^n\}$ with *n* elements set to *ref*.
5. Append *tableinst* to the tables of *S*.
6. Return *a*.

$$\text{alloctable}(S, \text{tabletype}, \text{ref}) = S', \text{tableaddr}$$

where:

$$\begin{aligned} \text{tabletype} &= \{\min n, \max m^?\} \text{ reftype} \\ \text{tableaddr} &= |S.\text{tables}| \\ \text{tableinst} &= \{\text{type } \text{tabletype}, \text{elem } \text{ref}^n\} \\ S' &= S \oplus \{\text{tables } \text{tableinst}\} \end{aligned}$$

Memories

1. Let *memtype* be the memory type to allocate.
2. Let $\{\min n, \max m^?\}$ be the structure of memory type *memtype*.
3. Let *a* be the first free memory address in *S*.
4. Let *meminst* be the memory instance $\{\text{type } \textit{memtype}, \text{data } (0x00)^{n \cdot 64 \text{ Ki}}\}$ that contains *n* pages of zeroed bytes.
5. Append *meminst* to the *mems* of *S*.
6. Return *a*.

$$\text{allocmem}(S, \textit{memtype}) = S', \textit{memaddr}$$

where:

$$\begin{aligned} \textit{memtype} &= \{\min n, \max m^?\} \\ \textit{memaddr} &= |S.\textit{mems}| \\ \textit{meminst} &= \{\text{type } \textit{memtype}, \text{data } (0x00)^{n \cdot 64 \text{ Ki}}\} \\ S' &= S \oplus \{\textit{mems } \textit{meminst}\} \end{aligned}$$

Globals

1. Let *globaltype* be the global type to allocate and *val* the value to initialize the global with.
2. Let *a* be the first free global address in *S*.
3. Let *globalinst* be the global instance $\{\text{type } \textit{globaltype}, \text{value } \textit{val}\}$.
4. Append *globalinst* to the *globals* of *S*.
5. Return *a*.

$$\text{allocglobal}(S, \textit{globaltype}, \textit{val}) = S', \textit{globaladdr}$$

where:

$$\begin{aligned} \textit{globaladdr} &= |S.\textit{globals}| \\ \textit{globalinst} &= \{\text{type } \textit{globaltype}, \text{value } \textit{val}\} \\ S' &= S \oplus \{\textit{globals } \textit{globalinst}\} \end{aligned}$$

Element segments

1. Let *reftype* be the elements' type and *ref** the vector of references to allocate.
2. Let *a* be the first free element address in *S*.
3. Let *eleminst* be the element instance $\{\text{type } \textit{reftype}, \text{elem } \textit{ref}^*\}$.
4. Append *eleminst* to the *elems* of *S*.
5. Return *a*.

$$\text{allocelem}(S, \textit{reftype}, \textit{ref}^*) = S', \textit{elemaddr}$$

where:

$$\begin{aligned} \textit{elemaddr} &= |S.\textit{elems}| \\ \textit{eleminst} &= \{\text{type } \textit{reftype}, \text{elem } \textit{ref}^*\} \\ S' &= S \oplus \{\textit{elems } \textit{eleminst}\} \end{aligned}$$

Data segments

1. Let b^* be the vector of **bytes** to allocate.
2. Let a be the first free **data address** in S .
3. Let $datainst$ be the **data instance** $\{\text{data } b^*\}$.
4. Append $datainst$ to the **datas** of S .
5. Return a .

$$\begin{aligned} \text{allocdata}(S, b^*) &= S', dataaddr \\ \text{where:} \\ dataaddr &= |S.datas| \\ datainst &= \{\text{data } b^*\} \\ S' &= S \oplus \{\text{datas } datainst\} \end{aligned}$$

Growing tables

1. Let $tableinst$ be the **table instance** to grow, n the number of elements by which to grow it, and ref the initialization value.
2. Let len be n added to the length of $tableinst.elem$.
3. If len is larger than or equal to 2^{32} , then fail.
4. Let $limits\ t$ be the structure of **table type** $tableinst.type$.
5. Let $limits'$ be $limits$ with **min** updated to len .
6. If $limits'$ is not **valid**, then fail.
7. Append ref^n to $tableinst.elem$.
8. Set $tableinst.type$ to the **table type** $limits'\ t$.

$$\begin{aligned} \text{growtable}(tableinst, n, ref) &= tableinst \text{ with type } = limits'\ t \text{ with elem } = tableinst.elem\ ref^n \\ &\quad (\text{if } len = n + |tableinst.elem| \\ &\quad \wedge len < 2^{32} \\ &\quad \wedge limits\ t = tableinst.type \\ &\quad \wedge limits' = limits \text{ with min } = len \\ &\quad \wedge \vdash limits' \text{ ok}) \end{aligned}$$

Growing memories

1. Let $meminst$ be the **memory instance** to grow and n the number of **pages** by which to grow it.
2. Assert: The length of $meminst.data$ is divisible by the **page size** 64 Ki.
3. Let len be n added to the length of $meminst.data$ divided by the **page size** 64 Ki.
4. If len is larger than 2^{16} , then fail.
5. Let $limits$ be the structure of **memory type** $meminst.type$.
6. Let $limits'$ be $limits$ with **min** updated to len .
7. If $limits'$ is not **valid**, then fail.
8. Append n times 64 Ki **bytes** with value 0x00 to $meminst.data$.
9. Set $meminst.type$ to the **memory type** $limits'$.

$$\begin{aligned}
\text{growmem}(\text{meminst}, n) = & \text{meminst with type} = \text{limits}' \text{ with data} = \text{meminst.data} \text{ (0x00)}^{n \cdot 64 \text{ Ki}} \\
& (\text{if } \text{len} = n + |\text{meminst.data}|/64 \text{ Ki} \\
& \wedge \text{len} \leq 2^{16} \\
& \wedge \text{limits} = \text{meminst.type} \\
& \wedge \text{limits}' = \text{limits with min} = \text{len} \\
& \wedge \vdash \text{limits}' \text{ ok})
\end{aligned}$$

Modules

The allocation function for **modules** requires a suitable list of **external values** that are assumed to **match** the **import** vector of the module, a list of initialization **values** for the module's **globals**, and list of **reference** vectors for the module's **element segments**.

1. Let *module* be the **module** to allocate and $\text{externval}_{\text{im}}^*$ the vector of **external values** providing the module's imports, *val*^{*} the initialization **values** of the module's **globals**, and $(\text{ref}^*)^*$ the **reference** vectors of the module's **element segments**.
2. For each function *func*_{*i*} in *module.funcs*, do:
 - a. Let *funcaddr*_{*i*} be the **function address** resulting from allocating *func*_{*i*} for the module instance *moduleinst* defined below.
3. For each table *table*_{*i*} in *module.tables*, do:
 - a. Let *limits*_{*i*} *t*_{*i*} be the table type *table*_{*i*}.type.
 - b. Let *tableaddr*_{*i*} be the **table address** resulting from allocating *table*_{*i*}.type with initialization value *ref.null t*_{*i*}.
4. For each memory *mem*_{*i*} in *module.mems*, do:
 - a. Let *memaddr*_{*i*} be the **memory address** resulting from allocating *mem*_{*i*}.type.
5. For each global *global*_{*i*} in *module.globals*, do:
 - a. Let *globaladdr*_{*i*} be the **global address** resulting from allocating *global*_{*i*}.type with initializer value *val*^{*}[*i*].
6. For each element segment *elem*_{*i*} in *module elems*, do:
 - a. Let *elemaddr*_{*i*} be the **element address** resulting from allocating an **element instance** of reference type *elem*_{*i*}.type with contents $(\text{ref}^*)^*[i]$.
7. For each data segment *data*_{*i*} in *module.datas*, do:
 - a. Let *dataaddr*_{*i*} be the **data address** resulting from allocating a **data instance** with contents *data*_{*i*}.init.
8. Let *funcaddr*^{*} be the concatenation of the **function addresses** *funcaddr*_{*i*} in index order.
9. Let *tableaddr*^{*} be the concatenation of the **table addresses** *tableaddr*_{*i*} in index order.
10. Let *memaddr*^{*} be the concatenation of the **memory addresses** *memaddr*_{*i*} in index order.
11. Let *globaladdr*^{*} be the concatenation of the **global addresses** *globaladdr*_{*i*} in index order.
12. Let *elemaddr*^{*} be the concatenation of the **element addresses** *elemaddr*_{*i*} in index order.
13. Let *dataaddr*^{*} be the concatenation of the **data addresses** *dataaddr*_{*i*} in index order.
14. Let *funcaddr*_{mod}^{*} be the list of **function addresses** extracted from $\text{externval}_{\text{im}}^*$, concatenated with *funcaddr*^{*}.
15. Let *tableaddr*_{mod}^{*} be the list of **table addresses** extracted from $\text{externval}_{\text{im}}^*$, concatenated with *tableaddr*^{*}.
16. Let *memaddr*_{mod}^{*} be the list of **memory addresses** extracted from $\text{externval}_{\text{im}}^*$, concatenated with *memaddr*^{*}.
17. Let *globaladdr*_{mod}^{*} be the list of **global addresses** extracted from $\text{externval}_{\text{im}}^*$, concatenated with *globaladdr*^{*}.

18. For each export $export_i$ in $module.exports$, do:
 - a. If $export_i$ is a function export for function index x , then let $externval_i$ be the external value $func(funcaddr_{mod}^*[x])$.
 - b. Else, if $export_i$ is a table export for table index x , then let $externval_i$ be the external value $table(tableaddr_{mod}^*[x])$.
 - c. Else, if $export_i$ is a memory export for memory index x , then let $externval_i$ be the external value $mem(memaddr_{mod}^*[x])$.
 - d. Else, if $export_i$ is a global export for global index x , then let $externval_i$ be the external value $global(globaladdr_{mod}^*[x])$.
 - e. Let $exportinst_i$ be the export instance $\{name(export_i.name), value externval_i\}$.
19. Let $exportinst^*$ be the concatenation of the export instances $exportinst_i$ in index order.
20. Let $moduleinst$ be the module instance $\{types(module.types), funcaddrs funcaddr_{mod}^*, tableaddrs tableaddr_{mod}^*, memaddrs memaddr_{mod}^*, globaladdrs globaladdr_{mod}^*, exports exportinst^*\}$.
21. Return $moduleinst$.

$$allocmodule(S, module, externval_{im}^*, val^*, (ref^*)^*) = S', moduleinst$$

where:

$$\begin{aligned}
 table^* &= module.tables \\
 mem^* &= module.mems \\
 global^* &= module.globals \\
 elem^* &= module.elems \\
 data^* &= module.datas \\
 export^* &= module.exports \\
 moduleinst &= \{ types module.types, \\
 &\quad funcaddrs funcs(externval_{im}^*) funcaddr^*, \\
 &\quad tableaddrs tables(externval_{im}^*) tableaddr^*, \\
 &\quad memaddrs mems(externval_{im}^*) memaddr^*, \\
 &\quad globaladdrs globals(externval_{im}^*) globaladdr^*, \\
 &\quad elemaddrs elemaddr^*, \\
 &\quad dataaddrs dataaddr^*, \\
 &\quad exports exportinst^* \} \\
 S_1, funcaddr^* &= allocfunc^*(S, module.funcs, moduleinst) \\
 S_2, tableaddr^* &= alloctable^*(S_1, (table.type)^*, (ref.null t)^*) \quad (\text{where } (table.type)^* = (limits t)^*) \\
 S_3, memaddr^* &= allocmem^*(S_2, (mem.type)^*) \\
 S_4, globaladdr^* &= allocglobal^*(S_3, (global.type)^*, val^*) \\
 S_5, elemaddr^* &= allocelem^*(S_4, (elem.type)^*, (ref^*)^*) \\
 S', dataaddr^* &= allocdata^*(S_5, (data.init)^*) \\
 exportinst^* &= \{ name(export.name), value externval_{ex}^* \}^* \\
 funcs(externval_{ex}^*) &= (moduleinst.funcaddrs[x])^* \quad (\text{where } x^* = funcs(export^*)) \\
 tables(externval_{ex}^*) &= (moduleinst.tableaddrs[x])^* \quad (\text{where } x^* = tables(export^*)) \\
 mems(externval_{ex}^*) &= (moduleinst.memaddrs[x])^* \quad (\text{where } x^* = mems(export^*)) \\
 globals(externval_{ex}^*) &= (moduleinst.globaladdrs[x])^* \quad (\text{where } x^* = globals(export^*))
 \end{aligned}$$

Here, the notation $allocx^*$ is shorthand for multiple allocations of object kind X , defined as follows:

$$\begin{aligned}
 allocx^*(S_0, X^n, \dots) &= S_n, a^n \\
 \text{where for all } i < n: \\
 S_{i+1}, a^n[i] &= allocx(S_i, X^n[i], \dots)
 \end{aligned}$$

Moreover, if the dots $\dots$ are a sequence A^n (as for globals or tables), then the elements of this sequence are passed to the allocation function pointwise.

Note: The definition of module allocation is mutually recursive with the allocation of its associated functions, because the resulting module instance *moduleinst* is passed to the function allocator as an argument, in order to form the necessary closures. In an implementation, this recursion is easily unraveled by mutating one or the other in a secondary step.

4.5.4 Instantiation

Given a store S , a module *module* is instantiated with a list of external values $externval^n$ supplying the required imports as follows.

Instantiation checks that the module is **valid** and the provided imports **match** the declared types, and may **fail** with an error otherwise. Instantiation can also result in a **trap** from initializing a table or memory from an active segment or from executing the start function. It is up to the **embedder** to define how such conditions are reported.

1. If *module* is not **valid**, then:
 - a. Fail.
2. Assert: *module* is **valid** with external types $externtype_{im}^m$ classifying its imports.
3. If the number m of imports is not equal to the number n of provided external values, then:
 - a. Fail.
4. For each external value $externval_i$ in $externval^n$ and external type $externtype'_i$ in $externtype_{im}^n$, do:
 - a. If $externval_i$ is not valid with an external type $externtype_i$ in store S , then:
 - i. Fail.
 - b. If $externtype_i$ does not **match** $externtype'_i$, then:
 - i. Fail.
5. Let $moduleinst_{init}$ be the auxiliary module instance $\{\text{globaladdrs } \text{globals}(externval^n), \text{funcaddrs } moduleinst.\text{funcaddrs}\}$ that only consists of the imported globals and the imported and allocated functions from the final module instance *moduleinst*, defined below.
6. Let F_{init} be the auxiliary frame $\{\text{module } moduleinst_{init}, \text{locals } \epsilon\}$.
7. Push the frame F_{init} to the stack.
8. Let val^* be the vector of global initialization values determined by *module* and $externval^n$. These may be calculated as follows.
 - a. For each global $global_i$ in *module.globals*, do:
 - i. Let val_i be the result of evaluating the initializer expression $global_i.\text{init}$.
 - b. Assert: due to **validation**, the frame F_{init} is now on the top of the stack.
 - c. Let val^* be the concatenation of val_i in index order.
9. Let $(ref^*)^*$ be the list of reference vectors determined by the element segments in *module*. These may be calculated as follows.
 - a. For each element segment $elem_i$ in *module.elems*, and for each element expression $expr_{ij}$ in $elem_i.\text{init}$, do:
 - i. Let ref_{ij} be the result of evaluating the initializer expression $expr_{ij}$.
 - b. Let ref_i^* be the concatenation of function elements ref_{ij} in order of index j .
 - c. Let $(ref^*)^*$ be the concatenation of function element vectors ref_i^* in order of index i .
10. Pop the frame F_{init} from the stack.

11. Let *moduleinst* be a new module instance allocated from *module* in store *S* with imports *externvalⁿ*, global initializer values *val^{*}*, and element segment contents *(ref^{*})^{*}*, and let *S'* be the extended store produced by module allocation.
12. Let *F* be the auxiliary frame {module *moduleinst*, locals ϵ }.
13. Push the frame *F* to the stack.
14. For each element segment *elem_i* in *module.elements* whose mode is of the form active {table *tableidx_i*, offset *einstr_i^{*}* end}, do:
 - a. Let *n* be the length of the vector *elem_i.init*.
 - b. Execute the instruction sequence *einstr_i^{*}*.
 - c. Execute the instruction i32.const 0.
 - d. Execute the instruction i32.const *n*.
 - e. Execute the instruction table.init *tableidx_i* *i*.
 - f. Execute the instruction elem.drop *i*.
15. For each element segment *elem_i* in *module.elements* whose mode is of the form declarative, do:
 - a. Execute the instruction elem.drop *i*.
16. For each data segment *data_i* in *module.datas* whose mode is of the form active {memory *memidx_i*, offset *dinstr_i^{*}* end}, do:
 - a. Assert: *memidx_i* is 0.
 - b. Let *n* be the length of the vector *data_i.init*.
 - c. Execute the instruction sequence *dinstr_i^{*}*.
 - d. Execute the instruction i32.const 0.
 - e. Execute the instruction i32.const *n*.
 - f. Execute the instruction memory.init *i*.
 - g. Execute the instruction data.drop *i*.
17. If the start function *module.start* is not empty, then:
 - a. Let *start* be the start function *module.start*.
 - b. Execute the instruction call *start.func*.
18. Assert: due to validation, the frame *F* is now on the top of the stack.
19. Pop the frame *F* from the stack.

$$\begin{aligned}
\text{instantiate}(S, \text{module}, \text{externval}^k) = & S'; F; \text{runelem}_0(\text{elem}^n[0]) \dots \text{runelem}_{n-1}(\text{elem}^n[n-1]) \\
& \text{rundata}_0(\text{data}^m[0]) \dots \text{rundata}_{m-1}(\text{data}^m[m-1]) \\
& (\text{call } \text{start.func})? \\
& (\text{if } \vdash \text{module} : \text{externtype}_{\text{im}}^k \rightarrow \text{externtype}_{\text{ex}}^* \\
& \wedge (S \vdash \text{externval} : \text{externtype})^k \\
& \wedge (\vdash \text{externtype} \leq \text{externtype}_{\text{im}})^k \\
& \wedge \text{module.globals} = \text{global}^* \\
& \wedge \text{module.elems} = \text{elem}^n \\
& \wedge \text{module.datas} = \text{data}^m \\
& \wedge \text{module.start} = \text{start}? \\
& \wedge (\text{expr}_g = \text{global.init})^* \\
& \wedge (\text{expr}_e = \text{elem.init})^n \\
& \wedge S', \text{moduleinst} = \text{allocmodule}(S, \text{module}, \text{externval}^k, \text{val}^*, (\text{ref}^*)^n) \\
& \wedge F = \{\text{module } \text{moduleinst}, \text{locals } \epsilon\} \\
& \wedge (S'; F; \text{expr}_g \hookrightarrow *S'; F; \text{val end})^* \\
& \wedge ((S'; F; \text{expr}_e \hookrightarrow *S'; F; \text{ref end})^*)^n)
\end{aligned}$$

where:

$$\begin{aligned}
\text{runelem}_i(\{\text{type } et, \text{init } \text{expr}^n, \text{mode passive}\}) &= \epsilon \\
\text{runelem}_i(\{\text{type } et, \text{init } \text{expr}^n, \text{mode active}\{\text{table } x, \text{offset } \text{instr}^* \text{end}\}\}) &= \\
& \text{instr}^* (\text{i32.const } 0) (\text{i32.const } n) (\text{table.init } x \ i) (\text{elem.drop } i) \\
\text{runelem}_i(\{\text{type } et, \text{init } \text{expr}^n, \text{mode declarative}\}) &= \\
& (\text{elem.drop } i) \\
\text{rundata}_i(\{\text{init } b^n, \text{mode passive}\}) &= \epsilon \\
\text{rundata}_i(\{\text{init } b^n, \text{mode active}\{\text{memory } 0, \text{offset } \text{instr}^* \text{end}\}\}) &= \\
& \text{instr}^* (\text{i32.const } 0) (\text{i32.const } n) (\text{memory.init } i) (\text{data.drop } i)
\end{aligned}$$

Note: Module allocation and the evaluation of global initializers and element segments are mutually recursive because the global initialization values val^* and element segment contents $(\text{ref}^*)^*$ are passed to the module allocator while depending on the module instance moduleinst and store S' returned by allocation. However, this recursion is just a specification device. In practice, the initialization values can be determined beforehand by staging module allocation such that first, the module's own function instances are pre-allocated in the store, then the initializer expressions are evaluated, then the rest of the module instance is allocated, and finally the new function instances' module fields are set to that module instance. This is possible because validation ensures that initialization expressions cannot actually call a function, only take their reference.

All failure conditions are checked before any observable mutation of the store takes place. Store mutation is not atomic; it happens in individual steps that may be interleaved with other threads.

Evaluation of constant expressions does not affect the store.

4.5.5 Invocation

Once a module has been instantiated, any exported function can be invoked externally via its function address funcaddr in the store S and an appropriate list val^* of argument values.

Invocation may fail with an error if the arguments do not fit the function type. Invocation can also result in a trap. It is up to the embedder to define how such conditions are reported.

Note: If the embedder API performs type checks itself, either statically or dynamically, before performing an invocation, then no failure other than traps can occur.

The following steps are performed:

1. Assert: $S.\text{funcs}[\text{funcaddr}]$ exists.
2. Let funcinst be the function instance $S.\text{funcs}[\text{funcaddr}]$.
3. Let $[t_1^n] \rightarrow [t_2^m]$ be the function type funcinst.type .
4. If the length $|\text{val}^*|$ of the provided argument values is different from the number n of expected arguments, then:
 - a. Fail.
5. For each value type t_i in t_1^n and corresponding value val_i in val^* , do:
 - a. If val_i is not valid with value type t_i , then:
 - i. Fail.
6. Let F be the dummy frame $\{\text{module } \{\}, \text{locals } \epsilon\}$.
7. Push the frame F to the stack.
8. Push the values val^* to the stack.
9. Invoke the function instance at address funcaddr .

Once the function has returned, the following steps are executed:

1. Assert: due to validation, m values are on the top of the stack.
2. Pop $\text{val}_{\text{res}}^m$ from the stack.

The values $\text{val}_{\text{res}}^m$ are returned as the results of the invocation.

$$\begin{aligned}
 \text{invoke}(S, \text{funcaddr}, \text{val}^n) &= S; F; \text{val}^n \text{ (invoke funcaddr)} \\
 &\text{(if } S.\text{funcs}[\text{funcaddr}].\text{type} = [t_1^n] \rightarrow [t_2^m] \\
 &\wedge (S \vdash \text{val} : t_1)^n \\
 &\wedge F = \{\text{module } \{\}, \text{locals } \epsilon\})
 \end{aligned}$$

5.1 Conventions

The binary format for WebAssembly **modules** is a dense linear *encoding* of their **abstract syntax**.²⁸

The format is defined by an *attribute grammar* whose only terminal symbols are **bytes**. A byte sequence is a well-formed encoding of a module if and only if it is generated by the grammar.

Each production of this grammar has exactly one synthesized attribute: the abstract syntax that the respective byte sequence encodes. Thus, the attribute grammar implicitly defines a *decoding* function (i.e., a parsing function for the binary format).

Except for a few exceptions, the binary grammar closely mirrors the grammar of the abstract syntax.

Note: Some phrases of abstract syntax have multiple possible encodings in the binary format. For example, numbers may be encoded as if they had optional leading zeros. Implementations of decoders must support all possible alternatives; implementations of encoders can pick any allowed encoding.

The recommended extension for files containing WebAssembly modules in binary format is “.wasm” and the recommended **Media Type**²⁷ is “application/wasm”.

5.1.1 Grammar

The following conventions are adopted in defining grammar rules for the binary format. They mirror the conventions used for **abstract syntax**. In order to distinguish symbols of the binary syntax from symbols of the abstract syntax, **typewriter** font is adopted for the former.

- Terminal symbols are **bytes** expressed in hexadecimal notation: 0x0F.
- Nonterminal symbols are written in typewriter font: `valtype`, `instr`.
- B^n is a sequence of $n \geq 0$ iterations of B .
- B^* is a possibly empty sequence of iterations of B . (This is a shorthand for B^n used where n is not relevant.)

²⁸ Additional encoding layers – for example, introducing compression – may be defined on top of the basic representation defined here. However, such layers are outside the scope of the current specification.

²⁷ <https://www.iana.org/assignments/media-types/media-types.xhtml>

- $B^?$ is an optional occurrence of B . (This is a shorthand for B^n where $n \leq 1$.)
- $x:B$ denotes the same language as the nonterminal B , but also binds the variable x to the attribute synthesized for B .
- Productions are written $\text{sym} ::= B_1 \Rightarrow A_1 \mid \dots \mid B_n \Rightarrow A_n$, where each A_i is the attribute that is synthesized for sym in the given case, usually from attribute variables bound in B_i .
- Some productions are augmented by side conditions in parentheses, which restrict the applicability of the production. They provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production (in the syntax or in an attribute), then all those occurrences must have the same instantiation. (This is a shorthand for a side condition requiring multiple different variables to be equal.)

Note: For example, the [binary grammar](#) for [number types](#) is given as follows:

$$\begin{array}{llll} \text{numtype} & ::= & 0x7F & \Rightarrow & i32 \\ & & | & & 0x7E & \Rightarrow & i64 \\ & & | & & 0x7D & \Rightarrow & f32 \\ & & | & & 0x7C & \Rightarrow & f64 \end{array}$$

Consequently, the byte 0x7F encodes the type [i32](#), 0x7E encodes the type [i64](#), and so forth. No other byte value is allowed as the encoding of a number type.

The [binary grammar](#) for [limits](#) is defined as follows:

$$\begin{array}{llll} \text{limits} & ::= & 0x00 \ n:\text{u32} & \Rightarrow & \{\min n, \max \epsilon\} \\ & & | & & 0x01 \ n:\text{u32} \ m:\text{u32} & \Rightarrow & \{\min n, \max m\} \end{array}$$

That is, a limits pair is encoded as either the byte 0x00 followed by the encoding of a [u32](#) value, or the byte 0x01 followed by two such encodings. The variables n and m name the attributes of the respective [u32](#) nonterminals, which in this case are the actual [unsigned integers](#) those decode into. The attribute of the complete production then is the abstract syntax for the limit, expressed in terms of the former values.

5.1.2 Auxiliary Notation

When dealing with binary encodings the following notation is also used:

- ϵ denotes the empty byte sequence.
- $||B||$ is the length of the byte sequence generated from the production B in a derivation.

5.1.3 Vectors

[Vectors](#) are encoded with their [u32](#) length followed by the encoding of their element sequence.

$$\text{vec}(B) ::= n:\text{u32} \ (x:B)^n \Rightarrow x^n$$

5.2 Values

5.2.1 Bytes

Bytes encode themselves.

$$\begin{array}{lcl} \text{byte} & ::= & 0x00 \Rightarrow 0x00 \\ & | & \dots \\ & | & 0xFF \Rightarrow 0xFF \end{array}$$

5.2.2 Integers

All integers are encoded using the [LEB128](#)²⁹ variable-length integer encoding, in either unsigned or signed variant.

Unsigned integers are encoded in [unsigned LEB128](#)³⁰ format. As an additional constraint, the total number of bytes encoding a value of type uN must not exceed $\text{ceil}(N/7)$ bytes.

$$\begin{array}{lcl} uN & ::= & n:\text{byte} \Rightarrow n \quad (\text{if } n < 2^7 \wedge n < 2^N) \\ & | & n:\text{byte } m:\text{u}(N-7) \Rightarrow 2^7 \cdot m + (n - 2^7) \quad (\text{if } n \geq 2^7 \wedge N > 7) \end{array}$$

Signed integers are encoded in [signed LEB128](#)³¹ format, which uses a two's complement representation. As an additional constraint, the total number of bytes encoding a value of type sN must not exceed $\text{ceil}(N/7)$ bytes.

$$\begin{array}{lcl} sN & ::= & n:\text{byte} \Rightarrow n \quad (\text{if } n < 2^6 \wedge n < 2^{N-1}) \\ & | & n:\text{byte} \Rightarrow n - 2^7 \quad (\text{if } 2^6 \leq n < 2^7 \wedge n \geq 2^7 - 2^{N-1}) \\ & | & n:\text{byte } m:\text{s}(N-7) \Rightarrow 2^7 \cdot m + (n - 2^7) \quad (\text{if } n \geq 2^7 \wedge N > 7) \end{array}$$

Uninterpreted integers are encoded as signed integers.

$$iN ::= n:sN \Rightarrow i \quad (\text{if } n = \text{signed}_N(i))$$

Note: The side conditions $N > 7$ in the productions for non-terminal bytes of the u and s encodings restrict the encoding's length. However, “trailing zeros” are still allowed within these bounds. For example, `0x03` and `0x83 0x00` are both well-formed encodings for the value 3 as a $u8$. Similarly, either of `0x7e` and `0xFE 0x7F` and `0xFE 0xFF 0x7F` are well-formed encodings of the value -2 as a $s16$.

The side conditions on the value n of terminal bytes further enforce that any unused bits in these bytes must be 0 for positive values and 1 for negative ones. For example, `0x83 0x10` is malformed as a $u8$ encoding. Similarly, both `0x83 0x3E` and `0xFF 0x7B` are malformed as $s8$ encodings.

5.2.3 Floating-Point

Floating-point values are encoded directly by their [IEEE 754](#)³² (Section 3.4) bit pattern in [little endian](#)³³ byte order:

$$fN ::= b*:\text{byte}^{N/8} \Rightarrow \text{bytes}_{fN}^{-1}(b*)$$

²⁹ <https://en.wikipedia.org/wiki/LEB128>

³⁰ https://en.wikipedia.org/wiki/LEB128#Unsigned_LEB128

³¹ https://en.wikipedia.org/wiki/LEB128#Signed_LEB128

³² <https://ieeexplore.ieee.org/document/8766229>

³³ <https://en.wikipedia.org/wiki/Endianness#Little-endian>

5.2.4 Names

Names are encoded as a **vector** of bytes containing the **Unicode**³⁴ (Section 3.9) UTF-8 encoding of the name's character sequence.

$$\text{name} ::= b^*.\text{vec}(\text{byte}) \Rightarrow \text{name} \quad (\text{if } \text{utf8}(\text{name}) = b^*)$$

The auxiliary **utf8** function expressing this encoding is defined as follows:

$$\begin{aligned} \text{utf8}(c^*) &= (\text{utf8}(c))^* \\ \text{utf8}(c) &= b && (\text{if } c < \text{U}+80 \\ &&& \wedge c = b) \\ \text{utf8}(c) &= b_1 b_2 && (\text{if } \text{U}+80 \leq c < \text{U}+800 \\ &&& \wedge c = 2^6(b_1 - 0\text{x}C0) + (b_2 - 0\text{x}80)) \\ \text{utf8}(c) &= b_1 b_2 b_3 && (\text{if } \text{U}+800 \leq c < \text{U}+\text{D}800 \vee \text{U}+\text{E}000 \leq c < \text{U}+10000 \\ &&& \wedge c = 2^{12}(b_1 - 0\text{x}E0) + 2^6(b_2 - 0\text{x}80) + (b_3 - 0\text{x}80)) \\ \text{utf8}(c) &= b_1 b_2 b_3 b_4 && (\text{if } \text{U}+10000 \leq c < \text{U}+110000 \\ &&& \wedge c = 2^{18}(b_1 - 0\text{x}F0) + 2^{12}(b_2 - 0\text{x}80) + 2^6(b_3 - 0\text{x}80) + (b_4 - 0\text{x}80)) \\ \text{where } b_2, b_3, b_4 &< 0\text{x}C0 \end{aligned}$$

Note: Unlike in some other formats, name strings are not 0-terminated.

5.3 Types

Note: In some places, possible types include both type constructors or types denoted by **type indices**. Thus, the binary format for type constructors corresponds to the encodings of small negative *sN* values, such that they can unambiguously occur in the same place as (positive) type indices.

5.3.1 Number Types

Number types are encoded by a single byte.

$$\begin{array}{lll} \text{numtype} & ::= & 0\text{x}7\text{F} \Rightarrow \text{i}32 \\ & | & 0\text{x}7\text{E} \Rightarrow \text{i}64 \\ & | & 0\text{x}7\text{D} \Rightarrow \text{f}32 \\ & | & 0\text{x}7\text{C} \Rightarrow \text{f}64 \end{array}$$

5.3.2 Vector Types

Vector types are also encoded by a single byte.

$$\text{vectype} ::= 0\text{x}7\text{B} \Rightarrow \text{v}128$$

³⁴ <https://www.unicode.org/versions/latest/>

5.3.3 Reference Types

Reference types are also encoded by a single byte.

```
reftype ::= 0x70 ⇒ funcref
          | 0x6F ⇒ externref
```

5.3.4 Value Types

Value types are encoded with their respective encoding as a number type, vector type, or reference type.

```
valtype ::= t:numtype ⇒ t
          | t:vectype ⇒ t
          | t:reftype ⇒ t
```

Note: Value types can occur in contexts where type indices are also allowed, such as in the case of block types. Thus, the binary format for types corresponds to the signed LEB128³⁵ encoding of small negative sN values, so that they can coexist with (positive) type indices in the future.

5.3.5 Result Types

Result types are encoded by the respective vectors of value types.

```
resulttype ::= t*:vec(valtype) ⇒ [t*]
```

5.3.6 Function Types

Function types are encoded by the byte 0x60 followed by the respective vectors of parameter and result types.

```
functype ::= 0x60 rt1:resulttype rt2:resulttype ⇒ rt1 → rt2
```

5.3.7 Limits

Limits are encoded with a preceding flag indicating whether a maximum is present.

```
limits ::= 0x00 n:u32 ⇒ {min n, max  $\epsilon$ }
          | 0x01 n:u32 m:u32 ⇒ {min n, max m}
```

5.3.8 Memory Types

Memory types are encoded with their limits.

```
memtype ::= lim:limits ⇒ lim
```

³⁵ https://en.wikipedia.org/wiki/LEB128#Signed_LEB128

5.3.9 Table Types

Table types are encoded with their [limits](#) and the encoding of their element [reference type](#).

$$\text{tabletype} ::= \text{et:reftype } \text{lim:limits} \Rightarrow \text{lim et}$$

5.3.10 Global Types

Global types are encoded by their [value type](#) and a flag for their [mutability](#).

$$\begin{aligned} \text{globaltype} &::= t:\text{valtype } m:\text{mut} \Rightarrow m t \\ \text{mut} &::= 0x00 \Rightarrow \text{const} \\ &| 0x01 \Rightarrow \text{var} \end{aligned}$$

5.4 Instructions

[Instructions](#) are encoded by *opcodes*. Each opcode is represented by a single byte, and is followed by the instruction's immediate arguments, where present. The only exception are [structured control instructions](#), which consist of several opcodes bracketing their nested instruction sequences.

Note: Gaps in the byte code ranges for encoding instructions are reserved for future extensions.

5.4.1 Control Instructions

[Control instructions](#) have varying encodings. For structured instructions, the instruction sequences forming nested blocks are terminated with explicit opcodes for [end](#) and [else](#).

[Block types](#) are encoded in special compressed form, by either the byte 0x40 indicating the empty type, as a single value type, or as a [type index](#) encoded as a positive [signed integer](#).

<code>blocktype</code>	<code>::= 0x40</code>	$\Rightarrow \epsilon$	
	<code> t:valtype</code>	$\Rightarrow t$	
	<code> x:s33</code>	$\Rightarrow x$	(if $x \geq 0$)
<code>instr</code>	<code>::= 0x00</code>	$\Rightarrow \text{unreachable}$	
	<code> 0x01</code>	$\Rightarrow \text{nop}$	
	<code> 0x02 bt:blocktype (in:instr)* 0x0B</code>	$\Rightarrow \text{block } bt \text{ in}^* \text{ end}$	
	<code> 0x03 bt:blocktype (in:instr)* 0x0B</code>	$\Rightarrow \text{loop } bt \text{ in}^* \text{ end}$	
	<code> 0x04 bt:blocktype (in:instr)* 0x0B</code>	$\Rightarrow \text{if } bt \text{ in}^* \text{ else } \epsilon \text{ end}$	
	<code> 0x04 bt:blocktype (in₁:instr)* 0x05 (in₂:instr)* 0x0B</code>	$\Rightarrow \text{if } bt \text{ in}_1^* \text{ else } \text{in}_2^* \text{ end}$	
	<code> 0x0C l:labelidx</code>	$\Rightarrow \text{br } l$	
	<code> 0x0D l:labelidx</code>	$\Rightarrow \text{br_if } l$	
	<code> 0x0E l*:vec(labelidx) l_N:labelidx</code>	$\Rightarrow \text{br_table } l^* l_N$	
	<code> 0x0F</code>	$\Rightarrow \text{return}$	
	<code> 0x10 x:funcidx</code>	$\Rightarrow \text{call } x$	
	<code> 0x11 y:typeidx x:tableidx</code>	$\Rightarrow \text{call_indirect } x y$	

Note: The [else](#) opcode 0x05 in the encoding of an [if](#) instruction can be omitted if the following instruction sequence is empty.

Unlike any [other occurrence](#), the [type index](#) in a [block type](#) is encoded as a positive [signed integer](#), so that its signed LEB128 bit pattern cannot collide with the encoding of [value types](#) or the special code 0x40, which correspond to

the LEB128 encoding of negative integers. To avoid any loss in the range of allowed indices, it is treated as a 33 bit signed integer.

5.4.2 Reference Instructions

Reference instructions are represented by single byte codes.

```
instr ::= ...
      | 0xD0 t:reftype ⇒ ref.null t
      | 0xD1           ⇒ ref.is_null
      | 0xD2 x:funcidx ⇒ ref.func x
```

5.4.3 Parametric Instructions

Parametric instructions are represented by single byte codes, possibly followed by a type annotation.

```
instr ::= ...
      | 0x1A           ⇒ drop
      | 0x1B           ⇒ select
      | 0x1C t*:vec(valtype) ⇒ select t*
```

5.4.4 Variable Instructions

Variable instructions are represented by byte codes followed by the encoding of the respective index.

```
instr ::= ...
      | 0x20 x:localidx ⇒ local.get x
      | 0x21 x:localidx ⇒ local.set x
      | 0x22 x:localidx ⇒ local.tee x
      | 0x23 x:globalidx ⇒ global.get x
      | 0x24 x:globalidx ⇒ global.set x
```

5.4.5 Table Instructions

Table instructions are represented either by a single byte or a one byte prefix followed by a variable-length unsigned integer.

```
instr ::= ...
      | 0x25 x:tableidx           ⇒ table.get x
      | 0x26 x:tableidx           ⇒ table.set x
      | 0xFC 12:u32 y:elemidx x:tableidx ⇒ table.init x y
      | 0xFC 13:u32 x:elemidx           ⇒ elem.drop x
      | 0xFC 14:u32 x:tableidx y:tableidx ⇒ table.copy x y
      | 0xFC 15:u32 x:tableidx           ⇒ table.grow x
      | 0xFC 16:u32 x:tableidx           ⇒ table.size x
      | 0xFC 17:u32 x:tableidx           ⇒ table.fill x
```

5.4.6 Memory Instructions

Each variant of [memory instruction](#) is encoded with a different byte code. Loads and stores are followed by the encoding of their *memarg* immediate.

<i>memarg</i>	::=	<i>a</i> :u32 <i>o</i> :u32	⇒	{align <i>a</i> , offset <i>o</i> }
<i>instr</i>	::=	...		
		0x28 <i>m</i> :memarg	⇒	i32.load <i>m</i>
		0x29 <i>m</i> :memarg	⇒	i64.load <i>m</i>
		0x2A <i>m</i> :memarg	⇒	f32.load <i>m</i>
		0x2B <i>m</i> :memarg	⇒	f64.load <i>m</i>
		0x2C <i>m</i> :memarg	⇒	i32.load8_s <i>m</i>
		0x2D <i>m</i> :memarg	⇒	i32.load8_u <i>m</i>
		0x2E <i>m</i> :memarg	⇒	i32.load16_s <i>m</i>
		0x2F <i>m</i> :memarg	⇒	i32.load16_u <i>m</i>
		0x30 <i>m</i> :memarg	⇒	i64.load8_s <i>m</i>
		0x31 <i>m</i> :memarg	⇒	i64.load8_u <i>m</i>
		0x32 <i>m</i> :memarg	⇒	i64.load16_s <i>m</i>
		0x33 <i>m</i> :memarg	⇒	i64.load16_u <i>m</i>
		0x34 <i>m</i> :memarg	⇒	i64.load32_s <i>m</i>
		0x35 <i>m</i> :memarg	⇒	i64.load32_u <i>m</i>
		0x36 <i>m</i> :memarg	⇒	i32.store <i>m</i>
		0x37 <i>m</i> :memarg	⇒	i64.store <i>m</i>
		0x38 <i>m</i> :memarg	⇒	f32.store <i>m</i>
		0x39 <i>m</i> :memarg	⇒	f64.store <i>m</i>
		0x3A <i>m</i> :memarg	⇒	i32.store8 <i>m</i>
		0x3B <i>m</i> :memarg	⇒	i32.store16 <i>m</i>
		0x3C <i>m</i> :memarg	⇒	i64.store8 <i>m</i>
		0x3D <i>m</i> :memarg	⇒	i64.store16 <i>m</i>
		0x3E <i>m</i> :memarg	⇒	i64.store32 <i>m</i>
		0x3F 0x00	⇒	memory.size
		0x40 0x00	⇒	memory.grow
		0xFC 8:u32 <i>x</i> :dataidx 0x00	⇒	memory.init <i>x</i>
		0xFC 9:u32 <i>x</i> :dataidx	⇒	data.drop <i>x</i>
		0xFC 10:u32 0x00 0x00	⇒	memory.copy
		0xFC 11:u32 0x00	⇒	memory.fill

Note: In future versions of WebAssembly, the additional zero bytes occurring in the encoding of the [memory.size](#), [memory.grow](#), [memory.copy](#), and [memory.fill](#) instructions may be used to index additional memories.

5.4.7 Numeric Instructions

All variants of [numeric instructions](#) are represented by separate byte codes.

The [const](#) instructions are followed by the respective literal.

<i>instr</i>	::=	...		
		0x41 <i>n</i> :i32	⇒	i32.const <i>n</i>
		0x42 <i>n</i> :i64	⇒	i64.const <i>n</i>
		0x43 <i>z</i> :f32	⇒	f32.const <i>z</i>
		0x44 <i>z</i> :f64	⇒	f64.const <i>z</i>

All other numeric instructions are plain opcodes without any immediates.

```

instr ::= ...
| 0x45 ⇒ i32.eqz
| 0x46 ⇒ i32.eq
| 0x47 ⇒ i32.ne
| 0x48 ⇒ i32.lt_s
| 0x49 ⇒ i32.lt_u
| 0x4A ⇒ i32.gt_s
| 0x4B ⇒ i32.gt_u
| 0x4C ⇒ i32.le_s
| 0x4D ⇒ i32.le_u
| 0x4E ⇒ i32.ge_s
| 0x4F ⇒ i32.ge_u

| 0x50 ⇒ i64.eqz
| 0x51 ⇒ i64.eq
| 0x52 ⇒ i64.ne
| 0x53 ⇒ i64.lt_s
| 0x54 ⇒ i64.lt_u
| 0x55 ⇒ i64.gt_s
| 0x56 ⇒ i64.gt_u
| 0x57 ⇒ i64.le_s
| 0x58 ⇒ i64.le_u
| 0x59 ⇒ i64.ge_s
| 0x5A ⇒ i64.ge_u

| 0x5B ⇒ f32.eq
| 0x5C ⇒ f32.ne
| 0x5D ⇒ f32.lt
| 0x5E ⇒ f32.gt
| 0x5F ⇒ f32.le
| 0x60 ⇒ f32.ge

| 0x61 ⇒ f64.eq
| 0x62 ⇒ f64.ne
| 0x63 ⇒ f64.lt
| 0x64 ⇒ f64.gt
| 0x65 ⇒ f64.le
| 0x66 ⇒ f64.ge

| 0x67 ⇒ i32.clz
| 0x68 ⇒ i32.ctz
| 0x69 ⇒ i32.popcnt
| 0x6A ⇒ i32.add
| 0x6B ⇒ i32.sub
| 0x6C ⇒ i32.mul
| 0x6D ⇒ i32.div_s
| 0x6E ⇒ i32.div_u
| 0x6F ⇒ i32.rem_s
| 0x70 ⇒ i32.rem_u
| 0x71 ⇒ i32.and
| 0x72 ⇒ i32.or
| 0x73 ⇒ i32.xor
| 0x74 ⇒ i32.shl
| 0x75 ⇒ i32.shr_s
| 0x76 ⇒ i32.shr_u
| 0x77 ⇒ i32.rotl
| 0x78 ⇒ i32.rotr

```

	0x79	⇒	i64.clz
	0x7A	⇒	i64.ctz
	0x7B	⇒	i64.popcnt
	0x7C	⇒	i64.add
	0x7D	⇒	i64.sub
	0x7E	⇒	i64.mul
	0x7F	⇒	i64.div_s
	0x80	⇒	i64.div_u
	0x81	⇒	i64.rem_s
	0x82	⇒	i64.rem_u
	0x83	⇒	i64.and
	0x84	⇒	i64.or
	0x85	⇒	i64.xor
	0x86	⇒	i64.shl
	0x87	⇒	i64.shr_s
	0x88	⇒	i64.shr_u
	0x89	⇒	i64.rotl
	0x8A	⇒	i64.rotr
	0x8B	⇒	f32.abs
	0x8C	⇒	f32.neg
	0x8D	⇒	f32.ceil
	0x8E	⇒	f32.floor
	0x8F	⇒	f32.trunc
	0x90	⇒	f32.nearest
	0x91	⇒	f32.sqrt
	0x92	⇒	f32.add
	0x93	⇒	f32.sub
	0x94	⇒	f32.mul
	0x95	⇒	f32.div
	0x96	⇒	f32.min
	0x97	⇒	f32.max
	0x98	⇒	f32.copysign
	0x99	⇒	f64.abs
	0x9A	⇒	f64.neg
	0x9B	⇒	f64.ceil
	0x9C	⇒	f64.floor
	0x9D	⇒	f64.trunc
	0x9E	⇒	f64.nearest
	0x9F	⇒	f64.sqrt
	0xA0	⇒	f64.add
	0xA1	⇒	f64.sub
	0xA2	⇒	f64.mul
	0xA3	⇒	f64.div
	0xA4	⇒	f64.min
	0xA5	⇒	f64.max
	0xA6	⇒	f64.copysign

0xA7	⇒	i32.wrap_i64
0xA8	⇒	i32.trunc_f32_s
0xA9	⇒	i32.trunc_f32_u
0xAA	⇒	i32.trunc_f64_s
0xAB	⇒	i32.trunc_f64_u
0xAC	⇒	i64.extend_i32_s
0xAD	⇒	i64.extend_i32_u
0xAE	⇒	i64.trunc_f32_s
0xAF	⇒	i64.trunc_f32_u
0xB0	⇒	i64.trunc_f64_s
0xB1	⇒	i64.trunc_f64_u
0xB2	⇒	f32.convert_i32_s
0xB3	⇒	f32.convert_i32_u
0xB4	⇒	f32.convert_i64_s
0xB5	⇒	f32.convert_i64_u
0xB6	⇒	f32.demote_f64
0xB7	⇒	f64.convert_i32_s
0xB8	⇒	f64.convert_i32_u
0xB9	⇒	f64.convert_i64_s
0xBA	⇒	f64.convert_i64_u
0xBB	⇒	f64.promote_f32
0xBC	⇒	i32.reinterpret_f32
0xBD	⇒	i64.reinterpret_f64
0xBE	⇒	f32.reinterpret_i32
0xBF	⇒	f64.reinterpret_i64
0xC0	⇒	i32.extend8_s
0xC1	⇒	i32.extend16_s
0xC2	⇒	i64.extend8_s
0xC3	⇒	i64.extend16_s
0xC4	⇒	i64.extend32_s

The saturating truncation instructions all have a one byte prefix, whereas the actual opcode is encoded by a variable-length [unsigned integer](#).

```
instr ::= ...
| 0xFC 0:u32 ⇒ i32.trunc_sat_f32_s
| 0xFC 1:u32 ⇒ i32.trunc_sat_f32_u
| 0xFC 2:u32 ⇒ i32.trunc_sat_f64_s
| 0xFC 3:u32 ⇒ i32.trunc_sat_f64_u
| 0xFC 4:u32 ⇒ i64.trunc_sat_f32_s
| 0xFC 5:u32 ⇒ i64.trunc_sat_f32_u
| 0xFC 6:u32 ⇒ i64.trunc_sat_f64_s
| 0xFC 7:u32 ⇒ i64.trunc_sat_f64_u
```

5.4.8 Vector Instructions

All variants of [vector instructions](#) are represented by separate byte codes. They all have a one byte prefix, whereas the actual opcode is encoded by a variable-length [unsigned integer](#).

Vector loads and stores are followed by the encoding of their *memory* immediate.

```

laneidx ::= l:byte ⇒ l
instr    ::= ...
          | 0xFD 0:u32 m:memarg ⇒ v128.load m
          | 0xFD 1:u32 m:memarg ⇒ v128.load8x8_s m
          | 0xFD 2:u32 m:memarg ⇒ v128.load8x8_u m
          | 0xFD 3:u32 m:memarg ⇒ v128.load16x4_s m
          | 0xFD 4:u32 m:memarg ⇒ v128.load16x4_u m
          | 0xFD 5:u32 m:memarg ⇒ v128.load32x2_s m
          | 0xFD 6:u32 m:memarg ⇒ v128.load32x2_u m
          | 0xFD 7:u32 m:memarg ⇒ v128.load8_splat m
          | 0xFD 8:u32 m:memarg ⇒ v128.load16_splat m
          | 0xFD 9:u32 m:memarg ⇒ v128.load32_splat m
          | 0xFD 10:u32 m:memarg ⇒ v128.load64_splat m
          | 0xFD 92:u32 m:memarg ⇒ v128.load32_zero m
          | 0xFD 93:u32 m:memarg ⇒ v128.load64_zero m
          | 0xFD 11:u32 m:memarg ⇒ v128.store m
          | 0xFD 84:u32 m:memarg l:laneidx ⇒ v128.load8_lane m l
          | 0xFD 85:u32 m:memarg l:laneidx ⇒ v128.load16_lane m l
          | 0xFD 86:u32 m:memarg l:laneidx ⇒ v128.load32_lane m l
          | 0xFD 87:u32 m:memarg l:laneidx ⇒ v128.load64_lane m l
          | 0xFD 88:u32 m:memarg l:laneidx ⇒ v128.store8_lane m l
          | 0xFD 89:u32 m:memarg l:laneidx ⇒ v128.store16_lane m l
          | 0xFD 90:u32 m:memarg l:laneidx ⇒ v128.store32_lane m l
          | 0xFD 91:u32 m:memarg l:laneidx ⇒ v128.store64_lane m l

```

The `const` instruction is followed by 16 immediate bytes, which are converted into a *i128* in *littleendian* byte order:

```

instr ::= ...
       | 0xFD 12:u32 (b:byte)16 ⇒ v128.const bytesi128-1(b0 ... b15)

```

The `shuffle` instruction is also followed by the encoding of 16 *laneidx* immediates.

```

instr ::= ...
       | 0xFD 13:u32 (l:laneidx)16 ⇒ i8x16.shuffle l16

```

`extract_lane` and `replace_lane` instructions are followed by the encoding of a *laneidx* immediate.

```

instr ::= ...
       | 0xFD 21:u32 l:laneidx ⇒ i8x16.extract_lane_s l
       | 0xFD 22:u32 l:laneidx ⇒ i8x16.extract_lane_u l
       | 0xFD 23:u32 l:laneidx ⇒ i8x16.replace_lane l
       | 0xFD 24:u32 l:laneidx ⇒ i16x8.extract_lane_s l
       | 0xFD 25:u32 l:laneidx ⇒ i16x8.extract_lane_u l
       | 0xFD 26:u32 l:laneidx ⇒ i16x8.replace_lane l
       | 0xFD 27:u32 l:laneidx ⇒ i32x4.extract_lane l
       | 0xFD 28:u32 l:laneidx ⇒ i32x4.replace_lane l
       | 0xFD 29:u32 l:laneidx ⇒ i64x2.extract_lane l
       | 0xFD 30:u32 l:laneidx ⇒ i64x2.replace_lane l
       | 0xFD 31:u32 l:laneidx ⇒ f32x4.extract_lane l
       | 0xFD 32:u32 l:laneidx ⇒ f32x4.replace_lane l
       | 0xFD 33:u32 l:laneidx ⇒ f64x2.extract_lane l
       | 0xFD 34:u32 l:laneidx ⇒ f64x2.replace_lane l

```

All other vector instructions are plain opcodes without any immediates.

```

instr ::= ...
| 0xFD 14:u32 ⇒ i8x16.swizzle
| 0xFD 15:u32 ⇒ i8x16.splat
| 0xFD 16:u32 ⇒ i16x8.splat
| 0xFD 17:u32 ⇒ i32x4.splat
| 0xFD 18:u32 ⇒ i64x2.splat
| 0xFD 19:u32 ⇒ f32x4.splat
| 0xFD 20:u32 ⇒ f64x2.splat

| 0xFD 35:u32 ⇒ i8x16.eq
| 0xFD 36:u32 ⇒ i8x16.ne
| 0xFD 37:u32 ⇒ i8x16.lt_s
| 0xFD 38:u32 ⇒ i8x16.lt_u
| 0xFD 39:u32 ⇒ i8x16.gt_s
| 0xFD 40:u32 ⇒ i8x16.gt_u
| 0xFD 41:u32 ⇒ i8x16.le_s
| 0xFD 42:u32 ⇒ i8x16.le_u
| 0xFD 43:u32 ⇒ i8x16.ge_s
| 0xFD 44:u32 ⇒ i8x16.ge_u

| 0xFD 45:u32 ⇒ i16x8.eq
| 0xFD 46:u32 ⇒ i16x8.ne
| 0xFD 47:u32 ⇒ i16x8.lt_s
| 0xFD 48:u32 ⇒ i16x8.lt_u
| 0xFD 49:u32 ⇒ i16x8.gt_s
| 0xFD 50:u32 ⇒ i16x8.gt_u
| 0xFD 51:u32 ⇒ i16x8.le_s
| 0xFD 52:u32 ⇒ i16x8.le_u
| 0xFD 53:u32 ⇒ i16x8.ge_s
| 0xFD 54:u32 ⇒ i16x8.ge_u

| 0xFD 55:u32 ⇒ i32x4.eq
| 0xFD 56:u32 ⇒ i32x4.ne
| 0xFD 57:u32 ⇒ i32x4.lt_s
| 0xFD 58:u32 ⇒ i32x4.lt_u
| 0xFD 59:u32 ⇒ i32x4.gt_s
| 0xFD 60:u32 ⇒ i32x4.gt_u
| 0xFD 61:u32 ⇒ i32x4.le_s
| 0xFD 62:u32 ⇒ i32x4.le_u
| 0xFD 63:u32 ⇒ i32x4.ge_s
| 0xFD 64:u32 ⇒ i32x4.ge_u

| 0xFD 214:u32 ⇒ i64x2.eq
| 0xFD 215:u32 ⇒ i64x2.ne
| 0xFD 216:u32 ⇒ i64x2.lt_s
| 0xFD 217:u32 ⇒ i64x2.gt_s
| 0xFD 218:u32 ⇒ i64x2.le_s
| 0xFD 219:u32 ⇒ i64x2.ge_s

| 0xFD 65:u32 ⇒ f32x4.eq
| 0xFD 66:u32 ⇒ f32x4.ne
| 0xFD 67:u32 ⇒ f32x4.lt
| 0xFD 68:u32 ⇒ f32x4.gt
| 0xFD 69:u32 ⇒ f32x4.le
| 0xFD 70:u32 ⇒ f32x4.ge

```

0xFD 71:u32	⇒	f64x2.eq
0xFD 72:u32	⇒	f64x2.ne
0xFD 73:u32	⇒	f64x2.lt
0xFD 74:u32	⇒	f64x2.gt
0xFD 75:u32	⇒	f64x2.le
0xFD 76:u32	⇒	f64x2.ge

0xFD 77:u32	⇒	v128.not
0xFD 78:u32	⇒	v128.and
0xFD 79:u32	⇒	v128.andnot
0xFD 80:u32	⇒	v128.or
0xFD 81:u32	⇒	v128.xor
0xFD 82:u32	⇒	v128.bitselect
0xFD 83:u32	⇒	v128.any_true

0xFD 96:u32	⇒	i8x16.abs
0xFD 97:u32	⇒	i8x16.neg
0xFD 98:u32	⇒	i8x16.popcnt
0xFD 99:u32	⇒	i8x16.all_true
0xFD 100:u32	⇒	i8x16.bitmask
0xFD 101:u32	⇒	i8x16.narrow_i16x8_s
0xFD 102:u32	⇒	i8x16.narrow_i16x8_u
0xFD 107:u32	⇒	i8x16.shl
0xFD 108:u32	⇒	i8x16.shr_s
0xFD 109:u32	⇒	i8x16.shr_u
0xFD 110:u32	⇒	i8x16.add
0xFD 111:u32	⇒	i8x16.add_sat_s
0xFD 112:u32	⇒	i8x16.add_sat_u
0xFD 113:u32	⇒	i8x16.sub
0xFD 114:u32	⇒	i8x16.sub_sat_s
0xFD 115:u32	⇒	i8x16.sub_sat_u
0xFD 118:u32	⇒	i8x16.min_s
0xFD 119:u32	⇒	i8x16.min_u
0xFD 120:u32	⇒	i8x16.max_s
0xFD 121:u32	⇒	i8x16.max_u
0xFD 123:u32	⇒	i8x16.avgr_u

0xFD 124:u32	⇒	i16x8.extadd_pairwise_i8x16_s
0xFD 125:u32	⇒	i16x8.extadd_pairwise_i8x16_u
0xFD 128:u32	⇒	i16x8.abs
0xFD 129:u32	⇒	i16x8.neg
0xFD 130:u32	⇒	i16x8.q15mulr_sat_s
0xFD 131:u32	⇒	i16x8.all_true
0xFD 132:u32	⇒	i16x8.bitmask
0xFD 133:u32	⇒	i16x8.narrow_i32x4_s
0xFD 134:u32	⇒	i16x8.narrow_i32x4_u
0xFD 135:u32	⇒	i16x8.extend_low_i8x16_s
0xFD 136:u32	⇒	i16x8.extend_high_i8x16_s
0xFD 137:u32	⇒	i16x8.extend_low_i8x16_u
0xFD 138:u32	⇒	i16x8.extend_high_i8x16_u
0xFD 139:u32	⇒	i16x8.shl
0xFD 140:u32	⇒	i16x8.shr_s
0xFD 141:u32	⇒	i16x8.shr_u
0xFD 142:u32	⇒	i16x8.add
0xFD 143:u32	⇒	i16x8.add_sat_s
0xFD 144:u32	⇒	i16x8.add_sat_u
0xFD 145:u32	⇒	i16x8.sub
0xFD 146:u32	⇒	i16x8.sub_sat_s
0xFD 147:u32	⇒	i16x8.sub_sat_u
0xFD 149:u32	⇒	i16x8.mul
0xFD 150:u32	⇒	i16x8.min_s
0xFD 151:u32	⇒	i16x8.min_u
0xFD 152:u32	⇒	i16x8.max_s
0xFD 153:u32	⇒	i16x8.max_u
0xFD 155:u32	⇒	i16x8.avgr_u
0xFD 156:u32	⇒	i16x8.extmul_low_i8x16_s
0xFD 157:u32	⇒	i16x8.extmul_high_i8x16_s
0xFD 158:u32	⇒	i16x8.extmul_low_i8x16_u
0xFD 159:u32	⇒	i16x8.extmul_high_i8x16_u
0xFD 126:u32	⇒	i32x4.extadd_pairwise_i16x8_s
0xFD 127:u32	⇒	i32x4.extadd_pairwise_i16x8_u
0xFD 160:u32	⇒	i32x4.abs
0xFD 161:u32	⇒	i32x4.neg
0xFD 163:u32	⇒	i32x4.all_true
0xFD 164:u32	⇒	i32x4.bitmask
0xFD 167:u32	⇒	i32x4.extend_low_i16x8_s
0xFD 168:u32	⇒	i32x4.extend_high_i16x8_s
0xFD 169:u32	⇒	i32x4.extend_low_i16x8_u
0xFD 170:u32	⇒	i32x4.extend_high_i16x8_u
0xFD 171:u32	⇒	i32x4.shl
0xFD 172:u32	⇒	i32x4.shr_s
0xFD 173:u32	⇒	i32x4.shr_u
0xFD 174:u32	⇒	i32x4.add
0xFD 177:u32	⇒	i32x4.sub
0xFD 181:u32	⇒	i32x4.mul
0xFD 182:u32	⇒	i32x4.min_s
0xFD 183:u32	⇒	i32x4.min_u
0xFD 184:u32	⇒	i32x4.max_s
0xFD 185:u32	⇒	i32x4.max_u
0xFD 186:u32	⇒	i32x4.dot_i16x8_s
0xFD 188:u32	⇒	i32x4.extmul_low_i16x8_s
0xFD 189:u32	⇒	i32x4.extmul_high_i16x8_s
0xFD 190:u32	⇒	i32x4.extmul_low_i16x8_u
0xFD 191:u32	⇒	i32x4.extmul_high_i16x8_u

0xFD 192:u32	⇒	i64x2.abs
0xFD 193:u32	⇒	i64x2.neg
0xFD 195:u32	⇒	i64x2.all_true
0xFD 196:u32	⇒	i64x2.bitmask
0xFD 199:u32	⇒	i64x2.extend_low_i32x4_s
0xFD 200:u32	⇒	i64x2.extend_high_i32x4_s
0xFD 201:u32	⇒	i64x2.extend_low_i32x4_u
0xFD 202:u32	⇒	i64x2.extend_high_i32x4_u
0xFD 203:u32	⇒	i64x2.shl
0xFD 204:u32	⇒	i64x2.shr_s
0xFD 205:u32	⇒	i64x2.shr_u
0xFD 206:u32	⇒	i64x2.add
0xFD 209:u32	⇒	i64x2.sub
0xFD 213:u32	⇒	i64x2.mul
0xFD 220:u32	⇒	i64x2.extmul_low_i32x4_s
0xFD 221:u32	⇒	i64x2.extmul_high_i32x4_s
0xFD 222:u32	⇒	i64x2.extmul_low_i32x4_u
0xFD 223:u32	⇒	i64x2.extmul_high_i32x4_u

0xFD 103:u32	⇒	f32x4.ceil
0xFD 104:u32	⇒	f32x4.floor
0xFD 105:u32	⇒	f32x4.trunc
0xFD 106:u32	⇒	f32x4.nearest
0xFD 224:u32	⇒	f32x4.abs
0xFD 225:u32	⇒	f32x4.neg
0xFD 227:u32	⇒	f32x4.sqrt
0xFD 228:u32	⇒	f32x4.add
0xFD 229:u32	⇒	f32x4.sub
0xFD 230:u32	⇒	f32x4.mul
0xFD 231:u32	⇒	f32x4.div
0xFD 232:u32	⇒	f32x4.min
0xFD 233:u32	⇒	f32x4.max
0xFD 234:u32	⇒	f32x4.pmin
0xFD 235:u32	⇒	f32x4.pmax

0xFD 116:u32	⇒	f64x2.ceil
0xFD 117:u32	⇒	f64x2.floor
0xFD 122:u32	⇒	f64x2.trunc
0xFD 148:u32	⇒	f64x2.nearest
0xFD 236:u32	⇒	f64x2.abs
0xFD 237:u32	⇒	f64x2.neg
0xFD 239:u32	⇒	f64x2.sqrt
0xFD 240:u32	⇒	f64x2.add
0xFD 241:u32	⇒	f64x2.sub
0xFD 242:u32	⇒	f64x2.mul
0xFD 243:u32	⇒	f64x2.div
0xFD 244:u32	⇒	f64x2.min
0xFD 245:u32	⇒	f64x2.max
0xFD 246:u32	⇒	f64x2.pmin
0xFD 247:u32	⇒	f64x2.pmax

0xFD 248:u32	⇒	i32x4.trunc_sat_f32x4_s
0xFD 249:u32	⇒	i32x4.trunc_sat_f32x4_u
0xFD 250:u32	⇒	f32x4.convert_i32x4_s
0xFD 251:u32	⇒	f32x4.convert_i32x4_u
0xFD 252:u32	⇒	i32x4.trunc_sat_f64x2_s_zero
0xFD 253:u32	⇒	i32x4.trunc_sat_f64x2_u_zero
0xFD 254:u32	⇒	f64x2.convert_low_i32x4_s
0xFD 255:u32	⇒	f64x2.convert_low_i32x4_u
0xFD 94:u32	⇒	f32x4.demote_f64x2_zero
0xFD 95:u32	⇒	f64x2.promote_low_f32x4

5.4.9 Expressions

Expressions are encoded by their instruction sequence terminated with an explicit 0x0B opcode for *end*.

$$\text{expr} ::= (\text{in}:\text{instr})^* \text{0x0B} \Rightarrow \text{in}^* \text{end}$$

5.5 Modules

The binary encoding of modules is organized into *sections*. Most sections correspond to one component of a *module* record, except that *function definitions* are split into two sections, separating their type declarations in the *function section* from their bodies in the *code section*.

Note: This separation enables *parallel* and *streaming* compilation of the functions in a module.

5.5.1 Indices

All *indices* are encoded with their respective value.

<i>typeid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>funcid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>tableid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>memid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>globalid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>elemid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>dataid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>localid</i>	::=	<i>x</i> :u32	⇒	<i>x</i>
<i>labelid</i>	::=	<i>l</i> :u32	⇒	<i>l</i>

5.5.2 Sections

Each section consists of

- a one-byte section *id*,
- the *u32* size of the contents, in bytes,
- the actual *contents*, whose structure is depended on the section *id*.

Every section is optional; an omitted section is equivalent to the section being present with empty contents.

The following parameterized grammar rule defines the generic structure of a section with id N and contents described by the grammar B .

$$\begin{array}{lcl} \text{section}_N(B) & ::= & N:\text{byte } size:\text{u32 } cont:B \Rightarrow cont \quad (\text{if } size = ||B||) \\ & | & \epsilon \quad \Rightarrow \epsilon \end{array}$$

For most sections, the contents B encodes a [vector](#). In these cases, the empty result ϵ is interpreted as the empty vector.

Note: Other than for unknown [custom sections](#), the *size* is not required for decoding, but can be used to skip sections when navigating through a binary. The module is malformed if the size does not match the length of the binary contents B .

The following section ids are used:

Id	Section
0	custom section
1	type section
2	import section
3	function section
4	table section
5	memory section
6	global section
7	export section
8	start section
9	element section
10	code section
11	data section
12	data count section

5.5.3 Custom Section

Custom sections have the id 0. They are intended to be used for debugging information or third-party extensions, and are ignored by the WebAssembly semantics. Their contents consist of a [name](#) further identifying the custom section, followed by an uninterpreted sequence of bytes for custom use.

$$\begin{array}{lcl} \text{customsec} & ::= & \text{section}_0(\text{custom}) \\ \text{custom} & ::= & \text{name byte}^* \end{array}$$

Note: If an implementation interprets the data of a custom section, then errors in that data, or the placement of the section, must not invalidate the module.

5.5.4 Type Section

The *type section* has the id 1. It decodes into a vector of [function types](#) that represent the [types](#) component of a [module](#).

$$\text{typesec} ::= ft^*:\text{section}_1(\text{vec}(\text{functype})) \Rightarrow ft^*$$

5.5.5 Import Section

The *import section* has the id 2. It decodes into a vector of **imports** that represent the **imports** component of a **module**.

```

importsec ::= im*:section2(vec(import)) ⇒ im*
import    ::= mod:name nm:name d:importdesc ⇒ {module mod, name nm, desc d}
importdesc ::= 0x00 x:typeidx                ⇒ func x
            | 0x01 tt:tabletype              ⇒ table tt
            | 0x02 mt:memtype                 ⇒ mem mt
            | 0x03 gt:globaltype              ⇒ global gt

```

5.5.6 Function Section

The *function section* has the id 3. It decodes into a vector of **type indices** that represent the **type** fields of the **functions** in the **funcs** component of a **module**. The **locals** and **body** fields of the respective functions are encoded separately in the **code section**.

```
funcsec ::= x*:section3(vec(typeidx)) ⇒ x*
```

5.5.7 Table Section

The *table section* has the id 4. It decodes into a vector of **tables** that represent the **tables** component of a **module**.

```

tablesec ::= tab*:section4(vec(table)) ⇒ tab*
table    ::= tt:tabletype                ⇒ {type tt}

```

5.5.8 Memory Section

The *memory section* has the id 5. It decodes into a vector of **memories** that represent the **mems** component of a **module**.

```

memsec ::= mem*:section5(vec(mem)) ⇒ mem*
mem    ::= mt:memtype                ⇒ {type mt}

```

5.5.9 Global Section

The *global section* has the id 6. It decodes into a vector of **globals** that represent the **globals** component of a **module**.

```

globalsec ::= glob*:section6(vec(global)) ⇒ glob*
global    ::= gt:globaltype e:expr          ⇒ {type gt, init e}

```

5.5.10 Export Section

The *export section* has the id 7. It decodes into a vector of [exports](#) that represent the [exports](#) component of a module.

```

exportsec ::= ex*:section7(vec(export)) ⇒ ex*
export    ::= nm:name d:exportdesc      ⇒ {name nm, desc d}
exportdesc ::= 0x00 x:funcidx           ⇒ func x
           | 0x01 x:tableidx           ⇒ table x
           | 0x02 x:memidx             ⇒ mem x
           | 0x03 x:globalidx          ⇒ global x

```

5.5.11 Start Section

The *start section* has the id 8. It decodes into an optional [start function](#) that represents the [start](#) component of a module.

```

startsec ::= st?:section8(start) ⇒ st?
start    ::= x:funcidx           ⇒ {func x}

```

5.5.12 Element Section

The *element section* has the id 9. It decodes into a vector of [element segments](#) that represent the [elems](#) component of a module.

```

elemsec ::= seg*:section9(vec(elem)) ⇒ seg*
elem    ::= 0:u32 e:expr y*:vec(funcidx) ⇒
           {type funcref, init ((ref.func y) end)*, mode active {table 0, offset e}}
           | 1:u32 et : elemkind y*:vec(funcidx) ⇒
           {type et, init ((ref.func y) end)*, mode passive}
           | 2:u32 x:tableidx e:expr et : elemkind y*:vec(funcidx) ⇒
           {type et, init ((ref.func y) end)*, mode active {table x, offset e}}
           | 3:u32 et : elemkind y*:vec(funcidx) ⇒
           {type et, init ((ref.func y) end)*, mode declarative}
           | 4:u32 e:expr el*:vec(expr) ⇒
           {type funcref, init el*, mode active {table 0, offset e}}
           | 5:u32 et : reftype el*:vec(expr) ⇒
           {type et, init el*, mode passive}
           | 6:u32 x:tableidx e:expr et : reftype el*:vec(expr) ⇒
           {type et, init el*, mode active {table x, offset e}}
           | 7:u32 et : reftype el*:vec(expr) ⇒
           {type et, init el*, mode declarative}
elemkind ::= 0x00 ⇒ funcref

```

Note: The initial integer can be interpreted as a bitfield. Bit 0 indicates a passive or declarative segment, bit 1 indicates the presence of an explicit table index for an active segment and otherwise distinguishes passive from declarative segments, bit 2 indicates the use of element type and element [expressions](#) instead of element kind and element indices.

Additional element kinds may be added in future versions of WebAssembly.

5.5.13 Code Section

The *code section* has the id 10. It decodes into a vector of *code* entries that are pairs of *value type* vectors and *expressions*. They represent the *locals* and *body* field of the *functions* in the *funcs* component of a *module*. The *type* fields of the respective functions are encoded separately in the *function section*.

The encoding of each code entry consists of

- the *u32 size* of the function code in bytes,
- the actual *function code*, which in turn consists of
 - the declaration of *locals*,
 - the function *body* as an *expression*.

Local declarations are compressed into a vector whose entries consist of

- a *u32 count*,
- a *value type*,

denoting *count* locals of the same value type.

<i>codesec</i>	::=	<i>code</i> *: <i>section</i> ₁₀ (<i>vec</i> (<i>code</i>))	⇒	<i>code</i> *	
<i>code</i>	::=	<i>size</i> : <i>u32</i> <i>code</i> : <i>func</i>	⇒	<i>code</i>	(if <i>size</i> = <i>func</i>)
<i>func</i>	::=	(<i>t</i> *)*: <i>vec</i> (<i>locals</i>) <i>e</i> : <i>expr</i>	⇒	<i>concat</i> ((<i>t</i> *)*), <i>e</i>	(if <i>concat</i> ((<i>t</i> *)*) < 2 ³²)
<i>locals</i>	::=	<i>n</i> : <i>u32</i> <i>t</i> : <i>valtype</i>	⇒	<i>t</i> ^{<i>n</i>}	

Here, *code* ranges over pairs (*valtype**, *expr*). The meta function *concat*((*t**)*) concatenates all sequences *t*_{*i*}* in (*t**)*. Any code for which the length of the resulting sequence is out of bounds of the maximum size of a *vector* is malformed.

Note: Like with *sections*, the code *size* is not needed for decoding, but can be used to skip functions when navigating through a binary. The module is malformed if a size does not match the length of the respective function code.

5.5.14 Data Section

The *data section* has the id 11. It decodes into a vector of *data segments* that represent the *datas* component of a *module*.

<i>datasec</i>	::=	<i>seg</i> *: <i>section</i> ₁₁ (<i>vec</i> (<i>data</i>))	⇒	<i>seg</i> *
<i>data</i>	::=	0: <i>u32</i> <i>e</i> : <i>expr</i> <i>b</i> *: <i>vec</i> (<i>byte</i>)	⇒	{init <i>b</i> *, mode active {memory 0, offset <i>e</i> }}
		1: <i>u32</i> <i>b</i> *: <i>vec</i> (<i>byte</i>)	⇒	{init <i>b</i> *, mode passive}
		2: <i>u32</i> <i>x</i> : <i>memidx</i> <i>e</i> : <i>expr</i> <i>b</i> *: <i>vec</i> (<i>byte</i>)	⇒	{init <i>b</i> *, mode active {memory <i>x</i> , offset <i>e</i> }}

Note: The initial integer can be interpreted as a bitfield. Bit 0 indicates a passive segment, bit 1 indicates the presence of an explicit memory index for an active segment.

In the current version of WebAssembly, at most one memory may be defined or imported in a single module, so all valid *active* data segments have a *memory* value of 0.

5.5.15 Data Count Section

The *data count section* has the id 12. It decodes into an optional `u32` that represents the number of `data segments` in the `data section`. If this count does not match the length of the data segment vector, the module is malformed.

$$\text{datacountsec} ::= n? : \text{section}_{12}(\text{u32}) \Rightarrow n?$$

Note: The data count section is used to simplify single-pass validation. Since the data section occurs after the code section, the `memory.init` and `data.drop` instructions would not be able to check whether the data segment index is valid until the data section is read. The data count section occurs before the code section, so a single-pass validator can use this count instead of deferring validation.

5.5.16 Modules

The encoding of a `module` starts with a preamble containing a 4-byte magic number (the string ‘\0asm’) and a version field. The current version of the WebAssembly binary format is 1.

The preamble is followed by a sequence of `sections`. `Custom sections` may be inserted at any place in this sequence, while other sections must occur at most once and in the prescribed order. All sections can be empty.

The lengths of vectors produced by the (possibly empty) `function` and `code` section must match up.

Similarly, the optional data count must match the length of the `data segment` vector. Furthermore, it must be present

if any `data index` occurs in the code section.

```

magic      ::= 0x00 0x61 0x73 0x6D
version    ::= 0x01 0x00 0x00 0x00
module     ::= magic
              version
              customsec*
              functype*:typesec
              customsec*
              import*:importsec
              customsec*
              typeidxn:funcsec
              customsec*
              table*:tablesec
              customsec*
              mem*:memsec
              customsec*
              global*:globalsec
              customsec*
              export*:exportsec
              customsec*
              start?:startsec
              customsec*
              elem*:elemsec
              customsec*
              m?:datacountsec
              customsec*
              coden:codesec
              customsec*
              datam:datasec
              customsec* ⇒ { types functype*,
                             funcs funcn,
                             tables table*,
                             mems mem*,
                             globals global*,
                             elems elem*,
                             datas datam,
                             start start?,
                             imports import*,
                             exports export* }
              (if m? ≠ ε ∨ dataidx(coden) = ∅)

```

where for each t_i^*, e_i in $code^n$,

$$func^n[i] = \{\text{type } typeidx^n[i], \text{locals } t_i^*, \text{body } e_i\}$$

Note: The version of the WebAssembly binary format may increase in the future if backward-incompatible changes have to be made to the format. However, such changes are expected to occur very infrequently, if ever. The binary format is intended to be forward-compatible, such that future extensions can be made without incrementing its version.

6.1 Conventions

The textual format for WebAssembly **modules** is a rendering of their **abstract syntax** into **S-expressions**³⁶.

Like the **binary format**, the text format is defined by an *attribute grammar*. A text string is a well-formed description of a module if and only if it is generated by the grammar. Each production of this grammar has at most one synthesized attribute: the abstract syntax that the respective character sequence expresses. Thus, the attribute grammar implicitly defines a *parsing* function. Some productions also take a **context** as an inherited attribute that records bound **identifiers**.

Except for a few exceptions, the core of the text grammar closely mirrors the grammar of the abstract syntax. However, it also defines a number of *abbreviations* that are “syntactic sugar” over the core syntax.

The recommended extension for files containing WebAssembly modules in text format is “.wat”. Files with this extension are assumed to be encoded in UTF-8, as per **Unicode**³⁷ (Section 2.5).

6.1.1 Grammar

The following conventions are adopted in defining grammar rules of the text format. They mirror the conventions used for **abstract syntax** and for the **binary format**. In order to distinguish symbols of the textual syntax from symbols of the abstract syntax, **typewriter** font is adopted for the former.

- Terminal symbols are either literal strings of characters enclosed in quotes or expressed as **Unicode**³⁸ scalar values: ‘module’, U+0A. (All characters written literally are unambiguously drawn from the 7-bit **ASCII**³⁹ subset of Unicode.)
- Nonterminal symbols are written in typewriter font: `valtype`, `instr`.
- T^n is a sequence of $n \geq 0$ iterations of T .
- T^* is a possibly empty sequence of iterations of T . (This is a shorthand for T^n used where n is not relevant.)
- T^+ is a sequence of one or more iterations of T . (This is a shorthand for T^n where $n \geq 1$.)
- $T^?$ is an optional occurrence of T . (This is a shorthand for T^n where $n \leq 1$.)

³⁶ <https://en.wikipedia.org/wiki/S-expression>

³⁷ <https://www.unicode.org/versions/latest/>

³⁸ <https://www.unicode.org/versions/latest/>

³⁹ <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

- $x:T$ denotes the same language as the nonterminal T , but also binds the variable x to the attribute synthesized for T .
- Productions are written $\text{sym} ::= T_1 \Rightarrow A_1 \mid \dots \mid T_n \Rightarrow A_n$, where each A_i is the attribute that is synthesized for sym in the given case, usually from attribute variables bound in T_i .
- Some productions are augmented by side conditions in parentheses, which restrict the applicability of the production. They provide a shorthand for a combinatorial expansion of the production into many separate cases.
- If the same meta variable or non-terminal symbol appears multiple times in a production (in the syntax or in an attribute), then all those occurrences must have the same instantiation.
- A distinction is made between *lexical* and *syntactic* productions. For the latter, arbitrary **white space** is allowed in any place where the grammar contains spaces. The productions defining **lexical syntax** and the syntax of **values** are considered lexical, all others are syntactic.

Note: For example, the **textual grammar** for **number types** is given as follows:

$$\begin{array}{llll} \text{numtype} & ::= & \text{'i32'} & \Rightarrow & \text{i32} \\ & & | & & \\ & & \text{'i64'} & \Rightarrow & \text{i64} \\ & & | & & \\ & & \text{'f32'} & \Rightarrow & \text{f32} \\ & & | & & \\ & & \text{'f64'} & \Rightarrow & \text{f64} \end{array}$$

The **textual grammar** for **limits** is defined as follows:

$$\begin{array}{llll} \text{limits} & ::= & n:\text{u32} & \Rightarrow & \{\min n, \max \epsilon\} \\ & & | & & \\ & & n:\text{u32} \ m:\text{u32} & \Rightarrow & \{\min n, \max m\} \end{array}$$

The variables n and m name the attributes of the respective **u32** nonterminals, which in this case are the actual **unsigned integers** those parse into. The attribute of the complete production then is the abstract syntax for the limit, expressed in terms of the former values.

6.1.2 Abbreviations

In addition to the core grammar, which corresponds directly to the **abstract syntax**, the textual syntax also defines a number of *abbreviations* that can be used for convenience and readability.

Abbreviations are defined by *rewrite rules* specifying their expansion into the core syntax:

$$\text{abbreviation syntax} \quad \equiv \quad \text{expanded syntax}$$

These expansions are assumed to be applied, recursively and in order of appearance, before applying the core grammar rules to construct the abstract syntax.

6.1.3 Contexts

The text format allows the use of symbolic **identifiers** in place of **indices**. To resolve these identifiers into concrete indices, some grammar productions are indexed by an *identifier context* I as a synthesized attribute that records the declared identifiers in each **index space**. In addition, the context records the types defined in the module, so that **parameter** indices can be computed for **functions**.

It is convenient to define identifier contexts as *records* I with abstract syntax as follows:

$$I ::= \{ \begin{array}{ll} \text{types} & (\text{id}^?)^*, \\ \text{funcs} & (\text{id}^?)^*, \\ \text{tables} & (\text{id}^?)^*, \\ \text{mems} & (\text{id}^?)^*, \\ \text{globals} & (\text{id}^?)^*, \\ \text{elem} & (\text{id}^?)^*, \\ \text{data} & (\text{id}^?)^*, \\ \text{locals} & (\text{id}^?)^*, \\ \text{labels} & (\text{id}^?)^*, \\ \text{typedefs} & \text{functype}^* \end{array} \}$$

For each index space, such a context contains the list of *identifiers* assigned to the defined indices. Unnamed indices are associated with empty (ϵ) entries in these lists.

An identifier context is *well-formed* if no index space contains duplicate identifiers.

Conventions

To avoid unnecessary clutter, empty components are omitted when writing out identifier contexts. For example, the record $\{\}$ is shorthand for an *identifier context* whose components are all empty.

6.1.4 Vectors

Vectors are written as plain sequences, but with a restriction on the length of these sequence.

$$\text{vec}(\mathbf{A}) ::= (x:\mathbf{A})^n \Rightarrow x^n \quad (\text{if } n < 2^{32})$$

6.2 Lexical Format

6.2.1 Characters

The text format assigns meaning to *source text*, which consists of a sequence of *characters*. Characters are assumed to be represented as valid *Unicode*⁴⁰ (Section 2.4) *scalar values*.

$$\begin{array}{ll} \text{source} & ::= \text{char}^* \\ \text{char} & ::= \text{U+00} \mid \dots \mid \text{U+D7FF} \mid \text{U+E000} \mid \dots \mid \text{U+10FFFF} \end{array}$$

Note: While source text may contain any Unicode character in *comments* or *string* literals, the rest of the grammar is formed exclusively from the characters supported by the 7-bit *ASCII*⁴¹ subset of Unicode.

⁴⁰ <https://www.unicode.org/versions/latest/>

⁴¹ <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

6.2.2 Tokens

The character stream in the source text is divided, from left to right, into a sequence of *tokens*, as defined by the following grammar.

```
token      ::= keyword | uN | sN | fN | string | id | '(' | ')' | reserved
keyword    ::= ('a' | ... | 'z') idchar*      (if occurring as a literal terminal in the grammar)
reserved   ::= (idchar | string)+
```

Tokens are formed from the input character stream according to the *longest match* rule. That is, the next token always consists of the longest possible sequence of characters that is recognized by the above lexical grammar. Tokens can be separated by *white space*, but except for strings, they cannot themselves contain whitespace.

Keyword tokens are defined either implicitly by an occurrence of a *terminal symbol* in literal form, such as ‘keyword’, in a *syntactic* production of this chapter, or explicitly where they arise in this chapter.

Any token that does not fall into any of the other categories is considered *reserved*, and cannot occur in source text.

Note: The effect of defining the set of reserved tokens is that all tokens must be separated by either parentheses, *white space*, or *comments*. For example, ‘0\$x’ is a single reserved token, as is “a”b”. Consequently, they are not recognized as two separate tokens ‘0’ and ‘\$x’, or “a” and “b”, respectively, but instead disallowed. This property of tokenization is not affected by the fact that the definition of reserved tokens overlaps with other token classes.

6.2.3 White Space

White space is any sequence of literal space characters, formatting characters, or *comments*. The allowed formatting characters correspond to a subset of the [ASCII⁴² format effectors](#), namely, *horizontal tabulation* (U+09), *line feed* (U+0A), and *carriage return* (U+0D).

```
space      ::= (' ' | format | comment)*
format     ::= U+09 | U+0A | U+0D
```

The only relevance of white space is to separate *tokens*. It is otherwise ignored.

6.2.4 Comments

A *comment* can either be a *line comment*, started with a double semicolon ‘;;’ and extending to the end of the line, or a *block comment*, enclosed in delimiters ‘(;’ ... ‘;)’ . Block comments can be nested.

```
comment     ::= linecomment | blockcomment
linecomment ::= ‘;;’ linechar* (U+0A | eof)
linechar    ::= c:char                      (if c ≠ U+0A)
blockcomment ::= ‘(;’ blockchar* ‘;)’
blockchar   ::= c:char                      (if c ≠ ‘;’ ∧ c ≠ ‘(’)
              | ‘;’                        (if the next character is not ‘;’)
              | ‘(’                        (if the next character is not ‘;’)
              | blockcomment
```

Here, the pseudo token *eof* indicates the end of the input. The *look-ahead* restrictions on the productions for *blockchar* disambiguate the grammar such that only well-bracketed uses of block comment delimiters are allowed.

Note: Any formatting and control characters are allowed inside comments.

⁴² <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

6.3 Values

The grammar productions in this section define *lexical syntax*, hence no *white space* is allowed.

6.3.1 Integers

All *integers* can be written in either decimal or hexadecimal notation. In both cases, digits can optionally be separated by underscores.

<i>sign</i>	::=	$\epsilon \Rightarrow + \mid '+' \Rightarrow + \mid '-' \Rightarrow -$	
<i>digit</i>	::=	$'0' \Rightarrow 0 \mid \dots \mid '9' \Rightarrow 9$	
<i>hexdigit</i>	::=	$d:\text{digit} \Rightarrow d$	
		$\mid 'A' \Rightarrow 10 \mid \dots \mid 'F' \Rightarrow 15$	
		$\mid 'a' \Rightarrow 10 \mid \dots \mid 'f' \Rightarrow 15$	
<i>num</i>	::=	$d:\text{digit} \Rightarrow d$	
		$\mid n:\text{num} \text{ '}'^? d:\text{digit} \Rightarrow 10 \cdot n + d$	
<i>hexnum</i>	::=	$h:\text{hexdigit} \Rightarrow h$	
		$\mid n:\text{hexnum} \text{ '}'^? h:\text{hexdigit} \Rightarrow 16 \cdot n + h$	

The allowed syntax for integer literals depends on size and signedness. Moreover, their value must lie within the range of the respective type.

<i>uN</i>	::=	$n:\text{num} \Rightarrow n$	(if $n < 2^N$)
		$\mid '0x' n:\text{hexnum} \Rightarrow n$	(if $n < 2^N$)
<i>sN</i>	::=	$\pm:\text{sign} n:\text{num} \Rightarrow \pm n$	(if $-2^{N-1} \leq \pm n < 2^{N-1}$)
		$\mid \pm:\text{sign} '0x' n:\text{hexnum} \Rightarrow \pm n$	(if $-2^{N-1} \leq \pm n < 2^{N-1}$)

Uninterpreted integers can be written as either signed or unsigned, and are normalized to unsigned in the abstract syntax.

<i>iN</i>	::=	$n:\text{uN} \Rightarrow n$	
		$\mid i:\text{sN} \Rightarrow n$	(if $i = \text{signed}(n)$)

6.3.2 Floating-Point

Floating-point values can be represented in either decimal or hexadecimal notation.

<i>frac</i>	::=	$d:\text{digit} \Rightarrow d/10$	
		$\mid d:\text{digit} \text{ '}'^? p:\text{frac} \Rightarrow (d + p/10)/10$	
<i>hexfrac</i>	::=	$h:\text{hexdigit} \Rightarrow h/16$	
		$\mid h:\text{hexdigit} \text{ '}'^? p:\text{hexfrac} \Rightarrow (h + p/16)/16$	
<i>float</i>	::=	$p:\text{num} \text{ '}'^? \Rightarrow p$	
		$\mid p:\text{num} \text{ '}'^? q:\text{frac} \Rightarrow p + q$	
		$\mid p:\text{num} \text{ '}'^? ('E' \mid 'e') \pm:\text{sign} e:\text{num} \Rightarrow p \cdot 10^{\pm e}$	
		$\mid p:\text{num} \text{ '}'^? q:\text{frac} ('E' \mid 'e') \pm:\text{sign} e:\text{num} \Rightarrow (p + q) \cdot 10^{\pm e}$	
<i>hexfloat</i>	::=	$'0x' p:\text{hexnum} \text{ '}'^? \Rightarrow p$	
		$\mid '0x' p:\text{hexnum} \text{ '}'^? q:\text{hexfrac} \Rightarrow p + q$	
		$\mid '0x' p:\text{hexnum} \text{ '}'^? ('P' \mid 'p') \pm:\text{sign} e:\text{num} \Rightarrow p \cdot 2^{\pm e}$	
		$\mid '0x' p:\text{hexnum} \text{ '}'^? q:\text{hexfrac} ('P' \mid 'p') \pm:\text{sign} e:\text{num} \Rightarrow (p + q) \cdot 2^{\pm e}$	

The value of a literal must not lie outside the representable range of the corresponding IEEE 754⁴³ type (that is, a numeric value must not overflow to $\pm\text{infinity}$), but it may be *rounded* to the nearest representable value.

⁴³ <https://ieeexplore.ieee.org/document/8766229>

Note: Rounding can be prevented by using hexadecimal notation with no more significant bits than supported by the required type.

Floating-point values may also be written as constants for *infinity* or *canonical NaN* (*not a number*). Furthermore, arbitrary NaN values may be expressed by providing an explicit payload value.

<code>f_N</code>	<code>::=</code>	<code>±:sign z:f_Nmag</code>	$\Rightarrow$	$\pm z$	
<code>f_Nmag</code>	<code>::=</code>	<code>z:float</code>	$\Rightarrow$	$\text{float}_N(z)$	(if $\text{float}_N(z) \neq \pm\infty$)
		<code>z:hexfloat</code>	$\Rightarrow$	$\text{float}_N(z)$	(if $\text{float}_N(z) \neq \pm\infty$)
		<code>'inf'</code>	$\Rightarrow$	∞	
		<code>'nan'</code>	$\Rightarrow$	$\text{nan}(\text{canon}_N)$	
		<code>'nan:0x' n:hexnum</code>	$\Rightarrow$	$\text{nan}(n)$	(if $1 \leq n < 2^{\text{signif}(N)}$)

6.3.3 Strings

Strings denote sequences of bytes that can represent both textual and binary data. They are enclosed in quotation marks and may contain any character other than ASCII⁴⁴ control characters, quotation marks (""), or backslash ('\'), except when expressed with an *escape sequence*.

<code>string</code>	<code>::=</code>	<code>"" (b*:stringelem)* ""</code>	$\Rightarrow$	$\text{concat}((b^*)^*)$	(if $ \text{concat}((b^*)^*) < 2^{32}$)
<code>stringelem</code>	<code>::=</code>	<code>c:stringchar</code>	$\Rightarrow$	$\text{utf8}(c)$	
		<code>'\ ' n:hexdigit m:hexdigit</code>	$\Rightarrow$	$16 \cdot n + m$	

Each character in a string literal represents the byte sequence corresponding to its UTF-8 Unicode⁴⁵ (Section 2.5) encoding, except for hexadecimal escape sequences `'\hh'`, which represent raw bytes of the respective value.

<code>stringchar</code>	<code>::=</code>	<code>c:char</code>	$\Rightarrow$	c	(if $c \geq \text{U}+20 \wedge c \neq \text{U}+7\text{F} \wedge c \neq \text{' '}' \wedge c \neq \text{'\ '}'$)
		<code>'\t'</code>	$\Rightarrow$	$\text{U}+09$	
		<code>'\n'</code>	$\Rightarrow$	$\text{U}+0\text{A}$	
		<code>'\r'</code>	$\Rightarrow$	$\text{U}+0\text{D}$	
		<code>'\"'</code>	$\Rightarrow$	$\text{U}+22$	
		<code>'\''</code>	$\Rightarrow$	$\text{U}+27$	
		<code>'\\'</code>	$\Rightarrow$	$\text{U}+5\text{C}$	
		<code>'\u{' n:hexnum '}'</code>	$\Rightarrow$	$\text{U}+(n)$	(if $n < 0\text{x}\text{D}800 \vee 0\text{x}\text{E}000 \leq n < 0\text{x}\text{110000}$)

6.3.4 Names

Names are strings denoting a literal character sequence. A name string must form a valid UTF-8 encoding as defined by Unicode⁴⁶ (Section 2.5) and is interpreted as a string of Unicode scalar values.

<code>name</code>	<code>::=</code>	<code>b*:string</code>	$\Rightarrow$	c^*	(if $b^* = \text{utf8}(c^*)$)
-------------------	------------------	------------------------	---------------	-------	--------------------------------

Note: Presuming the source text is itself encoded correctly, strings that do not contain any uses of hexadecimal byte escapes are always valid names.

⁴⁴ <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

⁴⁵ <https://www.unicode.org/versions/latest/>

⁴⁶ <https://www.unicode.org/versions/latest/>

6.3.5 Identifiers

Indices can be given in both numeric and symbolic form. Symbolic *identifiers* that stand in lieu of indices start with ‘\$’, followed by any sequence of printable [ASCII](https://www.ietf.org/rfc/rfc2012.txt)⁴⁷ characters that does not contain a space, quotation mark, comma, semicolon, or bracket.

```

id      ::= '$' idchar+
idchar  ::= '0' | ... | '9'
          | 'A' | ... | 'Z'
          | 'a' | ... | 'z'
          | '!' | '#' | '$' | '%' | '&' | "'" | '*' | '+' | '-' | '.' | '/'
          | ':' | '<' | '=' | '>' | '?' | '@' | '\' | '~' | '_' | '^' | '|' | '~'

```

Conventions

The expansion rules of some abbreviations require insertion of a *fresh* identifier. That may be any syntactically valid identifier that does not already occur in the given source text.

6.4 Types

6.4.1 Number Types

```

numtype ::= 'i32' ⇒ i32
         | 'i64' ⇒ i64
         | 'f32' ⇒ f32
         | 'f64' ⇒ f64

```

6.4.2 Vector Types

```

vectype ::= 'v128' ⇒ v128

```

6.4.3 Reference Types

```

reftype  ::= 'funcref' ⇒ funcref
         | 'externref' ⇒ externref
heaptype ::= 'func'    ⇒ funcref
         | 'extern'    ⇒ externref

```

6.4.4 Value Types

```

valtype ::= t:numtype ⇒ t
         | t:vectype  ⇒ t
         | t:reftype  ⇒ t

```

⁴⁷ <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

6.4.5 Function Types

$$\begin{aligned}
\text{functype} &::= (' \text{'func'} \ t_1^*:\text{vec}(\text{param}) \ t_2^*:\text{vec}(\text{result}) \ ') \Rightarrow [t_1^*] \rightarrow [t_2^*] \\
\text{param} &::= (' \text{'param'} \ \text{id}^? \ t:\text{valtype} \ ') \Rightarrow t \\
\text{result} &::= (' \text{'result'} \ t:\text{valtype} \ ') \Rightarrow t
\end{aligned}$$

Note: The optional identifier names for parameters in a function type only have documentation purpose. They cannot be referenced from anywhere.

Abbreviations

Multiple anonymous parameters or results may be combined into a single declaration:

$$\begin{aligned}
(' \text{'param'} \ \text{valtype}^* \ ') &\equiv ((' \text{'param'} \ \text{valtype} \ '))^* \\
(' \text{'result'} \ \text{valtype}^* \ ') &\equiv ((' \text{'result'} \ \text{valtype} \ '))^*
\end{aligned}$$

6.4.6 Limits

$$\begin{aligned}
\text{limits} &::= \begin{array}{l} n:\text{u32} \\ | \quad n:\text{u32} \ m:\text{u32} \end{array} \Rightarrow \begin{array}{l} \{\min n, \max \epsilon\} \\ \{\min n, \max m\} \end{array}
\end{aligned}$$

6.4.7 Memory Types

$$\text{memtype} ::= \text{lim}:\text{limits} \Rightarrow \text{lim}$$

6.4.8 Table Types

$$\text{tabletype} ::= \text{lim}:\text{limits} \ \text{et}:\text{reftype} \Rightarrow \text{lim} \ \text{et}$$

6.4.9 Global Types

$$\begin{aligned}
\text{globaltype} &::= \begin{array}{l} t:\text{valtype} \\ | \quad (' \text{'mut'} \ t:\text{valtype} \ ') \end{array} \Rightarrow \begin{array}{l} \text{const } t \\ \text{var } t \end{array}
\end{aligned}$$

6.5 Instructions

Instructions are syntactically distinguished into *plain* and *structured* instructions.

$$\begin{aligned}
\text{instr}_I &::= \begin{array}{l} \text{in}:\text{plaininstr}_I \\ | \quad \text{in}:\text{blockinstr}_I \end{array} \Rightarrow \begin{array}{l} \text{in} \\ \text{in} \end{array}
\end{aligned}$$

In addition, as a syntactic abbreviation, instructions can be written as S-expressions in *folded* form, to group them visually.

6.5.1 Labels

Structured control instructions can be annotated with a symbolic label identifier. They are the only symbolic identifiers that can be bound locally in an instruction sequence. The following grammar handles the corresponding update to the identifier context by composing the context with an additional label entry.

$$\begin{array}{lll} \text{label}_I & ::= & v:\text{id} \Rightarrow \{\text{labels } v\} \oplus I \quad (\text{if } v \notin I.\text{labels}) \\ & | & v:\text{id} \Rightarrow \{\text{labels } v\} \oplus (I \text{ with labels}[i] = \epsilon) \quad (\text{if } I.\text{labels}[i] = v) \\ & | & \epsilon \Rightarrow \{\text{labels } (\epsilon)\} \oplus I \end{array}$$

Note: The new label entry is inserted at the *beginning* of the label list in the identifier context. This effectively shifts all existing labels up by one, mirroring the fact that control instructions are indexed relatively not absolutely.

If a label with the same name already exists, then it is shadowed and the earlier label becomes inaccessible.

6.5.2 Control Instructions

Structured control instructions can bind an optional symbolic label identifier. The same label identifier may optionally be repeated after the corresponding end and else pseudo instructions, to indicate the matching delimiters.

Their block type is given as a type use, analogous to the type of functions. However, the special case of a type use that is syntactically empty or consists of only a single result is not regarded as an abbreviation for an inline function type, but is parsed directly into an optional value type.

$$\begin{array}{lll} \text{blocktype}_I & ::= & (t:\text{result})^? \Rightarrow t^? \\ & | & x, I':\text{typeuse}_I \Rightarrow x \quad (\text{if } I' = \{\text{locals } (\epsilon)^*\}) \\ \text{blockinstr}_I & ::= & \text{'block' } I':\text{label}_I \text{ bt}:\text{blocktype}_I (in:\text{instr}_{I'})^* \text{'end' } id^? \\ & \Rightarrow & \text{block } bt \text{ in}^* \text{ end} \quad (\text{if } id^? = \epsilon \vee id^? = \text{label}) \\ & | & \text{'loop' } I':\text{label}_I \text{ bt}:\text{blocktype}_I (in:\text{instr}_{I'})^* \text{'end' } id^? \\ & \Rightarrow & \text{loop } bt \text{ in}^* \text{ end} \quad (\text{if } id^? = \epsilon \vee id^? = \text{label}) \\ & | & \text{'if' } I':\text{label}_I \text{ bt}:\text{blocktype}_I (in_1:\text{instr}_{I'})^* \text{'else' } id_1^? (in_2:\text{instr}_{I'})^* \text{'end' } id_2^? \\ & \Rightarrow & \text{if } bt \text{ in}_1^* \text{ else } in_2^* \text{ end} \quad (\text{if } id_1^? = \epsilon \vee id_1^? = \text{label}, id_2^? = \epsilon \vee id_2^? = \text{label}) \end{array}$$

Note: The side condition stating that the identifier context I' must only contain unnamed entries in the rule for `typeuse` block types enforces that no identifier can be bound in any `param` declaration for a block type.

All other control instruction are represented verbatim.

$$\begin{array}{lll} \text{plaininstr}_I & ::= & \text{'unreachable'} \Rightarrow \text{unreachable} \\ & | & \text{'nop'} \Rightarrow \text{nop} \\ & | & \text{'br' } l:\text{labelidx}_I \Rightarrow \text{br } l \\ & | & \text{'br_if' } l:\text{labelidx}_I \Rightarrow \text{br_if } l \\ & | & \text{'br_table' } l^*:\text{vec}(\text{labelidx}_I) \text{ } l_N:\text{labelidx}_I \Rightarrow \text{br_table } l^* \text{ } l_N \\ & | & \text{'return'} \Rightarrow \text{return} \\ & | & \text{'call' } x:\text{funcidx}_I \Rightarrow \text{call } x \\ & | & \text{'call_indirect' } x:\text{tableidx} \text{ } y, I':\text{typeuse}_I \Rightarrow \text{call_indirect } x \text{ } y \quad (\text{if } I' = \{\text{locals } (\epsilon)^*\}) \end{array}$$

Note: The side condition stating that the identifier context I' must only contain unnamed entries in the rule for `call_indirect` enforces that no identifier can be bound in any `param` declaration appearing in the type annotation.

Abbreviations

The ‘else’ keyword of an ‘if’ instruction can be omitted if the following instruction sequence is empty.

‘if’ label blocktype instr* ‘end’ $\equiv$ ‘if’ label blocktype instr* ‘else’ ‘end’

Also, for backwards compatibility, the table index to ‘call_indirect’ can be omitted, defaulting to 0.

‘call_indirect’ typeuse $\equiv$ ‘call_indirect’ 0 typeuse

6.5.3 Reference Instructions

```
plaininstrI ::= ...
| 'ref.null' t:heaptypes ⇒ ref.null t
| 'ref.is_null' ⇒ ref.is_null
| 'ref.func' x:funcidx ⇒ ref.func x
```

6.5.4 Parametric Instructions

```
plaininstrI ::= ...
| 'drop' ⇒ drop
| 'select' ((t:result)*)? ⇒ select (t*)?
```

6.5.5 Variable Instructions

```
plaininstrI ::= ...
| 'local.get' x:localidxI ⇒ local.get x
| 'local.set' x:localidxI ⇒ local.set x
| 'local.tee' x:localidxI ⇒ local.tee x
| 'global.get' x:globalidxI ⇒ global.get x
| 'global.set' x:globalidxI ⇒ global.set x
```

6.5.6 Table Instructions

```
plaininstrI ::= ...
| 'table.get' x:tableidxI ⇒ table.get x
| 'table.set' x:tableidxI ⇒ table.set x
| 'table.size' x:tableidxI ⇒ table.size x
| 'table.grow' x:tableidxI ⇒ table.grow x
| 'table.fill' x:tableidxI ⇒ table.fill x
| 'table.copy' x:tableidxI y:tableidxI ⇒ table.copy x y
| 'table.init' x:tableidxI y:elemidxI ⇒ table.init x y
| 'elem.drop' x:elemidxI ⇒ elem.drop x
```

Abbreviations

For backwards compatibility, all **table indices** may be omitted from table instructions, defaulting to 0.

<code>'table.get'</code>	$\equiv$	<code>'table.get' 0</code>
<code>'table.set'</code>	$\equiv$	<code>'table.set' 0</code>
<code>'table.size'</code>	$\equiv$	<code>'table.size' 0</code>
<code>'table.grow'</code>	$\equiv$	<code>'table.grow' 0</code>
<code>'table.fill'</code>	$\equiv$	<code>'table.fill' 0</code>
<code>'table.copy'</code>	$\equiv$	<code>'table.copy' 0 0</code>
<code>'table.init' x:elemidx_I</code>	$\equiv$	<code>'table.init' 0 x:elemidx_I</code>

6.5.7 Memory Instructions

The offset and alignment immediates to memory instructions are optional. The offset defaults to 0, the alignment to the storage size of the respective memory access, which is its *natural alignment*. Lexically, an **offset** or **align** phrase is considered a single **keyword token**, so no **white space** is allowed around the '='.

<code>memarg_N</code>	$::=$	<code>o:offset a:align_N</code>	$\Rightarrow$	<code>{align n, offset o}</code> (if $a = 2^n$)
<code>offset</code>	$::=$	<code>'offset='o:u32</code>	$\Rightarrow$	<code>o</code>
		<code> </code>	$\Rightarrow$	<code>0</code>
<code>align_N</code>	$::=$	<code>'align='a:u32</code>	$\Rightarrow$	<code>a</code>
		<code> </code>	$\Rightarrow$	<code>N</code>
<code>plaininstr_I</code>	$::=$	<code>...</code>		
		<code>'i32.load' m:memarg₄</code>	$\Rightarrow$	<code>i32.load m</code>
		<code>'i64.load' m:memarg₈</code>	$\Rightarrow$	<code>i64.load m</code>
		<code>'f32.load' m:memarg₄</code>	$\Rightarrow$	<code>f32.load m</code>
		<code>'f64.load' m:memarg₈</code>	$\Rightarrow$	<code>f64.load m</code>
		<code>'i32.load8_s' m:memarg₁</code>	$\Rightarrow$	<code>i32.load8_s m</code>
		<code>'i32.load8_u' m:memarg₁</code>	$\Rightarrow$	<code>i32.load8_u m</code>
		<code>'i32.load16_s' m:memarg₂</code>	$\Rightarrow$	<code>i32.load16_s m</code>
		<code>'i32.load16_u' m:memarg₂</code>	$\Rightarrow$	<code>i32.load16_u m</code>
		<code>'i64.load8_s' m:memarg₁</code>	$\Rightarrow$	<code>i64.load8_s m</code>
		<code>'i64.load8_u' m:memarg₁</code>	$\Rightarrow$	<code>i64.load8_u m</code>
		<code>'i64.load16_s' m:memarg₂</code>	$\Rightarrow$	<code>i64.load16_s m</code>
		<code>'i64.load16_u' m:memarg₂</code>	$\Rightarrow$	<code>i64.load16_u m</code>
		<code>'i64.load32_s' m:memarg₄</code>	$\Rightarrow$	<code>i64.load32_s m</code>
		<code>'i64.load32_u' m:memarg₄</code>	$\Rightarrow$	<code>i64.load32_u m</code>
		<code>'i32.store' m:memarg₄</code>	$\Rightarrow$	<code>i32.store m</code>
		<code>'i64.store' m:memarg₈</code>	$\Rightarrow$	<code>i64.store m</code>
		<code>'f32.store' m:memarg₄</code>	$\Rightarrow$	<code>f32.store m</code>
		<code>'f64.store' m:memarg₈</code>	$\Rightarrow$	<code>f64.store m</code>
		<code>'i32.store8' m:memarg₁</code>	$\Rightarrow$	<code>i32.store8 m</code>
		<code>'i32.store16' m:memarg₂</code>	$\Rightarrow$	<code>i32.store16 m</code>
		<code>'i64.store8' m:memarg₁</code>	$\Rightarrow$	<code>i64.store8 m</code>
		<code>'i64.store16' m:memarg₂</code>	$\Rightarrow$	<code>i64.store16 m</code>
		<code>'i64.store32' m:memarg₄</code>	$\Rightarrow$	<code>i64.store32 m</code>
		<code>'memory.size'</code>	$\Rightarrow$	<code>memory.size</code>
		<code>'memory.grow'</code>	$\Rightarrow$	<code>memory.grow</code>
		<code>'memory.fill'</code>	$\Rightarrow$	<code>memory.fill</code>
		<code>'memory.copy'</code>	$\Rightarrow$	<code>memory.copy</code>
		<code>'memory.init' x:dataidx_I</code>	$\Rightarrow$	<code>memory.init x</code>
		<code>'data.drop' x:dataidx_I</code>	$\Rightarrow$	<code>data.drop x</code>

6.5.8 Numeric Instructions

```

plaininstrI ::= ...
| 'i32.const' n:i32 ⇒ i32.const n
| 'i64.const' n:i64 ⇒ i64.const n
| 'f32.const' z:f32 ⇒ f32.const z
| 'f64.const' z:f64 ⇒ f64.const z

| 'i32.clz' ⇒ i32.clz
| 'i32.ctz' ⇒ i32.ctz
| 'i32.popcnt' ⇒ i32.popcnt
| 'i32.add' ⇒ i32.add
| 'i32.sub' ⇒ i32.sub
| 'i32.mul' ⇒ i32.mul
| 'i32.div_s' ⇒ i32.div_s
| 'i32.div_u' ⇒ i32.div_u
| 'i32.rem_s' ⇒ i32.rem_s
| 'i32.rem_u' ⇒ i32.rem_u
| 'i32.and' ⇒ i32.and
| 'i32.or' ⇒ i32.or
| 'i32.xor' ⇒ i32.xor
| 'i32.shl' ⇒ i32.shl
| 'i32.shr_s' ⇒ i32.shr_s
| 'i32.shr_u' ⇒ i32.shr_u
| 'i32.rotl' ⇒ i32.rotl
| 'i32.rotr' ⇒ i32.rotr

| 'i64.clz' ⇒ i64.clz
| 'i64.ctz' ⇒ i64.ctz
| 'i64.popcnt' ⇒ i64.popcnt
| 'i64.add' ⇒ i64.add
| 'i64.sub' ⇒ i64.sub
| 'i64.mul' ⇒ i64.mul
| 'i64.div_s' ⇒ i64.div_s
| 'i64.div_u' ⇒ i64.div_u
| 'i64.rem_s' ⇒ i64.rem_s
| 'i64.rem_u' ⇒ i64.rem_u
| 'i64.and' ⇒ i64.and
| 'i64.or' ⇒ i64.or
| 'i64.xor' ⇒ i64.xor
| 'i64.shl' ⇒ i64.shl
| 'i64.shr_s' ⇒ i64.shr_s
| 'i64.shr_u' ⇒ i64.shr_u
| 'i64.rotl' ⇒ i64.rotl
| 'i64.rotr' ⇒ i64.rotr

```

'f32.abs'	⇒	f32.abs
'f32.neg'	⇒	f32.neg
'f32.ceil'	⇒	f32.ceil
'f32.floor'	⇒	f32.floor
'f32.trunc'	⇒	f32.trunc
'f32.nearest'	⇒	f32.nearest
'f32.sqrt'	⇒	f32.sqrt
'f32.add'	⇒	f32.add
'f32.sub'	⇒	f32.sub
'f32.mul'	⇒	f32.mul
'f32.div'	⇒	f32.div
'f32.min'	⇒	f32.min
'f32.max'	⇒	f32.max
'f32.copysign'	⇒	f32.copysign

'f64.abs'	⇒	f64.abs
'f64.neg'	⇒	f64.neg
'f64.ceil'	⇒	f64.ceil
'f64.floor'	⇒	f64.floor
'f64.trunc'	⇒	f64.trunc
'f64.nearest'	⇒	f64.nearest
'f64.sqrt'	⇒	f64.sqrt
'f64.add'	⇒	f64.add
'f64.sub'	⇒	f64.sub
'f64.mul'	⇒	f64.mul
'f64.div'	⇒	f64.div
'f64.min'	⇒	f64.min
'f64.max'	⇒	f64.max
'f64.copysign'	⇒	f64.copysign

'i32.eqz'	⇒	i32.eqz
'i32.eq'	⇒	i32.eq
'i32.ne'	⇒	i32.ne
'i32.lt_s'	⇒	i32.lt_s
'i32.lt_u'	⇒	i32.lt_u
'i32.gt_s'	⇒	i32.gt_s
'i32.gt_u'	⇒	i32.gt_u
'i32.le_s'	⇒	i32.le_s
'i32.le_u'	⇒	i32.le_u
'i32.ge_s'	⇒	i32.ge_s
'i32.ge_u'	⇒	i32.ge_u

'i64.eqz'	⇒	i64.eqz
'i64.eq'	⇒	i64.eq
'i64.ne'	⇒	i64.ne
'i64.lt_s'	⇒	i64.lt_s
'i64.lt_u'	⇒	i64.lt_u
'i64.gt_s'	⇒	i64.gt_s
'i64.gt_u'	⇒	i64.gt_u
'i64.le_s'	⇒	i64.le_s
'i64.le_u'	⇒	i64.le_u
'i64.ge_s'	⇒	i64.ge_s
'i64.ge_u'	⇒	i64.ge_u

	'f32.eq'	⇒	f32.eq
	'f32.ne'	⇒	f32.ne
	'f32.lt'	⇒	f32.lt
	'f32.gt'	⇒	f32.gt
	'f32.le'	⇒	f32.le
	'f32.ge'	⇒	f32.ge
	'f64.eq'	⇒	f64.eq
	'f64.ne'	⇒	f64.ne
	'f64.lt'	⇒	f64.lt
	'f64.gt'	⇒	f64.gt
	'f64.le'	⇒	f64.le
	'f64.ge'	⇒	f64.ge
	'i32.wrap_i64'	⇒	i32.wrap_i64
	'i32.trunc_f32_s'	⇒	i32.trunc_f32_s
	'i32.trunc_f32_u'	⇒	i32.trunc_f32_u
	'i32.trunc_f64_s'	⇒	i32.trunc_f64_s
	'i32.trunc_f64_u'	⇒	i32.trunc_f64_u
	'i32.trunc_sat_f32_s'	⇒	i32.trunc_sat_f32_s
	'i32.trunc_sat_f32_u'	⇒	i32.trunc_sat_f32_u
	'i32.trunc_sat_f64_s'	⇒	i32.trunc_sat_f64_s
	'i32.trunc_sat_f64_u'	⇒	i32.trunc_sat_f64_u
	'i64.extend_i32_s'	⇒	i64.extend_i32_s
	'i64.extend_i32_u'	⇒	i64.extend_i32_u
	'i64.trunc_f32_s'	⇒	i64.trunc_f32_s
	'i64.trunc_f32_u'	⇒	i64.trunc_f32_u
	'i64.trunc_f64_s'	⇒	i64.trunc_f64_s
	'i64.trunc_f64_u'	⇒	i64.trunc_f64_u
	'i64.trunc_sat_f32_s'	⇒	i64.trunc_sat_f32_s
	'i64.trunc_sat_f32_u'	⇒	i64.trunc_sat_f32_u
	'i64.trunc_sat_f64_s'	⇒	i64.trunc_sat_f64_s
	'i64.trunc_sat_f64_u'	⇒	i64.trunc_sat_f64_u
	'f32.convert_i32_s'	⇒	f32.convert_i32_s
	'f32.convert_i32_u'	⇒	f32.convert_i32_u
	'f32.convert_i64_s'	⇒	f32.convert_i64_s
	'f32.convert_i64_u'	⇒	f32.convert_i64_u
	'f32.demote_f64'	⇒	f32.demote_f64
	'f64.convert_i32_s'	⇒	f64.convert_i32_s
	'f64.convert_i32_u'	⇒	f64.convert_i32_u
	'f64.convert_i64_s'	⇒	f64.convert_i64_s
	'f64.convert_i64_u'	⇒	f64.convert_i64_u
	'f64.promote_f32'	⇒	f64.promote_f32
	'i32.reinterpret_f32'	⇒	i32.reinterpret_f32
	'i64.reinterpret_f64'	⇒	i64.reinterpret_f64
	'f32.reinterpret_i32'	⇒	f32.reinterpret_i32
	'f64.reinterpret_i64'	⇒	f64.reinterpret_i64
	'i32.extend8_s'	⇒	i32.extend8_s
	'i32.extend16_s'	⇒	i32.extend16_s
	'i64.extend8_s'	⇒	i64.extend8_s
	'i64.extend16_s'	⇒	i64.extend16_s
	'i64.extend32_s'	⇒	i64.extend32_s

6.5.9 Vector Instructions

Vector memory instructions have optional offset and alignment immediates, like the [memory instructions](#).

<code>plaininstr_I</code>	<code>::=</code>	<code>...</code>	
		<code>'v128.load' m:memarg₁₆</code>	$\Rightarrow$ <code>v128.load m</code>
		<code>'v128.load8x8_s' m:memarg₈</code>	$\Rightarrow$ <code>v128.load8x8_s m</code>
		<code>'v128.load8x8_u' m:memarg₈</code>	$\Rightarrow$ <code>v128.load8x8_u m</code>
		<code>'v128.load16x4_s' m:memarg₈</code>	$\Rightarrow$ <code>v128.load16x4_s m</code>
		<code>'v128.load16x4_u' m:memarg₈</code>	$\Rightarrow$ <code>v128.load16x4_u m</code>
		<code>'v128.load32x2_s' m:memarg₈</code>	$\Rightarrow$ <code>v128.load32x2_s m</code>
		<code>'v128.load32x2_u' m:memarg₈</code>	$\Rightarrow$ <code>v128.load32x2_u m</code>
		<code>'v128.load8_splat' m:memarg₁</code>	$\Rightarrow$ <code>v128.load8_splat m</code>
		<code>'v128.load16_splat' m:memarg₂</code>	$\Rightarrow$ <code>v128.load16_splat m</code>
		<code>'v128.load32_splat' m:memarg₄</code>	$\Rightarrow$ <code>v128.load32_splat m</code>
		<code>'v128.load64_splat' m:memarg₈</code>	$\Rightarrow$ <code>v128.load64_splat m</code>
		<code>'v128.load32_zero' m:memarg₄</code>	$\Rightarrow$ <code>v128.load32_zero m</code>
		<code>'v128.load64_zero' m:memarg₈</code>	$\Rightarrow$ <code>v128.load64_zero m</code>
		<code>'v128.store' m:memarg₁₆</code>	$\Rightarrow$ <code>v128.store m</code>
		<code>'v128.load8_lane' m:memarg₁ laneidx:u8</code>	$\Rightarrow$ <code>v128.load8_lane m laneidx</code>
		<code>'v128.load16_lane' m:memarg₂ laneidx:u8</code>	$\Rightarrow$ <code>v128.load16_lane m laneidx</code>
		<code>'v128.load32_lane' m:memarg₄ laneidx:u8</code>	$\Rightarrow$ <code>v128.load32_lane m laneidx</code>
		<code>'v128.load64_lane' m:memarg₈ laneidx:u8</code>	$\Rightarrow$ <code>v128.load64_lane m laneidx</code>
		<code>'v128.store8_lane' m:memarg₁ laneidx:u8</code>	$\Rightarrow$ <code>v128.store8_lane m laneidx</code>
		<code>'v128.store16_lane' m:memarg₂ laneidx:u8</code>	$\Rightarrow$ <code>v128.store16_lane m laneidx</code>
		<code>'v128.store32_lane' m:memarg₄ laneidx:u8</code>	$\Rightarrow$ <code>v128.store32_lane m laneidx</code>
		<code>'v128.store64_lane' m:memarg₈ laneidx:u8</code>	$\Rightarrow$ <code>v128.store64_lane m laneidx</code>

Vector constant instructions have a mandatory [shape](#) descriptor, which determines how the following values are parsed.

<code>'v128.const' 'i8x16' (n:i8)¹⁶</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{i8}(n)¹⁶)</code>
<code>'v128.const' 'i16x8' (n:i16)⁸</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{i16}(n)⁸)</code>
<code>'v128.const' 'i32x4' (n:i32)⁴</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{i32}(n)⁴)</code>
<code>'v128.const' 'i64x2' (n:i64)²</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{i64}(n)²)</code>
<code>'v128.const' 'f32x4' (z:f32)⁴</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{f32}(z)⁴)</code>
<code>'v128.const' 'f64x2' (z:f64)²</code>	$\Rightarrow$ <code>v128.const bytes_{i128}⁻¹(bytes_{f64}(z)²)</code>
<code>'i8x16.shuffle' (laneidx:u8)¹⁶</code>	$\Rightarrow$ <code>i8x16.shuffle laneidx¹⁶</code>
<code>'i8x16.swizzle'</code>	$\Rightarrow$ <code>i8x16.swizzle</code>
<code>'i8x16.splat'</code>	$\Rightarrow$ <code>i8x16.splat</code>
<code>'i16x8.splat'</code>	$\Rightarrow$ <code>i16x8.splat</code>
<code>'i32x4.splat'</code>	$\Rightarrow$ <code>i32x4.splat</code>
<code>'i64x2.splat'</code>	$\Rightarrow$ <code>i64x2.splat</code>
<code>'f32x4.splat'</code>	$\Rightarrow$ <code>f32x4.splat</code>
<code>'f64x2.splat'</code>	$\Rightarrow$ <code>f64x2.splat</code>

'i8x16.extract_lane_s' laneidx:u8	⇒	i8x16.extract_lane_s laneidx
'i8x16.extract_lane_u' laneidx:u8	⇒	i8x16.extract_lane_u laneidx
'i8x16.replace_lane' laneidx:u8	⇒	i8x16.replace_lane laneidx
'i16x8.extract_lane_s' laneidx:u8	⇒	i16x8.extract_lane_s laneidx
'i16x8.extract_lane_u' laneidx:u8	⇒	i16x8.extract_lane_u laneidx
'i16x8.replace_lane' laneidx:u8	⇒	i16x8.replace_lane laneidx
'i32x4.extract_lane' laneidx:u8	⇒	i32x4.extract_lane laneidx
'i32x4.replace_lane' laneidx:u8	⇒	i32x4.replace_lane laneidx
'i64x2.extract_lane' laneidx:u8	⇒	i64x2.extract_lane laneidx
'i64x2.replace_lane' laneidx:u8	⇒	i64x2.replace_lane laneidx
'f32x4.extract_lane' laneidx:u8	⇒	f32x4.extract_lane laneidx
'f32x4.replace_lane' laneidx:u8	⇒	f32x4.replace_lane laneidx
'f64x2.extract_lane' laneidx:u8	⇒	f64x2.extract_lane laneidx
'f64x2.replace_lane' laneidx:u8	⇒	f64x2.replace_lane laneidx
'i8x16.eq'	⇒	i8x16.eq
'i8x16.ne'	⇒	i8x16.ne
'i8x16.lt_s'	⇒	i8x16.lt_s
'i8x16.lt_u'	⇒	i8x16.lt_u
'i8x16.gt_s'	⇒	i8x16.gt_s
'i8x16.gt_u'	⇒	i8x16.gt_u
'i8x16.le_s'	⇒	i8x16.le_s
'i8x16.le_u'	⇒	i8x16.le_u
'i8x16.ge_s'	⇒	i8x16.ge_s
'i8x16.ge_u'	⇒	i8x16.ge_u
'i16x8.eq'	⇒	i16x8.eq
'i16x8.ne'	⇒	i16x8.ne
'i16x8.lt_s'	⇒	i16x8.lt_s
'i16x8.lt_u'	⇒	i16x8.lt_u
'i16x8.gt_s'	⇒	i16x8.gt_s
'i16x8.gt_u'	⇒	i16x8.gt_u
'i16x8.le_s'	⇒	i16x8.le_s
'i16x8.le_u'	⇒	i16x8.le_u
'i16x8.ge_s'	⇒	i16x8.ge_s
'i16x8.ge_u'	⇒	i16x8.ge_u
'i32x4.eq'	⇒	i32x4.eq
'i32x4.ne'	⇒	i32x4.ne
'i32x4.lt_s'	⇒	i32x4.lt_s
'i32x4.lt_u'	⇒	i32x4.lt_u
'i32x4.gt_s'	⇒	i32x4.gt_s
'i32x4.gt_u'	⇒	i32x4.gt_u
'i32x4.le_s'	⇒	i32x4.le_s
'i32x4.le_u'	⇒	i32x4.le_u
'i32x4.ge_s'	⇒	i32x4.ge_s
'i32x4.ge_u'	⇒	i32x4.ge_u
'i64x2.eq'	⇒	i64x2.eq
'i64x2.ne'	⇒	i64x2.ne
'i64x2.lt_s'	⇒	i64x2.lt_s
'i64x2.gt_s'	⇒	i64x2.gt_s
'i64x2.le_s'	⇒	i64x2.le_s
'i64x2.ge_s'	⇒	i64x2.ge_s

'f32x4.eq'	⇒	f32x4.eq
'f32x4.ne'	⇒	f32x4.ne
'f32x4.lt'	⇒	f32x4.lt
'f32x4.gt'	⇒	f32x4.gt
'f32x4.le'	⇒	f32x4.le
'f32x4.ge'	⇒	f32x4.ge
'f64x2.eq'	⇒	f64x2.eq
'f64x2.ne'	⇒	f64x2.ne
'f64x2.lt'	⇒	f64x2.lt
'f64x2.gt'	⇒	f64x2.gt
'f64x2.le'	⇒	f64x2.le
'f64x2.ge'	⇒	f64x2.ge
'v128.not'	⇒	v128.not
'v128.and'	⇒	v128.and
'v128.andnot'	⇒	v128.andnot
'v128.or'	⇒	v128.or
'v128.xor'	⇒	v128.xor
'v128.bitselect'	⇒	v128.bitselect
'v128.any_true'	⇒	v128.any_true
'i8x16.abs'	⇒	i8x16.abs
'i8x16.neg'	⇒	i8x16.neg
'i8x16.all_true'	⇒	i8x16.all_true
'i8x16.bitmask'	⇒	i8x16.bitmask
'i8x16.narrow_i16x8_s'	⇒	i8x16.narrow_i16x8_s
'i8x16.narrow_i16x8_u'	⇒	i8x16.narrow_i16x8_u
'i8x16.shl'	⇒	i8x16.shl
'i8x16.shr_s'	⇒	i8x16.shr_s
'i8x16.shr_u'	⇒	i8x16.shr_u
'i8x16.add'	⇒	i8x16.add
'i8x16.add_sat_s'	⇒	i8x16.add_sat_s
'i8x16.add_sat_u'	⇒	i8x16.add_sat_u
'i8x16.sub'	⇒	i8x16.sub
'i8x16.sub_sat_s'	⇒	i8x16.sub_sat_s
'i8x16.sub_sat_u'	⇒	i8x16.sub_sat_u
'i8x16.min_s'	⇒	i8x16.min_s
'i8x16.min_u'	⇒	i8x16.min_u
'i8x16.max_s'	⇒	i8x16.max_s
'i8x16.max_u'	⇒	i8x16.max_u
'i8x16.avgr_u'	⇒	i8x16.avgr_u
'i8x16.popcnt'	⇒	i8x16.popcnt

'i16x8.abs'	⇒	i16x8.abs
'i16x8.neg'	⇒	i16x8.neg
'i16x8.all_true'	⇒	i16x8.all_true
'i16x8.bitmask'	⇒	i16x8.bitmask
'i16x8.narrow_i32x4_s'	⇒	i16x8.narrow_i32x4_s
'i16x8.narrow_i32x4_u'	⇒	i16x8.narrow_i32x4_u
'i16x8.extend_low_i8x16_s'	⇒	i16x8.extend_low_i8x16_s
'i16x8.extend_high_i8x16_s'	⇒	i16x8.extend_high_i8x16_s
'i16x8.extend_low_i8x16_u'	⇒	i16x8.extend_low_i8x16_u
'i16x8.extend_high_i8x16_u'	⇒	i16x8.extend_high_i8x16_u
'i16x8.shl'	⇒	i16x8.shl
'i16x8.shr_s'	⇒	i16x8.shr_s
'i16x8.shr_u'	⇒	i16x8.shr_u
'i16x8.add'	⇒	i16x8.add
'i16x8.add_sat_s'	⇒	i16x8.add_sat_s
'i16x8.add_sat_u'	⇒	i16x8.add_sat_u
'i16x8.sub'	⇒	i16x8.sub
'i16x8.sub_sat_s'	⇒	i16x8.sub_sat_s
'i16x8.sub_sat_u'	⇒	i16x8.sub_sat_u
'i16x8.mul'	⇒	i16x8.mul
'i16x8.min_s'	⇒	i16x8.min_s
'i16x8.min_u'	⇒	i16x8.min_u
'i16x8.max_s'	⇒	i16x8.max_s
'i16x8.max_u'	⇒	i16x8.max_u
'i16x8.avgr_u'	⇒	i16x8.avgr_u
'i16x8.q15mulr_sat_s'	⇒	i16x8.q15mulr_sat_s
'i16x8.extmul_low_i8x16_s'	⇒	i16x8.extmul_low_i8x16_s
'i16x8.extmul_high_i8x16_s'	⇒	i16x8.extmul_high_i8x16_s
'i16x8.extmul_low_i8x16_u'	⇒	i16x8.extmul_low_i8x16_u
'i16x8.extmul_high_i8x16_u'	⇒	i16x8.extmul_high_i8x16_u
'i16x8.extadd_pairwise_i8x16_s'	⇒	i16x8.extadd_pairwise_i8x16_s
'i16x8.extadd_pairwise_i8x16_u'	⇒	i16x8.extadd_pairwise_i8x16_u
'i32x4.abs'	⇒	i32x4.abs
'i32x4.neg'	⇒	i32x4.neg
'i32x4.all_true'	⇒	i32x4.all_true
'i32x4.bitmask'	⇒	i32x4.bitmask
'i32x4.extadd_pairwise_i16x8_s'	⇒	i32x4.extadd_pairwise_i16x8_s
'i32x4.extend_low_i16x8_s'	⇒	i32x4.extend_low_i16x8_s
'i32x4.extend_high_i16x8_s'	⇒	i32x4.extend_high_i16x8_s
'i32x4.extend_low_i16x8_u'	⇒	i32x4.extend_low_i16x8_u
'i32x4.extend_high_i16x8_u'	⇒	i32x4.extend_high_i16x8_u
'i32x4.shl'	⇒	i32x4.shl
'i32x4.shr_s'	⇒	i32x4.shr_s
'i32x4.shr_u'	⇒	i32x4.shr_u
'i32x4.add'	⇒	i32x4.add
'i32x4.sub'	⇒	i32x4.sub
'i32x4.mul'	⇒	i32x4.mul
'i32x4.min_s'	⇒	i32x4.min_s
'i32x4.min_u'	⇒	i32x4.min_u
'i32x4.max_s'	⇒	i32x4.max_s
'i32x4.max_u'	⇒	i32x4.max_u
'i32x4.dot_i16x8_s'	⇒	i32x4.dot_i16x8_s
'i32x4.extmul_low_i16x8_s'	⇒	i32x4.extmul_low_i16x8_s
'i32x4.extmul_high_i16x8_s'	⇒	i32x4.extmul_high_i16x8_s
'i32x4.extmul_low_i16x8_u'	⇒	i32x4.extmul_low_i16x8_u
'i32x4.extmul_high_i16x8_u'	⇒	i32x4.extmul_high_i16x8_u

'i64x2.abs'	⇒ i64x2.abs
'i64x2.neg'	⇒ i64x2.neg
'i64x2.all_true'	⇒ i64x2.all_true
'i64x2.bitmask'	⇒ i64x2.bitmask
'i64x2.extend_low_i32x4_s'	⇒ i64x2.extend_low_i32x4_s
'i64x2.extend_high_i32x4_s'	⇒ i64x2.extend_high_i32x4_s
'i64x2.extend_low_i32x4_u'	⇒ i64x2.extend_low_i32x4_u
'i64x2.extend_high_i32x4_u'	⇒ i64x2.extend_high_i32x4_u
'i64x2.shl'	⇒ i64x2.shl
'i64x2.shr_s'	⇒ i64x2.shr_s
'i64x2.shr_u'	⇒ i64x2.shr_u
'i64x2.add'	⇒ i64x2.add
'i64x2.sub'	⇒ i64x2.sub
'i64x2.mul'	⇒ i64x2.mul
'i64x2.extmul_low_i32x4_s'	⇒ i64x2.extmul_low_i32x4_s
'i64x2.extmul_high_i32x4_s'	⇒ i64x2.extmul_high_i32x4_s
'i64x2.extmul_low_i32x4_u'	⇒ i64x2.extmul_low_i32x4_u
'i64x2.extmul_high_i32x4_u'	⇒ i64x2.extmul_high_i32x4_u
'f32x4.abs'	⇒ f32x4.abs
'f32x4.neg'	⇒ f32x4.neg
'f32x4.sqrt'	⇒ f32x4.sqrt
'f32x4.add'	⇒ f32x4.add
'f32x4.sub'	⇒ f32x4.sub
'f32x4.mul'	⇒ f32x4.mul
'f32x4.div'	⇒ f32x4.div
'f32x4.min'	⇒ f32x4.min
'f32x4.max'	⇒ f32x4.max
'f32x4.pmin'	⇒ f32x4.pmin
'f32x4.pmax'	⇒ f32x4.pmax
'f64x2.abs'	⇒ f64x2.abs
'f64x2.neg'	⇒ f64x2.neg
'f64x2.sqrt'	⇒ f64x2.sqrt
'f64x2.add'	⇒ f64x2.add
'f64x2.sub'	⇒ f64x2.sub
'f64x2.mul'	⇒ f64x2.mul
'f64x2.div'	⇒ f64x2.div
'f64x2.min'	⇒ f64x2.min
'f64x2.max'	⇒ f64x2.max
'f64x2.pmin'	⇒ f64x2.pmin
'f64x2.pmax'	⇒ f64x2.pmax
'i32x4.trunc_sat_f32x4_s'	⇒ i32x4.trunc_sat_f32x4_s
'i32x4.trunc_sat_f32x4_u'	⇒ i32x4.trunc_sat_f32x4_u
'i32x4.trunc_sat_f64x2_s_zero'	⇒ i32x4.trunc_sat_f64x2_s_zero
'i32x4.trunc_sat_f64x2_u_zero'	⇒ i32x4.trunc_sat_f64x2_u_zero
'f32x4.convert_i32x4_s'	⇒ f32x4.convert_i32x4_s
'f32x4.convert_i32x4_u'	⇒ f32x4.convert_i32x4_u
'f64x2.convert_low_i32x4_s'	⇒ f64x2.convert_low_i32x4_s
'f64x2.convert_low_i32x4_u'	⇒ f64x2.convert_low_i32x4_u
'f32x4.demote_f64x2_zero'	⇒ f32x4.demote_f64x2_zero
'f64x2.promote_low_f32x4'	⇒ f64x2.promote_low_f32x4

6.5.10 Folded Instructions

Instructions can be written as S-expressions by grouping them into *folded* form. In that notation, an instruction is wrapped in parentheses and optionally includes nested folded instructions to indicate its operands.

In the case of [block instructions](#), the folded form omits the ‘end’ delimiter. For *if* instructions, both branches have to be wrapped into nested S-expressions, headed by the keywords ‘then’ and ‘else’.

The set of all phrases defined by the following abbreviations recursively forms the auxiliary syntactic class *foldedinstr*. Such a folded instruction can appear anywhere a regular instruction can.

<code>(' plaininstr foldedinstr* ')</code>	$\equiv$	<code>foldedinstr* plaininstr</code>
<code>(' 'block' label blocktype instr* ')</code>	$\equiv$	<code>'block' label blocktype instr* 'end'</code>
<code>(' 'loop' label blocktype instr* ')</code>	$\equiv$	<code>'loop' label blocktype instr* 'end'</code>
<code>(' 'if' label blocktype foldedinstr* (' 'then' instr₁ ') (' 'else' instr₂ '))? ' ')</code>	$\equiv$	<code>foldedinstr* 'if' label blocktype instr₁ 'else' (instr₂)? 'end'</code>

Note: For example, the instruction sequence

`(local.get $x) (i32.const 2) i32.add (i32.const 3) i32.mul`

can be folded into

`(i32.mul (i32.add (local.get $x) (i32.const 2)) (i32.const 3))`

Folded instructions are solely syntactic sugar, no additional syntactic or type-based checking is implied.

6.5.11 Expressions

Expressions are written as instruction sequences. No explicit ‘end’ keyword is included, since they only occur in bracketed positions.

$$\text{expr}_I ::= (in:instr_I)^* \Rightarrow in^* \text{end}$$

6.6 Modules

6.6.1 Indices

Indices can be given either in raw numeric form or as symbolic **identifiers** when bound by a respective construct. Such identifiers are looked up in the suitable space of the **identifier context** I .

typeid_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{types}[x] = v$)
funcidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{funcs}[x] = v$)
tableidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{tables}[x] = v$)
memidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{mems}[x] = v$)
globalidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{globals}[x] = v$)
elemidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{elem}[x] = v$)
dataidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{data}[x] = v$)
localidx_I	$::=$	$x::u32$	$\Rightarrow$	x	
		$ $	$v::id$	$\Rightarrow$	x (if $I.\text{locals}[x] = v$)
labelidx_I	$::=$	$l::u32$	$\Rightarrow$	l	
		$ $	$v::id$	$\Rightarrow$	l (if $I.\text{labels}[l] = v$)

6.6.2 Types

Type definitions can bind a symbolic **type identifier**.

$$\text{type} ::= \text{'(' 'type' id? ft:functiontype ')'} \Rightarrow ft$$

6.6.3 Type Uses

A **type use** is a reference to a **type definition**. It may optionally be augmented by explicit inlined **parameter** and **result** declarations. That allows binding symbolic **identifiers** to name the **local indices** of parameters. If inline declarations are given, then their types must match the referenced **function type**.

$$\begin{aligned} \text{typeuse}_I &::= \text{'(' 'type' } x:\text{typeid}_I \text{')'} \Rightarrow x, I' \\ &\quad (\text{if } I.\text{typedefs}[x] = [t_1^*] \rightarrow [t_2^*] \wedge I' = \{\text{locals } (\epsilon)^n\}) \\ &| \text{'(' 'type' } x:\text{typeid}_I \text{')'} (t_1:\text{param})^* (t_2:\text{result})^* \Rightarrow x, I' \\ &\quad (\text{if } I.\text{typedefs}[x] = [t_1^*] \rightarrow [t_2^*] \wedge I' = \{\text{locals id}(\text{param})^*\} \text{ well-formed}) \end{aligned}$$

The synthesized attribute of a **typeuse** is a pair consisting of both the used **type index** and the local **identifier context** containing possible parameter identifiers. The following auxiliary function extracts optional identifiers from parameters:

$$\text{id}(\text{'(' 'param' id? ... ')'}) = \text{id?}$$

Note: Both productions overlap for the case that the function type is $[] \rightarrow []$. However, in that case, they also produce the same results, so that the choice is immaterial.

The **well-formedness** condition on I' ensures that the parameters do not contain duplicate identifiers.

Abbreviations

A **typeuse** may also be replaced entirely by inline **parameter** and **result** declarations. In that case, a **type index** is automatically inserted:

$$(t_1:\text{param})^* (t_2:\text{result})^* \equiv '(\text{'type' } x \text{'})' \text{ param}^* \text{ result}^*$$

where x is the smallest existing **type index** whose definition in the current module is the **function type** $[t_1^*] \rightarrow [t_2^*]$. If no such index exists, then a new **type definition** of the form

$$'(\text{'type' } '(\text{'func' } \text{param}^* \text{result}^* \text{'})' \text{'})'$$

is inserted at the end of the module.

Abbreviations are expanded in the order they appear, such that previously inserted type definitions are reused by consecutive expansions.

6.6.4 Imports

The descriptors in imports can bind a symbolic function, table, memory, or global **identifier**.

```

importI      ::= '(\text{'import' } mod:name nm:name d:importdescI \text{'})'
                  ⇒ {module mod, name nm, desc d}

importdescI ::= '(\text{'func' } id? x, I':typeuseI \text{'})'           ⇒ func x
                  | '(\text{'table' } id? tt:tabletype \text{'})'         ⇒ table tt
                  | '(\text{'memory' } id? mt:memtype \text{'})'           ⇒ mem mt
                  | '(\text{'global' } id? gt:globaltype \text{'})'         ⇒ global gt

```

Abbreviations

As an abbreviation, imports may also be specified inline with **function**, **table**, **memory**, or **global** definitions; see the respective sections.

6.6.5 Functions

Function definitions can bind a symbolic **function identifier**, and **local identifiers** for its **parameters** and **locals**.

```

funcI      ::= '(\text{'func' } id? x, I':typeuseI (t:local)* (in:instrI'')* \text{'})'
                  ⇒ {type x, locals t*, body in* end}
                  (if I'' = I ⊕ I' ⊕ {locals id(local)*} well-formed)

local      ::= '(\text{'local' } id? t:valtype \text{'})' ⇒ t

```

The definition of the local **identifier context** I'' uses the following auxiliary function to extract optional identifiers from locals:

$$\text{id}('(\text{'local' } id^? \dots \text{'})') = id^?$$

Note: The **well-formedness** condition on I'' ensures that parameters and locals do not contain duplicate identifiers.

Abbreviations

Multiple anonymous locals may be combined into a single declaration:

$$(' \text{ 'local' valtype}^* ') \equiv ((' \text{ 'local' valtype} ')^*)$$

Functions can be defined as [imports](#) or [exports](#) inline:

$$\begin{aligned} &(' \text{ 'func' id}^? (' \text{ 'import' name}_1 \text{ name}_2 ') \text{ typeuse} ') \equiv \\ &\quad (' \text{ 'import' name}_1 \text{ name}_2 (' \text{ 'func' id}^? \text{ typeuse} ') ') \\ &(' \text{ 'func' id}^? (' \text{ 'export' name} ') \dots ') \equiv \\ &\quad (' \text{ 'export' name} (' \text{ 'func' id}^? ') ') (' \text{ 'func' id}^? \dots ') \\ &\quad (\text{if id}^? \neq \epsilon \wedge \text{id}' = \text{id}^? \vee \text{id}^? = \epsilon \wedge \text{id}' \text{ fresh}) \end{aligned}$$

Note: The latter abbreviation can be applied repeatedly, if “...” contains additional export clauses. Consequently, a function declaration can contain any number of exports, possibly followed by an import.

6.6.6 Tables

Table definitions can bind a symbolic [table identifier](#).

$$\text{table}_I ::= (' \text{ 'table' id}^? \text{ tt:tabletype} ') \Rightarrow \{\text{type tt}\}$$

Abbreviations

An [element segment](#) can be given inline with a table definition, in which case its offset is 0 and the [limits](#) of the [table type](#) are inferred from the length of the given segment:

$$\begin{aligned} &(' \text{ 'table' id}^? \text{ reftype} (' \text{ 'elem' expr}^n \text{ :vec(elemexpr) } ') ') \equiv \\ &\quad (' \text{ 'table' id}^? n \text{ n reftype} ') \\ &\quad (' \text{ 'elem' } (' \text{ 'table' id}^? ') (' \text{ 'i32.const' '0' } ') \text{ reftype vec(elemexpr) } ') \\ &\quad (\text{if id}^? \neq \epsilon \wedge \text{id}' = \text{id}^? \vee \text{id}^? = \epsilon \wedge \text{id}' \text{ fresh}) \\ &(' \text{ 'table' id}^? \text{ reftype} (' \text{ 'elem' x}^n \text{ :vec(funcidx) } ') ') \equiv \\ &\quad (' \text{ 'table' id}^? n \text{ n reftype} ') \\ &\quad (' \text{ 'elem' } (' \text{ 'table' id}^? ') (' \text{ 'i32.const' '0' } ') \text{ 'func' vec(funcidx) } ') \\ &\quad (\text{if id}^? \neq \epsilon \wedge \text{id}' = \text{id}^? \vee \text{id}^? = \epsilon \wedge \text{id}' \text{ fresh}) \end{aligned}$$

Tables can be defined as [imports](#) or [exports](#) inline:

$$\begin{aligned} &(' \text{ 'table' id}^? (' \text{ 'import' name}_1 \text{ name}_2 ') \text{ tabletype} ') \equiv \\ &\quad (' \text{ 'import' name}_1 \text{ name}_2 (' \text{ 'table' id}^? \text{ tabletype} ') ') \\ &(' \text{ 'table' id}^? (' \text{ 'export' name} ') \dots ') \equiv \\ &\quad (' \text{ 'export' name} (' \text{ 'table' id}^? ') ') (' \text{ 'table' id}^? \dots ') \\ &\quad (\text{if id}^? \neq \epsilon \wedge \text{id}' = \text{id}^? \vee \text{id}^? = \epsilon \wedge \text{id}' \text{ fresh}) \end{aligned}$$

Note: The latter abbreviation can be applied repeatedly, if “...” contains additional export clauses. Consequently, a table declaration can contain any number of exports, possibly followed by an import.

6.6.7 Memories

Memory definitions can bind a symbolic [memory identifier](#).

$$\text{mem}_I ::= \text{'(' 'memory' id? mt:memtype ')'} \Rightarrow \{\text{type } mt\}$$

Abbreviations

A [data segment](#) can be given inline with a memory definition, in which case its offset is 0 and the [limits](#) of the [memory type](#) are inferred from the length of the data, rounded up to [page size](#):

$$\begin{aligned} \text{'(' 'memory' id? (' 'data' b^n:datastring ')')'} &\equiv \\ \text{'(' 'memory' id' m m ')'} & \\ \text{'(' 'data' (' 'memory' id' ') (' 'i32.const' '0' ') datastring ')'} & \\ \text{(if id? \neq \epsilon \wedge id' = id? \vee id? = \epsilon \wedge id' fresh, m = ceil(n/64 Ki))} & \end{aligned}$$

Memories can be defined as [imports](#) or [exports](#) inline:

$$\begin{aligned} \text{'(' 'memory' id? (' 'import' name_1 name_2 ') memtype ')'} &\equiv \\ \text{'(' 'import' name_1 name_2 (' 'memory' id? memtype ')')'} & \\ \text{'(' 'memory' id? (' 'export' name ') ... ')'} &\equiv \\ \text{'(' 'export' name (' 'memory' id' ')') (' 'memory' id' ... ')'} & \\ \text{(if id? \neq \epsilon \wedge id' = id? \vee id? = \epsilon \wedge id' fresh)} & \end{aligned}$$

Note: The latter abbreviation can be applied repeatedly, if “...” contains additional export clauses. Consequently, a memory declaration can contain any number of exports, possibly followed by an import.

6.6.8 Globals

Global definitions can bind a symbolic [global identifier](#).

$$\text{global}_I ::= \text{'(' 'global' id? gt:globaltype e:expr_I ')'} \Rightarrow \{\text{type } gt, \text{init } e\}$$

Abbreviations

Globals can be defined as [imports](#) or [exports](#) inline:

$$\begin{aligned} \text{'(' 'global' id? (' 'import' name_1 name_2 ') globaltype ')'} &\equiv \\ \text{'(' 'import' name_1 name_2 (' 'global' id? globaltype ')')'} & \\ \text{'(' 'global' id? (' 'export' name ') ... ')'} &\equiv \\ \text{'(' 'export' name (' 'global' id' ')') (' 'global' id' ... ')'} & \\ \text{(if id? \neq \epsilon \wedge id' = id? \vee id? = \epsilon \wedge id' fresh)} & \end{aligned}$$

Note: The latter abbreviation can be applied repeatedly, if “...” contains additional export clauses. Consequently, a global declaration can contain any number of exports, possibly followed by an import.

6.6.9 Exports

The syntax for exports mirrors their [abstract syntax](#) directly.

$$\begin{array}{llll}
 \text{export}_I & ::= & \text{'(' 'export' nm:name d:exportdesc}_I \text{')'} & \Rightarrow \{ \text{name nm, desc d} \} \\
 \text{exportdesc}_I & ::= & \text{'(' 'func' x:funcidx}_I \text{')'} & \Rightarrow \text{func } x \\
 & | & \text{'(' 'table' x:tableidx}_I \text{')'} & \Rightarrow \text{table } x \\
 & | & \text{'(' 'memory' x:memidx}_I \text{')'} & \Rightarrow \text{mem } x \\
 & | & \text{'(' 'global' x:globalidx}_I \text{')'} & \Rightarrow \text{global } x
 \end{array}$$

Abbreviations

As an abbreviation, exports may also be specified inline with [function](#), [table](#), [memory](#), or [global](#) definitions; see the respective sections.

6.6.10 Start Function

A [start function](#) is defined in terms of its index.

$$\text{start}_I ::= \text{'(' 'start' x:funcidx}_I \text{')'} \Rightarrow \{ \text{func } x \}$$

Note: At most one start function may occur in a module, which is ensured by a suitable side condition on the [module grammar](#).

6.6.11 Element Segments

Element segments allow for an optional [table index](#) to identify the table to initialize.

$$\begin{array}{llll}
 \text{elem}_I & ::= & \text{'(' 'elem' id? (et,y*):elemlist}_I \text{')'} & \Rightarrow \{ \text{type et, init y*, mode passive} \} \\
 & | & \text{'(' 'elem' id? x:tableuse}_I \text{' (' 'offset' e:expr}_I \text{')' (et,y*):elemlist}_I \text{')'} & \Rightarrow \{ \text{type et, init y*, mode active \{table x, offset e\}} \} \\
 & | & \text{'(' 'elem' id? 'declare' (et,y*):elemlist}_I \text{')'} & \Rightarrow \{ \text{type et, init y*, mode declarative} \} \\
 \text{elemlist}_I & ::= & t:\text{reftype } y*:\text{vec}(\text{elemexpr}_I) & \Rightarrow (\text{type } t, \text{init } y*) \\
 \text{elemexpr}_I & ::= & \text{'(' 'item' e:expr}_I \text{')'} & \Rightarrow e \\
 \text{tableuse}_I & ::= & \text{'(' 'table' x:tableidx}_I \text{')'} & \Rightarrow x
 \end{array}$$

Abbreviations

As an abbreviation, a single instruction may occur in place of the offset of an active element segment or as an element expression:

$$\begin{array}{ll}
 \text{'(' instr ')} & \equiv \text{'(' 'offset' instr ')} \\
 \text{'(' instr ')} & \equiv \text{'(' 'item' instr ')}
 \end{array}$$

Also, the element list may be written as just a sequence of [function indices](#):

$$\text{'func' vec(funcidx}_I) \equiv \text{'funcref' vec('(' 'ref.func' funcidx}_I \text{')')}$$

A table use can be omitted, defaulting to 0. Furthermore, for backwards compatibility with earlier versions of WebAssembly, if the table use is omitted, the 'func' keyword can be omitted as well.

$$\begin{array}{ll}
 \epsilon & \equiv \text{'(' 'table' '0' ')} \\
 \text{'(' 'elem' id? (' 'offset' expr}_I \text{')' vec(funcidx}_I \text{')'} & \equiv \text{'(' 'elem' id? (' 'table' '0' ')} \text{' (' 'offset' expr}_I \text{')'}
 \end{array}$$

As another abbreviation, element segments may also be specified inline with [table](#) definitions; see the respective section.

6.6.12 Data Segments

Data segments allow for an optional [memory index](#) to identify the memory to initialize. The data is written as a [string](#), which may be split up into a possibly empty sequence of individual string literals.

```

dataI      ::= '(' 'data' id? b*:datastring ')'
               ⇒ {init b*, mode passive}
               | '(' 'data' id? x:memuseI '(' 'offset' e:exprI ')' b*:datastring ')'
               ⇒ {init b*, mode active {memory x', offset e}}
datastring  ::= (b*:string)* ⇒ concat((b*)*)
memuseI    ::= '(' 'memory' x:memidxI ')' ⇒ x

```

Note: In the current version of WebAssembly, the only valid memory index is 0 or a symbolic [memory identifier](#) resolving to the same value.

Abbreviations

As an abbreviation, a single instruction may occur in place of the offset of an active data segment:

$$(' \text{instr} ') \equiv (' \text{'offset' instr} ')$$

Also, a memory use can be omitted, defaulting to 0.

$$\epsilon \equiv (' \text{'memory' '0'} ')$$

As another abbreviation, data segments may also be specified inline with [memory](#) definitions; see the respective section.

6.6.13 Modules

A module consists of a sequence of fields that can occur in any order. All definitions and their respective bound [identifiers](#) scope over the entire module, including the text preceding them.

A module may optionally bind an [identifier](#) that names the module. The name serves a documentary role only.

Note: Tools may include the module name in the [name section](#) of the [binary format](#).

```

module      ::= '(' 'module' id? (m:modulefieldI)* ')' ⇒ ⊕ m*
               (if I = ⊕ idc(modulefield)* well-formed)
modulefieldI ::= ty:type           ⇒ {types ty}
               | im:importI       ⇒ {imports im}
               | fn:funcI         ⇒ {funcs fn}
               | ta:tableI        ⇒ {tables ta}
               | me:memI          ⇒ {mems me}
               | gl:globalI       ⇒ {globals gl}
               | ex:exportI       ⇒ {exports ex}
               | st:startI        ⇒ {start st}
               | el:elemI         ⇒ {elems el}
               | da:dataI         ⇒ {datas da}

```

The following restrictions are imposed on the composition of [modules](#): $m_1 \oplus m_2$ is defined if and only if

- $m_1.\text{start} = \epsilon \vee m_2.\text{start} = \epsilon$
- $m_1.\text{funcs} = m_1.\text{tables} = m_1.\text{mems} = m_1.\text{globals} = \epsilon \vee m_2.\text{imports} = \epsilon$

Note: The first condition ensures that there is at most one start function. The second condition enforces that all **imports** must occur before any regular definition of a **function**, **table**, **memory**, or **global**, thereby maintaining the ordering of the respective **index spaces**.

The **well-formedness** condition on I in the grammar for **module** ensures that no namespace contains duplicate identifiers.

The definition of the initial **identifier context** I uses the following auxiliary definition which maps each relevant definition to a singular context with one (possibly empty) identifier:

<code>idc('(' 'type' id? <i>ft</i>:functype ')')</code>	<code>= {types (id?), typedefs <i>ft</i>}</code>
<code>idc('(' 'func' id? ... ')')</code>	<code>= {funcs (id?)}</code>
<code>idc('(' 'table' id? ... ')')</code>	<code>= {tables (id?)}</code>
<code>idc('(' 'memory' id? ... ')')</code>	<code>= {mems (id?)}</code>
<code>idc('(' 'global' id? ... ')')</code>	<code>= {globals (id?)}</code>
<code>idc('(' 'elem' id? ... ')')</code>	<code>= {elem (id?)}</code>
<code>idc('(' 'data' id? ... ')')</code>	<code>= {data (id?)}</code>
<code>idc('(' 'import' ... '(' 'func' id? ... ')')')</code>	<code>= {funcs (id?)}</code>
<code>idc('(' 'import' ... '(' 'table' id? ... ')')')</code>	<code>= {tables (id?)}</code>
<code>idc('(' 'import' ... '(' 'memory' id? ... ')')')</code>	<code>= {mems (id?)}</code>
<code>idc('(' 'import' ... '(' 'global' id? ... ')')')</code>	<code>= {globals (id?)}</code>
<code>idc('(' ... ')')</code>	<code>= {}</code>

Abbreviations

In a source file, the toplevel `(module ...)` surrounding the module body may be omitted.

$$\text{modulefield}^* \equiv '(' \text{'module' modulefield}^* ')'$$

7.1 Embedding

A WebAssembly implementation will typically be *embedded* into a *host* environment. An *embedder* implements the connection between such a host environment and the WebAssembly semantics as defined in the main body of this specification. An embedder is expected to interact with the semantics in well-defined ways.

This section defines a suitable interface to the WebAssembly semantics in the form of entry points through which an embedder can access it. The interface is intended to be complete, in the sense that an embedder does not need to reference other functional parts of the WebAssembly specification directly.

Note: On the other hand, an embedder does not need to provide the host environment with access to all functionality defined in this interface. For example, an implementation may not support [parsing](#) of the [text format](#).

7.1.1 Types

In the description of the embedder interface, syntactic classes from the [abstract syntax](#) and the [runtime's abstract machine](#) are used as names for variables that range over the possible objects from that class. Hence, these syntactic classes can also be interpreted as types.

For numeric parameters, notation like $n : u32$ is used to specify a symbolic name in addition to the respective value range.

7.1.2 Errors

Failure of an interface operation is indicated by an auxiliary syntactic class:

$$error ::= error$$

In addition to the error conditions specified explicitly in this section, implementations may also return errors when specific [implementation limitations](#) are reached.

Note: Errors are abstract and unspecific with this definition. Implementations can refine it to carry suitable classifications and diagnostic messages.

7.1.3 Pre- and Post-Conditions

Some operations state *pre-conditions* about their arguments or *post-conditions* about their results. It is the embedder's responsibility to meet the pre-conditions. If it does, the post conditions are guaranteed by the semantics.

In addition to pre- and post-conditions explicitly stated with each operation, the specification adopts the following conventions for *runtime objects* (*store*, *moduleinst*, *externval*, *addresses*):

- Every runtime object passed as a parameter must be *valid* per an implicit pre-condition.
- Every runtime object returned as a result is *valid* per an implicit post-condition.

Note: As long as an embedder treats runtime objects as abstract and only creates and manipulates them through the interface defined here, all implicit pre-conditions are automatically met.

7.1.4 Store

store_init() : *store*

1. Return the empty *store*.

$$\text{store_init}() = \{\text{funcs } \epsilon, \text{ mems } \epsilon, \text{ tables } \epsilon, \text{ globals } \epsilon\}$$

7.1.5 Modules

module_decode(byte)* : *module* | *error*

1. If there exists a derivation for the *byte* sequence *byte** as a *module* according to the *binary grammar* for *modules*, yielding a *module* *m*, then return *m*.
2. Else, return *error*.

$$\begin{aligned} \text{module_decode}(b^*) &= m && (\text{if } \text{module} \xRightarrow{*} m:b^*) \\ \text{module_decode}(b^*) &= \text{error} && (\text{otherwise}) \end{aligned}$$

module_parse(char)* : *module* | *error*

1. If there exists a derivation for the *source* *char** as a *module* according to the *text grammar* for *modules*, yielding a *module* *m*, then return *m*.
2. Else, return *error*.

$$\begin{aligned} \text{module_parse}(c^*) &= m && (\text{if } \text{module} \xRightarrow{*} m:c^*) \\ \text{module_parse}(c^*) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{module_validate}(\text{module}) : \text{error}^?$

1. If *module* is valid, then return nothing.
2. Else, return error.

$$\begin{aligned} \text{module_validate}(m) &= \epsilon && (\text{if } \vdash m : \text{externtype}^* \rightarrow \text{externtype}'^*) \\ \text{module_validate}(m) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{module_instantiate}(\text{store}, \text{module}, \text{externval}^*) : (\text{store}, \text{moduleinst} \mid \text{error})$

1. Try instantiating *module* in *store* with external values *externval*^{*} as imports:
 - a. If it succeeds with a module instance *moduleinst*, then let *result* be *moduleinst*.
 - b. Else, let *result* be error.
2. Return the new store paired with *result*.

$$\begin{aligned} \text{module_instantiate}(S, m, ev^*) &= (S', F.\text{module}) && (\text{if } \text{instantiate}(S, m, ev^*) \hookrightarrow *S'; F; \epsilon) \\ \text{module_instantiate}(S, m, ev^*) &= (S', \text{error}) && (\text{if } \text{instantiate}(S, m, ev^*) \hookrightarrow *S'; F; \text{trap}) \end{aligned}$$

Note: The store may be modified even in case of an error.

$\text{module_imports}(\text{module}) : (\text{name}, \text{name}, \text{externtype})^*$

1. Pre-condition: *module* is valid with external import types *externtype*^{*} and external export types *externtype*^{'*}.
2. Let *import*^{*} be the imports *module.imports*.
3. Assert: the length of *import*^{*} equals the length of *externtype*^{*}.
4. For each *import*_{*i*} in *import*^{*} and corresponding *externtype*_{*i*} in *externtype*^{*}, do:
 - a. Let *result*_{*i*} be the triple (*import*_{*i*}.module, *import*_{*i*}.name, *externtype*_{*i*}).
5. Return the concatenation of all *result*_{*i*}, in index order.
6. Post-condition: each *externtype*_{*i*} is valid.

$$\begin{aligned} \text{module_imports}(m) &= (im.\text{module}, im.\text{name}, \text{externtype})^* \\ &(\text{if } im^* = m.\text{imports} \wedge \vdash m : \text{externtype}^* \rightarrow \text{externtype}'^*) \end{aligned}$$

$\text{module_exports}(\text{module}) : (\text{name}, \text{externtype})^*$

1. Pre-condition: *module* is valid with external import types *externtype*^{*} and external export types *externtype*^{'*}.
2. Let *export*^{*} be the exports *module.exports*.
3. Assert: the length of *export*^{*} equals the length of *externtype*^{'*}.
4. For each *export*_{*i*} in *export*^{*} and corresponding *externtype*_{*i*}['] in *externtype*^{'*}, do:
 - a. Let *result*_{*i*} be the pair (*export*_{*i*}.name, *externtype*_{*i*}[']).
5. Return the concatenation of all *result*_{*i*}, in index order.
6. Post-condition: each *externtype*_{*i*}['] is valid.

$$\begin{aligned} \text{module_exports}(m) &= (ex.\text{name}, \text{externtype}')^* \\ &\quad (\text{if } ex^* = m.\text{exports} \wedge \vdash m : \text{externtype}^* \rightarrow \text{externtype}'^*) \end{aligned}$$

7.1.6 Module Instances

$\text{instance_export}(\text{moduleinst}, \text{name}) : \text{externval} \mid \text{error}$

1. Assert: due to **validity** of the module instance moduleinst , all its export names are different.
2. If there exists an exportinst_i in $\text{moduleinst}.\text{exports}$ such that $\text{name } \text{exportinst}_i.\text{name}$ equals name , then:
 - a. Return the external value $\text{exportinst}_i.\text{value}$.
3. Else, return **error**.

$$\begin{aligned} \text{instance_export}(m, \text{name}) &= m.\text{exports}[i].\text{value} && (\text{if } m.\text{exports}[i].\text{name} = \text{name}) \\ \text{instance_export}(m, \text{name}) &= \text{error} && (\text{otherwise}) \end{aligned}$$

7.1.7 Functions

$\text{func_alloc}(\text{store}, \text{functype}, \text{hostfunc}) : (\text{store}, \text{funcaddr})$

1. Pre-condition: functype is **valid**.
2. Let funcaddr be the result of allocating a host function in store with function type functype and host function code hostfunc .
3. Return the new store paired with funcaddr .

$$\text{func_alloc}(S, ft, \text{code}) = (S', a) \quad (\text{if } \text{allochostfunc}(S, ft, \text{code}) = S', a)$$

Note: This operation assumes that hostfunc satisfies the **pre- and post-conditions** required for a function instance with type functype .

Regular (non-host) function instances can only be created indirectly through **module instantiation**.

$\text{func_type}(\text{store}, \text{funcaddr}) : \text{functype}$

1. Return $S.\text{funcs}[a].\text{type}$.
2. Post-condition: the returned **function type** is **valid**.

$$\text{func_type}(S, a) = S.\text{funcs}[a].\text{type}$$

$\text{func_invoke}(\text{store}, \text{funcaddr}, \text{val}^*) : (\text{store}, \text{val}^* \mid \text{error})$

1. Try **invoking** the function funcaddr in store with values val^* as arguments:
 - a. If it succeeds with values val'^* as results, then let result be val'^* .
 - b. Else it has trapped, hence let result be **error**.
2. Return the new store paired with result .

$$\begin{aligned} \text{func_invoke}(S, a, v^*) &= (S', v'^*) && (\text{if } \text{invoke}(S, a, v^*) \hookrightarrow^* S'; F; v'^*) \\ \text{func_invoke}(S, a, v^*) &= (S', \text{error}) && (\text{if } \text{invoke}(S, a, v^*) \hookrightarrow^* S'; F; \text{trap}) \end{aligned}$$

Note: The store may be modified even in case of an error.

7.1.8 Tables

$\text{table_alloc}(\text{store}, \text{tabletype}, \text{ref}) : (\text{store}, \text{tableaddr})$

1. Pre-condition: tabletype is valid.
2. Let tableaddr be the result of allocating a table in store with table type tabletype and initialization value ref .
3. Return the new store paired with tableaddr .

$$\text{table_alloc}(S, tt, r) = (S', a) \quad (\text{if } \text{alloctable}(S, tt, r) = S', a)$$

$\text{table_type}(\text{store}, \text{tableaddr}) : \text{tabletype}$

1. Return $S.\text{tables}[a].\text{type}$.
2. Post-condition: the returned table type is valid.

$$\text{table_type}(S, a) = S.\text{tables}[a].\text{type}$$

$\text{table_read}(\text{store}, \text{tableaddr}, i : u32) : \text{ref} \mid \text{error}$

1. Let ti be the table instance $\text{store}.\text{tables}[\text{tableaddr}]$.
2. If i is larger than or equal to the length of $ti.\text{elem}$, then return **error**.
3. Else, return the reference value $ti.\text{elem}[i]$.

$$\begin{aligned} \text{table_read}(S, a, i) &= r && (\text{if } S.\text{tables}[a].\text{elem}[i] = r) \\ \text{table_read}(S, a, i) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{table_write}(\text{store}, \text{tableaddr}, i : u32, \text{ref}) : \text{store} \mid \text{error}$

1. Let ti be the table instance $\text{store}.\text{tables}[\text{tableaddr}]$.
2. If i is larger than or equal to the length of $ti.\text{elem}$, then return **error**.
3. Replace $ti.\text{elem}[i]$ with the reference value ref .
4. Return the updated store.

$$\begin{aligned} \text{table_write}(S, a, i, r) &= S' && (\text{if } S' = S \text{ with } \text{tables}[a].\text{elem}[i] = r) \\ \text{table_write}(S, a, i, r) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{table_size}(\text{store}, \text{tableaddr}) : u32$

1. Return the length of $\text{store.tables}[\text{tableaddr}].\text{elem}$.

$$\text{table_size}(S, a) = n \quad (\text{if } |S.\text{tables}[a].\text{elem}| = n)$$

$\text{table_grow}(\text{store}, \text{tableaddr}, n : u32, \text{ref}) : \text{store} \mid \text{error}$

1. Try growing the table instance $\text{store.tables}[\text{tableaddr}]$ by n elements with initialization value ref :
 - a. If it succeeds, return the updated store.
 - b. Else, return **error**.

$$\begin{aligned} \text{table_grow}(S, a, n, r) &= S' && (\text{if } S' = S \text{ with } \text{tables}[a] = \text{growtable}(S.\text{tables}[a], n, r)) \\ \text{table_grow}(S, a, n, r) &= \text{error} && (\text{otherwise}) \end{aligned}$$

7.1.9 Memories

$\text{mem_alloc}(\text{store}, \text{memtype}) : (\text{store}, \text{memaddr})$

1. Pre-condition: memtype is valid.
2. Let memaddr be the result of allocating a memory in store with memory type memtype .
3. Return the new store paired with memaddr .

$$\text{mem_alloc}(S, mt) = (S', a) \quad (\text{if } \text{allocmem}(S, mt) = S', a)$$

$\text{mem_type}(\text{store}, \text{memaddr}) : \text{memtype}$

1. Return $S.\text{mems}[a].\text{type}$.
2. Post-condition: the returned memory type is valid.

$$\text{mem_type}(S, a) = S.\text{mems}[a].\text{type}$$

$\text{mem_read}(\text{store}, \text{memaddr}, i : u32) : \text{byte} \mid \text{error}$

1. Let mi be the memory instance $\text{store.mems}[\text{memaddr}]$.
2. If i is larger than or equal to the length of $mi.\text{data}$, then return **error**.
3. Else, return the byte $mi.\text{data}[i]$.

$$\begin{aligned} \text{mem_read}(S, a, i) &= b && (\text{if } S.\text{mems}[a].\text{data}[i] = b) \\ \text{mem_read}(S, a, i) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{mem_write}(\text{store}, \text{memaddr}, i : u32, \text{byte}) : \text{store} \mid \text{error}$

1. Let mi be the memory instance $\text{store.mems}[\text{memaddr}]$.
2. If $u32$ is larger than or equal to the length of $mi.data$, then return **error**.
3. Replace $mi.data[i]$ with byte .
4. Return the updated store.

$$\begin{aligned} \text{mem_write}(S, a, i, b) &= S' && (\text{if } S' = S \text{ with } \text{mems}[a].\text{data}[i] = b) \\ \text{mem_write}(S, a, i, b) &= \text{error} && (\text{otherwise}) \end{aligned}$$

$\text{mem_size}(\text{store}, \text{memaddr}) : u32$

1. Return the length of $\text{store.mems}[\text{memaddr}].\text{data}$ divided by the page size.

$$\text{mem_size}(S, a) = n \quad (\text{if } |S.\text{mems}[a].\text{data}| = n \cdot 64 \text{ Ki})$$

$\text{mem_grow}(\text{store}, \text{memaddr}, n : u32) : \text{store} \mid \text{error}$

1. Try growing the memory instance $\text{store.mems}[\text{memaddr}]$ by n pages:
 - a. If it succeeds, return the updated store.
 - b. Else, return **error**.

$$\begin{aligned} \text{mem_grow}(S, a, n) &= S' && (\text{if } S' = S \text{ with } \text{mems}[a] = \text{growmem}(S.\text{mems}[a], n)) \\ \text{mem_grow}(S, a, n) &= \text{error} && (\text{otherwise}) \end{aligned}$$

7.1.10 Globals

$\text{global_alloc}(\text{store}, \text{globaltype}, \text{val}) : (\text{store}, \text{globaladdr})$

1. Pre-condition: globaltype is valid.
2. Let globaladdr be the result of allocating a global in store with global type globaltype and initialization value val .
3. Return the new store paired with globaladdr .

$$\text{global_alloc}(S, gt, v) = (S', a) \quad (\text{if } \text{allocglobal}(S, gt, v) = S', a)$$

$\text{global_type}(\text{store}, \text{globaladdr}) : \text{globaltype}$

1. Return $S.\text{globals}[a].\text{type}$.
2. Post-condition: the returned global type is valid.

$$\text{global_type}(S, a) = S.\text{globals}[a].\text{type}$$

$\text{global_read}(\text{store}, \text{globaladdr}) : \text{val}$

1. Let gi be the global instance $\text{store.globals}[\text{globaladdr}]$.
2. Return the value $gi.\text{value}$.

$$\text{global_read}(S, a) = v \quad (\text{if } S.\text{globals}[a].\text{value} = v)$$

$\text{global_write}(\text{store}, \text{globaladdr}, \text{val}) : \text{store} \mid \text{error}$

1. Let gi be the global instance $\text{store.globals}[\text{globaladdr}]$.
2. Let $\text{mut } t$ be the structure of the global type $gi.\text{type}$.
3. If mut is not `var`, then return `error`.
4. Replace $gi.\text{value}$ with the value val .
5. Return the updated store.

$$\begin{aligned} \text{global_write}(S, a, v) &= S' && (\text{if } S.\text{globals}[a].\text{type} = \text{var } t \wedge S' = S \text{ with } \text{globals}[a].\text{value} = v) \\ \text{global_write}(S, a, v) &= \text{error} && (\text{otherwise}) \end{aligned}$$

7.2 Implementation Limitations

Implementations typically impose additional restrictions on a number of aspects of a WebAssembly module or execution. These may stem from:

- physical resource limits,
- constraints imposed by the embedder or its environment,
- limitations of selected implementation strategies.

This section lists allowed limitations. Where restrictions take the form of numeric limits, no minimum requirements are given, nor are the limits assumed to be concrete, fixed numbers. However, it is expected that all implementations have “reasonably” large limits to enable common applications.

Note: A conforming implementation is not allowed to leave out individual *features*. However, designated subsets of WebAssembly may be specified in the future.

7.2.1 Syntactic Limits

Structure

An implementation may impose restrictions on the following dimensions of a module:

- the number of `types` in a `module`
- the number of `functions` in a `module`, including imports
- the number of `tables` in a `module`, including imports
- the number of `memories` in a `module`, including imports
- the number of `globals` in a `module`, including imports
- the number of `element segments` in a `module`
- the number of `data segments` in a `module`

- the number of [imports](#) to a [module](#)
- the number of [exports](#) from a [module](#)
- the number of parameters in a [function type](#)
- the number of results in a [function type](#)
- the number of parameters in a [block type](#)
- the number of results in a [block type](#)
- the number of [locals](#) in a [function](#)
- the size of a [function body](#)
- the size of a [structured control instruction](#)
- the number of [structured control instructions](#) in a [function](#)
- the nesting depth of [structured control instructions](#)
- the number of [label indices](#) in a [br_table](#) instruction
- the length of an [element segment](#)
- the length of a [data segment](#)
- the length of a [name](#)
- the range of [characters](#) in a [name](#)

If the limits of an implementation are exceeded for a given module, then the implementation may reject the [validation](#), compilation, or [instantiation](#) of that module with an embedder-specific error.

Note: The last item allows [embedders](#) that operate in limited environments without support for [Unicode](#)⁴⁸ to limit the names of [imports](#) and [exports](#) to common subsets like [ASCII](#)⁴⁹.

Binary Format

For a module given in [binary format](#), additional limitations may be imposed on the following dimensions:

- the size of a [module](#)
- the size of any [section](#)
- the size of an individual function's [code](#)
- the number of [sections](#)

Text Format

For a module given in [text format](#), additional limitations may be imposed on the following dimensions:

- the size of the [source text](#)
- the size of any syntactic element
- the size of an individual [token](#)
- the nesting depth of [folded instructions](#)
- the length of symbolic [identifiers](#)
- the range of literal [characters](#) allowed in the [source text](#)

⁴⁸ <https://www.unicode.org/versions/latest/>

⁴⁹ <https://webstore.ansi.org/RecordDetail.aspx?sku=INCITS+4-1986%5bR2012%5d>

7.2.2 Validation

An implementation may defer [validation](#) of individual [functions](#) until they are first [invoked](#).

If a function turns out to be invalid, then the invocation, and every consecutive call to the same function, results in a [trap](#).

Note: This is to allow implementations to use interpretation or just-in-time compilation for functions. The function must still be fully validated before execution of its body begins.

7.2.3 Execution

Restrictions on the following dimensions may be imposed during [execution](#) of a WebAssembly program:

- the number of allocated [module instances](#)
- the number of allocated [function instances](#)
- the number of allocated [table instances](#)
- the number of allocated [memory instances](#)
- the number of allocated [global instances](#)
- the size of a [table instance](#)
- the size of a [memory instance](#)
- the number of [frames](#) on the [stack](#)
- the number of [labels](#) on the [stack](#)
- the number of [values](#) on the [stack](#)

If the runtime limits of an implementation are exceeded during execution of a computation, then it may terminate that computation and report an embedder-specific error to the invoking code.

Some of the above limits may already be verified during instantiation, in which case an implementation may report exceedance in the same manner as for [syntactic limits](#).

Note: Concrete limits are usually not fixed but may be dependent on specifics, interdependent, vary over time, or depend on other implementation- or embedder-specific situations or events.

7.3 Validation Algorithm

The specification of WebAssembly [validation](#) is purely *declarative*. It describes the constraints that must be met by a [module](#) or [instruction](#) sequence to be valid.

This section sketches the skeleton of a sound and complete *algorithm* for effectively validating code, i.e., sequences of [instructions](#). (Other aspects of validation are straightforward to implement.)

In fact, the algorithm is expressed over the flat sequence of opcodes as occurring in the [binary format](#), and performs only a single pass over it. Consequently, it can be integrated directly into a decoder.

The algorithm is expressed in typed pseudo code whose semantics is intended to be self-explanatory.

7.3.1 Data Structures

Types are representable as an enumeration.

```
type val_type = I32 | I64 | F32 | F64 | V128 | Funcref | Externref

func is_num(t : val_type | Unknown) : bool =
  return t = I32 || t = I64 || t = F32 || t = F64 || t = Unknown

func is_vec(t : val_type | Unknown) : bool =
  return t = V128 || t = Unknown

func is_ref(t : val_type | Unknown) : bool =
  return t = Funcref || t = Externref || t = Unknown
```

The algorithm uses two separate stacks: the *value stack* and the *control stack*. The former tracks the types of operand values on the *stack*, the latter surrounding *structured control instructions* and their associated *blocks*.

```
type val_stack = stack(val_type | Unknown)

type ctrl_stack = stack(ctrl_frame)
type ctrl_frame = {
  opcode : opcode
  start_types : list(val_type)
  end_types : list(val_type)
  height : nat
  unreachable : bool
}
```

For each value, the value stack records its *value type*, or *Unknown* when the type is not known.

For each entered block, the control stack records a *control frame* with the originating opcode, the types on the top of the operand stack at the start and end of the block (used to check its result as well as branches), the height of the operand stack at the start of the block (used to check that operands do not underflow the current block), and a flag recording whether the remainder of the block is unreachable (used to handle *stack-polymorphic* typing after branches).

For the purpose of presenting the algorithm, the operand and control stacks are simply maintained as global variables:

```
var vals : val_stack
var ctrls : ctrl_stack
```

However, these variables are not manipulated directly by the main checking function, but through a set of auxiliary functions:

```
func push_val(type : val_type | Unknown) =
  vals.push(type)

func pop_val() : val_type | Unknown =
  if (vals.size() = ctrls[0].height && ctrls[0].unreachable) return Unknown
  error_if(vals.size() = ctrls[0].height)
  return vals.pop()

func pop_val(expect : val_type | Unknown) : val_type | Unknown =
  let actual = pop_val()
  error_if(actual != expect && actual != Unknown && expect != Unknown)
  return actual
```

(continues on next page)

(continued from previous page)

```

func push_vals(types : list(val_type)) = foreach (t in types) push_val(t)
func pop_vals(types : list(val_type)) : list(val_type) =
  var popped := []
  foreach (t in reverse(types)) popped.prepend(pop_val(t))
  return popped

```

Pushing an operand value simply pushes the respective type to the value stack.

Popping an operand value checks that the value stack does not underflow the current block and then removes one type. But first, a special case is handled where the block contains no known values, but has been marked as unreachable. That can occur after an unconditional branch, when the stack is typed [polymorphically](#). In that case, an unknown type is returned.

A second function for popping an operand value takes an expected type, which the actual operand type is checked against. The types may differ in case one of them is Unknown. The function returns the actual type popped from the stack.

Finally, there are accumulative functions for pushing or popping multiple operand types.

Note: The notation `stack[i]` is meant to index the stack from the top, so that, e.g., `ctrls[0]` accesses the element pushed last.

The control stack is likewise manipulated through auxiliary functions:

```

func push_ctrl(opcode : opcode, in : list(val_type), out : list(val_type)) =
  let frame = ctrl_frame(opcode, in, out, vals.size(), false)
  ctrls.push(frame)
  push_vals(in)

func pop_ctrl() : ctrl_frame =
  error_if(ctrls.is_empty())
  let frame = ctrls[0]
  pop_vals(frame.end_types)
  error_if(vals.size() != frame.height)
  ctrls.pop()
  return frame

func label_types(frame : ctrl_frame) : list(val_types) =
  return (if frame.opcode == loop then frame.start_types else frame.end_types)

func unreachable() =
  vals.resize(ctrls[0].height)
  ctrls[0].unreachable := true

```

Pushing a control frame takes the types of the label and result values. It allocates a new frame record recording them along with the current height of the operand stack and marks the block as reachable.

Popping a frame first checks that the control stack is not empty. It then verifies that the operand stack contains the right types of values expected at the end of the exited block and pops them off the operand stack. Afterwards, it checks that the stack has shrunk back to its initial height.

The type of the [label](#) associated with a control frame is either that of the stack at the start or the end of the frame, determined by the opcode that it originates from.

Finally, the current frame can be marked as unreachable. In that case, all existing operand types are purged from the value stack, in order to allow for the [stack-polymorphism](#) logic in `pop_val` to take effect. Because every function has an implicit outermost label that corresponds to an implicit block frame, it is an invariant of the validation algorithm that there always is at least one frame on the control stack when validating an instruction, and hence, `ctrls[0]` is always defined.

Note: Even with the unreachable flag set, consecutive operands are still pushed to and popped from the operand stack. That is necessary to detect invalid [examples](#) like ([unreachable](#) ([i32.const](#)) [i64.add](#)). However, a polymorphic stack cannot underflow, but instead generates `Unknown` types as needed.

7.3.2 Validation of Opcode Sequences

The following function shows the validation of a number of representative instructions that manipulate the stack. Other instructions are checked in a similar manner.

Note: Various instructions not shown here will additionally require the presence of a validation [context](#) for checking uses of [indices](#). That is an easy addition and therefore omitted from this presentation.

```
func validate(opcode) =
  switch (opcode)
  case (i32.add)
    pop_val(I32)
    pop_val(I32)
    push_val(I32)

  case (drop)
    pop_val()

  case (select)
    pop_val(I32)
    let t1 = pop_val()
    let t2 = pop_val()
    error_if(not ((is_num(t1) && is_num(t2)) || (is_vec(t1) && is_vec(t2))))
    error_if(t1 != t2 && t1 != Unknown && t2 != Unknown)
    push_val(if (t1 = Unknown) t2 else t1)

  case (select t)
    pop_val(I32)
    pop_val(t)
    pop_val(t)
    push_val(t)

  case (unreachable)
    unreachable()

  case (block t1*->t2*)
    pop_vals([t1*])
    push_ctrl(block, [t1*], [t2*])

  case (loop t1*->t2*)
    pop_vals([t1*])
    push_ctrl(loop, [t1*], [t2*])

  case (if t1*->t2*)
    pop_val(I32)
    pop_vals([t1*])
    push_ctrl(if, [t1*], [t2*])

  case (end)
```

(continues on next page)

(continued from previous page)

```

    let frame = pop_ctrl()
    push_vals(frame.end_types)

  case (else)
    let frame = pop_ctrl()
    error_if(frame.opcode != if)
    push_ctrl(else, frame.start_types, frame.end_types)

  case (br n)
    error_if(ctrls.size() < n)
    pop_vals(label_types(ctrls[n]))
    unreachable()

  case (br_if n)
    error_if(ctrls.size() < n)
    pop_val(I32)
    pop_vals(label_types(ctrls[n]))
    push_vals(label_types(ctrls[n]))

  case (br_table n* m)
    pop_val(I32)
    error_if(ctrls.size() < m)
    let arity = label_types(ctrls[m]).size()
    foreach (n in n*)
      error_if(ctrls.size() < n)
      error_if(label_types(ctrls[n]).size() != arity)
      push_vals(pop_vals(label_types(ctrls[n])))
    pop_vals(label_types(ctrls[m]))
    unreachable()

```

Note: It is an invariant under the current WebAssembly instruction set that an operand of `Unknown` type is never duplicated on the stack. This would change if the language were extended with stack instructions like `dup`. Under such an extension, the above algorithm would need to be refined by replacing the `Unknown` type with proper *type variables* to ensure that all uses are consistent.

7.4 Custom Sections

This appendix defines dedicated [custom sections](#) for WebAssembly's [binary format](#). Such sections do not contribute to, or otherwise affect, the WebAssembly semantics, and like any custom section they may be ignored by an implementation. However, they provide useful meta data that implementations can make use of to improve user experience or take compilation hints.

Currently, only one dedicated custom section is defined, the [name section](#).

7.4.1 Name Section

The *name section* is a [custom section](#) whose name string is itself ‘name’. The name section should appear only once in a module, and only after the [data section](#).

The purpose of this section is to attach printable names to definitions in a module, which e.g. can be used by a debugger or when parts of the module are to be rendered in [text form](#).

Note: All [names](#) are represented in [Unicode](#)⁵⁰ encoded in UTF-8. Names need not be unique.

Subsections

The [data](#) of a name section consists of a sequence of *subsections*. Each subsection consists of a

- a one-byte subsection *id*,
- the *u32* *size* of the contents, in bytes,
- the actual *contents*, whose structure is dependent on the subsection id.

```

namesec          ::= section0(namedata)
namedata         ::= n:name (if n = ‘name’)
                   modulenamesubsec?
                   funcnamesubsec?
                   localnamesubsec?
namesubsectionN(B) ::= N:byte size:u32 B (if size = ||B||)

```

The following subsection ids are used:

Id	Subsection
0	module name
1	function names
2	local names

Each subsection may occur at most once, and in order of increasing id.

Name Maps

A *name map* assigns [names](#) to [indices](#) in a given [index space](#). It consists of a [vector](#) of index/name pairs in order of increasing index value. Each index must be unique, but the assigned names need not be.

```

namemap          ::= vec(nameassoc)
nameassoc         ::= idx name

```

An *indirect name map* assigns [names](#) to a two-dimensional [index space](#), where secondary indices are *grouped* by primary indices. It consists of a vector of primary index/name map pairs in order of increasing index value, where each name map in turn maps secondary indices to names. Each primary index must be unique, and likewise each secondary index per individual name map.

```

indirectnamemap   ::= vec(indirectnameassoc)
indirectnameassoc ::= idx namemap

```

⁵⁰ <https://www.unicode.org/versions/latest/>

Module Names

The *module name subsection* has the id 0. It simply consists of a single **name** that is assigned to the module itself.

$$\text{modulenamesubsec} ::= \text{namesubsection}_0(\text{name})$$

Function Names

The *function name subsection* has the id 1. It consists of a **name map** assigning function names to **function indices**.

$$\text{funcnamesubsec} ::= \text{namesubsection}_1(\text{namemap})$$

Local Names

The *local name subsection* has the id 2. It consists of an **indirect name map** assigning local names to **local indices** grouped by **function indices**.

$$\text{localnamesubsec} ::= \text{namesubsection}_2(\text{indirectnamemap})$$

7.5 Soundness

The **type system** of WebAssembly is *sound*, implying both *type safety* and *memory safety* with respect to the WebAssembly semantics. For example:

- All types declared and derived during validation are respected at run time; e.g., every **local** or **global** variable will only contain type-correct values, every **instruction** will only be applied to operands of the expected type, and every **function invocation** always evaluates to a result of the right type (if it does not **trap** or **diverge**).
- No memory location will be read or written except those explicitly defined by the program, i.e., as a **local**, a **global**, an element in a **table**, or a location within a linear **memory**.
- There is no undefined behavior, i.e., the **execution rules** cover all possible cases that can occur in a **valid** program, and the rules are mutually consistent.

Soundness also is instrumental in ensuring additional properties, most notably, *encapsulation* of function and module scopes: no **locals** can be accessed outside their own function and no **module** components can be accessed outside their own module unless they are explicitly **exported** or **imported**.

The typing rules defining WebAssembly **validation** only cover the *static* components of a WebAssembly program. In order to state and prove soundness precisely, the typing rules must be extended to the *dynamic* components of the abstract **runtime**, that is, the **store**, **configurations**, and **administrative instructions**.⁵¹

⁵¹ The formalization and theorems are derived from the following article: Andreas Haas, Andreas Rossberg, Derek Schuff, Ben Titzer, Dan Gohman, Luke Wagner, Alon Zakai, JF Bastien, Michael Holman. [Bringing the Web up to Speed with WebAssembly](#)Page 200, ⁵². Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). ACM 2017.

⁵² <https://dl.acm.org/citation.cfm?doid=3062341.3062363>

7.5.1 Results

Results can be classified by **result types** as follows.

Results val^*

- For each value val_i in val^* :
 - The value val_i is **valid** with some **value type** t_i .
- Let t^* be the concatenation of all t_i .
- Then the result is valid with **result type** $[t^*]$.

$$\frac{(S \vdash val : t)^*}{S \vdash val^* : [t^*]}$$

Results trap

- The result is valid with **result type** $[t^*]$, for any sequence t^* of **value types**.

$$\overline{S \vdash \text{trap} : [t^*]}$$

7.5.2 Store Validity

The following typing rules specify when a runtime **store** S is **valid**. A valid store must consist of **function**, **table**, **memory**, **global**, and **module** instances that are themselves valid, relative to S .

To that end, each kind of instance is classified by a respective **function**, **table**, **memory**, or **global** type. Module instances are classified by *module contexts*, which are regular **contexts** repurposed as module types describing the **index spaces** defined by a module.

Store S

- Each **function** instance $funcinst_i$ in $S.funcs$ must be **valid** with some **function type** $functype_i$.
- Each **table** instance $tableinst_i$ in $S.tables$ must be **valid** with some **table type** $tabletype_i$.
- Each **memory** instance $meminst_i$ in $S.mems$ must be **valid** with some **memory type** $memtype_i$.
- Each **global** instance $globalinst_i$ in $S.globals$ must be **valid** with some **global type** $globaltype_i$.
- Each **element** instance $eleminst_i$ in $S.elems$ must be **valid** with some **reference type** $reftype_i$.
- Each **data** instance $datainst_i$ in $S.datas$ must be **valid**.
- Then the store is **valid**.

$$\frac{\begin{array}{cc} (S \vdash funcinst : functype)^* & (S \vdash tableinst : tabletype)^* \\ (S \vdash meminst : memtype)^* & (S \vdash globalinst : globaltype)^* \\ (S \vdash eleminst : reftype)^* & (S \vdash datainst \text{ ok})^* \end{array}}{S = \{funcs\ funcinst^*, tables\ tableinst^*, mems\ meminst^*, globals\ globalinst^*, elems\ eleminst^*, datas\ datainst^*\} \vdash S \text{ ok}}$$

Function Instances $\{\text{type } \textit{functype}, \text{module } \textit{moduleinst}, \text{code } \textit{func}\}$

- The function type $\textit{functype}$ must be valid.
- The module instance $\textit{moduleinst}$ must be valid with some context C .
- Under context C , the function $\textit{func}$ must be valid with function type $\textit{functype}$.
- Then the function instance is valid with function type $\textit{functype}$.

$$\frac{\vdash \textit{functype} \text{ ok} \quad S \vdash \textit{moduleinst} : C \quad C \vdash \textit{func} : \textit{functype}}{S \vdash \{\text{type } \textit{functype}, \text{module } \textit{moduleinst}, \text{code } \textit{func}\} : \textit{functype}}$$

Host Function Instances $\{\text{type } \textit{functype}, \text{hostcode } \textit{hf}\}$

- The function type $\textit{functype}$ must be valid.
- Let $[t_1^*] \rightarrow [t_2^*]$ be the function type $\textit{functype}$.
- For every valid store S_1 extending S and every sequence $\textit{val}^*$ of values whose types coincide with t_1^* :
 - Executing $\textit{hf}$ in store S_1 with arguments $\textit{val}^*$ has a non-empty set of possible outcomes.
 - For every element R of this set:
 - * Either R must be $\perp$ (i.e., divergence).
 - * Or R consists of a valid store S_2 extending S_1 and a result $\textit{result}$ whose type coincides with t_2^* .
- Then the function instance is valid with function type $\textit{functype}$.

$$\frac{\begin{array}{l} \forall S_1, \textit{val}^*, \vdash S_1 \text{ ok} \wedge \vdash S \preceq S_1 \wedge S_1 \vdash \textit{val}^* : [t_1^*] \implies \\ \quad \textit{hf}(S_1; \textit{val}^*) \supset \emptyset \wedge \\ \quad \forall R \in \textit{hf}(S_1; \textit{val}^*), R = \perp \vee \\ \quad \exists S_2, \textit{result}, \vdash S_2 \text{ ok} \wedge \vdash S_1 \preceq S_2 \wedge S_2 \vdash \textit{result} : [t_2^*] \wedge R = (S_2; \textit{result}) \end{array}}{\vdash [t_1^*] \rightarrow [t_2^*] \text{ ok} \quad S \vdash \{\text{type } [t_1^*] \rightarrow [t_2^*], \text{hostcode } \textit{hf}\} : [t_1^*] \rightarrow [t_2^*]}$$

Note: This rule states that, if appropriate pre-conditions about store and arguments are satisfied, then executing the host function must satisfy appropriate post-conditions about store and results. The post-conditions match the ones in the [execution rule](#) for invoking host functions.

Any store under which the function is invoked is assumed to be an extension of the current store. That way, the function itself is able to make sufficient assumptions about future stores.

Table Instances $\{\text{type } (\textit{limits } t), \text{elem } \textit{ref}^*\}$

- The table type $\textit{limits } t$ must be valid.
- The length of $\textit{ref}^*$ must equal $\textit{limits.min}$.
- For each reference $\textit{ref}_i$ in the table's elements $\textit{ref}^n$:
 - The reference $\textit{ref}_i$ must be valid with reference type t .
- Then the table instance is valid with table type $\textit{limits } t$.

$$\frac{\vdash \textit{limits } t \text{ ok} \quad n = \textit{limits.min} \quad (S \vdash \textit{ref} : t)^n}{S \vdash \{\text{type } (\textit{limits } t), \text{elem } \textit{ref}^n\} : \textit{limits } t}$$

Memory Instances $\{\text{type } \textit{limits}, \text{data } b^*\}$

- The memory type $\textit{limits}$ must be valid.
- The length of b^* must equal $\textit{limits}.\text{min}$ multiplied by the page size 64 Ki.
- Then the memory instance is valid with memory type $\textit{limits}$.

$$\frac{\vdash \textit{limits} \text{ ok} \quad n = \textit{limits}.\text{min} \cdot 64 \text{ Ki}}{S \vdash \{\text{type } \textit{limits}, \text{data } b^n\} : \textit{limits}}$$

Global Instances $\{\text{type } (\textit{mut } t), \text{value } \textit{val}\}$

- The global type $\textit{mut } t$ must be valid.
- The value $\textit{val}$ must be valid with value type t .
- Then the global instance is valid with global type $\textit{mut } t$.

$$\frac{\vdash \textit{mut } t \text{ ok} \quad S \vdash \textit{val} : t}{S \vdash \{\text{type } (\textit{mut } t), \text{value } \textit{val}\} : \textit{mut } t}$$

Element Instances $\{\text{type } t, \text{elem } \textit{ref}^*\}$

- For each reference $\textit{ref}_i$ in the elements $\textit{ref}^n$:
 - The reference $\textit{ref}_i$ must be valid with reference type t .
- Then the element instance is valid with reference type t .

$$\frac{(S \vdash \textit{ref} : t)^*}{S \vdash \{\text{type } t, \text{elem } \textit{ref}^*\} : t}$$

Data Instances $\{\text{data } b^*\}$

- The data instance is valid.

$$\overline{S \vdash \{\text{data } b^*\} \text{ ok}}$$

Export Instances $\{\text{name } \textit{name}, \text{value } \textit{externval}\}$

- The external value $\textit{externval}$ must be valid with some external type $\textit{externtype}$.
- Then the export instance is valid.

$$\frac{S \vdash \textit{externval} : \textit{externtype}}{S \vdash \{\text{name } \textit{name}, \text{value } \textit{externval}\} \text{ ok}}$$

Module Instances *moduleinst*

- Each function type $functype_i$ in $moduleinst.types$ must be valid.
- For each function address $funcaddr_i$ in $moduleinst.funcaddrs$, the external value $func\ funcaddr_i$ must be valid with some external type $func\ functype'_i$.
- For each table address $tableaddr_i$ in $moduleinst.tableaddrs$, the external value $table\ tableaddr_i$ must be valid with some external type $table\ tabletype_i$.
- For each memory address $memaddr_i$ in $moduleinst.memaddrs$, the external value $mem\ memaddr_i$ must be valid with some external type $mem\ memtype_i$.
- For each global address $globaladdr_i$ in $moduleinst.globaladdrs$, the external value $global\ globaladdr_i$ must be valid with some external type $global\ globaltype_i$.
- For each element address $elemaddr_i$ in $moduleinst.elemaddrs$, the element instance $S.elems[elemaddr_i]$ must be valid with some reference type $reftype_i$.
- For each data address $dataaddr_i$ in $moduleinst.dataaddrs$, the data instance $S.datas[dataaddr_i]$ must be valid.
- Each export instance $exportinst_i$ in $moduleinst.exports$ must be valid.
- For each export instance $exportinst_i$ in $moduleinst.exports$, the name $exportinst_i.name$ must be different from any other name occurring in $moduleinst.exports$.
- Let $functype'^*$ be the concatenation of all $functype'_i$ in order.
- Let $tabletype^*$ be the concatenation of all $tabletype_i$ in order.
- Let $memtype^*$ be the concatenation of all $memtype_i$ in order.
- Let $globaltype^*$ be the concatenation of all $globaltype_i$ in order.
- Let $reftype^*$ be the concatenation of all $reftype_i$ in order.
- Let n be the length of $moduleinst.dataaddrs$.
- Then the module instance is valid with context $\{types\ functype^*,\ funcs\ functype'^*,\ tables\ tabletype^*,\ mems\ memtype^*,\ globals\ globaltype^*,\ elems\ reftype^*,\ datas\ ok^n\}$.

$$\frac{
\begin{array}{l}
(\vdash functype\ ok)^* \\
(S \vdash func\ funcaddr : func\ functype')^* \quad (S \vdash table\ tableaddr : table\ tabletype)^* \\
(S \vdash mem\ memaddr : mem\ memtype)^* \quad (S \vdash global\ globaladdr : global\ globaltype)^* \\
(S \vdash S.elems[elemaddr] : reftype)^* \quad (S \vdash S.datas[dataaddr] ok)^n \\
(S \vdash exportinst\ ok)^* \quad (exportinst.name)^* \text{ disjoint}
\end{array}
}{
\begin{array}{l}
S \vdash \{types\ functype^*, \\
\quad funcaddrs\ funcaddr^*, \\
\quad tableaddrs\ tableaddr^*, \\
\quad memaddrs\ memaddr^*, \\
\quad globaladdrs\ globaladdr^*, \\
\quad elemaddrs\ elemaddr^*, \\
\quad dataaddrs\ dataaddr^n, \\
\quad exports\ exportinst^*\} : \{types\ functype^*, \\
\quad funcs\ functype'^*, \\
\quad tables\ tabletype^*, \\
\quad mems\ memtype^*, \\
\quad globals\ globaltype^*, \\
\quad elems\ reftype^*, \\
\quad datas\ ok^n\}
\end{array}$$

7.5.3 Configuration Validity

To relate the WebAssembly **type system** to its **execution semantics**, the **typing rules** for instructions must be extended to **configurations** $S; T$, which relates the **store** to execution **threads**.

Configurations and threads are classified by their **result type**. In addition to the store S , threads are typed under a **return type** $resulttype^?$, which controls whether and with which type a **return** instruction is allowed. This type is absent (ϵ) except for instruction sequences inside an administrative **frame** instruction.

Finally, **frames** are classified with **frame contexts**, which extend the **module contexts** of a frame's associated **module instance** with the **locals** that the frame contains.

Configurations $S; T$

- The **store** S must be **valid**.
- Under no allowed return type, the **thread** T must be **valid** with some **result type** $[t^*]$.
- Then the configuration is valid with the **result type** $[t^*]$.

$$\frac{\vdash S \text{ ok} \quad S; \epsilon \vdash T : [t^*]}{\vdash S; T : [t^*]}$$

Threads $F; instr^*$

- Let $resulttype^?$ be the current allowed return type.
- The **frame** F must be **valid** with a **context** C .
- Let C' be the same **context** as C , but with **return** set to $resulttype^?$.
- Under context C' , the instruction sequence $instr^*$ must be **valid** with some type $\square \rightarrow [t^*]$.
- Then the thread is valid with the **result type** $[t^*]$.

$$\frac{S \vdash F : C \quad S; C, \text{return } resulttype^? \vdash instr^* : \square \rightarrow [t^*]}{S; resulttype^? \vdash F; instr^* : [t^*]}$$

Frames $\{\text{locals } val^*, \text{module } moduleinst\}$

- The **module instance** $moduleinst$ must be **valid** with some **module context** C .
- Each **value** val_i in val^* must be **valid** with some **value type** t_i .
- Let t^* be the concatenation of all t_i in order.
- Let C' be the same **context** as C , but with the **value types** t^* prepended to the **locals** vector.
- Then the frame is valid with **frame context** C' .

$$\frac{S \vdash moduleinst : C \quad (S \vdash val : t)^*}{S \vdash \{\text{locals } val^*, \text{module } moduleinst\} : (C, \text{locals } t^*)}$$

7.5.4 Administrative Instructions

Typing rules for **administrative instructions** are specified as follows. In addition to the **context** C , typing of these instructions is defined under a given **store** S . To that end, all previous typing judgements $C \vdash prop$ are generalized to include the store, as in $S; C \vdash prop$, by implicitly adding S to all rules – S is never modified by the pre-existing rules, but it is accessed in the extra rules for **administrative instructions** given below.

trap

- The instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$, for any sequences of **value types** t_1^* and t_2^* .

$$\overline{S; C \vdash \text{trap} : [t_1^*] \rightarrow [t_2^*]}$$

ref.extern *externaddr*

- The instruction is valid with type $[] \rightarrow [\text{externref}]$.

$$\overline{S; C \vdash \text{ref.extern } \textit{externaddr} : [] \rightarrow [\text{externref}]}$$

ref.funcaddr

- The external function value **func** *funcaddr* must be valid with external function type **func** *func* *func* *type*.
- Then the instruction is valid with type $[] \rightarrow [\text{funcref}]$.

$$\frac{S \vdash \text{func } \textit{funcaddr} : \text{func } \textit{func} \textit{type}}{S; C \vdash \text{ref } \textit{funcaddr} : [] \rightarrow [\text{funcref}]}$$

invoke *funcaddr*

- The external function value **func** *funcaddr* must be valid with external function type **func** $([t_1^*] \rightarrow [t_2^*])$.
- Then the instruction is valid with type $[t_1^*] \rightarrow [t_2^*]$.

$$\frac{S \vdash \text{func } \textit{funcaddr} : \text{func } [t_1^*] \rightarrow [t_2^*]}{S; C \vdash \text{invoke } \textit{funcaddr} : [t_1^*] \rightarrow [t_2^*]}$$

label_n { *instr₀^{*}* } *instr^{*}* **end**

- The instruction sequence *instr₀^{*}* must be valid with some type $[t_1^n] \rightarrow [t_2^*]$.
- Let C' be the same **context** as C , but with the **result type** $[t_1^n]$ prepended to the **labels** vector.
- Under context C' , the instruction sequence *instr^{*}* must be valid with type $[] \rightarrow [t_2^*]$.
- Then the compound instruction is valid with type $[] \rightarrow [t_2^*]$.

$$\frac{S; C \vdash \textit{instr}_0^* : [t_1^n] \rightarrow [t_2^*] \quad S; C, \text{labels } [t_1^n] \vdash \textit{instr}^* : [] \rightarrow [t_2^*]}{S; C \vdash \text{label}_n \{ \textit{instr}_0^* \} \textit{instr}^* \text{ end} : [] \rightarrow [t_2^*]}$$

$\text{frame}_n\{F\} \text{ instr}^* \text{ end}$

- Under the return type $[t^n]$, the thread $F; \text{instr}^*$ must be valid with result type $[t^n]$.
- Then the compound instruction is valid with type $[] \rightarrow [t^n]$.

$$\frac{S; [t^n] \vdash F; \text{instr}^* : [t^n]}{S; C \vdash \text{frame}_n\{F\} \text{ instr}^* \text{ end} : [] \rightarrow [t^n]}$$

7.5.5 Store Extension

Programs can mutate the [store](#) and its contained instances. Any such modification must respect certain invariants, such as not removing allocated instances or changing immutable definitions. While these invariants are inherent to the execution semantics of WebAssembly [instructions](#) and [modules](#), [host functions](#) do not automatically adhere to them. Consequently, the required invariants must be stated as explicit constraints on the [invocation](#) of host functions. Soundness only holds when the [embedder](#) ensures these constraints.

The necessary constraints are codified by the notion of store *extension*: a store state S' extends state S , written $S \preceq S'$, when the following rules hold.

Note: Extension does not imply that the new store is valid, which is defined separately [above](#).

Store S

- The length of $S.\text{funcs}$ must not shrink.
- The length of $S.\text{tables}$ must not shrink.
- The length of $S.\text{mems}$ must not shrink.
- The length of $S.\text{globals}$ must not shrink.
- The length of $S.\text{elems}$ must not shrink.
- The length of $S.\text{datas}$ must not shrink.
- For each function instance funcinst_i in the original $S.\text{funcs}$, the new function instance must be an [extension](#) of the old.
- For each table instance tableinst_i in the original $S.\text{tables}$, the new table instance must be an [extension](#) of the old.
- For each memory instance meminst_i in the original $S.\text{mems}$, the new memory instance must be an [extension](#) of the old.
- For each global instance globalinst_i in the original $S.\text{globals}$, the new global instance must be an [extension](#) of the old.
- For each element instance eleminst_i in the original $S.\text{elems}$, the new global instance must be an [extension](#) of the old.
- For each data instance datainst_i in the original $S.\text{datas}$, the new global instance must be an [extension](#) of the old.

$$\frac{\begin{array}{lll} S_1.\text{funcs} = \text{funcinst}_1^* & S_2.\text{funcs} = \text{funcinst}_1'^* \text{ funcinst}_2^* & (\vdash \text{funcinst}_1 \preceq \text{funcinst}_1')^* \\ S_1.\text{tables} = \text{tableinst}_1^* & S_2.\text{tables} = \text{tableinst}_1'^* \text{ tableinst}_2^* & (\vdash \text{tableinst}_1 \preceq \text{tableinst}_1')^* \\ S_1.\text{mems} = \text{meminst}_1^* & S_2.\text{mems} = \text{meminst}_1'^* \text{ meminst}_2^* & (\vdash \text{meminst}_1 \preceq \text{meminst}_1')^* \\ S_1.\text{globals} = \text{globalinst}_1^* & S_2.\text{globals} = \text{globalinst}_1'^* \text{ globalinst}_2^* & (\vdash \text{globalinst}_1 \preceq \text{globalinst}_1')^* \\ S_1.\text{elems} = \text{eleminst}_1^* & S_2.\text{elems} = \text{eleminst}_1'^* \text{ eleminst}_2^* & (\vdash \text{eleminst}_1 \preceq \text{eleminst}_1')^* \\ S_1.\text{datas} = \text{datainst}_1^* & S_2.\text{datas} = \text{datainst}_1'^* \text{ datainst}_2^* & (\vdash \text{datainst}_1 \preceq \text{datainst}_1')^* \end{array}}{\vdash S_1 \preceq S_2}$$

Function Instance *funcinst*

- A function instance must remain unchanged.

$$\vdash \text{funcinst} \preceq \text{funcinst}$$

Table Instance *tableinst*

- The table type *tableinst.type* must remain unchanged.
- The length of *tableinst.elem* must not shrink.

$$\frac{n_1 \leq n_2}{\vdash \{\text{type } tt, \text{elem } (fa_1^?)^{n_1}\} \preceq \{\text{type } tt, \text{elem } (fa_2^?)^{n_2}\}}$$

Memory Instance *meminst*

- The memory type *meminst.type* must remain unchanged.
- The length of *meminst.data* must not shrink.

$$\frac{n_1 \leq n_2}{\vdash \{\text{type } mt, \text{data } b_1^{n_1}\} \preceq \{\text{type } mt, \text{data } b_2^{n_2}\}}$$

Global Instance *globalinst*

- The global type *globalinst.type* must remain unchanged.
- Let *mut t* be the structure of *globalinst.type*.
- If *mut* is *const*, then the value *globalinst.value* must remain unchanged.

$$\frac{\text{mut} = \text{var} \vee \text{val}_1 = \text{val}_2}{\vdash \{\text{type } (\text{mut } t), \text{value } \text{val}_1\} \preceq \{\text{type } (\text{mut } t), \text{value } \text{val}_2\}}$$

Element Instance *eleminst*

- The vector *eleminst.elem* must either remain unchanged or shrink to length 0.

$$\frac{fa_1^* = fa_2^* \vee fa_2^* = \epsilon}{\vdash \{\text{elem } fa_1^*\} \preceq \{\text{elem } fa_2^*\}}$$

Data Instance *datainst*

- The vector *datainst.data* must either remain unchanged or shrink to length 0.

$$\frac{b_1^* = b_2^* \vee b_2^* = \epsilon}{\vdash \{\text{data } b_1^*\} \preceq \{\text{data } b_2^*\}}$$

7.5.6 Theorems

Given the definition of **valid configurations**, the standard soundness theorems hold.⁵³⁵⁵

Theorem (Preservation). If a **configuration** $S;T$ is **valid** with **result type** $[t^*]$ (i.e., $\vdash S;T : [t^*]$), and steps to $S';T'$ (i.e., $S;T \hookrightarrow S';T'$), then $S';T'$ is a **valid configuration** with the same **result type** (i.e., $\vdash S';T' : [t^*]$). Furthermore, S' is an **extension** of S (i.e., $\vdash S \preceq S'$).

A **terminal thread** is one whose sequence of **instructions** is a **result**. A **terminal configuration** is a configuration whose thread is terminal.

Theorem (Progress). If a **configuration** $S;T$ is **valid** (i.e., $\vdash S;T : [t^*]$ for some **result type** $[t^*]$), then either it is terminal, or it can step to some configuration $S';T'$ (i.e., $S;T \hookrightarrow S';T'$).

From Preservation and Progress the soundness of the WebAssembly type system follows directly.

Corollary (Soundness). If a **configuration** $S;T$ is **valid** (i.e., $\vdash S;T : [t^*]$ for some **result type** $[t^*]$), then it either diverges or takes a finite number of steps to reach a **terminal configuration** $S';T'$ (i.e., $S;T \hookrightarrow^* S';T'$) that is **valid** with the same **result type** (i.e., $\vdash S';T' : [t^*]$) and where S' is an **extension** of S (i.e., $\vdash S \preceq S'$).

In other words, every thread in a **valid configuration** either runs forever, traps, or terminates with a **result** that has the expected type. Consequently, given a **valid store**, no computation defined by **instantiation** or **invocation** of a **valid module** can “crash” or otherwise (mis)behave in ways not covered by the **execution semantics** given in this specification.

7.6 Change History

Since the original release 1.0 of the WebAssembly specification, a number of proposals for extensions have been integrated. The following sections provide an overview of what has changed.

7.6.1 Release 2.0

Sign extension instructions

Added new numeric instructions for performing sign extension within integer representations⁵⁷.

- New **numeric instructions**: `inn.extendN_s`

Non-trapping float-to-int conversions

Added new conversion instructions that avoid trapping when converting a floating-point number to an integer⁵⁸.

- New **numeric instructions**: `inn.trunc_sat_fmm_sx`

⁵³ A machine-verified version of the formalization and soundness proof of the PLDI 2017 paper is described in the following article: Conrad Watt. *Mechanising and Verifying the WebAssembly Specification*^{Page 209, 54}. Proceedings of the 7th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2018). ACM 2018.

⁵⁴ <https://dl.acm.org/citation.cfm?id=3167082>

⁵⁵ Machine-verified formalizations and soundness proofs of the semantics from the official specification are described in the following article: Conrad Watt, Xiaojia Rao, Jean Pichon-Pharabod, Martin Bodin, Philippa Gardner. *Two Mechanisations of WebAssembly 1.0*⁵⁶. Proceedings of the 24th International Symposium on Formal Methods (FM 2021). Springer 2021.

⁵⁶ https://link.springer.com/chapter/10.1007/978-3-030-90870-6_4

⁵⁷ <https://github.com/WebAssembly/spec/tree/main/proposals/sign-extension-ops/>

⁵⁸ <https://github.com/WebAssembly/spec/tree/main/proposals/nontrapping-float-to-int-conversion/>

Multiple values

Generalized the result type of blocks and functions to allow for multiple values; in addition, introduced the ability to have block parameters⁵⁹.

- [Function types](#) allow more than one result
- [Block types](#) can be arbitrary function types

Reference types

Added [funcref](#) and [externref](#) as new value types and respective instructions⁶⁰.

- New value types: [reference types funcref](#) and [externref](#)
- New reference instructions: [ref.null](#), [ref.func](#), [ref.is_null](#)
- Enrich [parametric instruction](#): [select](#) with optional type immediate
- New [declarative form of element segment](#)

Table instructions

Added instructions to directly access and modify tables^{Page 210, 60}.

- [Table types](#) allow any [reference type](#) as element type
- New table instructions: [table.get](#), [table.set](#), [table.size](#), [table.grow](#)

Multiple tables

Added the ability to use multiple tables per module⁶⁰.

- [Modules](#) may [define](#), [import](#), and [export](#) multiple tables
- Table instructions take a [table index](#) immediate: [table.get](#), [table.set](#), [table.size](#), [table.grow](#), [call_indirect](#)
- [Element segments](#) take a [table index](#)

Bulk memory and table instructions

Added instructions that modify ranges of memory or table entries⁶⁰⁶¹

- New memory instructions: [memory.fill](#), [memory.init](#), [memory.copy](#), [data.drop](#)
- New table instructions: [table.fill](#), [table.init](#), [table.copy](#), [elem.drop](#)
- New [passive form of data segment](#)
- New [passive form of element segment](#)
- New [data count section](#) in binary format
- Active data and element segments boundaries are no longer checked at compile time but may trap instead

⁵⁹ <https://github.com/WebAssembly/spec/tree/main/proposals/multi-value/>

⁶⁰ <https://github.com/WebAssembly/spec/tree/main/proposals/reference-types/>

⁶¹ <https://github.com/WebAssembly/spec/tree/main/proposals/bulk-memory-operations/>

Vector instructions

Added vector type and instructions that manipulate multiple numeric values in parallel (also known as *SIMD*, single instruction multiple data)⁶²

- New value type: `v128`
- New memory instructions: `v128.load`, `v128.loadN_xM_sx`, `v128.loadN_zero`, `v128.loadN_splat`, `v128.loadN_lane`, `v128.store`, `v128.storeN_lane`
- New constant vector instruction: `v128.const`
- New unary vector instructions: `v128.not`, `iN_xM.abs`, `iN_xM.neg`, `i8x16.popcnt`, `fN_xM.abs`, `fN_xM.neg`, `fN_xM.sqrt`, `fN_xM.ceil`, `fN_xM.floor`, `fN_xM.trunc`, `fN_xM.nearest`
- New binary vector instructions: `v128.and`, `v128.andnot`, `v128.or`, `v128.xor`, `iN_xM.add`, `iN_xM.sub`, `iN_xM.mul`, `iN_xM.add_sat_sx`, `iN_xM.sub_sat_sx`, `iN_xM.min_sx`, `iN_xM.max_sx`, `iN_xM.shl`, `iN_xM.shr_sx`, `fN_xM.add`, `iN_xM.extmul_half_iN_xM'_sx`, `i16x8.q15mulr_sat_s`, `i32x4.dot_i16x8_s`, `i16x8.extadd_pairwise_i8x16_sx`, `i32x4.extadd_pairwise_i16x8_sx`, `i8x16.avgr_u`, `i16x8.avgr_u`, `fN_xM.sub`, `fN_xM.mul`, `fN_xM.div`, `fN_xM.min`, `fN_xM.max`, `fN_xM.pmin`, `fN_xM.pmax`
- New ternary vector instruction: `v128.bitselect`
- New test vector instructions: `v128.any_true`, `iN_xM.all_true`
- New relational vector instructions: `iN_xM.eq`, `iN_xM.ne`, `iN_xM.lt_sx`, `iN_xM.gt_sx`, `iN_xM.le_sx`, `iN_xM.ge_sx`, `fN_xM.eq`, `fN_xM.ne`, `fN_xM.lt`, `fN_xM.gt`, `fN_xM.le`, `fN_xM.ge`
- New conversion vector instructions: `i32x4.trunc_sat_f32x4_sx`, `i32x4.trunc_sat_f64x2_sx_zero`, `f32x4.convert_i32x4_sx`, `f32x4.demote_f64x2_zero`, `f64x2.convert_low_i32x4_sx`, `f64x2.promote_low_f32x4`
- New lane access vector instructions: `iN_xM.extract_lane_sx?`, `iN_xM.replace_lane`, `fN_xM.extract_lane`, `fN_xM.replace_lane`
- New lane splitting/combining vector instructions: `iN_xM.extend_half_iN_xM'_sx`, `i8x16.narrow_i16x8_sx`, `i16x8.narrow_i32x4_sx`
- New byte reordering vector instructions: `i8x16.shuffle`, `i8x16.swizzle`
- New injection/projection vector instructions: `iN_xM.splat`, `fN_xM.splat`, `iN_xM.bitmask`

⁶² <https://github.com/WebAssembly/spec/tree/main/proposals/simd/>

Symbols

: abstract syntax
administrative instruction, 62

A

abbreviations, 158
abstract syntax, 5, 133, 157, 192
 block type, 17, 28
 byte, 7
 data, 21, 50
 data address, 58
 data index, 19
 data instance, 60
 element, 21, 49
 element address, 58
 element index, 19
 element instance, 60
 element mode, 21
 export, 22, 50
 export instance, 60
 expression, 18, 46, 120
 external type, 11, 29
 external value, 60
 floating-point number, 7
 frame, 61
 function, 20, 47
 function address, 58
 function index, 19
 function instance, 59
 function type, 10, 28
 global, 21, 48
 global address, 58
 global index, 19
 global instance, 60
 global type, 11, 29
 grammar, 5
 host address, 58
 import, 22, 51
 instruction, 11–13, 15–17, 32–34, 37, 38, 40, 43, 87, 89, 90, 97–99, 105, 114
 integer, 7
 label, 61
 label index, 19
 limits, 10, 28
 local, 20
 local index, 19
 memory, 20, 48
 memory address, 58
 memory index, 19
 memory instance, 59
 memory type, 10, 29
 module, 19, 52
 module instance, 59
 mutability, 11
 name, 8
 notation, 5
 number type, 9
 reference type, 9
 result, 58
 result type, 10
 signed integer, 7
 start function, 22, 50
 store, 58
 table, 20, 48
 table address, 58
 table index, 19
 table instance, 59
 table type, 11, 29
 type, 9
 type definition, 20
 type index, 19
 uninterpreted integer, 7
 unsigned integer, 7
 value, 6, 57
 value type, 10
 vector, 6, 8
activation, 61
active, 21, 21
address, 58, 98, 99, 105, 114, 122
 data, 58
 element, 58
 function, 58
 global, 58
 host, 58
 memory, 58
 table, 58
administrative instruction, 204, 205

- : abstract syntax, 62
- administrative instructions, 62
- algorithm, 194
- allocation, 58, 122, 186, 194
- arithmetic NaN, 7
- ASCII, 159, 160, 162

B

- binary format, 8, 133, 186, 193, 194, 198
 - block type, 138
 - byte, 135
 - custom section, 150
 - data, 153
 - data count, 153
 - data index, 149
 - element, 152
 - element index, 149
 - export, 151
 - expression, 149
 - floating-point number, 135
 - function, 151, 152
 - function index, 149
 - function type, 137
 - global, 151
 - global index, 149
 - global type, 138
 - grammar, 133
 - import, 150
 - instruction, 138–140, 143
 - integer, 135
 - label index, 149
 - limits, 137
 - local, 152
 - local index, 149
 - memory, 151
 - memory index, 149
 - memory type, 137
 - module, 154
 - mutability, 138
 - name, 135
 - notation, 133
 - number type, 136
 - reference type, 136
 - result type, 137
 - section, 149
 - signed integer, 135
 - start function, 152
 - table, 151
 - table index, 149
 - table type, 137
 - type, 136
 - type index, 149
 - type section, 150
 - uninterpreted integer, 135
 - unsigned integer, 135
 - value, 134
 - value type, 137
 - vector, 134

- vector type, 136
- bit, 65
- bit width, 7, 9, 64, 105
- block, 17, 43, 114, 118, 138, 165, 209
 - type, 17
- block context, 63
- block type, 17, 28, 43, 138
 - abstract syntax, 17
 - binary format, 138
 - validation, 28
- Boolean, 3, 65, 66
- bottom type, 31
- branch, 17, 43, 63, 114, 138, 165
- byte, 7, 8, 21, 50, 59, 60, 66, 123, 133, 135, 153, 162, 180, 181, 190, 202, 203
 - abstract syntax, 7
 - binary format, 135
 - text format, 162

C

- call, 61, 62, 118
- canonical NaN, 7
- changes, 209
- character, 2, 8, 159, 159, 160, 162, 192, 193
 - text format, 159
- closure, 59
- code, 11, 193
 - section, 152
- code section, 152
- comment, 159, 160
- concepts, 3
- configuration, 56, 63, 204, 208
- constant, 18, 21, 47, 57
- context, 25, 31, 37, 38, 40, 43, 52, 154, 203, 205
- control instruction, 17
- control instructions, 43, 114, 138, 165
- custom section, 150, 198
 - binary format, 150

D

- data, 18–20, 21, 50, 52, 62, 124, 153, 154, 180–182, 192
 - abstract syntax, 21
 - address, 58
 - binary format, 153
 - index, 19
 - instance, 60
 - section, 153
 - segment, 21, 50, 153, 180, 181
 - text format, 180, 181
 - validation, 50
- data address, 59, 124
 - abstract syntax, 58
- data count, 153
 - binary format, 153
 - section, 153
- data count section, 153
- data index, 19, 21, 149, 177

- abstract syntax, 19
- binary format, 149
- text format, 177
- data instance, 58, 59, **60**, 124, 203, 208
 - abstract syntax, 60
- data section, **153**
- data segment, 59, 60, 153, 210
- declarative, **21**
- decoding, 4
- default value, **57**
- design goals, 1
- determinism, 64, 87

E

- element, 11, 18–20, **21**, 49, 52, 62, 124, 152, 154, 179, 181, 182, 189, 192
 - abstract syntax, 21
 - address, 58
 - binary format, 152
 - index, 19
 - instance, 60
 - mode, 21
 - section, 152
 - segment, 21, 49, 152, 179, 181
 - text format, 179, 181
 - validation, 49
- element address, 59, 99, 124
 - abstract syntax, 58
- element expression, 60
- element index, **19**, 21, 149, 177
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
- element instance, 58, 59, **60**, 99, 124, 203, 208
 - abstract syntax, 60
- element mode, **21**
 - abstract syntax, 21
- element section, **152**
- element segment, 59, 60, 210
- element type, 31
- embedder, 2, **3**, 58–60, 185
- embedding, **185**
- evaluation context, 56, **63**
- execution, 4, 9, 10, **55**, 194
 - expression, 120
 - instruction, 87, 89, 90, 97–99, 105, 114
- exponent, 7, 66
- export, 19, **22**, 50, 52, 60, 126, 130, 151, 154, 178–180, 182, 187, 188, 192
 - abstract syntax, 22
 - binary format, 151
 - instance, 60
 - section, 151
 - text format, 178–180
 - validation, 50
- export instance, 59, **60**, 126, 188, 203
 - abstract syntax, 60
- export section, **151**
- expression, **18**, 20, 21, 46–50, 120, 149, 151–153, 176, 180, 181
 - abstract syntax, 18
 - binary format, 149
 - constant, 18, 46, 149, 176
 - execution, 120
 - text format, 176
 - validation, 46
- extern address, 206
- extern type, 206
- extern value, 206
- external
 - type, 11
 - value, 60
- external type, **11**, 29, 30, 121, 126, 203
 - abstract syntax, 11
 - validation, 29
- external value, 11, **60**, 60, 121, 126, 203
 - abstract syntax, 60

F

- file extension, 133, 157
- floating point, 2
- floating-point, 3, 7, 8, 9, 12, 57, 64, 65, 74, 209
- floating-point number, 135, 161
 - abstract syntax, 7
 - binary format, 135
 - text format, 161
- folded instruction, **175**
- frame, **61**, 62, 63, 98, 99, 105, 114, 118, 194, 204–206
 - abstract syntax, 61
- function, 2, 3, 9, 10, 17, 19, **20**, 22, 25, 47, 52, 59–62, 118, 122, 126, 130, 151, 152, 154, 178, 182, 188, 192, 193, 200, 209
 - abstract syntax, 20, 47
 - address, 58
 - binary format, 151, 152
 - export, 22
 - import, 22
 - index, 19
 - instance, 59
 - section, 151
 - text format, 178
 - type, 10
- function address, 59, 60, 62, 121–123, 126, 130, 188, 189, 202, 206
 - abstract syntax, 58
- function index, 17, **19**, 20–22, 43, 47, 49, 50, 114, 126, 138, 149, 151, 152, 165, 177–181, 200
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
- function instance, 58, **59**, 59, 62, 118, 122, 123, 126, 130, 188, 194, 201, 202, 207
 - abstract syntax, 59
- function section, **151**
- function type, 9, **10**, 11, 17, 19, 20, 22, 25, 28–31,

- 47, 51, 52, 59, 87, 121–123, 130, 137, 150–152, 154, 163, 178, 182, 188, 201, 202, 206
 - abstract syntax, 10
 - binary format, 137
 - text format, 163
 - validation, 28
- ## G
- global, 11, 16, 18, 19, **21**, 22, 48, 52, 60, 124, 126, 151, 154, 180, 182, 191, 192
 - abstract syntax, 21
 - address, 58
 - binary format, 151
 - export, 22
 - import, 22
 - index, 19
 - instance, 60
 - mutability, 11
 - section, 151
 - text format, 180
 - type, 11
 - validation, 48
 - global address, 59, 60, 98, 121, 124, 126, 191
 - abstract syntax, 58
 - global index, 16, **19**, 21, 22, 37, 50, 98, 126, 139, 149, 151, 166, 177, 180
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
 - global instance, 58, 59, **60**, 98, 124, 126, 191, 194, 201, 203, 207, 208
 - abstract syntax, 60
 - global section, **151**
 - global type, **11**, 11, 21, 22, 25, 29, 31, 48, 51, 121, 124, 138, 150, 151, 164, 178, 180, 191, 201, 203
 - abstract syntax, 11
 - binary format, 138
 - text format, 164
 - validation, 29
 - globaltype, 25
 - grammar notation, **5**, 133, 157
 - grow, 125
- ## H
- host, 2, 185
 - address, 58
 - host address, **57**
 - abstract syntax, 58
 - host function, **59**, 119, 123, 188, 202
- ## I
- identifier, 157, 158, 177–180, 182, 193
 - identifier context, **158**, 182
 - identifiers, **162**
 - text format, 162
 - IEEE 754, 2, 3, 7, 9, 66, 74
 - implementation, 185, 192
 - implementation limitations, **192**
 - import, 2, 11, 19–21, **22**, 47, 51, 52, 121, 126, 150, 154, 178–180, 182, 187, 192
 - abstract syntax, 22
 - binary format, 150
 - section, 150
 - text format, 178–180
 - validation, 51
 - import section, **150**
 - index, **19**, 22, 50, 59, 149, 151, 158, 164, 177–180, 199
 - data, 19
 - element, 19
 - function, 19
 - global, 19
 - label, 19
 - local, 19
 - memory, 19
 - table, 19
 - type, 19
 - index space, **19**, 22, 25, 158, 199
 - instance, **59**, 128
 - data, 60
 - element, 60
 - export, 60
 - function, 59
 - global, 60
 - memory, 59
 - module, 59
 - table, 59
 - instantiation, 4, 9, 22, **128**, 187, 208
 - instantiation. module, 25
 - instruction, 3, 10, **11**, 18, 31, 46, 59–63, 87, 118, 138, 164, 192, 194, 205, 206, 209, 210
 - abstract syntax, 11–13, 15–17
 - binary format, 138–140, 143
 - execution, 87, 89, 90, 97–99, 105, 114
 - text format, 165–167, 170
 - validation, 32–34, 37, 38, 40, 43
 - instruction sequence, 46, 118
 - instructions, 210
 - integer, 3, **7**, 8, 9, 12, 57, 64–66, 99, 105, 135, 161, 209
 - abstract syntax, 7
 - binary format, 135
 - signed, 7
 - text format, 161
 - uninterpreted, 7
 - unsigned, 7
 - invocation, 4, 59, **130**, 188, 208
- ## K
- keyword, **159**
- ## L
- label, **17**, 43, **61**, 62, 63, 114, 118, 138, 165, 194, 206
 - abstract syntax, 61
 - index, 19
 - label index, 17, **19**, 43, 114, 138, 149, 164, 165, 177

- abstract syntax, 19
 - binary format, 149
 - text format, 164, 177
 - lane, 8, 66
 - LEB128, 135, 138
 - lexical format, 159
 - limits, **10**, 10, 11, 20, 28–31, 99, 105, 123, 125, 137, 164, 202
 - abstract syntax, 10
 - binary format, 137
 - memory, 10
 - table, 11
 - text format, 164
 - validation, 28
 - linear memory, 3
 - little endian, 16, 66, 135
 - local, 16, 19, **20**, 47, 61, 152, 178, 192, 200, 205
 - abstract syntax, 20
 - binary format, 152
 - index, 19
 - text format, 178
 - local index, 16, **19**, 20, 37, 47, 98, 139, 149, 166, 177, 200
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
- ## M
- magnitude, 7
 - matching, **30**, 126
 - memory, 3, 9, 10, 16, 19, **20**, 21, 22, 48, 50, 52, 59, 60, 62, 66, 123, 125, 126, 151, 153, 154, 179–182, 190, 192, 210
 - abstract syntax, 20
 - address, 58
 - binary format, 151
 - data, 21, 50, 153, 180, 181
 - export, 22
 - import, 22
 - index, 19
 - instance, 59
 - limits, 10
 - section, 151
 - text format, 179
 - type, 10
 - validation, 48
 - memory address, 59, 60, 105, 121, 123, 125, 126, 190
 - abstract syntax, 58
 - memory index, 16, **19**, 20–22, 40, 50, 105, 126, 139, 149, 151, 153, 167, 177, 180, 181
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
 - memory instance, 58, **59**, 59, 62, 105, 123, 125, 126, 190, 194, 201, 202, 207, 208
 - abstract syntax, 59
 - memory instruction, **16**, 40, 105, 139, 167
 - memory section, **151**
 - memory type, **10**, 10, 11, 20, 22, 25, 29, 31, 48, 51, 59, 121, 123, 137, 150, 151, 164, 178, 179, 190, 201, 202
 - abstract syntax, 10
 - binary format, 137
 - text format, 164
 - validation, 29
 - module, 2, 3, **19**, 25, 52, 58, 59, 126, 128, 130, 133, 154, 182, 186, 188, 192–194, 199, 208
 - abstract syntax, 19
 - binary format, 154
 - instance, 59
 - text format, 182
 - validation, 52
 - module instance, 59, 61, 122, 126, 130, 187, 188, 194, 203, 205
 - abstract syntax, 59
 - module instruction, 63
 - mutability, **11**, 11, 21, 29, 31, 60, 121, 124, 138, 164, 203, 208
 - abstract syntax, 11
 - binary format, 138
 - global, 11
 - text format, 164
- ## N
- name, 2, **8**, 22, 50, 51, 59, 60, 135, 150, 151, 162, 178–180, 192, 198, 203
 - abstract syntax, 8
 - binary format, 135
 - text format, 162
 - name map, **199**
 - name section, 182, **198**
 - NaN, **7**, 64, 75, 87
 - arithmetic, 7
 - canonical, 7
 - payload, 7
 - notation, 5, 133, 157
 - abstract syntax, 5
 - binary format, 133
 - text format, 157
 - null, 9, 15
 - number, 13, 57
 - type, 9
 - number type, **9**, 10, 57, 136, 137, 163
 - abstract syntax, 9
 - binary format, 136
 - text format, 163
 - numeric instruction, **12**, 32, 87, 140, 167
 - numeric vector, **8**, 13, 65, 66
- ## O
- offset, 18
 - opcode, **138**, 194, 197
 - operand, 11
 - operand stack, 11, 31

P

page size, 10, 16, 20, **59**, 137, 164, 180
parameter, 10, 19, 192
parametric instruction, **15**, 139, 166
parametric instructions, 37, 97
passive, **21**, 21
payload, 7
phases, 4
polymorphism, **31**, 37, 43, 138, 139, 165, 166
portability, 1
preservation, **208**
progress, **208**

R

reduction rules, **56**
reference, 9, 15, 57, 89, 99, 203, 210
 type, 9
reference instruction, **15**, 139, 166
reference instructions, 33, 89
reference type, **9**, 10, 11, 29, 33, 57, 99, 136, 137,
 163, 164, 210
 abstract syntax, 9
 binary format, 136
 text format, 163
result, 10, **58**, 188, 192, 200
 abstract syntax, 58
 type, 10
result type, **10**, 10, 25, 46, 114, 137, 138, 163, 165,
 200, 205, 206, 209
 abstract syntax, 10
 binary format, 137
resulttype, 25
rewrite rule, 158
rounding, 74
runtime, **57**

S

S-expression, 157, 175
section, **149**, 154, 193, 198
 binary format, 149
 code, 152
 custom, 150
 data, 153
 data count, 153
 element, 152
 export, 151
 function, 151
 global, 151
 import, 150
 memory, 151
 name, 182
 start, 152
 table, 151
 type, 150
security, **2**
segment, 62
shape, 66
sign, 66

signed integer, 7, 66, 135, 161
 abstract syntax, 7
 binary format, 135
 text format, 161
significand, 7, 66
SIMD, 8, 9, 13, 210
soundness, **200**, 208
source text, **159**, 159, 193
stack, 55, **61**, 130, 194
stack machine, 11
stack type, 17
start function, 19, **22**, 50, 52, 152, 154, 181, 182
 abstract syntax, 22
 binary format, 152
 section, 152
 text format, 181
 validation, 50
start section, **152**
store, 55, **58**, 58, 60, 61, 63, 87, 98, 99, 105, 114, 119,
 121, 122, 128, 130, 186, 188–191, 201, 204,
 205, 207
 abstract syntax, 58
store extension, **207**
string, **162**
 text format, 162
structured control, **17**, 43, 114, 138, 165
structured control instruction, 192

T

table, 3, 9, 11, 16, 17, 19, **20**, 21, 22, 48, 49, 52, 59, 60,
 62, 123, 125, 126, 151, 154, 179, 182, 189,
 192, 210
 abstract syntax, 20
 address, 58
 binary format, 151
 element, 21, 49, 152, 179, 181
 export, 22
 import, 22
 index, 19
 instance, 59
 limits, 10, 11
 section, 151
 text format, 179
 type, 11
 validation, 48
table address, 59, 60, 99, 114, 121, 123, 125, 126,
 189
 abstract syntax, 58
table index, 16, **19**, 20–22, 38, 49, 50, 99, 126, 139,
 149, 151, 152, 166, 177, 179–181, 210
 abstract syntax, 19
 binary format, 149
 text format, 177
table instance, 58, **59**, 59, 62, 99, 114, 123, 125,
 126, 189, 194, 201, 202, 207, 208
 abstract syntax, 59
table instruction, **16**, 38, 99, 139, 166
table section, **151**

- table type, 10, **11**, 11, 20, 22, 25, 29, 31, 48, 51, 59, 121, 123, 137, 150, 151, 164, 178, 179, 189, 201, 202, 210
 - abstract syntax, 11
 - binary format, 137
 - text format, 164
 - validation, 29
 - terminal configuration, 208
 - text format, 2, **157**, 186, 193
 - byte, 162
 - character, 159
 - comment, 160
 - data, 180, 181
 - data index, 177
 - element, 179, 181
 - element index, 177
 - export, 178–180
 - expression, 176
 - floating-point number, 161
 - function, 178
 - function index, 177
 - function type, 163
 - global, 180
 - global index, 177
 - global type, 164
 - grammar, 157
 - identifiers, 162
 - import, 178–180
 - instruction, 165–167, 170
 - integer, 161
 - label index, 164, 177
 - limits, 164
 - local, 178
 - local index, 177
 - memory, 179
 - memory index, 177
 - memory type, 164
 - module, 182
 - mutability, 164
 - name, 162
 - notation, 157
 - number type, 163
 - reference type, 163
 - signed integer, 161
 - start function, 181
 - string, 162
 - table, 179
 - table index, 177
 - table type, 164
 - token, 159
 - type, 163
 - type definition, 177
 - type index, 177
 - type use, 177
 - uninterpreted integer, 161
 - unsigned integer, 161
 - value, 160
 - value type, 163
 - vector, 159
 - vector type, 163
 - white space, 160
 - thread, **63**, 205, 208
 - token, **159**, 193
 - trap, 3, 16, 17, 58, 62, 63, 87, 128, 130, 200, 206, 209
 - two's complement, 3, 7, 12, 66, 135
 - type, **9**, 126, 136, 163, 192
 - abstract syntax, 9
 - binary format, 136
 - block, 17
 - external, 11
 - function, 10
 - global, 11
 - index, 19
 - memory, 10
 - number, 9
 - reference, 9
 - result, 10
 - section, 150
 - table, 11
 - text format, 163
 - value, 10
 - type definition, 19, **20**, 52, 150, 154, 177, 182
 - abstract syntax, 20
 - text format, 177
 - type index, 17, **19**, 20, 22, 43, 47, 114, 138, 149, 151, 152, 165, 177, 178
 - abstract syntax, 19
 - binary format, 149
 - text format, 177
 - type section, **150**
 - binary format, 150
 - type system, **25**, 200
 - type use, 177
 - text format, 177
 - typing rules, **27**
- ## U
- Unicode, 2, 8, 135, 157, 159, 162, 192
 - unicode, 193
 - Unicode UTF-8, 198
 - uninterpreted integer, 7, 66, 135, 161
 - abstract syntax, 7
 - binary format, 135
 - text format, 161
 - unsigned integer, 7, 66, 135, 161
 - abstract syntax, 7
 - binary format, 135
 - text format, 161
 - unwinding, **17**
 - UTF-8, 2, 8, **135**, 157, 162
- ## V
- validation, 4, 9, **25**, 87, 121, 186, 193, 194
 - block type, 28
 - data, 50
 - element, 49

- export, 50
- expression, 46
- external type, 29
- function type, 28
- global, 48
- global type, 29
- import, 51
- instruction, 32–34, 37, 38, 40, 43
- limits, 28
- memory, 48
- memory type, 29
- module, 52
- start function, 50
- table, 48
- table type, 29
- validity, 208
- valtype, 25
- value, 3, 6, 12, 13, 21, 31, 57, 58, 60, 63, 64, 87, 97–99, 105, 121, 124, 130, 134, 160, 188, 191, 194, 200, 203, 205, 208
 - abstract syntax, 6, 57
 - binary format, 134
 - external, 60
 - text format, 160
 - type, 10
- value type, 10, 10–13, 15, 17, 20, 25, 29, 31, 37, 47, 57, 87, 105, 121, 124, 137–139, 163, 164, 166, 194, 200, 205, 209, 210
 - abstract syntax, 10
 - binary format, 137
 - text format, 163
- variable instruction, 16
- variable instructions, 37, 98, 139, 166
- vector, 6, 10, 17, 21, 43, 114, 134, 138, 159, 165
 - abstract syntax, 6, 8
 - binary format, 134
 - text format, 159
- vector instruction, 13, 34, 90, 143, 170
- vector number, 57
- vector type, 9, 10, 57, 136, 163, 210
 - binary format, 136
 - text format, 163
- version, 154

W

- white space, 159, 160